Lab2_FFNN_RNN_GNN

May 28, 2026

1 Laboratory 2 - Model Engineering

This notebook implements the second lab of the *AI and Cybersecurity* course.

It follows the official brief and develops a full **Machine Learning and Deep Learning pipeline** using PyTorch and PyTorch Geometric to explore, train, and evaluate models for malware classification from API-call sequences.

This lab is organized into tasks: - Task 1: Frequency-based baseline with sparse API-call count vectors and Logistic Regression - Task 2: Feed Forward Neural Networks with fixed-length sequences, sequential identifiers, learnable embeddings, and embedding-size analysis - Task 3: Recurrent models with dynamic padding, Simple RNN, Bidirectional RNN, and Bidirectional LSTM - Task 4: Graph-based sequence representation and GNN architectures, including GCN, GraphSAGE, and GAT

1.1 Setup

[1]:
```
# --- Check Python runtime ---
import sys
print(sys.executable)
print(sys.version)
```

```
Python 3.12.13
Requirement already satisfied: pip in /usr/local/lib/python3.12/dist-packages
(24.1.2)
Collecting pip
  Downloading pip-26.1.1-py3-none-any.whl.metadata (4.6 kB)
Downloading pip-26.1.1-py3-none-any.whl (1.8 MB)
                           1.8/1.8 MB
35.0 MB/s eta 0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 24.1.2
    Uninstalling pip-24.1.2:
      Successfully uninstalled pip-24.1.2
Successfully installed pip-26.1.1
```

[2]:
```
# --- Dependencies ---
```

```
# This local version expects dependencies to be installed in the active Python␣
 ↪environment.
# From the `Laboratory2/lab` directory, install them with:
#   pip install -r requirements.txt
```

```
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages
(2.10.0+cu128)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-
packages (from torch) (3.29.0)
Requirement already satisfied: typing-extensions>=4.10.0 in
/usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-
packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-
packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in
/usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages
(from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-
packages (from torch) (2025.3.0)
Requirement already satisfied: cuda-bindings==12.9.4 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.9.4)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in
/usr/local/lib/python3.12/dist-packages (from torch) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.8.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in
/usr/local/lib/python3.12/dist-packages (from torch) (11.3.3.83)
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in
/usr/local/lib/python3.12/dist-packages (from torch) (10.3.9.90)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in
/usr/local/lib/python3.12/dist-packages (from torch) (11.7.3.90)
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.5.8.93)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in
/usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in
/usr/local/lib/python3.12/dist-packages (from torch) (2.27.5)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in
/usr/local/lib/python3.12/dist-packages (from torch) (3.4.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in
```

```
/usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in
/usr/local/lib/python3.12/dist-packages (from torch) (1.13.1.3)
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-
packages (from torch) (3.6.0)
Requirement already satisfied: cuda-pathfinder~=1.1 in
/usr/local/lib/python3.12/dist-packages (from cuda-bindings==12.9.4->torch)
(1.5.3)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages
(2.0.2)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages
(2.2.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-
packages (1.6.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-
packages (3.10.0)
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-
packages (0.13.2)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-
packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-
packages (from pandas) (2026.1)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-
packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-
packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in
/usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: contourpy>=1.0.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-
packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (26.1)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-
packages (from matplotlib) (11.3.0)
```

```
Requirement already satisfied: pyparsing>=2.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-
packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages
(4.67.3)
Collecting torch_geometric
  Downloading torch_geometric-2.7.0-py3-none-any.whl.metadata (63 kB)
Requirement already satisfied: aiohttp in /usr/local/lib/python3.12/dist-
packages (from torch_geometric) (3.13.5)
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages
(from torch_geometric) (2025.3.0)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages
(from torch_geometric) (3.1.6)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages
(from torch_geometric) (2.0.2)
Requirement already satisfied: psutil>=5.8.0 in /usr/local/lib/python3.12/dist-
packages (from torch_geometric) (5.9.5)
Requirement already satisfied: pyparsing in /usr/local/lib/python3.12/dist-
packages (from torch_geometric) (3.3.2)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-
packages (from torch_geometric) (2.32.4)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages
(from torch_geometric) (4.67.3)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages
(from torch_geometric) (3.6.0)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->torch_geometric) (2.6.1)
Requirement already satisfied: aiosignal>=1.4.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->torch_geometric) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-
packages (from aiohttp->torch_geometric) (26.1.0)
Requirement already satisfied: frozenlist>=1.1.1 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->torch_geometric) (1.8.0)
Requirement already satisfied: multidict<7.0,>=4.5 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->torch_geometric) (6.7.1)
Requirement already satisfied: propcache>=0.2.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->torch_geometric) (0.4.1)
Requirement already satisfied: yarl<2.0,>=1.17.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp->torch_geometric) (1.23.0)
Requirement already satisfied: idna>=2.0 in /usr/local/lib/python3.12/dist-
packages (from yarl<2.0,>=1.17.0->aiohttp->torch_geometric) (3.13)
Requirement already satisfied: typing-extensions>=4.2 in
/usr/local/lib/python3.12/dist-packages (from
aiosignal>=1.4.0->aiohttp->torch_geometric) (4.15.0)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.12/dist-packages (from jinja2->torch_geometric) (3.0.3)
Requirement already satisfied: charset_normalizer<4,>=2 in
```

```
/usr/local/lib/python3.12/dist-packages (from requests->torch_geometric) (3.4.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.12/dist-packages (from requests->torch_geometric) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.12/dist-packages (from requests->torch_geometric)
(2026.4.22)
Downloading torch_geometric-2.7.0-py3-none-any.whl (1.3 MB)
                         1.3/1.3 MB
10.0 MB/s  0:00:00
Installing collected packages: torch_geometric
Successfully installed torch_geometric-2.7.0
```

[3]:
```python
# --- Import libraries ---
import os
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import json
import math

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler,␣
 ↪LabelEncoder
from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import classification_report, confusion_matrix,␣
 ↪accuracy_score, f1_score, log_loss
from sklearn.decomposition import PCA
from sklearn.impute import SimpleImputer

import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, TensorDataset
from torch.nn.utils.rnn import pad_sequence, pack_padded_sequence,␣
 ↪pad_packed_sequence
import torch_geometric
from torch_geometric.nn import GCNConv, SAGEConv, GATConv, global_mean_pool,␣
 ↪global_max_pool
from torch_geometric.loader import DataLoader as PyGDataLoader
from torch_geometric.utils import to_networkx
from torch_geometric.data import Data
```

```
import networkx as nx

from tqdm import tqdm
```

1.1.1 Runtime information

[4]:
```
# --- Check device availability ---
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"The device is set to: {device}")
if torch.cuda.is_available():
    print(torch.cuda.get_device_name(0))
```

```
Sun May 24 08:51:06 2026
+-----------------------------------------------------------------------------
----------+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version:
13.0     |
+-----------------------------------------+------------------------+-----------
----------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile
Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util
Compute M. |
|                                         |                        |
MIG M. |
|=========================================+========================+===========
==========|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |
0 |
| N/A   53C    P8             10W /   70W |       3MiB /  15360MiB |      0%
Default |
|                                         |                        |
N/A |
+-----------------------------------------+------------------------+-----------
----------+

+-----------------------------------------------------------------------------
----------+
| Processes:
|
|  GPU   GI   CI              PID   Type   Process name
GPU Memory |
|        ID   ID
Usage      |
|=============================================================================
==========|
|  No running processes found
|
```

```
+-----------------------------------------------------------------------------
----------+
```

[5]:
```
# --- Check RAM availability ---
from psutil import virtual_memory
ram_gb = virtual_memory().total / 1e9
print('Your runtime has {:.1f} gigabytes of available RAM\n'.format(ram_gb))

if ram_gb < 20:
  print('Not using a high-RAM runtime')
else:
  print('You are using a high-RAM runtime!')
```

```
Your runtime has 13.6 gigabytes of available RAM

Not using a high-RAM runtime
```

1.1.2 Paths setup

[6]:
```
# --- Local filesystem setup ---
# Google Drive mounting is not required in the local version of this notebook.
print("Using local filesystem paths; Google Drive mount skipped.")
```

```
Mounted at /content/drive
```

[7]:
```
# --- Define Paths ---
from pathlib import Path

# Locate the Laboratory2/lab directory from the current working directory.
cwd = Path.cwd().resolve()
project_root = None

for candidate in [cwd, *cwd.parents]:
    if (candidate / 'data' / 'train.json').exists() and (candidate / 'data' /␣
 ↪'test.json').exists():
        project_root = candidate
        break
    lab_candidate = candidate / 'Laboratory2' / 'lab'
    if (lab_candidate / 'data' / 'train.json').exists() and (lab_candidate /␣
 ↪'data' / 'test.json').exists():
        project_root = lab_candidate
        break

if project_root is None:
    raise FileNotFoundError(
        "Could not locate Laboratory2/lab/data/train.json and test.json. "
        "Run the notebook from the repository/lab tree or adjust project_root␣
 ↪manually."
```

```
    )

data_dir = project_root / 'data'
results_dir = project_root / 'results'

# Keep trailing separators because later cells build paths by string␣
 ↪concatenation.
project_path = str(project_root) + os.sep
data_path = str(data_dir) + os.sep
results_path = str(results_dir) + os.sep

# Ensure directories exist
os.makedirs(project_path, exist_ok=True)
os.makedirs(data_path, exist_ok=True)
os.makedirs(results_path, exist_ok=True)

print(f"Project path: {project_path}")
print(f"Data path: {data_path}")
print(f"Results path: {results_path}")
```

```
Project path: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/
Data path: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/data/
Results path: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/
```

[8]:
```
# --- Set visual style ---
sns.set(style="whitegrid", palette="muted", font_scale=1.1)

def save_plot(fig: plt.Figure, filename: str, path: str = "./plots/", fmt: str␣
 ↪= "png", dpi: int = 300, close_fig: bool = False) -> None:
    """
    Save a Matplotlib figure in a specific to a specified directory.

    Args:
        fig (plt.Figure): Matplotlib figure object to save.
        filename (str): Name of the file to save (e.g., 'plot.png').
        path (str, optional): Directory path to save the figure. Defaults to './
 ↪plots/'.
        fmt (str, optional): File format for the saved figure. Defaults to␣
 ↪'png'.
        dpi (int, optional): Dots per inch for the saved figure. Defaults to␣
 ↪300.

    Returns:
        None
    """
    # Ensure the directory exists
    os.makedirs(path, exist_ok=True)
```

```
    save_path = os.path.join(path, f"{filename}.{fmt}")

    # Save the figure
    fig.savefig(save_path, bbox_inches='tight', pad_inches=0.1, dpi=dpi,␣
↪format=fmt)
    # plt.close(fig) # Removed to display plots in notebook

    if close_fig:
        plt.close(fig)

    print(f"Saved plot: {save_path}")
```

[9]:
```
def _internal_plot_cm(y_true, y_pred, model_name, save_dir):
    """Internal helper to draw the confusion matrix plot."""
    cm = confusion_matrix(y_true, y_pred)

    # Create figure explicitly
    fig, ax = plt.subplots(figsize=(5, 4))

    sns.heatmap(
        cm,
        annot=True,
        fmt='d',
        cmap='Blues',
        cbar=False,
        xticklabels=['Benign (0)', 'Malware (1)'],
        yticklabels=['Benign (0)', 'Malware (1)'],
        ax=ax
    )

    ax.set_xlabel('Predicted Label', fontsize=12)
    ax.set_ylabel('True Label', fontsize=12)
    ax.set_title(f'Confusion Matrix: {model_name}', fontsize=14)

    if save_dir:
        clean_name = model_name.lower().replace(" ", "_").replace("-", "_")
        save_plot(
            fig=fig,
            filename=f"cm_{clean_name}",
            path=save_dir,
            fmt="png",
            dpi=300,
            close_fig=False
        )

    plt.show()
```

1.2 Task 1 — Frequency-based baseline

We implement a simple frequency-based baseline.
- Transform sequences into feature vectors counting API call occurrences.
- Helps evaluate whether simple approaches already perform well before using complex models.

[10]:
```
# Create directory for plots
save_dir = results_path + 'images/' + 'task1_plots/'
os.makedirs(save_dir, exist_ok=True)
```

1.2.1 Frequency-Based Approach

- Extract vocabulary from train and test datasets.
- Use vocabulary to create feature vectors (frequency counts per API call).
- Output dataframe: one row per sequence, one column per API call.

[11]:
```
# --- Load dataset and perform initial inspection ---

# Train set
file_path_train = data_path + 'train.json'
df_train = pd.read_json(file_path_train)

# Basic info
print("Train set:")
print("  Shape (raw):", df_train.shape)
print("  Columns:", list(df_train.columns))

# Test set
file_path_test = data_path + 'test.json'
df_test = pd.read_json(file_path_test)

# Basic info
print("\nTest set:")
print("  Shape (raw):", df_test.shape)
print("  Columns:", list(df_test.columns))
```

```
Train set:
  Shape (raw): (16325, 2)
  Columns: ['api_call_sequence', 'is_malware']

Test set:
  Shape (raw): (6505, 2)
  Columns: ['api_call_sequence', 'is_malware']
```

[12]:
```
df_train
```

[12]:

	api_call_sequence	is_malware
0	[LdrGetDllHandle, LdrGetProcedureAddress, LdrL…	1
1	[NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr…	1

```
2      [FindResourceExW, LoadResource, FindResourceEx…           1
3      [FindResourceExW, LoadResource, FindResourceEx…           1
4      [LdrGetProcedureAddress, SetErrorMode, LdrLoad…           1
…                                                    …          …
16320  [LdrGetProcedureAddress, LdrLoadDll, LdrGetPro…           1
16321  [NtClose, LdrGetProcedureAddress, CryptCreateH…           1
16322  [LdrGetProcedureAddress, LdrGetDllHandle, LdrG…           1
16323  [LdrGetProcedureAddress, LdrGetDllHandle, LdrG…           1
16324  [SetFilePointer, NtReadFile, SetFilePointer, N…           1

[16325 rows x 2 columns]
```

[13]:
```python
malware_counts = df_train['is_malware'].value_counts()
num_total_samples = len(df_train)

print("Malware vs. Benign Samples in Training Set:")
print(malware_counts)

print("\nPercentages:")
for is_malware, count in malware_counts.items():
    percentage = (count / num_total_samples) * 100
    if is_malware == 1:
        print(f"  Malware (1): {count} samples ({percentage:.2f}%) - {(count / num_total_samples)*100:.2f}% of total")
    else:
        print(f"  Benign (0): {count} samples ({percentage:.2f}%) - {(count / num_total_samples)*100:.2f}% of total")
```

```
Malware vs. Benign Samples in Training Set:
is_malware
1    15714
0      611
Name: count, dtype: int64

Percentages:
  Malware (1): 15714 samples (96.26%) - 96.26% of total
  Benign (0): 611 samples (3.74%) - 3.74% of total
```

[14]:
```python
df_test
```

[14]:
```
                                      api_call_sequence  is_malware
0      [NtQueryValueKey, NtClose, NtOpenKey, NtQueryV…           1
1      [LdrGetProcedureAddress, NtClose, NtOpenKey, N…           1
2      [NtOpenKey, NtQueryValueKey, NtClose, NtOpenKe…           1
3      [NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr…           1
4      [NtOpenKey, NtQueryValueKey, NtClose, LdrGetPr…           1
…                                                    …          …
6500   [SetErrorMode, NtOpenFile, NtClose, SHGetFolde…           0
```

```
6501  [NtProtectVirtualMemory, RegOpenKeyExW, RegQue…          1
6502  [RegOpenKeyExW, RegQueryValueExW, RegCloseKey,…          1
6503  [NtQueryValueKey, NtClose, RegOpenKeyExA, RegQ…          1
6504  [NtClose, LdrGetProcedureAddress, CryptCreateH…          1

[6505 rows x 2 columns]
```

[15]:
```
malware_counts = df_test['is_malware'].value_counts()
num_total_samples = len(df_test)

print("Malware vs. Benign Samples in Test Set:")
print(malware_counts)

print("\nPercentages:")
for is_malware, count in malware_counts.items():
    percentage = (count / num_total_samples) * 100
    if is_malware == 1:
        print(f"  Malware (1): {count} samples ({percentage:.2f}%) - {(count /␣
 ↪num_total_samples)*100:.2f}% of total")
    else:
        print(f"  Benign (0): {count} samples ({percentage:.2f}%) - {(count /␣
 ↪num_total_samples)*100:.2f}% of total")
```

```
Malware vs. Benign Samples in Test Set:
is_malware
1    6262
0     243
Name: count, dtype: int64

Percentages:
  Malware (1): 6262 samples (96.26%) - 96.26% of total
  Benign (0): 243 samples (3.74%) - 3.74% of total
```

1.2.2 Extract the vocabularies

[16]:
```
# Extract vocabulary from training set
train_vocab = set(api_call for seq in df_train['api_call_sequence'] for␣
 ↪api_call in seq)

# Count unique API calls
print(f"Train: {len(train_vocab)}")
```

```
Train: 258
```

[17]:
```
# Extract vocabulary from test set
test_vocab = set(api_call for seq in df_test['api_call_sequence'] for api_call␣
 ↪in seq)
```

```
# Count unique API calls
print(f"Test: {len(test_vocab)}")
```

```
Test: 232
```

Q: How many unique API calls does the training set contain? How many in the test set?

- **Training set**: there are **258 unique API calls**.
- **Test set**: there are **232 unique API calls**.

[18]:
```
# --- Find API calls only in the training set ---

only_in_train = train_vocab - test_vocab

# Count and print them
print(f"Number of API calls only in training set: {len(only_in_train)}")
print("API calls only in training set:", only_in_train)
```

```
Number of API calls only in training set: 29
API calls only in training set: {'NtLoadKey', 'NtWriteVirtualMemory',
'RtlRemoveVectoredExceptionHandler', 'GetFileInformationByHandleEx',
'WSASocketW', 'ObtainUserAgentString', 'SendNotifyMessageW', 'NtDeleteFile',
'sendto', 'InternetOpenUrlW', 'CryptProtectMemory', 'NetUserGetInfo',
'IWbemServices_ExecMethod', 'InternetReadFile', 'GetKeyboardState',
'NetUserGetLocalGroups', 'CryptDecodeObjectEx', 'CertOpenStore',
'CreateRemoteThread', 'InternetSetStatusCallback', 'RegDeleteKeyA',
'CertOpenSystemStoreW', 'FindFirstFileExA', 'GetUserNameExA', 'GetAddrInfoW',
'listen', 'GetUserNameExW', 'NtReadVirtualMemory', 'recvfrom'}
```

[19]:
```
# --- Find API calls only in the test set ---

only_in_test = test_vocab - train_vocab

# Count and print them
print(f"Number of API calls only in test set: {len(only_in_test)}")
print("API calls only in test set:", only_in_test)
```

```
Number of API calls only in test set: 3
API calls only in test set: {'ControlService', 'NtDeleteKey', 'WSASocketA'}
```

[20]:
```
# Create a boolean series to identify samples in df_test that contain any of␣
 ↪these API calls
has_test_only_apis = df_test['api_call_sequence'].apply(
    lambda seq: any(api in only_in_test for api in seq)
)

# Count the samples
samples_with_test_only_apis = has_test_only_apis.sum()
```

```
print(f"Number of test set samples with API calls unique to the test set:␣
 ↪{samples_with_test_only_apis}")

# Display the actual rows
print("\nTest samples containing API calls unique to the test set:")
display(df_test[has_test_only_apis])
```

```
Number of test set samples with API calls unique to the test set: 3

Test samples containing API calls unique to the test set:
```

	api_call_sequence	is_malware
2873	[LdrGetDllHandle, LdrGetProcedureAddress, LdrG…	1
4105	[NtProtectVirtualMemory, CreateThread, NtAlloc…	1
5948	[NtDelayExecution, gethostbyname, NtDelayExecu…	1

Q: Are there any API calls that appear only in the test set (but not in the training set)? If yes, how many? Which ones? Yes — there are API calls that appear only in the test set and not in the training set.

- **Number of such API calls**: 3
- **API calls only in the test set**: `NtDeleteKey`, `ControlService`, `WSASocketA`

1.2.3 New frequency-based dataframes

- Create dataframe using training vocabulary as features.
- Count occurrences of each API call per sequence.

[21]:
```
# Create frequency-based dataframe for training data
train_df_freq = pd.DataFrame([{api: seq.count(api) for api in train_vocab} for␣
 ↪seq in df_train['api_call_sequence']])
```

[22]:
```
# Dataframe characteristics
print("Shape (raw):", train_df_freq.shape)
print("Columns:", list(train_df_freq.columns))
```

```
Shape (raw): (16325, 258)
Columns: ['CreateDirectoryW', 'GetForegroundWindow', 'SearchPathW',
'OpenSCManagerW', 'CoInitializeSecurity', 'FindResourceA', 'GetUserNameA',
'CryptExportKey', 'RtlRemoveVectoredExceptionHandler', 'NtResumeThread',
'GetComputerNameA', 'InternetSetOptionA', 'getsockname', 'Module32FirstW',
'RegEnumKeyExA', 'HttpOpenRequestW', 'CryptProtectData', 'Thread32First',
'NtClose', 'ObtainUserAgentString', 'DrawTextExW', 'FindFirstFileExW',
'ioctlsocket', 'Process32FirstW', 'InternetCloseHandle',
'NtQueryInformationFile', 'SendNotifyMessageW', 'RegDeleteValueA', 'sendto',
'closesocket', 'InternetOpenW', 'LookupAccountSidW',
'SHGetSpecialFolderLocation', 'WriteConsoleW', 'NtOpenSection',
'NtSuspendThread', 'NtQueryDirectoryFile', 'LookupPrivilegeValueW',
```

'DrawTextExA', 'EnumServicesStatusA', 'DnsQuery_A', 'IWbemServices_ExecMethod',
'NtQuerySystemInformation', 'GetSystemWindowsDirectoryA',
'GlobalMemoryStatusEx', 'GetAdaptersInfo', 'RegQueryValueExW',
'InternetCrackUrlW', 'CopyFileW', 'NtReadVirtualMemory', 'CreateRemoteThread',
'RegDeleteKeyA', 'NtCreateThreadEx', 'OleInitialize', 'SetFileAttributesW',
'NtGetContextThread', 'CreateProcessInternalW', 'MessageBoxTimeoutW',
'CryptDecrypt', 'WriteConsoleA', 'RegEnumValueW', 'NtQueryAttributesFile',
'DeviceIoControl', 'CryptCreateHash', 'NtOpenMutant', 'InternetConnectA',
'RegEnumKeyExW', 'SHGetFolderPathW', 'SetFileTime', 'SetStdHandle',
'GetFileSize', 'NtCreateMutant', 'NtWriteFile', 'CoCreateInstanceEx',
'NtEnumerateKey', 'GetDiskFreeSpaceExW', 'GetTempPathW',
'NtOpenDirectoryObject', 'GetVolumeNameForVolumeMountPointW',
'GetFileInformationByHandle', 'DeleteUrlCacheEntryA', 'GetFileType',
'CoUninitialize', 'FindWindowA', 'recv', 'setsockopt',
'GetFileInformationByHandleEx', 'OpenSCManagerA', 'GetUserNameExA',
'NtOpenFile', 'connect', 'CryptAcquireContextA', 'GetSystemDirectoryA',
'FindWindowW', 'GetVolumePathNameW', 'RegSetValueExW', 'NtProtectVirtualMemory',
'InternetConnectW', 'RegEnumValueA', 'Module32NextW', 'CopyFileA', 'NtReadFile',
'RegQueryInfoKeyW', 'RemoveDirectoryA', 'DeleteUrlCacheEntryW',
'NetUserGetInfo', 'RegQueryValueExA', 'RegDeleteValueW', 'RegOpenKeyExW',
'DeleteFileW', 'RegCloseKey', 'GetVolumePathNamesForVolumeNameW',
'NtAllocateVirtualMemory', 'SetUnhandledExceptionFilter', 'SetFilePointerEx',
'CryptEncrypt', 'CertOpenStore', 'FindResourceExA', 'GetComputerNameW',
'GetTimeZoneInformation', 'CertOpenSystemStoreW', 'NtQueryKey', 'GetFileSizeEx',
'RtlAddVectoredExceptionHandler', 'listen', 'LdrGetProcedureAddress',
'GetAsyncKeyState', 'FindResourceExW', 'GetUserNameW', 'NtSetValueKey',
'RegCreateKeyExW', 'recvfrom', 'SetFilePointer', 'MoveFileWithProgressW',
'NtUnmapViewOfSection', 'send', 'NtTerminateThread', 'GetSystemDirectoryW',
'OutputDebugStringA', 'GetFileVersionInfoW', 'timeGetTime',
'IWbemServices_ExecQuery', 'CreateServiceW', 'NtSetContextThread',
'HttpOpenRequestA', 'NtDeviceIoControlFile', 'IsDebuggerPresent',
'GetFileVersionInfoSizeW', 'GetSystemTimeAsFileTime', 'NtOpenProcess',
'CoCreateInstance', 'NtEnumerateValueKey', 'SetWindowsHookExA', 'StartServiceW',
'gethostbyname', 'NtDeleteFile', 'InternetQueryOptionA', 'InternetOpenUrlW',
'CoInitializeEx', 'GetSystemWindowsDirectoryW', 'RtlDecompressBuffer',
'LoadStringW', '__exception__', 'CreateActCtxW', 'SizeofResource',
'GlobalMemoryStatus', 'NtQueryValueKey', 'NtDelayExecution', 'GetSystemInfo',
'GetAdaptersAddresses', 'SetEndOfFile', 'GetKeyboardState', 'LoadStringA',
'RegSetValueExA', 'NetUserGetLocalGroups', 'GetDiskFreeSpaceW', 'StartServiceA',
'LdrGetDllHandle', 'GetNativeSystemInfo', 'InternetGetConnectedState',
'InternetSetStatusCallback', 'CopyFileExW', 'UuidCreate', 'GetBestInterfaceEx',
'WriteProcessMemory', 'HttpSendRequestA', 'WSARecv', 'FindWindowExA',
'CryptHashData', 'GetShortPathNameW', 'Thread32Next', 'InternetOpenA',
'NtLoadKey', 'GetFileAttributesW', 'NtWriteVirtualMemory', 'RegDeleteKeyW',
'UnhookWindowsHookEx', 'NtFreeVirtualMemory', 'NtOpenThread', 'LdrLoadDll',
'WSASocketW', 'LoadResource', 'SetErrorMode', 'NtOpenKeyEx', 'RegOpenKeyExA',
'Process32NextW', 'RegisterHotKey', 'GetFileAttributesExW', 'GetKeyState',
'InternetCrackUrlA', 'HttpQueryInfoA', 'FindWindowExW', 'RegCreateKeyExA',

```
'NtTerminateProcess', 'CreateServiceA', 'EnumWindows', 'CoGetClassObject',
'OpenServiceW', 'CryptProtectMemory', 'shutdown', 'FindResourceW',
'NtCreateFile', 'GetFileVersionInfoExW', 'RemoveDirectoryW', 'InternetReadFile',
'NtSetInformationFile', 'getaddrinfo', 'GetSystemMetrics', 'socket',
'CryptGenKey', 'ShellExecuteExW', 'GetCursorPos', 'NtDuplicateObject',
'CryptDecodeObjectEx', 'LdrUnloadDll', 'CreateJobObjectW', 'NtQueueApcThread',
'CreateThread', 'WSAStartup', 'FindFirstFileExA', 'select', 'SetWindowsHookExW',
'RegQueryInfoKeyA', 'GetAddrInfoW', 'MessageBoxTimeoutA', 'InternetOpenUrlA',
'NtCreateSection', 'RegEnumKeyW', 'CreateToolhelp32Snapshot', 'OpenServiceA',
'ReadProcessMemory', 'NtCreateKey', 'NtMapViewOfSection', 'GetUserNameExW',
'GetFileVersionInfoSizeExW', 'CryptAcquireContextW', 'NtOpenKey', 'bind']
```

[23]:
```
train_df_freq.head(10)
```

[23]:

	CreateDirectoryW	GetForegroundWindow	SearchPathW	OpenSCManagerW	\
0	0	0	0	0	
1	0	0	0	0	
2	0	0	0	0	
3	0	0	0	0	
4	0	0	0	0	
5	0	0	0	0	
6	0	0	0	0	
7	0	0	0	0	
8	0	0	0	0	
9	0	0	0	0	

	CoInitializeSecurity	FindResourceA	GetUserNameA	CryptExportKey	\
0	0	0	0	0	
1	0	0	0	0	
2	0	0	0	0	
3	0	1	0	0	
4	0	0	0	0	
5	0	1	0	0	
6	0	0	0	0	
7	0	0	0	0	
8	0	1	0	0	
9	0	0	0	0	

	RtlRemoveVectoredExceptionHandler	NtResumeThread	…	\
0	0	0	…	
1	0	0	…	
2	0	0	…	
3	0	0	…	
4	0	0	…	
5	0	0	…	
6	0	0	…	
7	0	0	…	

```
8                                   0               0  …
9                                   0               0  …

   CreateToolhelp32Snapshot  OpenServiceA  ReadProcessMemory  NtCreateKey  \
0                         0             0                  0            0
1                         0             0                  0            0
2                         0             0                  0            0
3                         0             0                  0            0
4                         0             0                  0            0
5                         0             0                  0            0
6                         0             0                  0            0
7                         0             0                  0            0
8                         0             0                  0            0
9                         0             0                  0            0

   NtMapViewOfSection  GetUserNameExW  GetFileVersionInfoSizeExW  \
0                   0               0                          0
1                   0               0                          0
2                   0               0                          0
3                   1               0                          0
4                   0               0                          0
5                   1               0                          0
6                   0               0                          0
7                   1               0                          0
8                   1               0                          0
9                   0               0                          0

   CryptAcquireContextW  NtOpenKey  bind
0                     0          0     0
1                     0         14     0
2                     0          0     0
3                     0          0     0
4                     0          0     0
5                     0          0     0
6                     0          0     0
7                     0          2     0
8                     0          0     0
9                     0          2     0

[10 rows x 258 columns]
```

[24]:
```
# Create frequency-based dataframe for test data
test_df_freq = pd.DataFrame([{api: seq.count(api) for api in test_vocab} for␣
 ↪seq in df_test['api_call_sequence']])
```

[25]:
```
# Dataframe characteristics
print("Shape (raw):", test_df_freq.shape)
```

```
print("Columns:", list(test_df_freq.columns))
```

```
Shape (raw): (6505, 232)
Columns: ['CreateDirectoryW', 'GetForegroundWindow', 'SearchPathW',
'OpenSCManagerW', 'CoInitializeSecurity', 'FindResourceA', 'GetUserNameA',
'CryptExportKey', 'NtResumeThread', 'GetComputerNameA', 'InternetSetOptionA',
'getsockname', 'RegEnumKeyExA', 'Module32FirstW', 'HttpOpenRequestW',
'CryptProtectData', 'Thread32First', 'NtClose', 'DrawTextExW',
'FindFirstFileExW', 'ioctlsocket', 'Process32FirstW', 'InternetCloseHandle',
'NtQueryInformationFile', 'RegDeleteValueA', 'InternetOpenW', 'closesocket',
'LookupAccountSidW', 'WriteConsoleW', 'SHGetSpecialFolderLocation',
'NtOpenSection', 'NtSuspendThread', 'NtQueryDirectoryFile',
'LookupPrivilegeValueW', 'DnsQuery_A', 'DrawTextExA', 'EnumServicesStatusA',
'NtQuerySystemInformation', 'GetSystemWindowsDirectoryA',
'GlobalMemoryStatusEx', 'GetAdaptersInfo', 'RegQueryValueExW',
'InternetCrackUrlW', 'CopyFileW', 'NtCreateThreadEx', 'OleInitialize',
'SetFileAttributesW', 'MessageBoxTimeoutW', 'CreateProcessInternalW',
'NtGetContextThread', 'CryptDecrypt', 'WriteConsoleA', 'RegEnumValueW',
'NtQueryAttributesFile', 'DeviceIoControl', 'CryptCreateHash', 'NtOpenMutant',
'InternetConnectA', 'RegEnumKeyExW', 'SHGetFolderPathW', 'SetFileTime',
'SetStdHandle', 'GetFileSize', 'NtCreateMutant', 'NtWriteFile',
'CoCreateInstanceEx', 'NtEnumerateKey', 'GetDiskFreeSpaceExW', 'GetTempPathW',
'NtOpenDirectoryObject', 'GetVolumeNameForVolumeMountPointW',
'GetFileInformationByHandle', 'DeleteUrlCacheEntryA', 'GetFileType',
'CoUninitialize', 'FindWindowA', 'setsockopt', 'recv', 'OpenSCManagerA',
'NtOpenFile', 'connect', 'CryptAcquireContextA', 'GetSystemDirectoryA',
'FindWindowW', 'GetVolumePathNameW', 'RegSetValueExW', 'NtProtectVirtualMemory',
'InternetConnectW', 'RegEnumValueA', 'Module32NextW', 'CopyFileA', 'NtReadFile',
'RegQueryInfoKeyW', 'RemoveDirectoryA', 'DeleteUrlCacheEntryW',
'RegQueryValueExA', 'RegDeleteValueW', 'RegOpenKeyExW', 'DeleteFileW',
'RegCloseKey', 'GetVolumePathNamesForVolumeNameW', 'NtDeleteKey',
'NtAllocateVirtualMemory', 'SetUnhandledExceptionFilter', 'CryptEncrypt',
'SetFilePointerEx', 'FindResourceExA', 'GetComputerNameW',
'GetTimeZoneInformation', 'NtQueryKey', 'RtlAddVectoredExceptionHandler',
'GetFileSizeEx', 'LdrGetProcedureAddress', 'GetAsyncKeyState',
'FindResourceExW', 'WSASocketA', 'GetUserNameW', 'NtSetValueKey',
'RegCreateKeyExW', 'SetFilePointer', 'MoveFileWithProgressW',
'NtUnmapViewOfSection', 'send', 'NtTerminateThread', 'GetSystemDirectoryW',
'OutputDebugStringA', 'GetFileVersionInfoW', 'timeGetTime', 'CreateServiceW',
'IWbemServices_ExecQuery', 'NtSetContextThread', 'HttpOpenRequestA',
'NtDeviceIoControlFile', 'IsDebuggerPresent', 'GetFileVersionInfoSizeW',
'GetSystemTimeAsFileTime', 'CoCreateInstance', 'NtOpenProcess',
'NtEnumerateValueKey', 'SetWindowsHookExA', 'StartServiceW', 'gethostbyname',
'InternetQueryOptionA', 'CoInitializeEx', 'GetSystemWindowsDirectoryW',
'ControlService', 'RtlDecompressBuffer', 'LoadStringW', '__exception__',
'SizeofResource', 'CreateActCtxW', 'GlobalMemoryStatus', 'NtQueryValueKey',
'NtDelayExecution', 'GetSystemInfo', 'GetAdaptersAddresses', 'SetEndOfFile',
'LoadStringA', 'RegSetValueExA', 'GetDiskFreeSpaceW', 'StartServiceA',
```

```
'LdrGetDllHandle', 'GetNativeSystemInfo', 'InternetGetConnectedState',
'CopyFileExW', 'UuidCreate', 'GetBestInterfaceEx', 'WriteProcessMemory',
'WSARecv', 'HttpSendRequestA', 'FindWindowExA', 'CryptHashData',
'GetShortPathNameW', 'Thread32Next', 'InternetOpenA', 'GetFileAttributesW',
'RegDeleteKeyW', 'UnhookWindowsHookEx', 'NtFreeVirtualMemory', 'NtOpenThread',
'LdrLoadDll', 'LoadResource', 'SetErrorMode', 'NtOpenKeyEx', 'RegOpenKeyExA',
'Process32NextW', 'RegisterHotKey', 'GetKeyState', 'InternetCrackUrlA',
'GetFileAttributesExW', 'HttpQueryInfoA', 'FindWindowExW', 'RegCreateKeyExA',
'NtTerminateProcess', 'CreateServiceA', 'EnumWindows', 'CoGetClassObject',
'OpenServiceW', 'shutdown', 'FindResourceW', 'NtCreateFile',
'GetFileVersionInfoExW', 'RemoveDirectoryW', 'NtSetInformationFile',
'getaddrinfo', 'GetSystemMetrics', 'socket', 'CryptGenKey', 'ShellExecuteExW',
'GetCursorPos', 'NtDuplicateObject', 'LdrUnloadDll', 'CreateJobObjectW',
'NtQueueApcThread', 'WSAStartup', 'CreateThread', 'SetWindowsHookExW', 'select',
'RegQueryInfoKeyA', 'MessageBoxTimeoutA', 'InternetOpenUrlA', 'NtCreateSection',
'RegEnumKeyW', 'CreateToolhelp32Snapshot', 'OpenServiceA', 'ReadProcessMemory',
'NtCreateKey', 'NtMapViewOfSection', 'GetFileVersionInfoSizeExW',
'CryptAcquireContextW', 'NtOpenKey', 'bind']
```

[26]:
```
test_df_freq.head(10)
```

[26]:

	CreateDirectoryW	GetForegroundWindow	SearchPathW	OpenSCManagerW	\
0	0	0	0	0	
1	0	0	0	0	
2	0	0	0	0	
3	0	0	0	0	
4	0	0	0	0	
5	0	0	0	0	
6	0	0	0	0	
7	0	0	1	0	
8	0	0	0	0	
9	0	0	0	0	

	CoInitializeSecurity	FindResourceA	GetUserNameA	CryptExportKey	\
0	0	1	0	0	
1	0	0	0	0	
2	0	0	0	0	
3	0	0	0	0	
4	0	0	0	0	
5	0	0	0	0	
6	0	0	0	0	
7	0	12	0	0	
8	0	0	0	0	
9	0	0	0	0	

	NtResumeThread	GetComputerNameA	…	RegEnumKeyW	\
0	0	0	…	0	

```
1                0                0  …            0
2                0                0  …            0
3                1                0  …            2
4                0                0  …            2
5                0                0  …            0
6                0                0  …            2
7                0                0  …            0
8                1                0  …            0
9                0                0  …            2

   CreateToolhelp32Snapshot  OpenServiceA  ReadProcessMemory  NtCreateKey  \
0                         0             0                  0            0
1                         0             0                  0            0
2                         0             0                  0            0
3                         0             0                  0            0
4                         0             0                  0            0
5                         0             0                  0            0
6                         0             0                  0            0
7                         0             0                  0            0
8                         0             0                  0            0
9                         0             0                  0            0

   NtMapViewOfSection  GetFileVersionInfoSizeExW  CryptAcquireContextW  \
0                   1                          0                     0
1                   1                          0                     0
2                   0                          0                     0
3                   1                          0                     1
4                   0                          0                     0
5                   0                          0                     0
6                   0                          0                     1
7                   0                          0                     0
8                   0                          0                     1
9                   0                          0                     1

   NtOpenKey  bind
0          1     0
1          2     0
2          3     0
3          1     0
4          1     0
5          3     0
6          2     0
7          1     0
8          0     0
9          2     0

[10 rows x 232 columns]
```

Q: Can you use the test vocabulary to build the new test dataframe? If not, how do you handle API calls in the test set that do not exist in the training vocabulary? No, you cannot use the test vocabulary to build the test dataframe.

Machine learning models require the feature space (columns) during inference (testing) to be identical to the feature space used during training.

Common strategies for handling API calls in the test set that did not exist in the training vocabulary include:

1. **Add an "Unknown" Column**: Map all unseen API calls to a single generic token (e.g., <UNK>) or a specific "is_unknown" feature. This preserves the information that *an* action occurred, even if the specific action is unrecognized.
2. **Ignore Them**: Simply discard or zero-out any API calls that were not seen during training.

Decision: In this specific case, since there are **very few unseen API calls** (i.e. *only 3* samples) in the test set, we decided to **ignore them**. This approach simplifies the pipeline without significantly impacting the model's performance.

Sparse Vectors Analysis

[27]:
```
# --- Non-zero elements analysis for training data ---

# Calculate average non-zero elements per row
avg_nonzero_train = (train_df_freq != 0).sum(axis=1).mean()
print(f"Average non-zero elements per row in train_df_freq: {avg_nonzero_train:.
  ↪2f}")

# Calculate ratio of non-zero elements
ratio_nonzero_train = avg_nonzero_train / train_df_freq.shape[1]
print(f"Ratio of non-zero elements per row in train_df_freq:␣
  ↪{ratio_nonzero_train:.4f}")
```

```
Average non-zero elements per row in train_df_freq: 21.95
Ratio of non-zero elements per row in train_df_freq: 0.0851
```

[28]:
```
# --- Non-zero elements analysis for test data ---

# Calculate average non-zero elements per row
avg_nonzero_test = (test_df_freq != 0).sum(axis=1).mean()
print(f"Average non-zero elements per row in test_df_freq: {avg_nonzero_test:.
  ↪2f}")

# Calculate ratio of non-zero elements
ratio_nonzero_test = avg_nonzero_test / test_df_freq.shape[1]
print(f"Ratio of non-zero elements per row in test_df_freq: {ratio_nonzero_test:
  ↪.4f}")
```

```
Average non-zero elements per row in test_df_freq: 24.28
Ratio of non-zero elements per row in test_df_freq: 0.1046
```

Q: How many non-zero elements per row do you have on average in the training set? How many in the test set? What is the ratio with respect to the number of elements per row? On average:

- **Training set**: ~22 non-zero elements per row, which is **8.5%** of all elements.
- **Test set**: ~24 non-zero elements per row, which is **10.5%** of all elements.

Q: The original API sequences were ordered. Is it still the case now in the frequency-based dataframe? Why? No, the original order of API calls is **not preserved** in the frequency-based dataframe.

This is because the frequency-based representation **counts how many times each API call occurs** in a sequence, without keeping track of their positions. Each row now simply contains **one column per API call in the vocabulary**, with the value representing its count, so the sequential order is lost.

This approach converts sequences into fixed-dimensional vectors, simplifying the computation but discarding temporal information. As a result, complex behavioral patterns based on the chronology of calls cannot be captured by this representation.

1.2.4 Feed the frequency-based datasets to a classifier

- Any classifier can be used (shallow/deep neural or non-neural).
- Goal: evaluate baseline performance on sparse vectors without sequence information.

Training

[29]:
```
# --- Training with a classifier ---

# Split data into features and target
X = train_df_freq
y = df_train['is_malware']

# Create a validation set from training set
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.4, random_state=42, stratify=y
)

# Construct test set with features aligned to train_vocab
X_test = pd.DataFrame([{api: seq.count(api) for api in train_vocab} for seq in␣
 ↪df_test['api_call_sequence']])
y_test = df_test['is_malware']

# Train a simple classifier (using default hyperparameters for a simple␣
 ↪baseline)
clf = LogisticRegression(random_state=42, solver='liblinear') # 'liblinear'␣
 ↪solver is good for small datasets and sparse data
clf.fit(X_train, y_train)

# Compute training and validation loss
```

```
y_train_proba = clf.predict_proba(X_train)
y_val_proba = clf.predict_proba(X_val)

print(f"Training log-loss: {log_loss(y_train, y_train_proba):.4f}")
print(f"Validation log-loss: {log_loss(y_val, y_val_proba):.4f}")
```

```
Training log-loss: 0.0806
Validation log-loss: 0.1225
```

Evaluation

[30]:
```
def evaluate_model(model, X, y_true, model_name: str = "Unnamed model"):
    """
    Evaluate a trained scikit-learn model on a given dataset and return the␣
 ↪classification report.
    Handles missing predicted classes gracefully (zero_division=0).
    """
    # Get predictions
    y_pred = model.predict(X)

    # Identify missing classes (not predicted at all)
    missing_classes = set(np.unique(y_true)) - set(np.unique(y_pred))
    if missing_classes:
        missing_classes = [int(x) for x in sorted(missing_classes)]
        print(f"Warning: {model_name} made no predictions for classes:␣
 ↪{missing_classes}")

    # Generate classification report
    report = classification_report(y_true, y_pred, digits=4, zero_division=0)

    return report
```

[31]:
```
def plot_cm_task1(model, X_test, y_test, model_name, save_dir=None):
    """
    Specific for Task 1 (Scikit-Learn models).
    Generates predictions using .predict() and then plots.
    """
    # 1. Get predictions (Scikit-learn models work directly on X_test)
    y_pred = model.predict(X_test)

    # 2. Use the core helper to plot
    # Note: y_test might need to be converted to a simple list/array if it's a␣
 ↪pandas Series
    _internal_plot_cm(y_test, y_pred, model_name, save_dir)
```

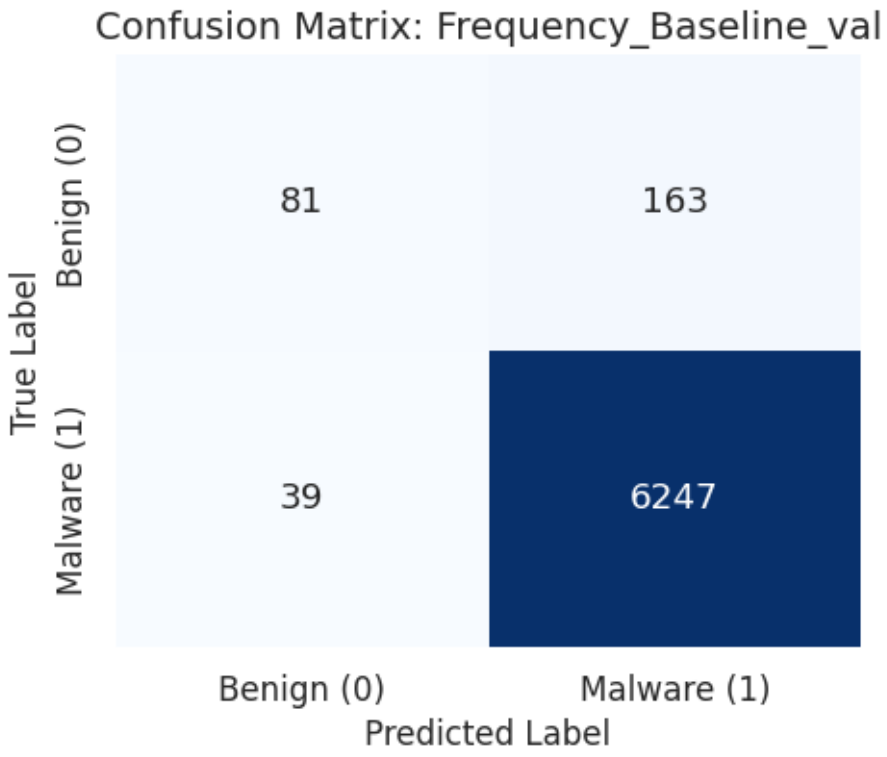
[32]:
```
# --- Evaluate validation and test set and print classification reports ---

print(f"\nValidation classification report:\n")
```

```
report = evaluate_model(clf, X_val, y_val, "Logistic Regression (Validation)")
print(report)

print(f"\nTest classification report:\n")
report = evaluate_model(clf, X_test, y_test, "Logistic Regression (Test)")
print(report)
```

```
Validation classification report:

              precision    recall  f1-score   support

           0     0.6750    0.3320    0.4451       244
           1     0.9746    0.9938    0.9841      6286

    accuracy                         0.9691      6530
   macro avg     0.8248    0.6629    0.7146      6530
weighted avg     0.9634    0.9691    0.9639      6530


Test classification report:

              precision    recall  f1-score   support

           0     0.6885    0.3457    0.4603       243
           1     0.9751    0.9939    0.9844      6262

    accuracy                         0.9697      6505
   macro avg     0.8318    0.6698    0.7223      6505
weighted avg     0.9644    0.9697    0.9648      6505
```

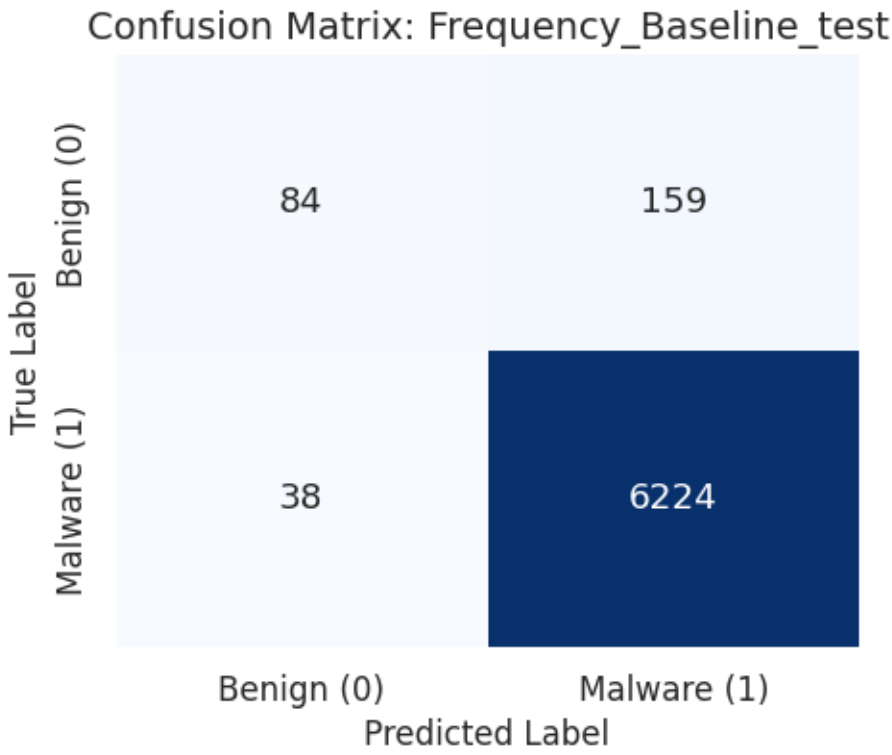
[33]:
```
plot_cm_task1(
    model=clf,
    X_test=X_val,
    y_test=y_val,
    model_name="Frequency_Baseline_val",
    save_dir=results_path + 'images/task1_plots/'
)
```

```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task1_plots/cm_frequency_baseline_val.png
```



[34]:
```
plot_cm_task1(
    model=clf,
    X_test=X_test,
    y_test=y_test,
    model_name="Frequency_Baseline_test",
    save_dir=results_path + 'images/task1_plots/'
)
```

```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task1_plots/cm_frequency_baseline_test.png
```



Q: Report how you chose the hyperparameters of your classifier, and the final performance on the test set. We used Logistic Regression as a deliberately simple frequency-based baseline. This choice is useful because the goal of Task 1 is not to maximize complexity, but to understand how much discriminative information is already contained in API-call counts before introducing sequence-aware models.

The model was trained with the `liblinear` solver, default L2 regularization (`C=1.0`), no class weighting, and `random_state=42` for reproducibility. We did not perform an extensive hyperparameter search at this stage, because the classifier is meant to be a stable reference point for the later FFNN, RNN, and GNN experiments.

The final test performance is strong in terms of global accuracy: the model reaches `96.97%` accuracy and macro F1 `0.7223`. However, the class-wise report shows a clear imbalance effect. Malware is detected very well, with precision `0.9751` and recall `0.9939`, while the benign class has good precision (`0.6885`) but low recall (`0.3457`). Therefore, the baseline is effective at identifying malware, but it tends to classify many benign samples as malware.

Q: Is the final performance good, even ignoring the order of API calls and handling very sparse vectors? Yes, the final performance is good for a simple frequency-based baseline. Even though the representation is sparse and completely discards the order of API calls, the model still reaches `96.97%` accuracy and macro F1 `0.7223` on the test set. This indicates that the presence and frequency of API calls already contain substantial information for distinguishing malware from benign software.

At the same time, the per-class metrics show the limitation of this approach. The model obtains excellent malware recall (`0.9939`), but benign recall is only `0.3457`. This means that the high accuracy is partly driven by the majority malware class. The baseline is therefore useful and competitive, but not fully balanced: later models should try to improve minority-class recall and macro F1, even if this slightly reduces overall accuracy.

1.3 Task 2 - Feed Forward Neural Network (FFNN)

[35]:
```
# Create directory for plots
save_dir = results_path + 'images/' + 'task2_plots/'
os.makedirs(save_dir, exist_ok=True)
```

1.3.1 API Call Statistics

[36]:
```
# --- Compute the length (number of API calls) per sequence ---
df_train['seq_length'] = df_train['api_call_sequence'].apply(len)
df_test['seq_length'] = df_test['api_call_sequence'].apply(len)

# --- Basic descriptive statistics ---
print("Train set sequence length stats:")
print(df_train['seq_length'].describe(), "\n")

print("Test set sequence length stats:")
print(df_test['seq_length'].describe(), "\n")

# --- Check if all sequences have the same length ---
same_length_train = df_train['seq_length'].nunique() == 1
same_length_test = df_test['seq_length'].nunique() == 1
print(f"All train sequences same length? {same_length_train}")
print(f"All test sequences same length? {same_length_test}\n")
```

```
Train set sequence length stats:
count    16325.000000
mean        75.030812
std          8.951411
min         60.000000
25%         67.000000
50%         75.000000
75%         83.000000
max         90.000000
Name: seq_length, dtype: float64

Test set sequence length stats:
count    6505.000000
mean       86.331130
std         9.113511
min        70.000000
25%        79.000000
```

```
50%        87.000000
75%        94.000000
max       100.000000
Name: seq_length, dtype: float64

All train sequences same length? False
All test sequences same length? False
```

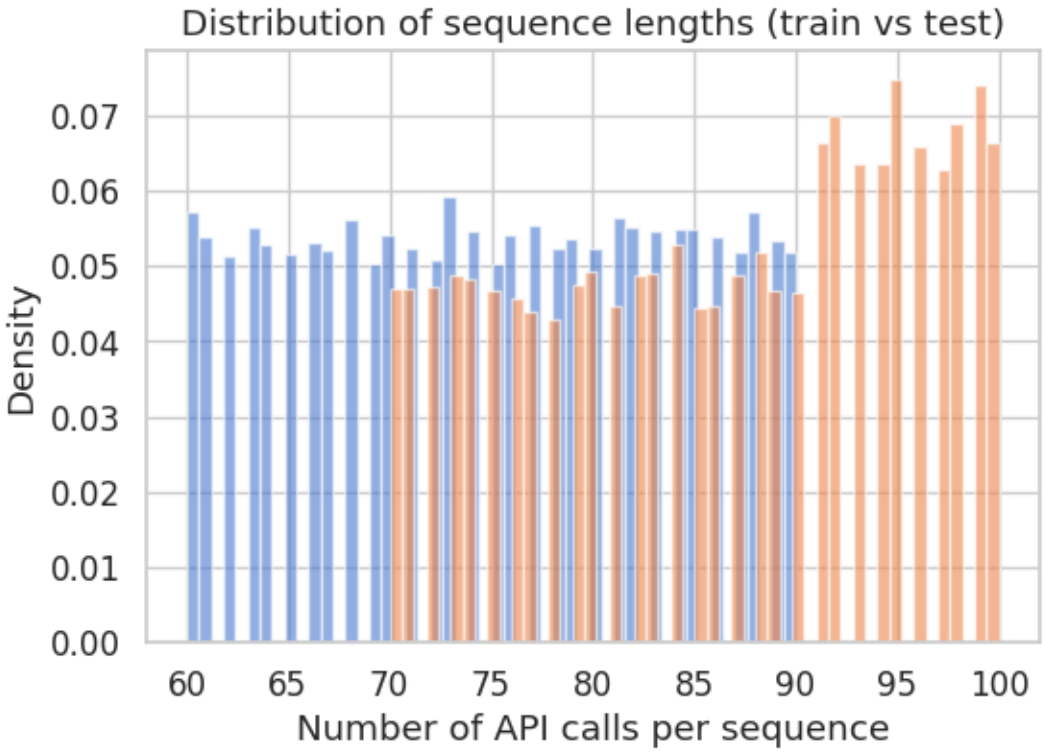
[37]:
```
df_train
```

[37]:
```
                                        api_call_sequence  is_malware  \
0      [LdrGetDllHandle, LdrGetProcedureAddress, LdrL…           1
1      [NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr…           1
2      [FindResourceExW, LoadResource, FindResourceEx…           1
3      [FindResourceExW, LoadResource, FindResourceEx…           1
4      [LdrGetProcedureAddress, SetErrorMode, LdrLoad…           1
…                                                    …          …
16320  [LdrGetProcedureAddress, LdrLoadDll, LdrGetPro…           1
16321  [NtClose, LdrGetProcedureAddress, CryptCreateH…           1
16322  [LdrGetProcedureAddress, LdrGetDllHandle, LdrG…           1
16323  [LdrGetProcedureAddress, LdrGetDllHandle, LdrG…           1
16324  [SetFilePointer, NtReadFile, SetFilePointer, N…           1

       seq_length
0              73
1              88
2              79
3              71
4              63
…             …
16320          64
16321          78
16322          84
16323          62
16324          77

[16325 rows x 3 columns]
```

[38]:
```
# --- Compare distributions visually ---

plt.figure(figsize=(6, 4))
plt.hist(df_train['seq_length'], bins=50, alpha=0.6, label='Train',␣
 ↪density=True)
plt.hist(df_test['seq_length'], bins=50, alpha=0.6, label='Test', density=True)
plt.xlabel("Number of API calls per sequence")
plt.ylabel("Density")
```

```
plt.title("Distribution of sequence lengths (train vs test)")

# Save the plot to the specified path
save_plot(plt.gcf(), f"distribution_sequence_lengths_train_test", save_dir)

plt.show()
```

```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/distribution_sequence_lengths_train_test.png
```



Q: Do you have the same number of API calls per sequence? If not, is the distribution of API calls per sequence the same for training and test sets? No, the sequences **do not all have the same length** in either the training or test sets.

- **Training set**: sequence lengths range from 60 to 90 API calls (mean 75).
- **Test set**: sequence lengths range from 70 to 100 API calls (mean 86).

The **sequences in the test set** are not only longer on average, but also fall within a range (90–100) that is completely absent from the training set. This discrepancy means that the model will have to generalize to sequence lengths it has never encountered during training, posing an additional challenge for sequential architectures.

Q: Can a FFNN handle a variable number of elements? If not, why? No, a feedforward neural network (FFNN) **cannot directly handle a variable number** of elements per input.

- FFNNs require a **fixed-size input vector**, because each input neuron corresponds to a specific feature, and the weight matrix has a fixed shape.
- If the number of elements (features) changes between samples, the network **cannot align inputs with the learned weights**, so it won't work properly.

Due to this architectural constraint, in order to process variable-length API sequences using a FFNN, they must be converted into fixed-length vectors using padding (to extend short sequences) or truncation (to shorten longer ones).

1.3.2 Fixed-Size Sequences

Q: How to estimate a fixed-size candidate? Which partition do you use to estimate it? To estimate a fixed-size input for an FFNN, it's possible to choose a representative sequence length from the training set only, since the model should not use information from the test set.

Common strategies include: 1. **Maximum length**: ensures all training sequences fit, but can be too large and waste memory. 2. **Median length**: gives a typical sequence size, but some longer sequences will need truncation. 3. **75th percentile length**: a compromise that covers most sequences without being overly large.

Among these strategies, we opted for the maximum training set length (90). This choice is justified by the fact that the distribution of lengths (as seen earlier) is fairly uniform and does not contain any extreme outliers; therefore, the computational "cost" of using 90 as a fixed size is offset by the advantage of not having to truncate (and thus lose) information in any training sequence.

[39]:
```python
# Compute sequence lengths in training set
seq_lengths = df_train['api_call_sequence'].apply(len)

# Option 1: maximum length
max_len = seq_lengths.max()
print(f"Max length: {max_len}")

# Option 2: median
median_len = int(seq_lengths.median())
print(f"Median length: {median_len}")

# Option 3: 75th percentile
p75_len = int(np.percentile(seq_lengths, 75))
print(f"75th percentile length: {p75_len}")
```

```
Max length: 90
Median length: 75
75th percentile length: 83
```

Q: Given the estimate, what technique could you use to obtain the same number of API calls per sequence? To obtain the same number of API calls per sequence, the main strategies are:

1. **Padding**: Add "dummy" or special tokens to sequences **shorter than the fixed length** so they reach the target size.

2. **Truncation**: **Cut sequences longer than the fixed length** to fit the target size.

By combining padding and truncation based on the estimated fixed length, you can ensure that **all sequences have the same number of elements**, which is required for an FFNN.

[40]:
```python
def pad_truncate_sequence(seq, fixed_len, pad_value=0):
    """
    Convert a variable-length sequence into a fixed-length sequence.

    Args:
        seq (list): original sequence (list of API calls)
        fixed_len (int): target length L
        pad_value: value to use for padding (0 is common for integers)

    Returns:
        list of length fixed_len
    """
    if len(seq) < fixed_len:
        # Pad at the end
        seq_padded = seq + [pad_value] * (fixed_len - len(seq))
    else:
        # Truncate at the end
        seq_padded = seq[:fixed_len]
    return seq_padded
```

[41]:
```python
# --- Perform padding or truncating on the training data ---

L = max_len # chosen fixed size

# Apply the padding or truncating to all the dataframe
df_train['seq_fixed'] = df_train['api_call_sequence'].apply(lambda x:␣
 ↪pad_truncate_sequence(x, L))
```

[42]:
```python
# --- Inspect the train set (e.g. the first row) ---

print("First row of the train set before padding:")
print(df_train['api_call_sequence'].iloc[0])
print(f"Lenght of the sequence: {len(df_train['api_call_sequence'].iloc[0])}")␣
 ↪# should be less than L

print("\nFirst row of the train set after padding:")
print(df_train['seq_fixed'].iloc[0])
print(f"Lenght of the sequence: {len(df_train['seq_fixed'].iloc[0])}") # should␣
 ↪be exactly L
```

```
First row of the train set before padding:
['LdrGetDllHandle', 'LdrGetProcedureAddress', 'LdrLoadDll',
'LdrGetProcedureAddress', 'LdrGetDllHandle', 'LdrGetProcedureAddress',
'GetTimeZoneInformation', 'LoadStringW', 'RegOpenKeyExW', 'RegQueryValueExW',
```

```
'RegCloseKey', 'LdrGetDllHandle', 'LdrGetProcedureAddress', 'LdrGetDllHandle',
'LdrGetProcedureAddress', 'SetErrorMode', 'LdrLoadDll', 'SetErrorMode',
'LdrLoadDll', 'SetErrorMode', 'GetSystemMetrics', 'FindResourceExW',
'OleInitialize', 'FindResourceExW', 'LoadResource', 'FindResourceExW',
'LoadResource', 'NtClose', 'NtAllocateVirtualMemory', 'GetSystemMetrics',
'NtAllocateVirtualMemory', 'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrLoadDll',
'LdrGetProcedureAddress', 'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrLoadDll',
'LdrGetProcedureAddress', 'LdrLoadDll', 'LdrGetProcedureAddress',
'LookupAccountSidW', 'LdrGetProcedureAddress', 'LookupAccountSidW',
'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrLoadDll', 'LdrGetProcedureAddress',
'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrGetDllHandle', 'FindResourceExW',
'LoadResource', 'FindResourceExW', 'LoadResource', 'DrawTextExW',
'GetSystemMetrics', 'FindResourceExW', 'LoadResource', 'GetSystemMetrics',
'DrawTextExW', 'LdrGetDllHandle', 'FindResourceExW', 'LoadResource',
'FindResourceExW', 'LoadResource', 'FindResourceExW', 'LoadResource',
'LdrGetDllHandle', 'LdrGetProcedureAddress', 'LoadStringW', 'LdrGetDllHandle',
'LdrGetProcedureAddress']
Lenght of the sequence: 73

First row of the train set after padding:
['LdrGetDllHandle', 'LdrGetProcedureAddress', 'LdrLoadDll',
'LdrGetProcedureAddress', 'LdrGetDllHandle', 'LdrGetProcedureAddress',
'GetTimeZoneInformation', 'LoadStringW', 'RegOpenKeyExW', 'RegQueryValueExW',
'RegCloseKey', 'LdrGetDllHandle', 'LdrGetProcedureAddress', 'LdrGetDllHandle',
'LdrGetProcedureAddress', 'SetErrorMode', 'LdrLoadDll', 'SetErrorMode',
'LdrLoadDll', 'SetErrorMode', 'GetSystemMetrics', 'FindResourceExW',
'OleInitialize', 'FindResourceExW', 'LoadResource', 'FindResourceExW',
'LoadResource', 'NtClose', 'NtAllocateVirtualMemory', 'GetSystemMetrics',
'NtAllocateVirtualMemory', 'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrLoadDll',
'LdrGetProcedureAddress', 'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrLoadDll',
'LdrGetProcedureAddress', 'LdrLoadDll', 'LdrGetProcedureAddress',
'LookupAccountSidW', 'LdrGetProcedureAddress', 'LookupAccountSidW',
'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrLoadDll', 'LdrGetProcedureAddress',
'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrGetDllHandle', 'FindResourceExW',
'LoadResource', 'FindResourceExW', 'LoadResource', 'DrawTextExW',
'GetSystemMetrics', 'FindResourceExW', 'LoadResource', 'GetSystemMetrics',
'DrawTextExW', 'LdrGetDllHandle', 'FindResourceExW', 'LoadResource',
'FindResourceExW', 'LoadResource', 'FindResourceExW', 'LoadResource',
'LdrGetDllHandle', 'LdrGetProcedureAddress', 'LoadStringW', 'LdrGetDllHandle',
'LdrGetProcedureAddress', 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
Lenght of the sequence: 90
```

Q: If at test time you have more API calls than the fixed-size, what do you do with the exceeding API calls? At test time, if a sequence has **more API calls than the fixed size**, the exceeding API calls should be **truncated** to fit the fixed length. **Sequences shorter** than the fixed size are **padded** with dummy tokens.

We have decided to perform the same pad or truncate action on the test set, so that all sequences

have a consistent length compatible with the FFNN.

[43]:
```
# To ensure consistency, we apply the padding or truncating to also on the test␣
 ↪df
df_test['seq_fixed'] = df_test['api_call_sequence'].apply(lambda x:␣
 ↪pad_truncate_sequence(x, L))
```

[44]:
```
df_train
```

[44]:
```
                                        api_call_sequence  is_malware  \
0      [LdrGetDllHandle, LdrGetProcedureAddress, LdrL…           1
1      [NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr…           1
2      [FindResourceExW, LoadResource, FindResourceEx…           1
3      [FindResourceExW, LoadResource, FindResourceEx…           1
4      [LdrGetProcedureAddress, SetErrorMode, LdrLoad…           1
…                                                    …         …
16320  [LdrGetProcedureAddress, LdrLoadDll, LdrGetPro…           1
16321  [NtClose, LdrGetProcedureAddress, CryptCreateH…           1
16322  [LdrGetProcedureAddress, LdrGetDllHandle, LdrG…           1
16323  [LdrGetProcedureAddress, LdrGetDllHandle, LdrG…           1
16324  [SetFilePointer, NtReadFile, SetFilePointer, N…           1

       seq_length                                          seq_fixed
0              73  [LdrGetDllHandle, LdrGetProcedureAddress, LdrL…
1              88  [NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr…
2              79  [FindResourceExW, LoadResource, FindResourceEx…
3              71  [FindResourceExW, LoadResource, FindResourceEx…
4              63  [LdrGetProcedureAddress, SetErrorMode, LdrLoad…
…             …                                                 …
16320          64  [LdrGetProcedureAddress, LdrLoadDll, LdrGetPro…
16321          78  [NtClose, LdrGetProcedureAddress, CryptCreateH…
16322          84  [LdrGetProcedureAddress, LdrGetDllHandle, LdrG…
16323          62  [LdrGetProcedureAddress, LdrGetDllHandle, LdrG…
16324          77  [SetFilePointer, NtReadFile, SetFilePointer, N…

[16325 rows x 4 columns]
```

[45]:
```
df_test
```

[45]:
```
                                      api_call_sequence  is_malware  \
0     [NtQueryValueKey, NtClose, NtOpenKey, NtQueryV…           1
1     [LdrGetProcedureAddress, NtClose, NtOpenKey, N…           1
2     [NtOpenKey, NtQueryValueKey, NtClose, NtOpenKe…           1
3     [NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr…           1
4     [NtOpenKey, NtQueryValueKey, NtClose, LdrGetPr…           1
…                                                   …         …
6500  [SetErrorMode, NtOpenFile, NtClose, SHGetFolde…           0
6501  [NtProtectVirtualMemory, RegOpenKeyExW, RegQue…           1
```

```
6502  [RegOpenKeyExW, RegQueryValueExW, RegCloseKey,…           1
6503  [NtQueryValueKey, NtClose, RegOpenKeyExA, RegQ…           1
6504  [NtClose, LdrGetProcedureAddress, CryptCreateH…           1

      seq_length                                        seq_fixed
0             98  [NtQueryValueKey, NtClose, NtOpenKey, NtQueryV…
1             98  [LdrGetProcedureAddress, NtClose, NtOpenKey, N…
2             99  [NtOpenKey, NtQueryValueKey, NtClose, NtOpenKe…
3             85  [NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr…
4             80  [NtOpenKey, NtQueryValueKey, NtClose, LdrGetPr…
…            …                                                 …
6500          75  [SetErrorMode, NtOpenFile, NtClose, SHGetFolde…
6501          83  [NtProtectVirtualMemory, RegOpenKeyExW, RegQue…
6502          70  [RegOpenKeyExW, RegQueryValueExW, RegCloseKey,…
6503          96  [NtQueryValueKey, NtClose, RegOpenKeyExA, RegQ…
6504          78  [NtClose, LdrGetProcedureAddress, CryptCreateH…

[6505 rows x 4 columns]
```

1.3.3 Handling Categorical Features

[46]:
```python
# --- Build a unified vocabulary ---

all_sequences = df_train['seq_fixed']

# Flatten all sequences into a single list of API calls
all_api_calls = [api for seq in all_sequences for api in seq]
```

[47]:
```python
# --- Build vocabulary with padding handling ---

# 1. Filter strings only. We ignore 0 here because we will manually assign it␣
 ↪later.
unique_api_calls = sorted(list(set(api for api in all_api_calls if␣
 ↪isinstance(api, str))))

# 2. Define Special Tokens
PAD_IDX = 0  # <PAD>
UNK_IDX = 1  # <UNK>

# 3. Map API call -> sequential ID (Start from 2 to save space for PAD and UNK)
api_to_id = {api: i + 2 for i, api in enumerate(unique_api_calls)}

# 4. Add the special tokens to the dictionary
# (Optional: helpful if you need to reverse look up '<PAD>' later)
api_to_id['<PAD>'] = PAD_IDX
api_to_id['<UNK>'] = UNK_IDX
```

```
# Map sequential ID -> API call (for verification/debugging)
id_to_api = {i: api for api, i in api_to_id.items()}

print(f"Number of unique API calls (including special tokens):␣
 ↪{len(api_to_id)}")
print("\nExample mapping (API -> ID):", list(api_to_id.items())[:5])
print("\nExample mapping (ID -> API):", list(id_to_api.items())[:5])
```

```
Number of unique API calls (including special tokens): 260

Example mapping (API -> ID): [('CertOpenStore', 2), ('CertOpenSystemStoreW', 3),
('CoCreateInstance', 4), ('CoCreateInstanceEx', 5), ('CoGetClassObject', 6)]

Example mapping (ID -> API): [(2, 'CertOpenStore'), (3, 'CertOpenSystemStoreW'),
(4, 'CoCreateInstance'), (5, 'CoCreateInstanceEx'), (6, 'CoGetClassObject')]
```

Sequential Identifiers Approach

[48]:
```
def sequence_to_ids(seq, mapping, pad_idx, unk_idx):
    """
    Convert a sequence of API calls to their corresponding IDs.
    - If item is 0 (integer padding), map to pad_idx.
    - If item is string but not in mapping, map to unk_idx.
    """
    ids = []
    for item in seq:
        # Case A: It is existing padding (integer 0)
        if item == 0:
            ids.append(pad_idx)
        # Case B: It is a known string API
        elif item in mapping:
            ids.append(mapping[item])
        # Case C: It is an unknown string API (Test set OOV)
        else:
            ids.append(unk_idx)
    return ids
```

[49]:
```
# --- Convert sequences to sequential IDs ---

# Apply conversion to train and test sets
# Note: We pass UNK_IDX (1) so unknown calls in Test set become 1.
df_train['seq_sequential_ids'] = df_train['seq_fixed'].apply(
    lambda seq: sequence_to_ids(seq, api_to_id, PAD_IDX, UNK_IDX)
)

df_test['seq_sequential_ids'] = df_test['seq_fixed'].apply(
    lambda seq: sequence_to_ids(seq, api_to_id, PAD_IDX, UNK_IDX)
)
```

```
# Quick check
print("\nSequential IDs - first row of train set:")
print(df_train['seq_sequential_ids'].iloc[0])
print("Length of the sequence:", len(df_train['seq_sequential_ids'].iloc[0]))

print("\nSequential IDs - first row of test set:")
print(df_test['seq_sequential_ids'].iloc[0])
print("Length of the sequence:", len(df_test['seq_sequential_ids'].iloc[0]))

# Verify that the logic works (check for any None values)
# If this prints nothing, your data is clean!
for seq in df_test['seq_sequential_ids']:
    if None in seq:
        print("Error: Found None in sequence! Check mapping.")
        break
```

```
Sequential IDs - first row of train set:
[117, 118, 119, 118, 117, 118, 86, 123, 201, 205, 188, 117, 118, 117, 118, 219,
119, 219, 119, 219, 81, 46, 179, 46, 121, 46, 121, 134, 133, 81, 133, 119, 118,
119, 118, 119, 118, 119, 118, 119, 118, 124, 118, 124, 119, 118, 119, 118, 119,
118, 117, 46, 121, 46, 121, 39, 81, 46, 121, 81, 39, 117, 46, 121, 46, 121, 46,
121, 117, 118, 123, 117, 118, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
Length of the sequence: 90

Sequential IDs - first row of test set:
[164, 134, 152, 164, 134, 159, 122, 133, 122, 117, 118, 117, 118, 117, 122, 118,
81, 46, 121, 46, 121, 46, 121, 46, 121, 46, 121, 46, 121, 46, 121, 46, 121, 46,
121, 46, 121, 46, 121, 46, 121, 46, 121, 46, 121, 118, 46, 121, 46, 121, 133,
81, 134, 81, 133, 119, 118, 117, 46, 121, 46, 121, 39, 81, 46, 121, 81, 39, 117,
46, 121, 46, 121, 46, 121, 117, 118, 122, 117, 118, 117, 118, 81, 44, 121, 229,
133, 119, 118, 84]
Length of the sequence: 90
```

[50]:
```
df_train
```

[50]:

	api_call_sequence	is_malware	\
0	[LdrGetDllHandle, LdrGetProcedureAddress, LdrL…	1	
1	[NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr…	1	
2	[FindResourceExW, LoadResource, FindResourceEx…	1	
3	[FindResourceExW, LoadResource, FindResourceEx…	1	
4	[LdrGetProcedureAddress, SetErrorMode, LdrLoad…	1	
…	…	…	
16320	[LdrGetProcedureAddress, LdrLoadDll, LdrGetPro…	1	
16321	[NtClose, LdrGetProcedureAddress, CryptCreateH…	1	
16322	[LdrGetProcedureAddress, LdrGetDllHandle, LdrG…	1	
16323	[LdrGetProcedureAddress, LdrGetDllHandle, LdrG…	1	

```
16324  [SetFilePointer, NtReadFile, SetFilePointer, N…           1

       seq_length                                          seq_fixed  \
0              73  [LdrGetDllHandle, LdrGetProcedureAddress, LdrL…
1              88  [NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr…
2              79  [FindResourceExW, LoadResource, FindResourceEx…
3              71  [FindResourceExW, LoadResource, FindResourceEx…
4              63  [LdrGetProcedureAddress, SetErrorMode, LdrLoad…
…           …                                                   …
16320          64  [LdrGetProcedureAddress, LdrLoadDll, LdrGetPro…
16321          78  [NtClose, LdrGetProcedureAddress, CryptCreateH…
16322          84  [LdrGetProcedureAddress, LdrGetDllHandle, LdrG…
16323          62  [LdrGetProcedureAddress, LdrGetDllHandle, LdrG…
16324          77  [SetFilePointer, NtReadFile, SetFilePointer, N…

                                    seq_sequential_ids
0      [117, 118, 119, 118, 117, 118, 86, 123, 201, 2…
1      [133, 119, 118, 82, 80, 137, 82, 152, 153, 163…
2      [46, 121, 46, 121, 46, 121, 46, 121, 46, 121, …
3      [46, 121, 46, 121, 46, 121, 46, 121, 46, 121, …
4      [118, 219, 119, 219, 81, 46, 121, 46, 121, 46,…
…                                                    …
16320  [118, 119, 118, 119, 118, 119, 118, 119, 118, …
16321  [134, 118, 24, 118, 30, 118, 135, 66, 221, 166…
16322  [118, 117, 118, 81, 216, 119, 118, 46, 121, 46…
16323  [118, 117, 118, 117, 118, 117, 118, 117, 118, …
16324  [221, 166, 221, 166, 221, 45, 121, 133, 146, 1…

[16325 rows x 5 columns]
```

[51]:
```
df_test
```

[51]:
```
                                    api_call_sequence  is_malware  \
0     [NtQueryValueKey, NtClose, NtOpenKey, NtQueryV…           1
1     [LdrGetProcedureAddress, NtClose, NtOpenKey, N…           1
2     [NtOpenKey, NtQueryValueKey, NtClose, NtOpenKe…           1
3     [NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr…           1
4     [NtOpenKey, NtQueryValueKey, NtClose, LdrGetPr…           1
…                                                    …         …
6500  [SetErrorMode, NtOpenFile, NtClose, SHGetFolde…           0
6501  [NtProtectVirtualMemory, RegOpenKeyExW, RegQue…           1
6502  [RegOpenKeyExW, RegQueryValueExW, RegCloseKey,…           1
6503  [NtQueryValueKey, NtClose, RegOpenKeyExA, RegQ…           1
6504  [NtClose, LdrGetProcedureAddress, CryptCreateH…           1

      seq_length                                          seq_fixed  \
0             98  [NtQueryValueKey, NtClose, NtOpenKey, NtQueryV…
```

```
1              98  [LdrGetProcedureAddress, NtClose, NtOpenKey, N…
2              99  [NtOpenKey, NtQueryValueKey, NtClose, NtOpenKe…
3              85  [NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr…
4              80  [NtOpenKey, NtQueryValueKey, NtClose, LdrGetPr…
…             …                                                  …
6500           75  [SetErrorMode, NtOpenFile, NtClose, SHGetFolde…
6501           83  [NtProtectVirtualMemory, RegOpenKeyExW, RegQue…
6502           70  [RegOpenKeyExW, RegQueryValueExW, RegCloseKey,…
6503           96  [NtQueryValueKey, NtClose, RegOpenKeyExA, RegQ…
6504           78  [NtClose, LdrGetProcedureAddress, CryptCreateH…

                                        seq_sequential_ids
0     [164, 134, 152, 164, 134, 159, 122, 133, 122, …
1     [118, 134, 152, 164, 134, 152, 164, 134, 119, …
2     [152, 164, 134, 152, 164, 134, 117, 118, 117, …
3     [133, 119, 118, 119, 118, 82, 200, 204, 188, 1…
4     [152, 164, 134, 118, 24, 118, 30, 118, 135, 66…
…                                                    …
6500  [219, 151, 134, 214, 219, 63, 219, 151, 134, 1…
6501  [158, 201, 205, 188, 140, 133, 158, 133, 146, …
6502  [201, 205, 188, 201, 205, 188, 80, 117, 118, 2…
6503  [164, 134, 200, 204, 188, 153, 164, 134, 119, …
6504  [134, 118, 24, 118, 30, 118, 135, 66, 221, 166…

[6505 rows x 5 columns]
```

Learnable Embeddings Approach

[52]:
```
# The learnable embeddings approach is implemented in the first layer of the␣
↪neural network model.
# The nn.Embedding layer in the MalwareFFNN class handles this process.
```

1.3.4 FFNN Training

[53]:
```
class MalwareFFNN(nn.Module):
    def __init__(self, seq_len, hidden_dims=[128,64,32],
                 dropout_rates=None, batchnorm_flags=None,
                 use_embedding=True, vocab_size=None, embedding_dim=16,
                 dropout_embedding_rate=0.1):
        """
        Flexible FFNN for malware classification.

        Parameters:
        - seq_len: length of each API call sequence
        - vocab_size: number of unique API calls (required if␣
↪use_embedding=True)
        - hidden_dims: list of hidden layer sizes
```

```
        - embedding_dim: size of embedding vectors (only used if␣
↪use_embedding=True)
        - use_embedding: whether to use an embedding layer (True) or raw␣
↪sequential IDs / one-hot (False)
        """
        super(MalwareFFNN, self).__init__()
        self.use_embedding = use_embedding

        if self.use_embedding:
            if vocab_size is None:
                raise ValueError("vocab_size must be provided if␣
↪use_embedding=True")

            # Embedding layer for sequential IDs
            self.embedding = nn.Embedding(num_embeddings=vocab_size,␣
↪embedding_dim=embedding_dim, padding_idx=0)
            input_dim = seq_len * embedding_dim

            self.dropout_embed = nn.Dropout(dropout_embedding_rate)
        else:
            # Input will be one-hot encoded or numeric vector
            input_dim = seq_len # assumes IDs are already numeric

        # Ensure defaults if none are provided
        if dropout_rates is None:
            dropout_rates = [0.0] * len(hidden_dims)      # no dropout by default
        if batchnorm_flags is None:
            batchnorm_flags = [False] * len(hidden_dims) # no batchnorm by␣
↪default

        # Safety check for lengths
        assert len(dropout_rates) == len(hidden_dims), \
            "dropout_rates length must match hidden_dims length"
        assert len(batchnorm_flags) == len(hidden_dims), \
            "batchnorm_flags length must match hidden_dims length"

        # Build hidden layers
        layers = []
        prev_dim = input_dim
        for h_dim, dropout_rate, use_bn in zip(hidden_dims, dropout_rates,␣
↪batchnorm_flags):
            layers.append(nn.Linear(prev_dim, h_dim))
            if use_bn:
                layers.append(nn.BatchNorm1d(h_dim))
            layers.append(nn.ReLU())
            if dropout_rate > 0:
```

```
                layers.append(nn.Dropout(dropout_rate))
            prev_dim = h_dim
        self.ffnn = nn.Sequential(*layers)

        # Output layer: binary classification
        self.output_layer = nn.Linear(prev_dim, 1)

    def forward(self, x):
        if self.use_embedding:
            # x: [batch_size, seq_len] of IDs
            x = self.embedding(x)     # [batch_size, seq_len, embedding_dim]
            x = self.dropout_embed(x)
            x = x.view(x.size(0), -1) # flatten: [batch_size, seq_len*embedding_dim]
        else:
            # x: [batch_size, seq_len] or [batch_size, seq_len*vocab_size] if one-hot flattened
            x = x.float()

        x = self.ffnn(x)
        out = self.output_layer(x) # [batch_size, 1]
        return out
```

[54]:
```
# --- Training function with early stopping ---

def train_model(model, train_loader, val_loader, epochs, optimizer, criterion, min_delta=None, patience=None, scheduler=None):
    """
    Train a PyTorch model with early stopping and metric tracking.
    Returns: model, history dictionary (losses + metrics), and elapsed time.
    """

    # --- DEVICE DETECTION ---
    device = next(model.parameters()).device

    # --- HISTORY STORAGE ---
    history = {
        'train_loss': [],
        'val_loss': [],
        'val_f1_macro': [],
        'val_f1_weighted': [],
    }

    best_val_loss = float('inf')
    best_model_state = None
    trigger_times = 0
```

```
    start_time = time.time()

    for epoch in range(epochs):
        # --- TRAINING ---
        model.train()
        batch_losses = []
        for X_batch, y_batch in train_loader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            optimizer.zero_grad()
            outputs = model(X_batch)
            loss = criterion(outputs, y_batch)
            loss.backward()
            optimizer.step()
            batch_losses.append(loss.item())
        train_loss = np.mean(batch_losses) if batch_losses else 0.0

        # --- VALIDATION & METRICS ---
        model.eval()
        val_batch_losses = []
        all_preds = []
        all_targets = []

        with torch.no_grad():
            for X_batch, y_batch in val_loader:
                X_batch, y_batch = X_batch.to(device), y_batch.to(device)
                outputs = model(X_batch)
                loss = criterion(outputs, y_batch)
                val_batch_losses.append(loss.item())

                # Convert logits to probabilities and then binary predictions␣
↪(0 or 1)
                # Assumes Binary Classification (BCEWithLogitsLoss)
                probs = torch.sigmoid(outputs)
                preds = (probs > 0.5).float()

                # Store for epoch-level metric calculation
                all_preds.extend(preds.cpu().numpy())
                all_targets.extend(y_batch.cpu().numpy())

        val_loss = np.mean(val_batch_losses) if val_batch_losses else 0.0

        # Calculate Metrics for this Epoch
        epoch_f1_macro = f1_score(all_targets, all_preds, average='macro',␣
↪zero_division=0)
        epoch_f1_weighted = f1_score(all_targets, all_preds,␣
↪average='weighted', zero_division=0)
```

```
        # Update History
        history['train_loss'].append(train_loss)
        history['val_loss'].append(val_loss)
        history['val_f1_macro'].append(epoch_f1_macro)
        history['val_f1_weighted'].append(epoch_f1_weighted)

        # --- SCHEDULER STEP ---
        if scheduler is not None:
            last_lr = optimizer.param_groups[0]['lr']
            if isinstance(scheduler, torch.optim.lr_scheduler.
↪ReduceLROnPlateau):
                scheduler.step(val_loss)
            else:
                scheduler.step()
            curr_lr = optimizer.param_groups[0]['lr']
            if curr_lr != last_lr:
                print(f"Epoch {epoch}: LR reduced from {last_lr:.6f} to␣
↪{curr_lr:.6f}")

        # --- EARLY STOPPING ---
        if min_delta is not None:
            if val_loss < best_val_loss - min_delta:
                best_val_loss = val_loss
                # Save best model state
                best_model_state = {k: v.cpu().clone() for k, v in model.
↪state_dict().items()}
                trigger_times = 0
            else:
                trigger_times += 1
                if trigger_times >= patience:
                    print(f"Early stopping at epoch {epoch+1} (best val loss:␣
↪{best_val_loss:.6f})")
                    break

        if (epoch+1) % 5 == 0 or epoch == 0 or epoch == epochs:
            print(f"Epoch {epoch+1}/{epochs} - Train Loss: {train_loss:.4f},␣
↪Val Loss: {val_loss:.4f}, Val F1(W): {epoch_f1_weighted:.4f}")

    # Restore best model after training loop completes
    if best_model_state is not None:
        model.load_state_dict(best_model_state)
        model.to(device)

    elapsed = time.time() - start_time
    elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed))
    print(f'\nTraining completed in {elapsed_str} (hh:mm:ss) - {elapsed:.2f}␣
↪seconds')
```

```
    return model, history, elapsed
```

[55]:
```
def evaluate_model(model, X_tensor, y_true, model_name: str = "Unnamed model"):
    """
    Evaluate a trained model on a given dataset and return the classification␣
 ↪report.

    Handles missing predicted classes gracefully (zero_division=0) and
    reports which classes were not predicted, along with the model/config name.
    """
    model.eval()
    with torch.no_grad():
        outputs = model(X_tensor)
        probs = torch.sigmoid(outputs)
        y_pred = (probs > 0.5).long().cpu().numpy().flatten() # Binary␣
 ↪prediction (0 or 1)

    # Convert y_true to numpy if it's a tensor
    if isinstance(y_true, torch.Tensor):
        y_true = y_true.cpu().numpy().flatten()

    # Identify missing classes (not predicted at all)
    missing_classes = set(np.unique(y_true)) - set(np.unique(y_pred))
    if missing_classes:
        missing_classes = [int(x) for x in sorted(missing_classes)]
        print(f"Warning: {model_name} made no predictions for classes:␣
 ↪{missing_classes}")

    # Generate classification report without raising warnings
    report = classification_report(y_true, y_pred, digits=4, zero_division=0)

    return report
```

[56]:
```
def plot_cm_task2(model, X_tensor, y_tensor, model_name, save_dir=None):
    """
    Specific for FFNN. Takes tensors, moves to device, predicts, and plots.
    """
    # Ensure model is in eval mode
    model.eval()

    # Detect device (cuda or cpu) from the model parameters
    device = next(model.parameters()).device

    # Move inputs to the same device as model
    X_input = X_tensor.to(device)
```

```
    with torch.no_grad():
        # Get logits
        outputs = model(X_input)
        # Apply sigmoid to get probabilities
        probs = torch.sigmoid(outputs)
        # Convert to binary predictions (0 or 1)
        y_pred = (probs > 0.5).long().cpu().numpy().flatten()

    # Ensure y_true is on CPU and numpy
    y_true = y_tensor.cpu().numpy().flatten()

    # Plot
    _internal_plot_cm(y_true, y_pred, model_name, save_dir)
```

[57]:
```
def plot_training_metrics(histories, model_names=None, save_dir=None,␣
↪plot_name="validation_f1_macro_comparison"):
    """
    Compare multiple models' validation F1 Macro across epochs.
    Only shows validation F1 macro (no weighted, no markers for cleaner␣
↪plots).
    """
    # Normalize input
    if isinstance(histories, dict):
        hist_map = histories
        names = [f"lr_{k:.0e}" if isinstance(k, float) else str(k) for k in␣
↪hist_map.keys()]
        hist_map = dict(zip(names, hist_map.values()))
    else:
        if model_names is None:
            names = [f"Model_{i+1}" for i in range(len(histories))]
        else:
            names = list(model_names)
        hist_map = dict(zip(names, histories))

    # Validate
    for name, h in hist_map.items():
        if 'val_f1_macro' not in h:
            raise KeyError(f"History for '{name}' must contain 'val_f1_macro'.
↪")

    colors = ['#E74C3C', '#F39C12', '#27AE60', '#3498DB', '#9B59B6',␣
↪'#34495E']

    fig, ax = plt.subplots(figsize=(10, 5))

    for idx, (name, h) in enumerate(hist_map.items()):
        epochs = range(1, len(h['val_f1_macro']) + 1)
```

```
        color = colors[idx % len(colors)]
        ax.plot(epochs, h['val_f1_macro'], label=name, linewidth=2,␣
↪color=color)

    ax.set_title("Validation F1 Macro Comparison", fontsize=13)
    ax.set_xlabel("Epochs", fontsize=11)
    ax.set_ylabel("F1 Macro", fontsize=11)
    ax.grid(True, alpha=0.3)
    ax.legend(fontsize=9)
    plt.tight_layout()

    if save_dir:
        save_plot(fig, filename=plot_name, path=save_dir, fmt="png",␣
↪dpi=300, close_fig=False)
    plt.show()
```

[58]:
```
# --- Plotting function for LR comparison ---
def plot_lr_comparison(histories_dict, model_name, save_dir=None):
    """
    Plot ONLY validation losses for different learning rates.
    (When multiple LRs are present, we show only val curves for readability.)
    """
    plt.style.use('seaborn-v0_8-paper')

    fig, ax = plt.subplots(figsize=(8, 5))

    colors = ['#E74C3C', '#F39C12', '#27AE60', '#3498DB', '#9B59B6',␣
↪'#34495E']

    for idx, (lr, history) in enumerate(sorted(histories_dict.items(),␣
↪reverse=True)):
        color = colors[idx % len(colors)]
        label = f'lr_{lr:.0e}'
        ax.plot(history['val_loss'], color=color, linewidth=2, label=label,␣
↪alpha=0.8)

    ax.set_title(f'{model_name} - Validation Loss (LR Comparison)',␣
↪fontsize=13, pad=10)
    ax.set_ylabel('Validation Loss', fontsize=10)
    ax.set_xlabel('Epochs', fontsize=10)
    ax.grid(True, linestyle='-', alpha=0.3)
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
    ax.legend(fontsize=9, loc='best')

    plt.tight_layout()
```

```
    if save_dir:
        save_plot(fig, f'{model_name.lower().replace(" ",␣
↪"_")}_lr_comparison', save_dir)

    plt.show()
```

[59]:
```
# --- Plot Train vs Val ---

def plot_train_val_grid(histories_dict, model_name, save_dir=None):
    """
    Plots a grid (1 row, N columns) showing Train vs Val loss for EACH learning␣
↪rate.
    This helps diagnose overfitting/underfitting for each specific setting.
    Works for both FFNN (Task 2) and RNN (Task 3) models.

    Args:
        histories_dict: Dictionary with LR values as keys and history dicts as␣
↪values
        model_name: Name of the model architecture
        save_dir: Directory to save the plot
    """
    lrs = sorted(histories_dict.keys(), reverse=True) # Highest LR first
    num_lrs = len(lrs)

    # Set style
    plt.style.use('seaborn-v0_8-paper')

    # Create figure with 1 row and N columns
    fig, axes = plt.subplots(1, num_lrs, figsize=(5 * num_lrs, 4), sharey=True)

    # Handle case if there is only 1 LR (axes is not a list)
    if num_lrs == 1:
        axes = [axes]

    # Common colors
    col_train = '#2980B9' # Blue
    col_val = '#E74C3C'   # Red

    for idx, lr in enumerate(lrs):
        ax = axes[idx]
        history = histories_dict[lr]
        epochs = range(1, len(history['train_loss']) + 1)

        # Plot Lines
        ax.plot(epochs, history['train_loss'], label='Train Loss',␣
↪color=col_train, linewidth=2)
```

```
        ax.plot(epochs, history['val_loss'], label='Val Loss', color=col_val,␣
↪linewidth=2)

        # Find best epoch (min val loss)
        min_val_loss = min(history['val_loss'])
        best_epoch = history['val_loss'].index(min_val_loss) + 1

        # Mark best epoch
        ax.axvline(x=best_epoch, color='green', linestyle=':', alpha=0.6,␣
↪label=f'Best Ep: {best_epoch}')
        ax.scatter(best_epoch, min_val_loss, color='green', zorder=5)

        # Styling
        ax.set_title(f'LR: {lr:.0e}', fontsize=12, fontweight='bold')
        ax.set_xlabel('Epochs')
        if idx == 0:
            ax.set_ylabel('Loss')

        ax.legend(fontsize=9)
        ax.grid(True, linestyle='-', alpha=0.3)
        ax.spines['top'].set_visible(False)
        ax.spines['right'].set_visible(False)

    fig.suptitle(f'{model_name} - Overfitting Diagnosis (Train vs Val)',␣
↪fontsize=14, y=1.05)
    plt.tight_layout()

    if save_dir:
        save_plot(fig, f'{model_name.lower().replace(" ", "_")}_diagnosis',␣
↪save_dir)

    plt.show()
```

Training with Sequential Identifiers Approach

[60]:
```
# --- Create a validation set ---

# Convert sequences to a numpy array (shape: [samples, seq_len])
X_train = np.array(df_train['seq_sequential_ids'].tolist())
X_test = np.array(df_test['seq_sequential_ids'].tolist())

# Labels
y_train = df_train['is_malware'].values
y_test = df_test['is_malware'].values

# Split train into train + val
X_train_split, X_val, y_train_split, y_val = train_test_split(
```

```
    X_train, y_train, test_size=0.2, random_state=42, stratify=y_train
)

print("Train set:", X_train_split.shape, y_train_split.shape)
print("Validation set:", X_val.shape, y_val.shape)
```

```
Train set: (13060, 90) (13060,)
Validation set: (3265, 90) (3265,)
```

[61]:
```
# --- Prepare PyTorch datasets and loaders ---

# Convert to tensors
X_train_tensor = torch.tensor(X_train_split.astype(np.int64), dtype=torch.long)
y_train_tensor = torch.tensor(y_train_split, dtype=torch.float32).unsqueeze(1)

X_val_tensor = torch.tensor(X_val.astype(np.int64), dtype=torch.long)
y_val_tensor = torch.tensor(y_val, dtype=torch.float32).unsqueeze(1)

X_test_tensor = torch.tensor(X_test.astype(np.int64), dtype=torch.long)
y_test_tensor = torch.tensor(y_test, dtype=torch.float32).unsqueeze(1)

# Create datasets and loaders
batch_size = 64

train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
val_dataset = TensorDataset(X_val_tensor, y_val_tensor)
test_dataset = TensorDataset(X_test_tensor, y_test_tensor)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=True)
```

[62]:
```
unique, counts = np.unique(y_train_split, return_counts=True)
print(dict(zip(unique, counts)))
```

```
{np.int64(0): np.int64(489), np.int64(1): np.int64(12571)}
```

[63]:
```
def compute_pos_weight(y):
    """
    Compute positive class weight for BCEWithLogitsLoss.
    Returns a tensor > 1 if positive class is minority.
    """
    y_np = np.array(y, dtype=int)
    class_counts = np.bincount(y_np)

    if len(class_counts) < 2:
        raise ValueError("Dataset must contain both classes 0 and 1.")
```

```
    count_neg, count_pos = class_counts[0], class_counts[1]

    # if 1 = malware (majority class)
    ratio = count_neg / count_pos

    # Optionally stabilize the weight
    ratio = max(ratio, 0.2)

    print(f"Negatives: {count_neg}, Positives: {count_pos}")
    print(f"pos_weight (applied to y=1 samples): {ratio:.4f}")

    # return torch.tensor([ratio], dtype=torch.float32)
    return torch.tensor([ratio], dtype=torch.float32)
```

[64]:
```
# --- Configuration Constants ---
seq_len = X_train.shape[1]
hidden_dims = [256, 128, 64]
dropout_rates = [0.4, 0.35, 0.3]
batchnorm_flags = [True, True, True]
min_delta = 0.00001
patience_early_stop = 15
epochs = 100

# --- Device Setup ---
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")

# Compute weights and move to device
pos_weight = compute_pos_weight(y_train_split).to(device)

# --- Define LR Candidates ---
learning_rates_seq = [0.0001, 0.00005, 0.00001]
histories_seq = {}     # Store results here
models_seq = {}        # Store models here
elapsed_seq_times = {} # Store elapsed times here

print(f"\n{'='*80}")
print("Training FFNN (Sequential IDs) with different learning rates")
print(f"{'='*80}")

for lr in learning_rates_seq:
    print(f"\n--- Training FFNN (Sequential IDs) with lr={lr:.0e} ---")

    # 1. Re-Initialize Model and move to device
    model_seq = MalwareFFNN(
        seq_len,
        hidden_dims,
```

```
        dropout_rates,
        batchnorm_flags,
        use_embedding=False,
        dropout_embedding_rate=0.1
    ).to(device)

    # 2. Re-Initialize Criterion
    criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)

    # 3. Initialize Optimizer with current LR
    optimizer = optim.AdamW(model_seq.parameters(), lr=lr, weight_decay=1e-5)

    # 4. Initialize Scheduler
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
        optimizer,
        mode='min',
        factor=0.7,
        patience=10,
        min_lr=1e-6
    )

    # 5. Train
    model_seq, history, elapsed_seq = train_model(
        model_seq,
        train_loader,
        val_loader,
        epochs,
        optimizer,
        criterion,
        min_delta,
        patience=patience_early_stop,
        scheduler=scheduler
    )

    # Store model and history with numeric LR key
    models_seq[lr] = model_seq
    histories_seq[lr] = history
    elapsed_seq_times[lr] = elapsed_seq

    elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed_seq))
    print(f"Completed. Training time: {elapsed_str} ({elapsed_seq:.2f}s) -␣
↪Final val loss: {history['val_loss'][-1]:.4f}")
```

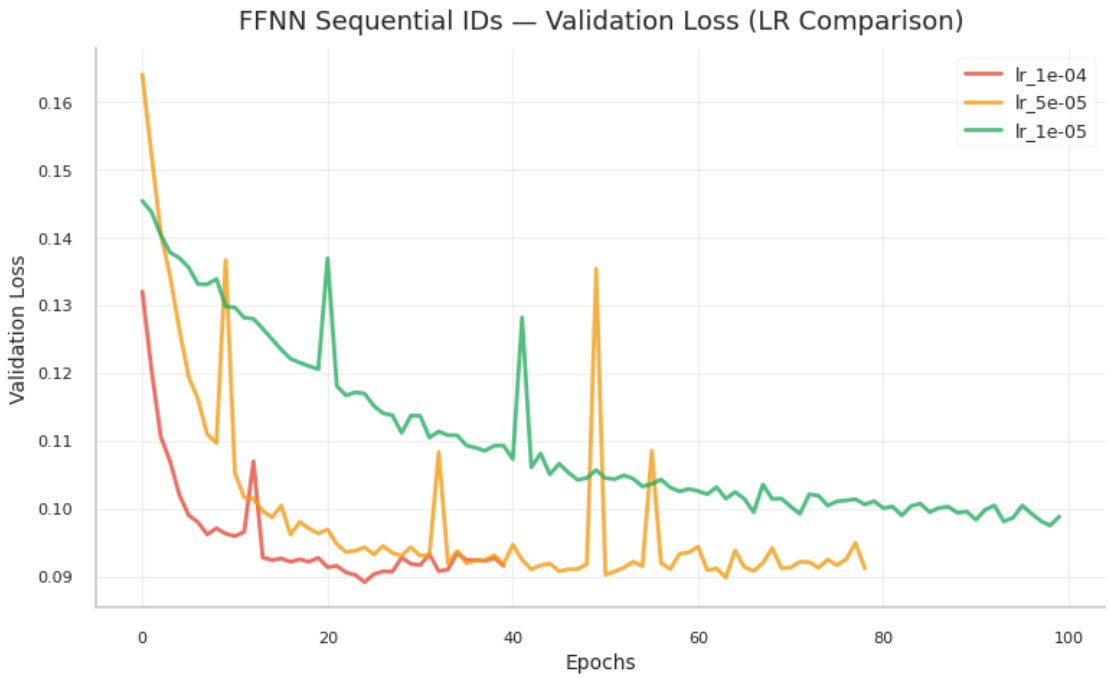
```
Using device: cuda
Negatives: 489, Positives: 12571
pos_weight (applied to y=1 samples): 0.2000

================================================================================
```

```
Training FFNN (Sequential IDs) with different learning rates
================================================================================

--- Training FFNN (Sequential IDs) with lr=1e-04 ---
Epoch 1/100 - Train Loss: 0.1380, Val Loss: 0.1320, Val F1(W): 0.9388
Epoch 5/100 - Train Loss: 0.0977, Val Loss: 0.1021, Val F1(W): 0.9425
Epoch 10/100 - Train Loss: 0.0906, Val Loss: 0.0963, Val F1(W): 0.9376
Epoch 15/100 - Train Loss: 0.0872, Val Loss: 0.0924, Val F1(W): 0.9331
Epoch 20/100 - Train Loss: 0.0843, Val Loss: 0.0927, Val F1(W): 0.9253
Epoch 25/100 - Train Loss: 0.0796, Val Loss: 0.0891, Val F1(W): 0.9298
Epoch 30/100 - Train Loss: 0.0778, Val Loss: 0.0918, Val F1(W): 0.9221
Epoch 35/100 - Train Loss: 0.0754, Val Loss: 0.0935, Val F1(W): 0.9145
Epoch 35: LR reduced from 0.000100 to 0.000070
Early stopping at epoch 40 (best val loss: 0.089145)

Training completed in 00:00:48 (hh:mm:ss) - 48.88 seconds
Completed. Training time: 00:00:48 (48.88s) - Final val loss: 0.0916

--- Training FFNN (Sequential IDs) with lr=5e-05 ---
Epoch 1/100 - Train Loss: 0.1773, Val Loss: 0.1641, Val F1(W): 0.4722
Epoch 5/100 - Train Loss: 0.1176, Val Loss: 0.1265, Val F1(W): 0.9407
Epoch 10/100 - Train Loss: 0.0997, Val Loss: 0.1367, Val F1(W): 0.9435
Epoch 15/100 - Train Loss: 0.0939, Val Loss: 0.0987, Val F1(W): 0.9431
Epoch 20/100 - Train Loss: 0.0910, Val Loss: 0.0963, Val F1(W): 0.9392
Epoch 25/100 - Train Loss: 0.0892, Val Loss: 0.0943, Val F1(W): 0.9325
Epoch 30/100 - Train Loss: 0.0861, Val Loss: 0.0943, Val F1(W): 0.9307
Epoch 35/100 - Train Loss: 0.0843, Val Loss: 0.0937, Val F1(W): 0.9265
Epoch 40/100 - Train Loss: 0.0834, Val Loss: 0.0918, Val F1(W): 0.9244
Epoch 45/100 - Train Loss: 0.0798, Val Loss: 0.0919, Val F1(W): 0.9216
Epoch 50/100 - Train Loss: 0.0815, Val Loss: 0.1354, Val F1(W): 0.9174
Epoch 55/100 - Train Loss: 0.0774, Val Loss: 0.0915, Val F1(W): 0.9172
Epoch 60/100 - Train Loss: 0.0749, Val Loss: 0.0936, Val F1(W): 0.9159
Epoch 61: LR reduced from 0.000050 to 0.000035
Epoch 65/100 - Train Loss: 0.0749, Val Loss: 0.0938, Val F1(W): 0.9176
Epoch 70/100 - Train Loss: 0.0744, Val Loss: 0.0912, Val F1(W): 0.9177
Epoch 74: LR reduced from 0.000035 to 0.000024
Epoch 75/100 - Train Loss: 0.0713, Val Loss: 0.0925, Val F1(W): 0.9136
Early stopping at epoch 79 (best val loss: 0.089840)

Training completed in 00:01:01 (hh:mm:ss) - 61.43 seconds
Completed. Training time: 00:01:01 (61.43s) - Final val loss: 0.0912

--- Training FFNN (Sequential IDs) with lr=1e-05 ---
Epoch 1/100 - Train Loss: 0.1493, Val Loss: 0.1454, Val F1(W): 0.8859
Epoch 5/100 - Train Loss: 0.1371, Val Loss: 0.1370, Val F1(W): 0.9327
Epoch 10/100 - Train Loss: 0.1246, Val Loss: 0.1299, Val F1(W): 0.9405
Epoch 15/100 - Train Loss: 0.1161, Val Loss: 0.1250, Val F1(W): 0.9420
Epoch 20/100 - Train Loss: 0.1103, Val Loss: 0.1206, Val F1(W): 0.9425
```

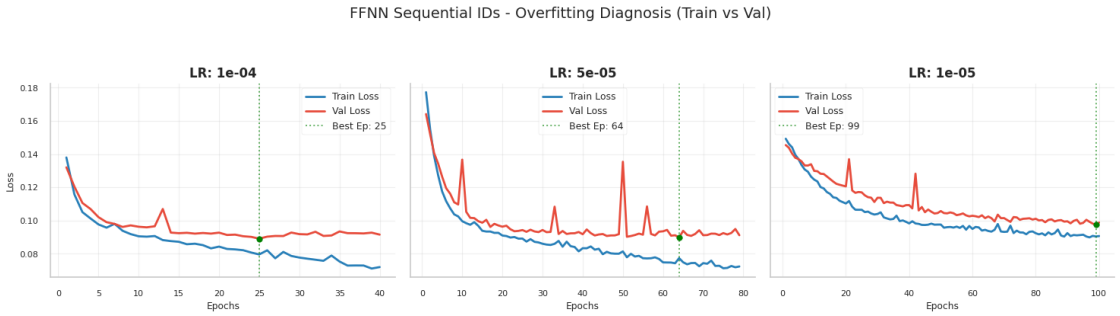
```
Epoch 25/100 - Train Loss: 0.1065, Val Loss: 0.1170, Val F1(W): 0.9432
Epoch 30/100 - Train Loss: 0.1040, Val Loss: 0.1137, Val F1(W): 0.9432
Epoch 35/100 - Train Loss: 0.1010, Val Loss: 0.1108, Val F1(W): 0.9425
Epoch 40/100 - Train Loss: 0.0983, Val Loss: 0.1093, Val F1(W): 0.9432
Epoch 45/100 - Train Loss: 0.0974, Val Loss: 0.1051, Val F1(W): 0.9438
Epoch 50/100 - Train Loss: 0.0976, Val Loss: 0.1057, Val F1(W): 0.9436
Epoch 55/100 - Train Loss: 0.0964, Val Loss: 0.1033, Val F1(W): 0.9425
Epoch 60/100 - Train Loss: 0.0950, Val Loss: 0.1029, Val F1(W): 0.9414
Epoch 65/100 - Train Loss: 0.0937, Val Loss: 0.1025, Val F1(W): 0.9414
Epoch 70/100 - Train Loss: 0.0932, Val Loss: 0.1015, Val F1(W): 0.9406
Epoch 75/100 - Train Loss: 0.0929, Val Loss: 0.1005, Val F1(W): 0.9416
Epoch 80/100 - Train Loss: 0.0924, Val Loss: 0.1011, Val F1(W): 0.9379
Epoch 85/100 - Train Loss: 0.0917, Val Loss: 0.1007, Val F1(W): 0.9367
Epoch 90/100 - Train Loss: 0.0925, Val Loss: 0.0996, Val F1(W): 0.9347
Epoch 95/100 - Train Loss: 0.0915, Val Loss: 0.0987, Val F1(W): 0.9358
Epoch 100/100 - Train Loss: 0.0907, Val Loss: 0.0988, Val F1(W): 0.9343

Training completed in 00:01:19 (hh:mm:ss) - 79.77 seconds
Completed. Training time: 00:01:19 (79.77s) - Final val loss: 0.0988
```

[65]:
```python
# --- Plot loss curves ---

# Plot comparison
plot_lr_comparison(histories_seq, "FFNN Sequential IDs", save_dir)

# Plot Train vs Validation
plot_train_val_grid(histories_seq, "FFNN Sequential IDs", save_dir)
```

```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/ffnn_sequential_ids_lr_comparison.png
```



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task2_plots/ffnn_sequential_ids_diagnosis.png



[66]:
```
# --- Evaluate validation and test sets for ALL learning rates ---

print("\n" + "=" * 80)
print("Evaluation for ALL FFNN (Sequential IDs) learning rates")
print("=" * 80)

for lr in learning_rates_seq:
    model = models_seq[lr]

    print(f"\n{'='*60}")
```

```
    print(f"FFNN (Sequential IDs) with lr={lr:.0e}")
    print(f"{'='*60}")

    # Validation evaluation
    print(f'\nValidation report (FFNN Sequential IDs - lr={lr:.0e}):')
    report = evaluate_model(model, X_val_tensor.to(device), y_val) # Moved␣
 ↪X_val_tensor to device
    print(report)

    # Test evaluation
    print(f'Test report (FFNN Sequential IDs - lr={lr:.0e}):')
    report_test = evaluate_model(model, X_test_tensor.to(device), y_test) #␣
 ↪Moved X_test_tensor to device
    print(report_test)

# Find and highlight best model
best_lr_seq = min(histories_seq.items(), key=lambda x: min(x[1]['val_loss']))
print(f"\n{'='*60}")
print(f"  Best FFNN (Sequential IDs) - LR: {best_lr_seq[0]:.0e} (Min Val Loss:␣
 ↪{min(best_lr_seq[1]['val_loss']):.4f})")
print(f"{'='*60}")
```

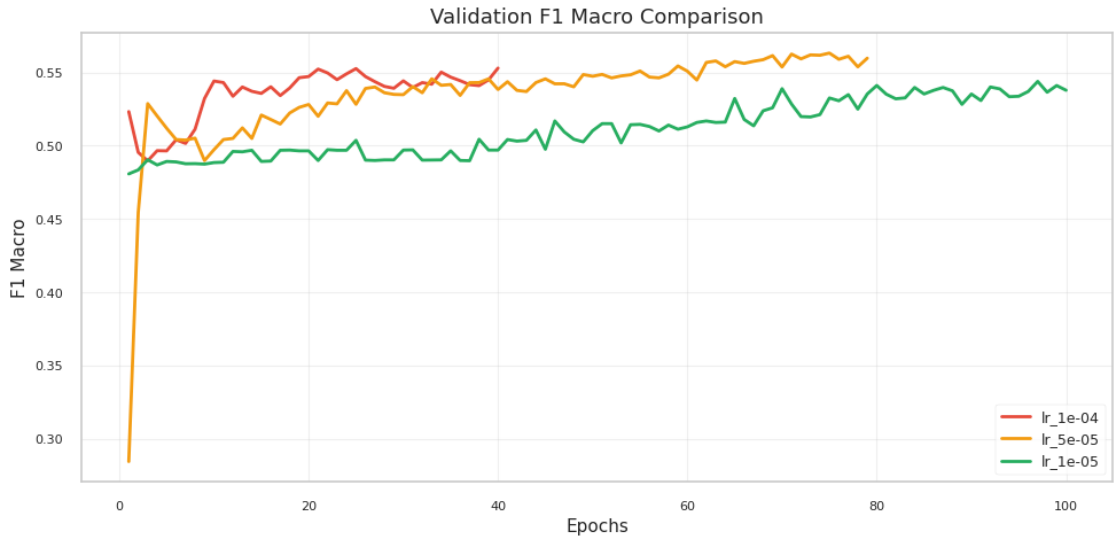
```
================================================================================
Evaluation for ALL FFNN (Sequential IDs) learning rates
================================================================================

============================================================
FFNN (Sequential IDs) with lr=1e-04
============================================================

Validation report (FFNN Sequential IDs - lr=1e-04):
              precision    recall  f1-score   support

           0     0.1250    0.1721    0.1448       122
           1     0.9674    0.9532    0.9603      3143

    accuracy                         0.9240      3265
   macro avg     0.5462    0.5627    0.5525      3265
weighted avg     0.9359    0.9240    0.9298      3265

Test report (FFNN Sequential IDs - lr=1e-04):
              precision    recall  f1-score   support

           0     0.1379    0.1317    0.1347       243
           1     0.9664    0.9681    0.9672      6262
```

```
    accuracy                           0.9368      6505
   macro avg     0.5521    0.5499    0.5510      6505
weighted avg     0.9354    0.9368    0.9361      6505


============================================================
FFNN (Sequential IDs) with lr=5e-05
============================================================

Validation report (FFNN Sequential IDs - lr=5e-05):
              precision    recall  f1-score   support

           0     0.1204    0.2131    0.1538       122
           1     0.9685    0.9395    0.9538      3143

    accuracy                           0.9124      3265
   macro avg     0.5444    0.5763    0.5538      3265
weighted avg     0.9368    0.9124    0.9239      3265

Test report (FFNN Sequential IDs - lr=5e-05):
              precision    recall  f1-score   support

           0     0.1260    0.1975    0.1538       243
           1     0.9682    0.9468    0.9574      6262

    accuracy                           0.9188      6505
   macro avg     0.5471    0.5722    0.5556      6505
weighted avg     0.9367    0.9188    0.9274      6505


============================================================
FFNN (Sequential IDs) with lr=1e-05
============================================================

Validation report (FFNN Sequential IDs - lr=1e-05):
              precision    recall  f1-score   support

           0     0.1226    0.1066    0.1140       122
           1     0.9655    0.9704    0.9679      3143

    accuracy                           0.9381      3265
   macro avg     0.5441    0.5385    0.5410      3265
weighted avg     0.9340    0.9381    0.9360      3265

Test report (FFNN Sequential IDs - lr=1e-05):
              precision    recall  f1-score   support

           0     0.1284    0.0782    0.0972       243
```

```
           1       0.9648    0.9794    0.9720      6262

    accuracy                         0.9457      6505
   macro avg       0.5466    0.5288    0.5346      6505
weighted avg       0.9335    0.9457    0.9393      6505


============================================================
 Best FFNN (Sequential IDs) - LR: 1e-04 (Min Val Loss: 0.0891)
============================================================
```

[67]:
```
# --- Plot metric curves for ALL learning rates (Sequential IDs) ---

print("\n" + "=" * 80)
print("Metric Curves for ALL FFNN (Sequential IDs) learning rates")
print("=" * 80)

plot_training_metrics(histories_seq, save_dir=save_dir,␣
↪plot_name="ffnn_sequential_ids_f1_comparison")
```

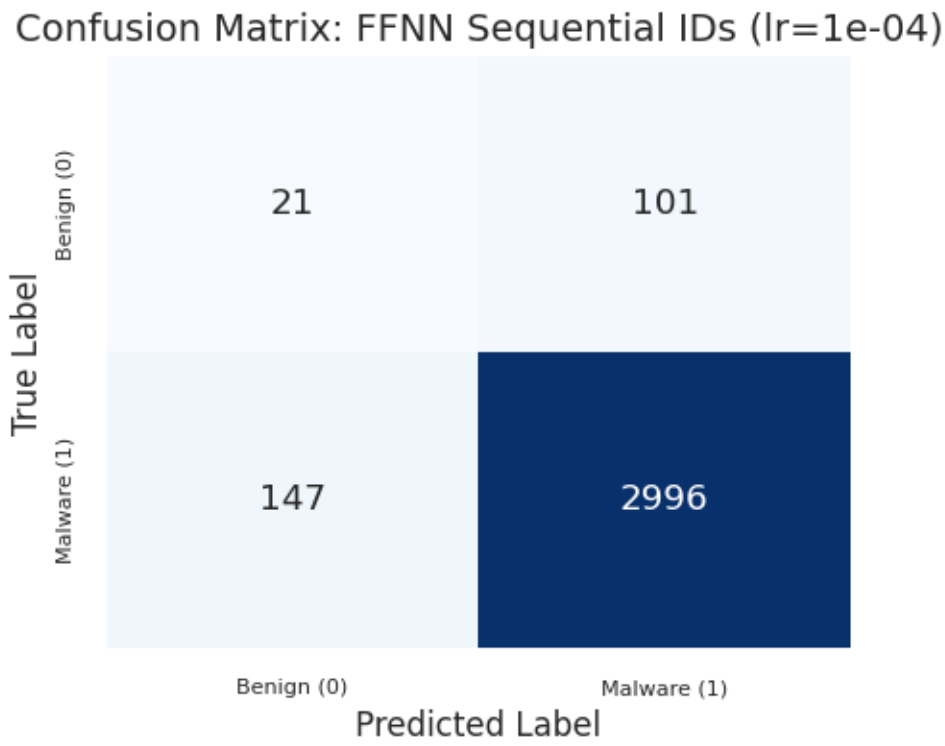
```
================================================================================
Metric Curves for ALL FFNN (Sequential IDs) learning rates
================================================================================
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/ffnn_sequential_ids_f1_comparison.png
```



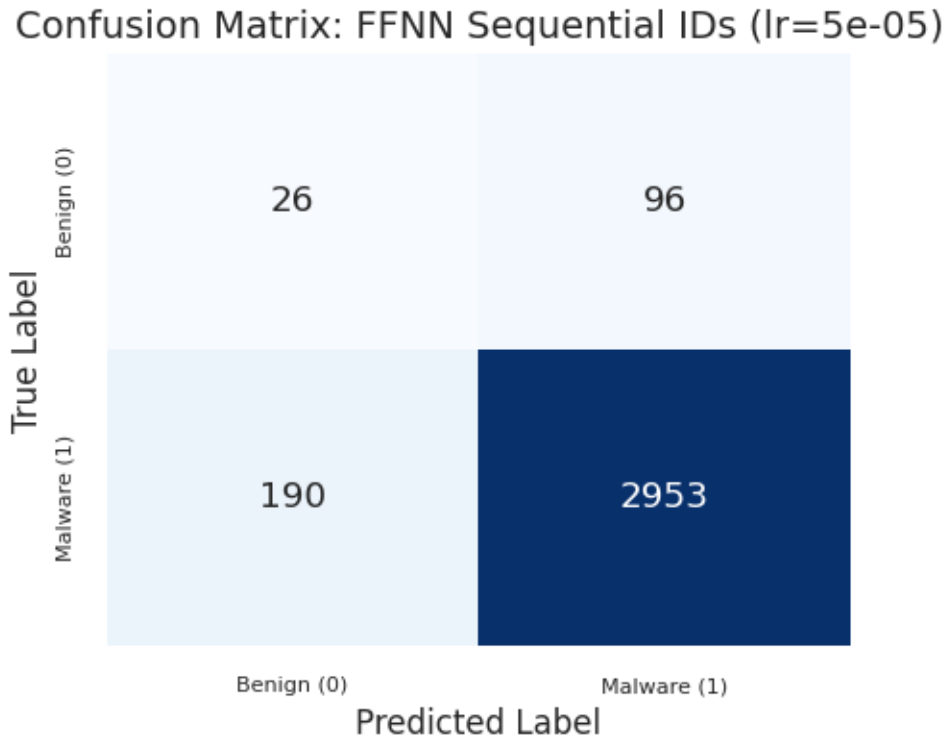
[68]:
```
# --- Plot confusion matrices for ALL learning rates (Sequential IDs) -␣
↪VALIDATION ---
```

```
print("\n" + "=" * 80)
print("Confusion matrices for ALL FFNN (Sequential IDs) learning rates - VALIDATION")
print("=" * 80)

for lr in learning_rates_seq:
    model = models_seq[lr]
    print(f"\nGenerating confusion matrix for FFNN (Sequential IDs) with lr={lr:.0e}...")

    plot_cm_task2(model=model, X_tensor=X_val_tensor, y_tensor=y_val_tensor, model_name=f"FFNN Sequential IDs (lr={lr:.0e})", save_dir=save_dir)
```

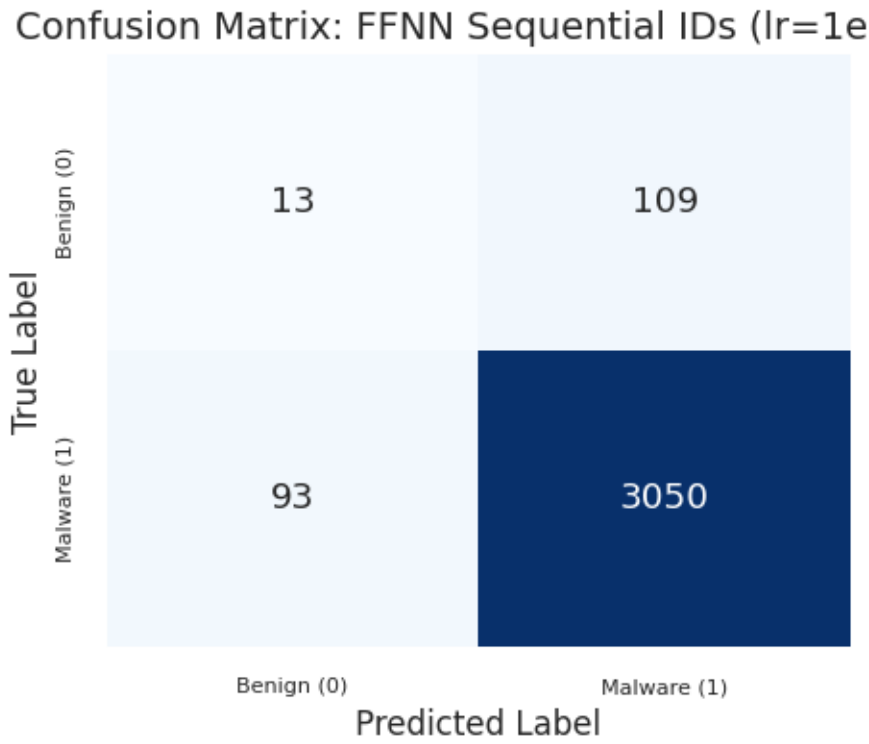
```
================================================================================
Confusion matrices for ALL FFNN (Sequential IDs) learning rates - VALIDATION
================================================================================

Generating confusion matrix for FFNN (Sequential IDs) with lr=1e-04...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task2_plots/cm_ffnn_sequential_ids_(lr=1e_04).png
```



```
Generating confusion matrix for FFNN (Sequential IDs) with lr=5e-05…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_sequential_ids_(lr=5e_05).png
```

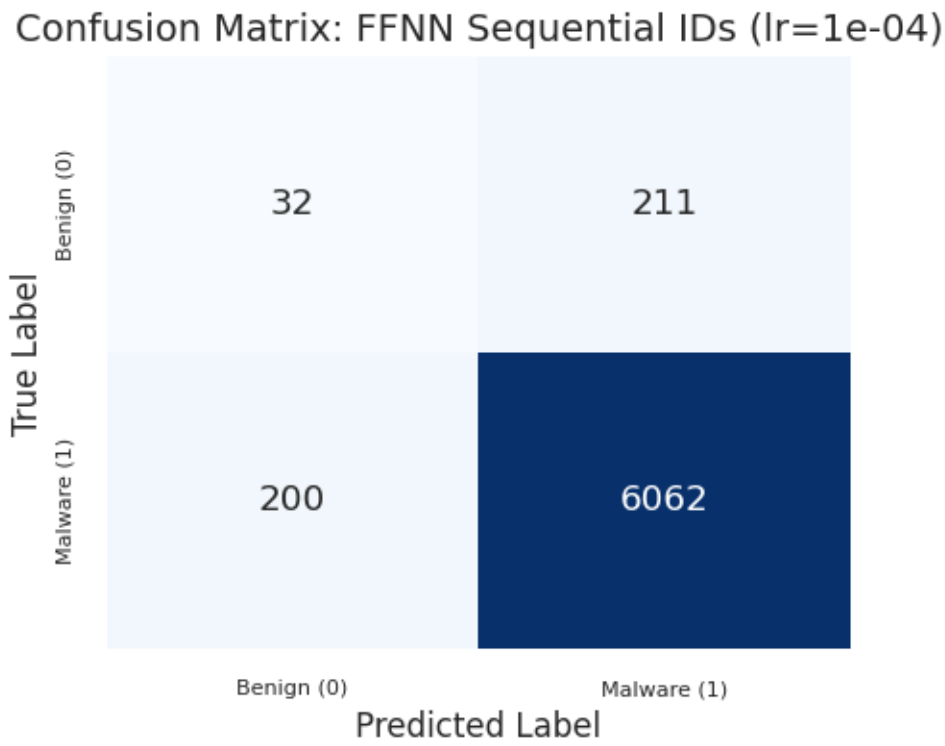



```
Generating confusion matrix for FFNN (Sequential IDs) with lr=1e-05…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_sequential_ids_(lr=1e_05).png
```

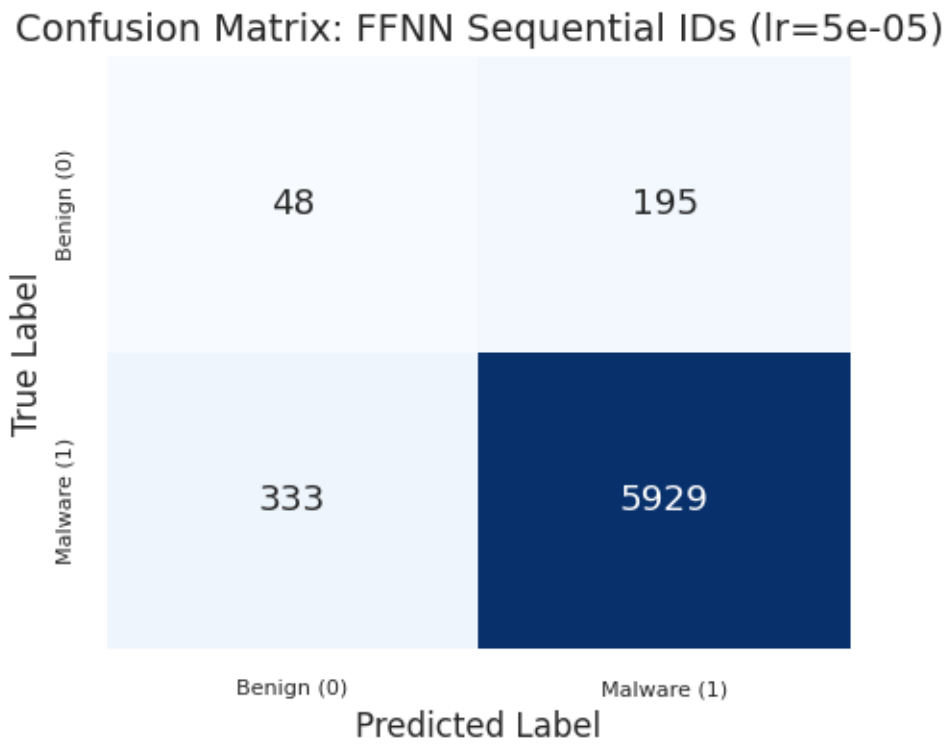


[69]:
```
# --- Plot confusion matrices for ALL learning rates (Sequential IDs) - TEST ---

print("\n" + "=" * 80)
print("Confusion matrices for ALL FFNN (Sequential IDs) learning rates - TEST")
print("=" * 80)

for lr in learning_rates_seq:
    model = models_seq[lr]
    print(f"\nGenerating confusion matrix for FFNN (Sequential IDs) with lr={lr:
 ↪.0e}...")

    plot_cm_task2(model=model, X_tensor=X_test_tensor, y_tensor=y_test_tensor,␣
 ↪model_name=f"FFNN Sequential IDs (lr={lr:.0e})", save_dir=save_dir)
```

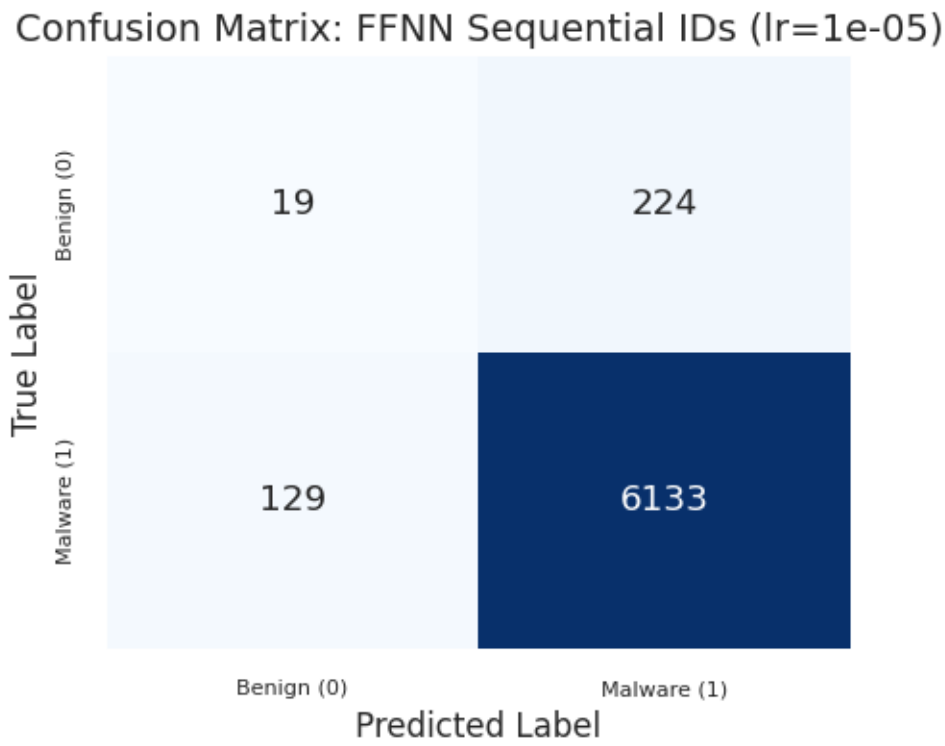
```
================================================================================
Confusion matrices for ALL FFNN (Sequential IDs) learning rates - TEST
================================================================================

Generating confusion matrix for FFNN (Sequential IDs) with lr=1e-04…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_sequential_ids_(lr=1e_04).png
```




```
Generating confusion matrix for FFNN (Sequential IDs) with lr=5e-05…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_sequential_ids_(lr=5e_05).png
```




```
Generating confusion matrix for FFNN (Sequential IDs) with lr=1e-05…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_sequential_ids_(lr=1e_05).png
```



Training with Learnable Embeddings Approach

[70]:
```
# --- Configuration Constants ---
seq_len = X_train.shape[1]
hidden_dims = [128, 64, 32]
dropout_rates = [0.4, 0.4, 0.4]
batchnorm_flags = [True, True, True]
min_delta = 0.00001
patience_early_stop = 15
epochs = 100

vocab_size = len(api_to_id)
embedding_dim = 128

# --- Device Setup (reuse from previous section if available) ---
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")

# Compute weights and move to device
pos_weight = compute_pos_weight(y_train_split).to(device)

# --- Define LR Candidates ---
learning_rates_emb = [0.00005, 0.00001, 0.000005]
```

```python
histories_emb = {}      # Store results here
models_emb = {}         # Store models here
elapsed_emb_times = {} # Store elapsed times here

print(f"\n{'='*80}")
print("Training FFNN (Learnable Embeddings) with different learning rates")
print(f"{'='*80}")

for lr in learning_rates_emb:
    print(f"\n--- Training FFNN (Learnable Embeddings) with lr={lr:.0e} ---")

    # Initialize model with embedding layer and move to device
    model_emb = MalwareFFNN(
        seq_len,
        hidden_dims,
        dropout_rates,
        batchnorm_flags,
        use_embedding=True,
        vocab_size=vocab_size,
        embedding_dim=embedding_dim,
    ).to(device)

    # Criterion (pos_weight already on device)
    criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)

    # Initialize Optimizer
    optimizer = optim.AdamW(model_emb.parameters(), lr=lr, weight_decay=1e-4)

    # Setup Scheduler
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
        optimizer,
        mode='min',
        factor=0.5,
        patience=5,
        min_lr=1e-6
    )

    # Train the model
    model_emb, history, elapsed_emb = train_model(
        model_emb,
        train_loader,
        val_loader,
        epochs,
        optimizer,
        criterion,
        min_delta,
        patience_early_stop,
```

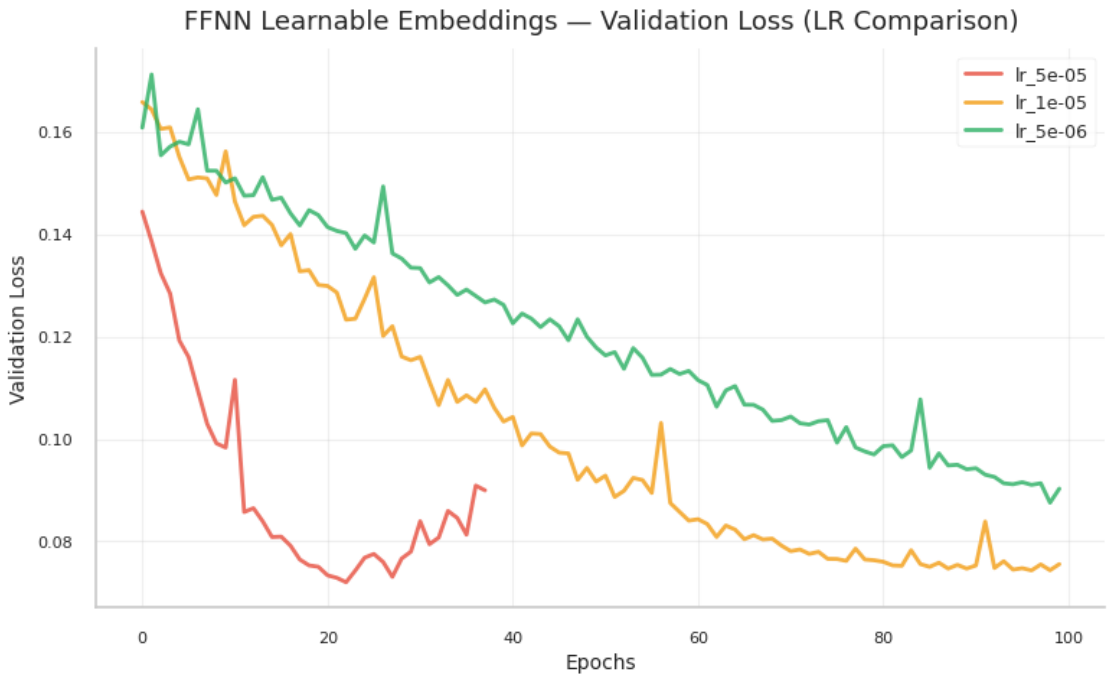
```
        scheduler=None
    )

    # Store model and history with numeric LR key
    models_emb[lr] = model_emb
    histories_emb[lr] = history
    elapsed_emb_times[lr] = elapsed_emb

    elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed_emb))
    print(f"Completed. Training time: {elapsed_str} ({elapsed_emb:.2f}s) -␣
↪Final val loss: {history['val_loss'][-1]:.4f}")
```

```
Using device: cuda
Negatives: 489, Positives: 12571
pos_weight (applied to y=1 samples): 0.2000

================================================================================
Training FFNN (Learnable Embeddings) with different learning rates
================================================================================

--- Training FFNN (Learnable Embeddings) with lr=5e-05 ---
Epoch 1/100 - Train Loss: 0.1520, Val Loss: 0.1444, Val F1(W): 0.8207
Epoch 5/100 - Train Loss: 0.1138, Val Loss: 0.1192, Val F1(W): 0.9218
Epoch 10/100 - Train Loss: 0.0853, Val Loss: 0.0983, Val F1(W): 0.9311
Epoch 15/100 - Train Loss: 0.0631, Val Loss: 0.0808, Val F1(W): 0.9429
Epoch 20/100 - Train Loss: 0.0452, Val Loss: 0.0750, Val F1(W): 0.9446
Epoch 25/100 - Train Loss: 0.0361, Val Loss: 0.0768, Val F1(W): 0.9508
Epoch 30/100 - Train Loss: 0.0257, Val Loss: 0.0780, Val F1(W): 0.9499
Epoch 35/100 - Train Loss: 0.0217, Val Loss: 0.0846, Val F1(W): 0.9548
Early stopping at epoch 38 (best val loss: 0.071992)

Training completed in 00:00:35 (hh:mm:ss) - 35.49 seconds
Completed. Training time: 00:00:35 (35.49s) - Final val loss: 0.0899

--- Training FFNN (Learnable Embeddings) with lr=1e-05 ---
Epoch 1/100 - Train Loss: 0.1721, Val Loss: 0.1659, Val F1(W): 0.4619
Epoch 5/100 - Train Loss: 0.1575, Val Loss: 0.1551, Val F1(W): 0.6919
Epoch 10/100 - Train Loss: 0.1433, Val Loss: 0.1562, Val F1(W): 0.7644
Epoch 15/100 - Train Loss: 0.1306, Val Loss: 0.1418, Val F1(W): 0.8124
Epoch 20/100 - Train Loss: 0.1197, Val Loss: 0.1301, Val F1(W): 0.8724
Epoch 25/100 - Train Loss: 0.1108, Val Loss: 0.1274, Val F1(W): 0.8785
Epoch 30/100 - Train Loss: 0.1024, Val Loss: 0.1154, Val F1(W): 0.9070
Epoch 35/100 - Train Loss: 0.0936, Val Loss: 0.1072, Val F1(W): 0.9205
Epoch 40/100 - Train Loss: 0.0863, Val Loss: 0.1034, Val F1(W): 0.9161
Epoch 45/100 - Train Loss: 0.0793, Val Loss: 0.0985, Val F1(W): 0.9247
Epoch 50/100 - Train Loss: 0.0714, Val Loss: 0.0917, Val F1(W): 0.9383
Epoch 55/100 - Train Loss: 0.0662, Val Loss: 0.0920, Val F1(W): 0.9358
Epoch 60/100 - Train Loss: 0.0623, Val Loss: 0.0840, Val F1(W): 0.9418
```

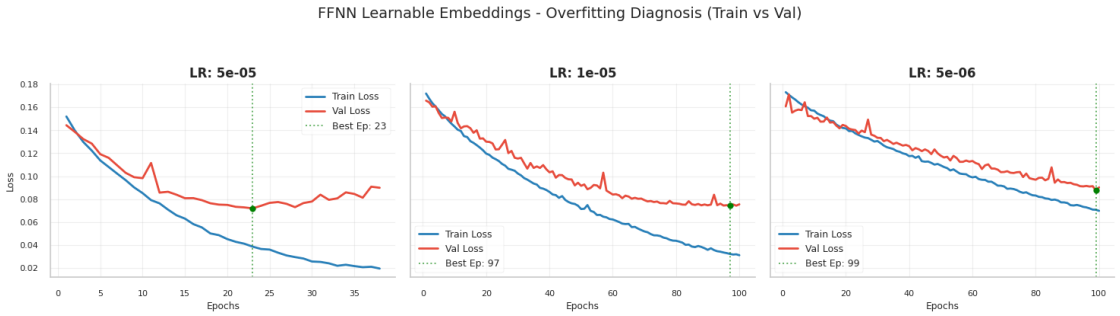
```
Epoch 65/100 - Train Loss: 0.0585, Val Loss: 0.0823, Val F1(W): 0.9387
Epoch 70/100 - Train Loss: 0.0519, Val Loss: 0.0792, Val F1(W): 0.9430
Epoch 75/100 - Train Loss: 0.0479, Val Loss: 0.0766, Val F1(W): 0.9491
Epoch 80/100 - Train Loss: 0.0438, Val Loss: 0.0763, Val F1(W): 0.9465
Epoch 85/100 - Train Loss: 0.0387, Val Loss: 0.0755, Val F1(W): 0.9455
Epoch 90/100 - Train Loss: 0.0359, Val Loss: 0.0747, Val F1(W): 0.9464
Epoch 95/100 - Train Loss: 0.0335, Val Loss: 0.0745, Val F1(W): 0.9494
Epoch 100/100 - Train Loss: 0.0312, Val Loss: 0.0755, Val F1(W): 0.9486

Training completed in 00:01:32 (hh:mm:ss) - 92.84 seconds
Completed. Training time: 00:01:32 (92.84s) - Final val loss: 0.0755

--- Training FFNN (Learnable Embeddings) with lr=5e-06 ---
Epoch 1/100 - Train Loss: 0.1732, Val Loss: 0.1609, Val F1(W): 0.5950
Epoch 5/100 - Train Loss: 0.1645, Val Loss: 0.1581, Val F1(W): 0.6463
Epoch 10/100 - Train Loss: 0.1571, Val Loss: 0.1501, Val F1(W): 0.7481
Epoch 15/100 - Train Loss: 0.1485, Val Loss: 0.1467, Val F1(W): 0.7947
Epoch 20/100 - Train Loss: 0.1417, Val Loss: 0.1438, Val F1(W): 0.8341
Epoch 25/100 - Train Loss: 0.1347, Val Loss: 0.1398, Val F1(W): 0.8562
Epoch 30/100 - Train Loss: 0.1307, Val Loss: 0.1335, Val F1(W): 0.8962
Epoch 35/100 - Train Loss: 0.1237, Val Loss: 0.1282, Val F1(W): 0.9022
Epoch 40/100 - Train Loss: 0.1175, Val Loss: 0.1262, Val F1(W): 0.8987
Epoch 45/100 - Train Loss: 0.1127, Val Loss: 0.1234, Val F1(W): 0.9104
Epoch 50/100 - Train Loss: 0.1094, Val Loss: 0.1179, Val F1(W): 0.9174
Epoch 55/100 - Train Loss: 0.1041, Val Loss: 0.1159, Val F1(W): 0.9167
Epoch 60/100 - Train Loss: 0.0990, Val Loss: 0.1133, Val F1(W): 0.9167
Epoch 65/100 - Train Loss: 0.0953, Val Loss: 0.1103, Val F1(W): 0.9195
Epoch 70/100 - Train Loss: 0.0911, Val Loss: 0.1037, Val F1(W): 0.9318
Epoch 75/100 - Train Loss: 0.0864, Val Loss: 0.1037, Val F1(W): 0.9275
Epoch 80/100 - Train Loss: 0.0832, Val Loss: 0.0970, Val F1(W): 0.9360
Epoch 85/100 - Train Loss: 0.0793, Val Loss: 0.1077, Val F1(W): 0.9402
Epoch 90/100 - Train Loss: 0.0766, Val Loss: 0.0941, Val F1(W): 0.9363
Epoch 95/100 - Train Loss: 0.0733, Val Loss: 0.0911, Val F1(W): 0.9432
Epoch 100/100 - Train Loss: 0.0699, Val Loss: 0.0903, Val F1(W): 0.9431

Training completed in 00:01:32 (hh:mm:ss) - 92.02 seconds
Completed. Training time: 00:01:32 (92.02s) - Final val loss: 0.0903
```

[71]:
```python
# --- Plot loss curves ---

# Plot comparison
plot_lr_comparison(histories_emb, "FFNN Learnable Embeddings", save_dir)

# Plot Train vs Validation
plot_train_val_grid(histories_emb, "FFNN Learnable Embeddings", save_dir)
```

```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/ffnn_learnable_embeddings_lr_comparison.png
```



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images /task2_plots/ffnn_learnable_embeddings_diagnosis.png



[72]:
```
# --- Evaluate validation and test sets for ALL learning rates ---

print("\n" + "=" * 80)
print("Evaluation for ALL FFNN (Learnable Embeddings) learning rates")
print("=" * 80)

for lr in learning_rates_emb:
    model = models_emb[lr]

    print(f"\n{'='*60}")
```

```
    print(f"FFNN (Learnable Embeddings) with lr={lr:.0e}")
    print(f"{'='*60}")

    # Validation evaluation
    print(f'\nValidation report (FFNN Learnable Embeddings - lr={lr:.0e}):')
    report = evaluate_model(model, X_val_tensor.to(device), y_val) # Moved␣
 ↪X_val_tensor to device
    print(report)

    # Test evaluation
    print(f'Test report (FFNN Learnable Embeddings - lr={lr:.0e}):')
    report_test = evaluate_model(model, X_test_tensor.to(device), y_test) #␣
 ↪Moved X_test_tensor to device
    print(report_test)

# Find and highlight best model
best_lr_emb = min(histories_emb.items(), key=lambda x: min(x[1]['val_loss']))
print(f"\n{'='*60}")
print(f"  Best FFNN (Learnable Embeddings) - LR: {best_lr_emb[0]:.0e} (Min Val␣
 ↪Loss: {min(best_lr_emb[1]['val_loss']):.4f})")
print(f"{'='*60}")
```

```
================================================================================
Evaluation for ALL FFNN (Learnable Embeddings) learning rates
================================================================================

============================================================
FFNN (Learnable Embeddings) with lr=5e-05
============================================================

Validation report (FFNN Learnable Embeddings - lr=5e-05):
              precision    recall  f1-score   support

           0     0.2924    0.4098    0.3413       122
           1     0.9767    0.9615    0.9691      3143

    accuracy                         0.9409      3265
   macro avg     0.6346    0.6857    0.6552      3265
weighted avg     0.9512    0.9409    0.9456      3265

Test report (FFNN Learnable Embeddings - lr=5e-05):
              precision    recall  f1-score   support

           0     0.1446    0.2922    0.1935       243
           1     0.9714    0.9329    0.9518      6262
```

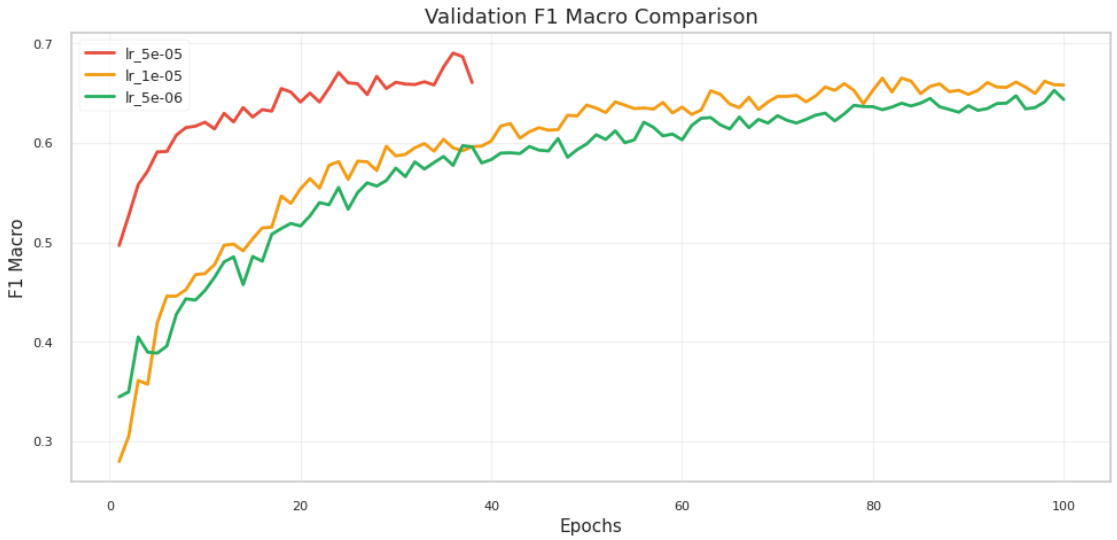
```
    accuracy                           0.9090      6505
   macro avg     0.5580    0.6126    0.5726      6505
weighted avg     0.9405    0.9090    0.9234      6505


============================================================
FFNN (Learnable Embeddings) with lr=1e-05
============================================================

Validation report (FFNN Learnable Embeddings - lr=1e-05):
              precision    recall  f1-score   support

           0     0.3014    0.3607    0.3284       122
           1     0.9750    0.9675    0.9713      3143

    accuracy                           0.9449      3265
   macro avg     0.6382    0.6641    0.6498      3265
weighted avg     0.9498    0.9449    0.9472      3265

Test report (FFNN Learnable Embeddings - lr=1e-05):
              precision    recall  f1-score   support

           0     0.1804    0.2881    0.2219       243
           1     0.9717    0.9492    0.9603      6262

    accuracy                           0.9245      6505
   macro avg     0.5761    0.6186    0.5911      6505
weighted avg     0.9422    0.9245    0.9328      6505


============================================================
FFNN (Learnable Embeddings) with lr=5e-06
============================================================

Validation report (FFNN Learnable Embeddings - lr=5e-06):
              precision    recall  f1-score   support

           0     0.2899    0.4016    0.3368       122
           1     0.9764    0.9618    0.9691      3143

    accuracy                           0.9409      3265
   macro avg     0.6332    0.6817    0.6529      3265
weighted avg     0.9508    0.9409    0.9454      3265

Test report (FFNN Learnable Embeddings - lr=5e-06):
              precision    recall  f1-score   support

           0     0.1794    0.3004    0.2246       243
```

```
              1       0.9721    0.9467    0.9592      6262

       accuracy                            0.9225      6505
      macro avg       0.5757    0.6235    0.5919      6505
   weighted avg       0.9425    0.9225    0.9318      6505
```

```
============================================================
 Best FFNN (Learnable Embeddings) - LR: 5e-05 (Min Val Loss: 0.0720)
============================================================
```

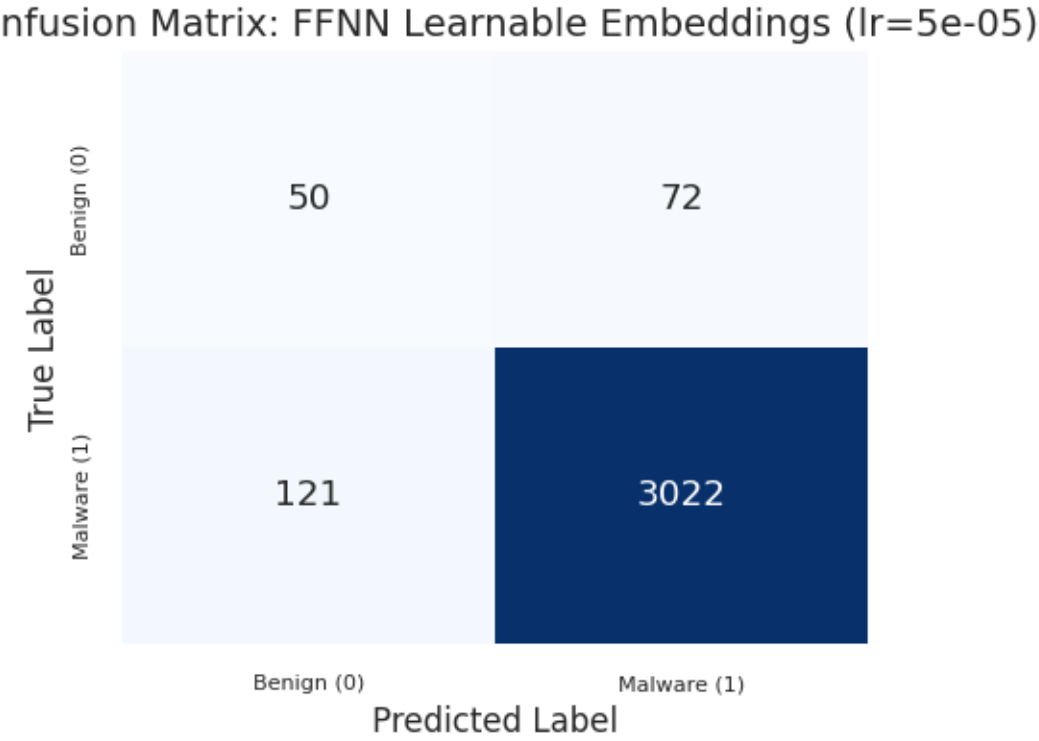
[73]:
```
# --- Plot metric curves for ALL learning rates (Learnable Embeddings) ---

print("\n" + "=" * 80)
print("Metric Curves for ALL FFNN (Learnable Embeddings) learning rates")
print("=" * 80)

# The plot_training_metrics function is designed to compare multiple histories.
plot_training_metrics(histories_emb, save_dir=save_dir,␣
 ↪plot_name="ffnn_learnable_embeddings_f1_comparison")
```

```
================================================================================
Metric Curves for ALL FFNN (Learnable Embeddings) learning rates
================================================================================
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/ffnn_learnable_embeddings_f1_comparison.png
```

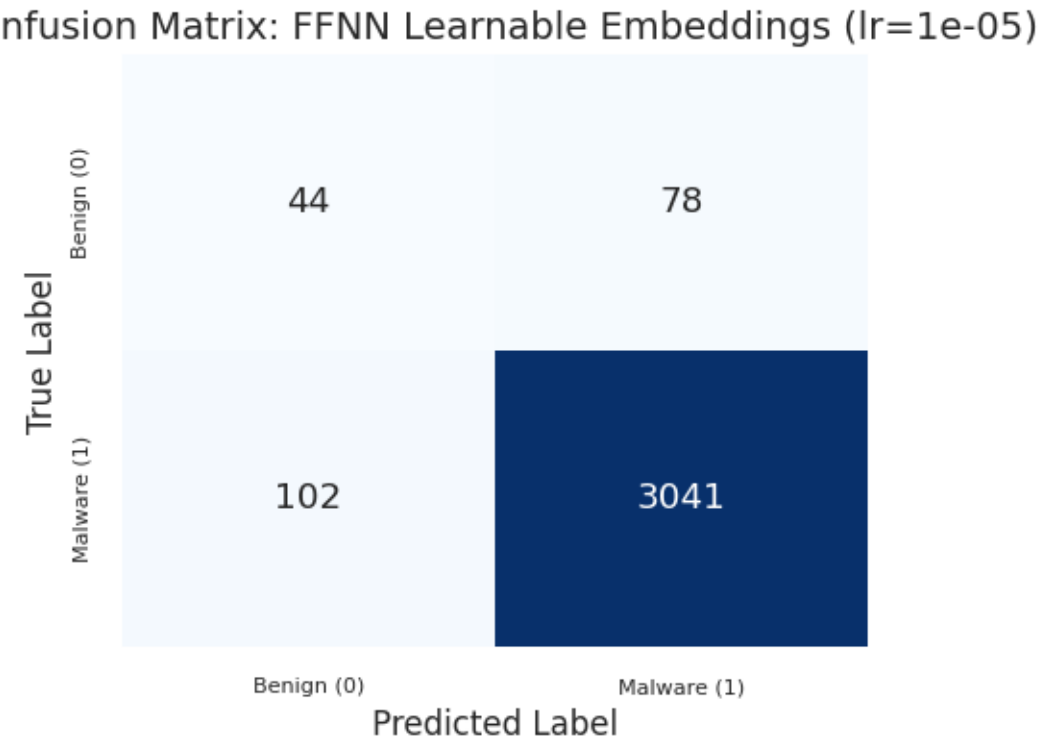


[74]:
```
# --- Plot confusion matrices for ALL learning rates (Learnable Embeddings) -␣
 ↪VALIDATION ---

print("\n" + "=" * 80)
print("Confusion matrices for ALL FFNN (Learnable Embeddings) learning rates -␣
 ↪VALIDATION")
print("=" * 80)

for lr in learning_rates_emb:
    model = models_emb[lr]
    print(f"\nGenerating confusion matrix for FFNN (Learnable Embeddings) with␣
 ↪lr={lr:.0e}...")

    plot_cm_task2(model=model, X_tensor=X_val_tensor, y_tensor=y_val_tensor,␣
 ↪model_name=f"FFNN Learnable Embeddings (lr={lr:.0e})", save_dir=save_dir)
```

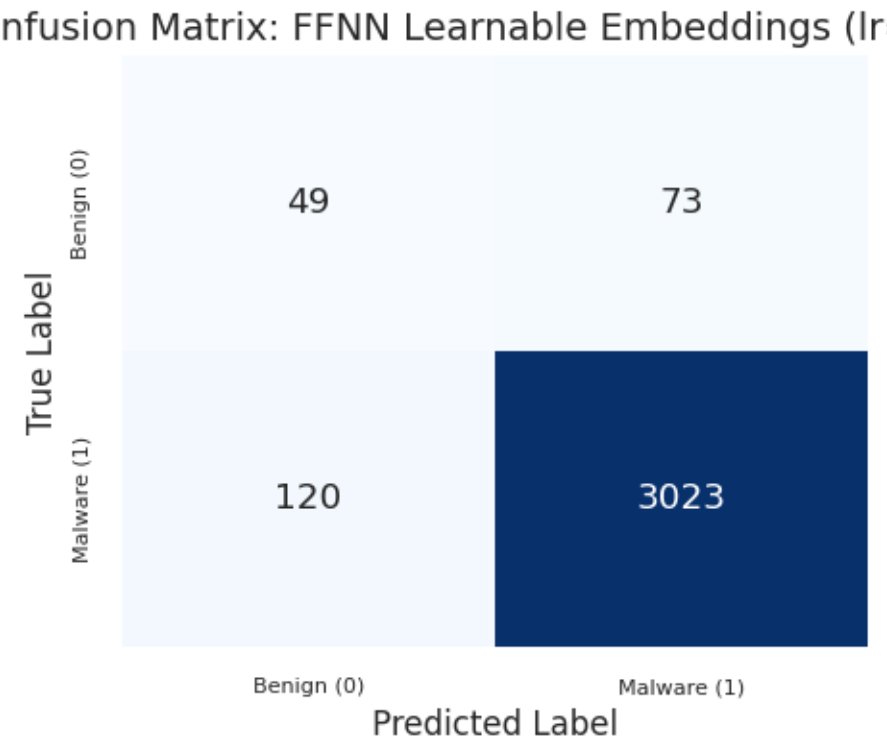
```
================================================================================
Confusion matrices for ALL FFNN (Learnable Embeddings) learning rates -
VALIDATION
================================================================================

Generating confusion matrix for FFNN (Learnable Embeddings) with lr=5e-05…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_learnable_embeddings_(lr=5e_05).png
```



```
Generating confusion matrix for FFNN (Learnable Embeddings) with lr=1e-05…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_learnable_embeddings_(lr=1e_05).png
```



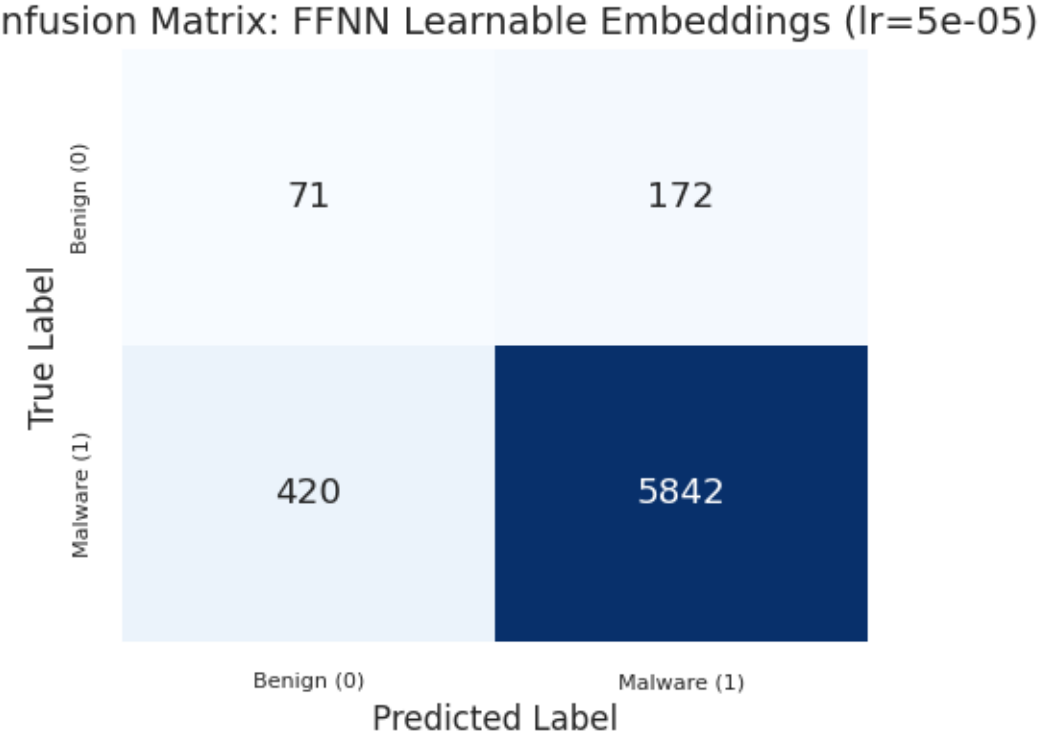

```
Generating confusion matrix for FFNN (Learnable Embeddings) with lr=5e-06…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_learnable_embeddings_(lr=5e_06).png
```



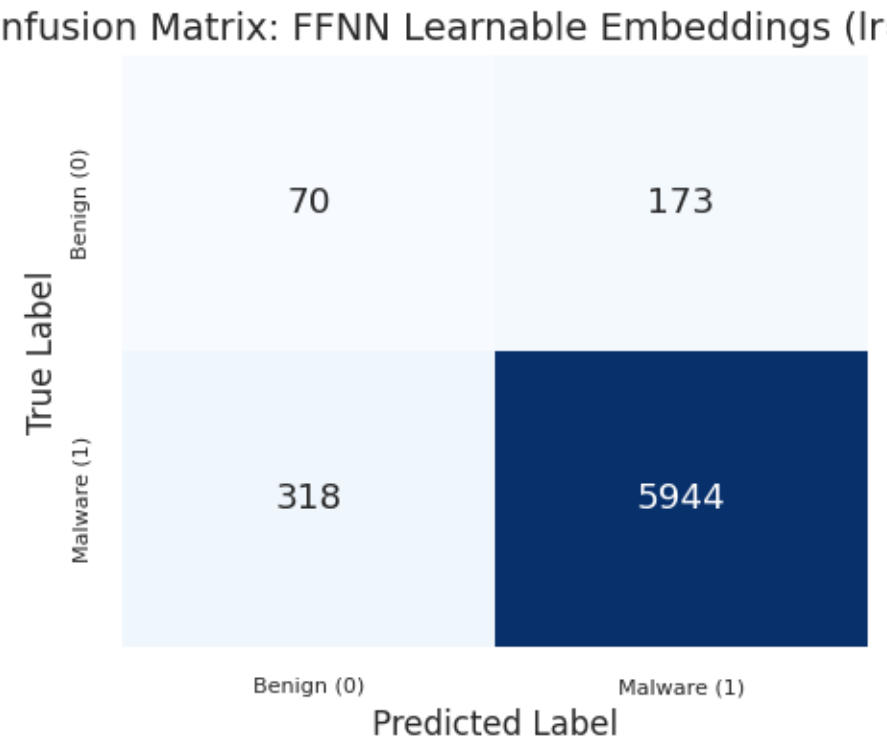
[75]:
```
# --- Plot confusion matrices for ALL learning rates (Learnable Embeddings) -␣
 ↪TEST ---

print("\n" + "=" * 80)
print("Confusion matrices for ALL FFNN (Learnable Embeddings) learning rates -␣
 ↪TEST")
print("=" * 80)

for lr in learning_rates_emb:
    model = models_emb[lr]
    print(f"\nGenerating confusion matrix for FFNN (Learnable Embeddings) with␣
 ↪lr={lr:.0e}...")

    plot_cm_task2(model=model, X_tensor=X_test_tensor, y_tensor=y_test_tensor,␣
 ↪model_name=f"FFNN Learnable Embeddings (lr={lr:.0e})", save_dir=save_dir)
```

```
================================================================================
Confusion matrices for ALL FFNN (Learnable Embeddings) learning rates - TEST
================================================================================

Generating confusion matrix for FFNN (Learnable Embeddings) with lr=5e-05…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
```

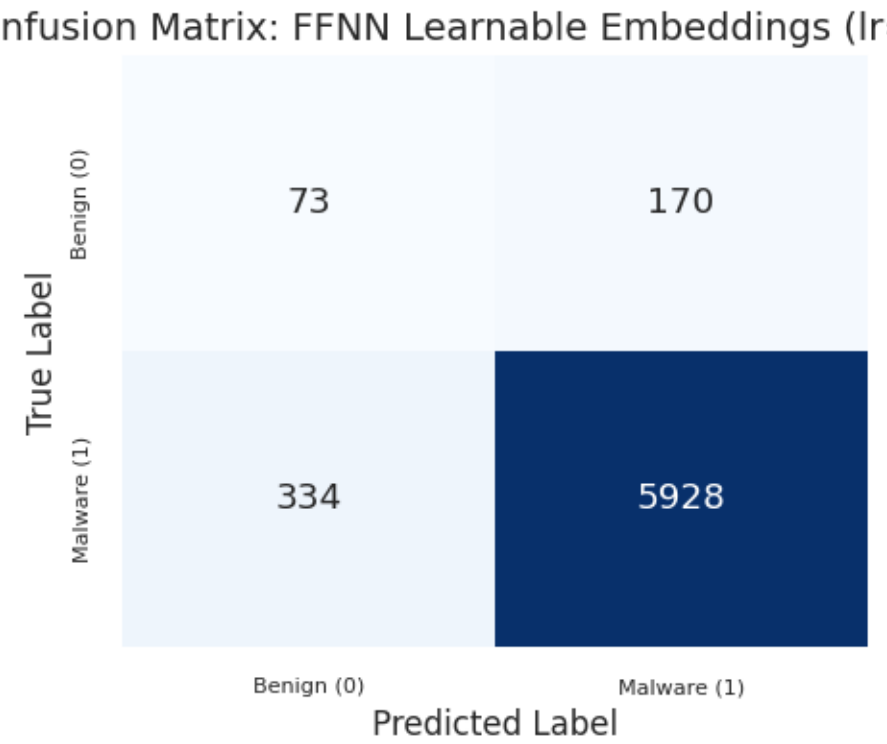
/task2_plots/cm_ffnn_learnable_embeddings_(lr=5e_05).png



Generating confusion matrix for FFNN (Learnable Embeddings) with lr=1e-05…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_learnable_embeddings_(lr=1e_05).png




```
Generating confusion matrix for FFNN (Learnable Embeddings) with lr=5e-06…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_learnable_embeddings_(lr=5e_06).png
```



Embedding size sensitivity analysis To complete the learnable-embedding experiment, we keep the selected FFNN architecture and learning rate fixed, and vary only the embedding dimensionality. This isolates the effect of representation capacity from the effect of the optimizer or hidden-layer design.

[76]:
```
# --- Embedding size sensitivity analysis ---

# The LR grid above selects the best learning rate for the learnable-embedding␣
 ↪FFNN.
# Here we keep that LR and the rest of the architecture fixed, changing only␣
 ↪embedding_dim.
embedding_sizes = [8, 16, 32, 64, 128]
if 'best_lr_emb' not in globals():
    raise RuntimeError('Run the learnable-embedding LR selection cell before␣
 ↪the embedding-size sweep.')
selected_embedding_lr = best_lr_emb[0]

histories_emb_size = {}
models_emb_size = {}
elapsed_emb_size_times = {}
embedding_size_rows = []
```

```
def predict_ffnn(model, X_tensor):
    """Return binary predictions for an FFNN model."""
    model.eval()
    model_device = next(model.parameters()).device
    with torch.no_grad():
        logits = model(X_tensor.to(model_device))
        probs = torch.sigmoid(logits)
        preds = (probs > 0.5).long().cpu().numpy().flatten()
    return preds

print(f"\n{'='*80}")
print("Embedding size sensitivity analysis for FFNN Learnable Embeddings")
print(f"Fixed learning rate: {selected_embedding_lr:.0e}")
print(f"Embedding sizes: {embedding_sizes}")
print(f"{'='*80}")

for emb_dim in embedding_sizes:
    print(f"\n--- Training FFNN Learnable Embeddings with␣
 ↪embedding_dim={emb_dim} ---")

    model_emb_size = MalwareFFNN(
        seq_len,
        hidden_dims,
        dropout_rates,
        batchnorm_flags,
        use_embedding=True,
        vocab_size=vocab_size,
        embedding_dim=emb_dim,
    ).to(device)

    criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)
    optimizer = optim.AdamW(model_emb_size.parameters(),␣
 ↪lr=selected_embedding_lr, weight_decay=1e-4)

    model_emb_size, history, elapsed_emb_size = train_model(
        model_emb_size,
        train_loader,
        val_loader,
        epochs,
        optimizer,
        criterion,
        min_delta,
        patience=patience_early_stop,
        scheduler=None,
    )

    histories_emb_size[emb_dim] = history
```

```
    models_emb_size[emb_dim] = model_emb_size
    elapsed_emb_size_times[emb_dim] = elapsed_emb_size

    val_pred = predict_ffnn(model_emb_size, X_val_tensor)
    test_pred = predict_ffnn(model_emb_size, X_test_tensor)

    val_report = classification_report(y_val, val_pred, output_dict=True,␣
 ↪zero_division=0)
    test_report = classification_report(y_test, test_pred, output_dict=True,␣
 ↪zero_division=0)

    embedding_size_rows.append({
        'embedding_dim': emb_dim,
        'min_val_loss': min(history['val_loss']),
        'best_val_macro_f1_curve': max(history['val_f1_macro']),
        'val_macro_f1': val_report['macro avg']['f1-score'],
        'val_benign_recall': val_report['0']['recall'],
        'test_macro_f1': test_report['macro avg']['f1-score'],
        'test_benign_recall': test_report['0']['recall'],
        'training_time_s': elapsed_emb_size,
    })

embedding_size_results = pd.DataFrame(embedding_size_rows)
embedding_size_results = embedding_size_results.sort_values('embedding_dim').
 ↪reset_index(drop=True)

print("\nEmbedding size comparison:")
display(embedding_size_results)

best_embedding_dim_row = embedding_size_results.
 ↪loc[embedding_size_results['min_val_loss'].idxmin()]
best_embedding_dim = int(best_embedding_dim_row['embedding_dim'])
print(
    f"\nBest embedding size by minimum validation loss: {best_embedding_dim} "
    f"(min val loss = {best_embedding_dim_row['min_val_loss']:.4f}, "
    f"test macro F1 = {best_embedding_dim_row['test_macro_f1']:.4f})"
)
```

```
================================================================================
Embedding size sensitivity analysis for FFNN Learnable Embeddings
Fixed learning rate: 5e-05
Embedding sizes: [8, 16, 32, 64, 128]
================================================================================

--- Training FFNN Learnable Embeddings with embedding_dim=8 ---
Epoch 1/100 - Train Loss: 0.1387, Val Loss: 0.1432, Val F1(W): 0.9100
```

```
Epoch 5/100 - Train Loss: 0.1167, Val Loss: 0.1262, Val F1(W): 0.9437
Epoch 10/100 - Train Loss: 0.1000, Val Loss: 0.1103, Val F1(W): 0.9437
Epoch 15/100 - Train Loss: 0.0897, Val Loss: 0.1045, Val F1(W): 0.9452
Epoch 20/100 - Train Loss: 0.0852, Val Loss: 0.1000, Val F1(W): 0.9473
Epoch 25/100 - Train Loss: 0.0775, Val Loss: 0.0971, Val F1(W): 0.9419
Epoch 30/100 - Train Loss: 0.0717, Val Loss: 0.0945, Val F1(W): 0.9354
Epoch 35/100 - Train Loss: 0.0655, Val Loss: 0.0940, Val F1(W): 0.9177
Epoch 40/100 - Train Loss: 0.0618, Val Loss: 0.0885, Val F1(W): 0.9184
Epoch 45/100 - Train Loss: 0.0562, Val Loss: 0.0889, Val F1(W): 0.8974
Epoch 50/100 - Train Loss: 0.0520, Val Loss: 0.0873, Val F1(W): 0.9008
Epoch 55/100 - Train Loss: 0.0484, Val Loss: 0.0871, Val F1(W): 0.8974
Epoch 60/100 - Train Loss: 0.0434, Val Loss: 0.0877, Val F1(W): 0.8964
Epoch 65/100 - Train Loss: 0.0385, Val Loss: 0.0833, Val F1(W): 0.9121
Epoch 70/100 - Train Loss: 0.0397, Val Loss: 0.0849, Val F1(W): 0.9138
Epoch 75/100 - Train Loss: 0.0330, Val Loss: 0.0837, Val F1(W): 0.9153
Early stopping at epoch 78 (best val loss: 0.082490)

Training completed in 00:01:08 (hh:mm:ss) - 68.49 seconds

--- Training FFNN Learnable Embeddings with embedding_dim=16 ---
Epoch 1/100 - Train Loss: 0.1622, Val Loss: 0.1613, Val F1(W): 0.5660
Epoch 5/100 - Train Loss: 0.1256, Val Loss: 0.1354, Val F1(W): 0.8869
Epoch 10/100 - Train Loss: 0.1015, Val Loss: 0.1158, Val F1(W): 0.9292
Epoch 15/100 - Train Loss: 0.0891, Val Loss: 0.1013, Val F1(W): 0.9354
Epoch 20/100 - Train Loss: 0.0777, Val Loss: 0.0958, Val F1(W): 0.9136
Epoch 25/100 - Train Loss: 0.0680, Val Loss: 0.0866, Val F1(W): 0.9235
Epoch 30/100 - Train Loss: 0.0584, Val Loss: 0.0834, Val F1(W): 0.9280
Epoch 35/100 - Train Loss: 0.0525, Val Loss: 0.0790, Val F1(W): 0.9256
Epoch 40/100 - Train Loss: 0.0420, Val Loss: 0.0796, Val F1(W): 0.9241
Epoch 45/100 - Train Loss: 0.0381, Val Loss: 0.0777, Val F1(W): 0.9333
Epoch 50/100 - Train Loss: 0.0330, Val Loss: 0.0789, Val F1(W): 0.9368
Epoch 55/100 - Train Loss: 0.0302, Val Loss: 0.0833, Val F1(W): 0.9406
Early stopping at epoch 58 (best val loss: 0.077617)

Training completed in 00:00:51 (hh:mm:ss) - 51.45 seconds

--- Training FFNN Learnable Embeddings with embedding_dim=32 ---
Epoch 1/100 - Train Loss: 0.1682, Val Loss: 0.1635, Val F1(W): 0.4878
Epoch 5/100 - Train Loss: 0.1290, Val Loss: 0.1350, Val F1(W): 0.9099
Epoch 10/100 - Train Loss: 0.1041, Val Loss: 0.1137, Val F1(W): 0.9283
Epoch 15/100 - Train Loss: 0.0864, Val Loss: 0.0995, Val F1(W): 0.9367
Epoch 20/100 - Train Loss: 0.0732, Val Loss: 0.0909, Val F1(W): 0.9296
Epoch 25/100 - Train Loss: 0.0616, Val Loss: 0.0840, Val F1(W): 0.9336
Epoch 30/100 - Train Loss: 0.0500, Val Loss: 0.0821, Val F1(W): 0.9278
Epoch 35/100 - Train Loss: 0.0427, Val Loss: 0.0799, Val F1(W): 0.9403
Epoch 40/100 - Train Loss: 0.0344, Val Loss: 0.0824, Val F1(W): 0.9438
Epoch 45/100 - Train Loss: 0.0290, Val Loss: 0.0872, Val F1(W): 0.9502
Early stopping at epoch 48 (best val loss: 0.078990)
```

```
Training completed in 00:00:41 (hh:mm:ss) - 41.88 seconds

--- Training FFNN Learnable Embeddings with embedding_dim=64 ---
Epoch 1/100 - Train Loss: 0.1735, Val Loss: 0.1509, Val F1(W): 0.7736
Epoch 5/100 - Train Loss: 0.1252, Val Loss: 0.1225, Val F1(W): 0.8972
Epoch 10/100 - Train Loss: 0.0919, Val Loss: 0.0968, Val F1(W): 0.9293
Epoch 15/100 - Train Loss: 0.0685, Val Loss: 0.0846, Val F1(W): 0.9437
Epoch 20/100 - Train Loss: 0.0516, Val Loss: 0.0794, Val F1(W): 0.9437
Epoch 25/100 - Train Loss: 0.0414, Val Loss: 0.0820, Val F1(W): 0.9512
Epoch 30/100 - Train Loss: 0.0324, Val Loss: 0.0843, Val F1(W): 0.9481
Epoch 35/100 - Train Loss: 0.0284, Val Loss: 0.0921, Val F1(W): 0.9542
Early stopping at epoch 38 (best val loss: 0.078601)

Training completed in 00:00:33 (hh:mm:ss) - 33.75 seconds

--- Training FFNN Learnable Embeddings with embedding_dim=128 ---
Epoch 1/100 - Train Loss: 0.1553, Val Loss: 0.1473, Val F1(W): 0.7891
Epoch 5/100 - Train Loss: 0.1144, Val Loss: 0.1166, Val F1(W): 0.9066
Epoch 10/100 - Train Loss: 0.0837, Val Loss: 0.0919, Val F1(W): 0.9350
Epoch 15/100 - Train Loss: 0.0609, Val Loss: 0.0784, Val F1(W): 0.9453
Epoch 20/100 - Train Loss: 0.0459, Val Loss: 0.0708, Val F1(W): 0.9478
Epoch 25/100 - Train Loss: 0.0336, Val Loss: 0.0816, Val F1(W): 0.9504
Epoch 30/100 - Train Loss: 0.0271, Val Loss: 0.0793, Val F1(W): 0.9522
Epoch 35/100 - Train Loss: 0.0223, Val Loss: 0.0838, Val F1(W): 0.9538
Early stopping at epoch 38 (best val loss: 0.069708)

Training completed in 00:00:35 (hh:mm:ss) - 35.40 seconds

Embedding size comparison:
```

	embedding_dim	min_val_loss	best_val_macro_f1_curve	val_macro_f1	\
0	8	0.082490	0.610624	0.589613	
1	16	0.077617	0.641823	0.626873	
2	32	0.078990	0.661026	0.632967	
3	64	0.078601	0.676398	0.630676	
4	128	0.069708	0.686089	0.673148	

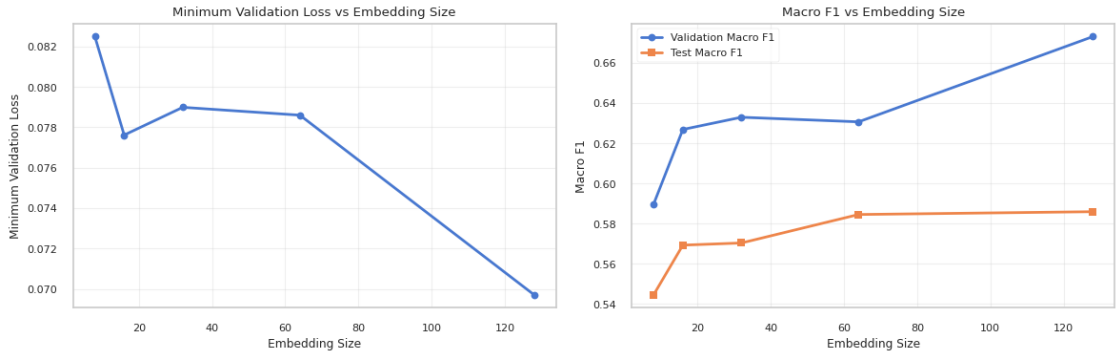
	val_benign_recall	test_macro_f1	test_benign_recall	training_time_s
0	0.483607	0.544464	0.353909	68.494550
1	0.475410	0.569241	0.308642	51.454352
2	0.409836	0.570320	0.325103	41.878716
3	0.303279	0.584506	0.246914	33.750937
4	0.401639	0.585908	0.271605	35.399494

```
Best embedding size by minimum validation loss: 128 (min val loss = 0.0697, test
macro F1 = 0.5859)
```

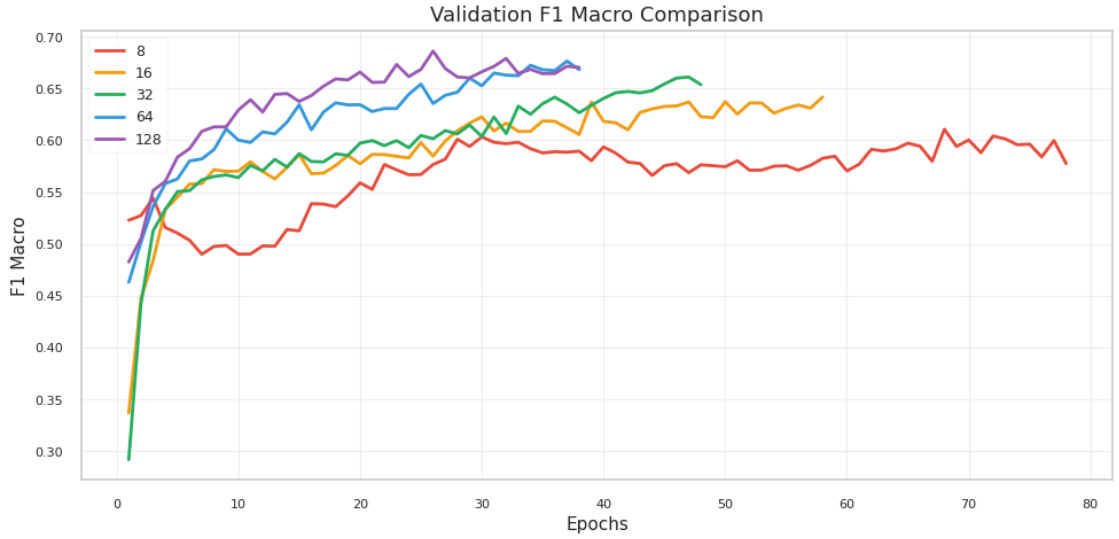
[77]:
```
# --- Plot embedding size comparison ---

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(
    embedding_size_results['embedding_dim'],
    embedding_size_results['min_val_loss'],
    marker='o',
    linewidth=2,
)
axes[0].set_title('Minimum Validation Loss vs Embedding Size')
axes[0].set_xlabel('Embedding Size')
axes[0].set_ylabel('Minimum Validation Loss')
axes[0].grid(True, alpha=0.3)

axes[1].plot(
    embedding_size_results['embedding_dim'],
    embedding_size_results['val_macro_f1'],
    marker='o',
    linewidth=2,
    label='Validation Macro F1',
)
axes[1].plot(
    embedding_size_results['embedding_dim'],
    embedding_size_results['test_macro_f1'],
    marker='s',
    linewidth=2,
    label='Test Macro F1',
)
axes[1].set_title('Macro F1 vs Embedding Size')
axes[1].set_xlabel('Embedding Size')
axes[1].set_ylabel('Macro F1')
axes[1].grid(True, alpha=0.3)
axes[1].legend()

plt.tight_layout()
if save_dir:
    save_plot(fig, 'ffnn_embedding_size_comparison', save_dir)
plt.show()

# Optional: compare the validation macro F1 curves across embedding sizes.
plot_training_metrics(
    histories_emb_size,
    save_dir=save_dir,
    plot_name='ffnn_embedding_size_f1_comparison'
)
```

```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
```

/task2_plots/ffnn_embedding_size_comparison.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task2_plots/ffnn_embedding_size_f1_comparison.png



Q: Use a FFNN in both cases. Report how you selected the hyperparameters of your final model, and justify your choices. For Task 2, we implemented two FFNN configurations: one using raw sequential identifiers and one using learnable embeddings. In both cases, model selection used validation loss as the primary criterion, while validation macro F1 was monitored because the dataset is strongly imbalanced and accuracy alone would hide poor behavior on the benign class.

Sequential identifiers. The sequential-ID model receives a fixed-length vector of 90 API identifiers. We used a funnel-shaped architecture with hidden dimensions `[256, 128, 64]`, Batch Normalization, Dropout, AdamW, and `BCEWithLogitsLoss`. The tested learning rates were `1e-4`, `5e-5`, and `1e-5`. The selected configuration is `lr=1e-4`, which reaches the lowest validation loss

(0.0891) and validation macro F1 0.5525. On the test set, it obtains accuracy 93.68% and macro F1 0.5510.

Learnable embeddings. The embedding-based model first maps each API identifier into a continuous vector and then flattens the embedded sequence before the dense layers. We used hidden dimensions [128, 64, 32], Batch Normalization, Dropout, AdamW, and the same early-stopping strategy. The tested learning rates were 5e-5, 1e-5, and 5e-6; the selected learning rate is 5e-5, with validation loss 0.0720 and validation macro F1 0.6552. On the test set, this selected model reaches accuracy 90.90% and macro F1 0.5726.

To address the requirement of testing different embedding sizes, we then fixed the selected learning rate (5e-5) and repeated the experiment with embedding dimensions [8, 16, 32, 64, 128]. The validation macro F1 increases from 0.5896 with dimension 8 to 0.6731 with dimension 128, showing that richer API-call representations help the FFNN. The best validation behavior is obtained with embedding dimension 128, which gives the lowest validation loss (0.0697) and the highest validation macro F1. On the test set, dimension 128 reaches macro F1 0.5859, only slightly above dimension 64 (0.5845), so the gain is real but modest.

The loss uses pos_weight=0.20 because label 1 corresponds to malware, which is the majority class. In BCEWithLogitsLoss, this down-weights malware errors and gives more relative influence to the minority benign samples during optimization.

Q: Can you obtain the same results for sequential identifiers and learnable embeddings? If not, why? No. The learnable-embedding representation performs better than raw sequential identifiers, especially when looking at validation macro F1 and benign-class behavior. The sequential-ID model treats API identifiers as ordinary numerical values, implicitly suggesting that nearby IDs have a meaningful numerical relationship. This is not true: the assignment of IDs is arbitrary, so the network may exploit accidental magnitudes rather than semantic similarity between API calls.

Learnable embeddings reduce this limitation by mapping each API call into a continuous vector space. During training, the model can learn representations that make API calls with similar roles or similar discriminative behavior easier to use jointly. This is reflected in the selected validation results: the best sequential-ID model reaches validation macro F1 0.5525, while the embedding-based model reaches 0.6552 with the selected learning rate.

The embedding-size experiment also shows that the embedding mechanism is useful, but not unlimited. Very small embeddings, such as dimension 8, do not provide enough representation capacity and obtain lower macro F1. Larger embeddings improve validation performance, with dimension 128 giving the best validation macro F1 (0.6731) and the lowest validation loss (0.0697). On the test set, however, dimension 128 (0.5859) is only slightly better than dimension 64 (0.5845), suggesting that extra capacity helps but does not fully solve the FFNN limitations.

Overall, learnable embeddings are preferable to sequential identifiers, but the FFNN remains limited by the fixed-length representation. Since sequences longer than 90 calls must be truncated, and since the dense network cannot explicitly model temporal dependencies, both FFNN variants remain weaker than the later RNN and GNN models.

[78]:
```
# ============================================================================
# FINAL SUMMARY
```

```
# ================================================================================

# Helper function to extract macro avg from the best FFNN model
def get_macro_avg_stats_ffnn(model, X_tensor):
    """
    Predict and get macro average statistics for an FFNN model.
    """
    # Get the device from the model itself
    device = next(model.parameters()).device

    model.eval()
    with torch.no_grad():
        outputs = model(X_tensor.to(device)) # Move input tensor to device
        probs = torch.sigmoid(outputs)
        y_pred = (probs > 0.5).long().cpu().numpy().flatten()

    y_true = y_val  # Using validation set
    report_dict = classification_report(y_true, y_pred, output_dict=True,␣
 ↪zero_division=0)
    return report_dict['macro avg']

# --- 1. Prepare Stats for FFNN Sequential IDs ---
best_model_seq_obj = models_seq[best_lr_seq[0]]
macro_seq = get_macro_avg_stats_ffnn(best_model_seq_obj, X_val_tensor)

# --- 2. Prepare Stats for FFNN Learnable Embeddings ---
best_model_emb_obj = models_emb[best_lr_emb[0]]
macro_emb = get_macro_avg_stats_ffnn(best_model_emb_obj, X_val_tensor)

# --- PRINTING THE REPORT ---

print("\n" + "=" * 100)
print("TASK 2 SUMMARY: Best Learning Rates & Macro Avg Performance")
print("=" * 100)
# Header for the manual table
print(f"{'Model':<30} | {'Best LR':<10} | {'Val Loss':<10} | {'Macro P':<8}␣
 ↪{'Macro R':<8} {'Macro F1':<8} | {'Time':<10}")
print("-" * 100)

# 1. FFNN Sequential IDs
t_seq = time.strftime('%H:%M:%S', time.
 ↪gmtime(elapsed_seq_times[best_lr_seq[0]]))
print(f"{'FFNN (Sequential IDs)':<30} | {best_lr_seq[0]:.0e}      |␣
 ↪{min(best_lr_seq[1]['val_loss']):.4f}     | "
      f"{macro_seq['precision']:.4f}   {macro_seq['recall']:.4f}  ␣
 ↪{macro_seq['f1-score']:.4f}   | {t_seq}")
```

```
# 2. FFNN Learnable Embeddings
t_emb = time.strftime('%H:%M:%S', time.
 ↪gmtime(elapsed_emb_times[best_lr_emb[0]]))
print(f"{'FFNN (Learnable Embeddings)':<30} | {best_lr_emb[0]:.0e}      |␣
 ↪{min(best_lr_emb[1]['val_loss']):.4f}     | "
      f"{macro_emb['precision']:.4f}   {macro_emb['recall']:.4f}  ␣
 ↪{macro_emb['f1-score']:.4f}   | {t_emb}")

# --- 3. Optional Embedding Size Summary ---
if 'embedding_size_results' in globals():
    best_emb_loss_row = embedding_size_results.
 ↪loc[embedding_size_results['min_val_loss'].idxmin()]
    best_emb_f1_row = embedding_size_results.
 ↪loc[embedding_size_results['val_macro_f1'].idxmax()]

    print("\n" + "=" * 100)
    print("EMBEDDING SIZE SENSITIVITY SUMMARY")
    print("=" * 100)
    print(
        f"Best by min validation loss:␣
 ↪dim={int(best_emb_loss_row['embedding_dim'])} | "
        f"val loss={best_emb_loss_row['min_val_loss']:.4f} | "
        f"val macro F1={best_emb_loss_row['val_macro_f1']:.4f} | "
        f"test macro F1={best_emb_loss_row['test_macro_f1']:.4f} | "
        f"time={best_emb_loss_row['training_time_s']:.2f}s"
    )
    print(
        f"Best by validation macro F1:␣
 ↪dim={int(best_emb_f1_row['embedding_dim'])} | "
        f"val loss={best_emb_f1_row['min_val_loss']:.4f} | "
        f"val macro F1={best_emb_f1_row['val_macro_f1']:.4f} | "
        f"test macro F1={best_emb_f1_row['test_macro_f1']:.4f} | "
        f"time={best_emb_f1_row['training_time_s']:.2f}s"
    )
```

```
================================================================================
====================
TASK 2 SUMMARY: Best Learning Rates & Macro Avg Performance
================================================================================
====================
Model                          | Best LR    | Val Loss   | Macro P  Macro R
Macro F1 | Time
--------------------------------------------------------------------------------
--------------------
FFNN (Sequential IDs)          | 1e-04      | 0.0891     | 0.5462   0.5627
0.5525   | 00:00:48
```

```
FFNN (Learnable Embeddings)    | 5e-05       | 0.0720      | 0.6346   0.6857
0.6552   | 00:00:35

================================================================================
====================
EMBEDDING SIZE SENSITIVITY SUMMARY
================================================================================
====================
Best by min validation loss: dim=128 | val loss=0.0697 | val macro F1=0.6731 |
test macro F1=0.5859 | time=35.40s
Best by validation macro F1: dim=128 | val loss=0.0697 | val macro F1=0.6731 |
test macro F1=0.5859 | time=35.40s
```

1.4 Task 3 - Recursive Neural Network (RNN)

[79]:
```python
# Create directory for plots
save_dir = results_path + 'images/' + 'task3_plots/'
os.makedirs(save_dir, exist_ok=True)
```

[80]:
```python
# --- Choose number of epochs depending on device (GPU vs CPU) ---

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# Use fewer epochs when running on CPU to avoid extremely long runs
if torch.cuda.is_available():
    num_epochs = 150
else:
    num_epochs = 15

print(f"Number of epochs: {num_epochs}")
```

```
Number of epochs: 150
```

1.4.1 Sequence Modeling with RNNs

[81]:
```python
# --- Build vocabulary and API-to-ID mappings from training data ---

def build_vocab_from_train(df_train, seq_col="api_call_sequence"):
    """
    Extract the set of unique API calls from the training set only.
    Returns a sorted list of API call names (vocab).
    """
    vocab = sorted({api for seq in df_train[seq_col] for api in seq})
    return vocab

def build_api_mappings(vocab):
    """
    Create dictionaries mapping API names to integer IDs and vice versa.
```

```
    Reserves indices 0 and 1 for <PAD> and <UNK> special tokens.
    """
    api_to_id = {"<PAD>": PAD_IDX, "<UNK>": UNK_IDX}
    for i, api in enumerate(vocab):
        api_to_id[api] = i + 2  # start after PAD/UNK
    id_to_api = {i: api for api, i in api_to_id.items()}
    return api_to_id, id_to_api

# Build vocabulary from training data only (no data leakage)
vocab_train = build_vocab_from_train(df_train)
api_to_id, id_to_api = build_api_mappings(vocab_train)
```

[82]:
```
# --- Encode API call sequences to integer IDs ---

def encode_sequences(df, api_to_id, seq_col="api_call_sequence",␣
 ↪label_col="is_malware", max_len=None):
    """
    Convert API call sequences to lists of integer IDs using the mapping.
    Unknown API calls (not seen in training) are mapped to <UNK> (PAD_IDX).
    Optionally truncate sequences to max_len (but we preserve full length for␣
 ↪RNNs).
    """
    seqs, labels = [], []
    for seq, y in zip(df[seq_col], df[label_col]):
        ids = [api_to_id.get(api, PAD_IDX) for api in seq]
        if max_len is not None:
            ids = ids[:max_len]  # only if you explicitly choose to truncate
        seqs.append(ids)
        labels.append(int(y))
    return seqs, labels

# Encode training and test sequences (no truncation - RNNs handle variable␣
 ↪lengths)
train_seqs, train_labels = encode_sequences(df_train, api_to_id)
test_seqs, test_labels = encode_sequences(df_test, api_to_id)
```

[83]:
```
# --- Split training data into train/validation subsets ---

# Create indices for stratified train/val split (preserves class balance)
idx = list(range(len(train_seqs)))
train_idx, val_idx = train_test_split(
    idx, test_size=0.2, random_state=42, stratify=train_labels
)

def subset_by_idx(seqs, labels, indices):
    """Helper to extract sequences and labels at given indices."""
    return [seqs[i] for i in indices], [labels[i] for i in indices]
```

```
# Create training and validation subsets
train_seqs_sub, train_labels_sub = subset_by_idx(train_seqs, train_labels,␣
 ↪train_idx)
val_seqs_sub, val_labels_sub = subset_by_idx(train_seqs, train_labels, val_idx)
```

[84]:
```
# --- Define PyTorch Dataset for variable-length sequences ---

class SeqDataset(torch.utils.data.Dataset):
    """
    A PyTorch Dataset that wraps sequences and their labels.
    Each item returns a tensor of API IDs and a scalar label.
    Note: Sequences have variable lengths at this stage.
    """
    def __init__(self, sequences, labels):
        self.sequences = sequences
        self.labels = labels

    def __len__(self):
        return len(self.sequences)

    def __getitem__(self, idx):
        x = torch.tensor(self.sequences[idx], dtype=torch.long)
        y = torch.tensor(self.labels[idx], dtype=torch.float32) # scalar
        return x, y
```

[85]:
```
# --- Custom collate function for padding and packing ---

def collate_fn(batch):
    """
    Custom collate function for DataLoader:
    1. Sorts sequences by length (descending) - required for␣
 ↪pack_padded_sequence
    2. Pads all sequences to the max length in this batch
    3. Returns padded sequences, original lengths, and labels (CPU tensors)
    Note: Device transfer is handled by the training loop, not the collate␣
 ↪function.
    """
    sequences, labels = zip(*batch)

    # Compute original lengths before padding
    lengths = torch.tensor([len(s) for s in sequences], dtype=torch.long)
    # Sort by length (descending) for pack_padded_sequence compatibility
    lengths_sorted, perm_idx = lengths.sort(descending=True)

    # Reorder sequences and labels according to sorted indices
    sequences = [sequences[i] for i in perm_idx]
```

```
    labels = torch.stack([labels[i] for i in perm_idx]).unsqueeze(1)  # (B,1)

    # Pad sequences to max length in batch (dynamic padding)
    padded = pad_sequence(sequences, batch_first=True, padding_value=PAD_IDX)

    # Return CPU tensors - training loop handles device transfer
    return padded, lengths_sorted, labels
```

[86]:
```
# --- Create DataLoaders with custom collate function ---

# Wrap sequences in Dataset objects
train_ds = SeqDataset(train_seqs_sub, train_labels_sub)
val_ds = SeqDataset(val_seqs_sub, val_labels_sub)
test_ds = SeqDataset(test_seqs, test_labels)

# Create DataLoaders with dynamic padding via collate_fn
# Note: collate_fn handles variable-length sequences by padding per batch
train_loader = DataLoader(train_ds, batch_size=64, shuffle=True,␣
 ↪collate_fn=collate_fn)
val_loader = DataLoader(val_ds, batch_size=64, shuffle=False,␣
 ↪collate_fn=collate_fn)
test_loader = DataLoader(test_ds, batch_size=64, shuffle=False,␣
 ↪collate_fn=collate_fn)
```

Q: With RNNs, do you still have to pad your data? If yes, how? Yes. RNNs can process sequences of different lengths in principle, but mini-batch training still requires tensors with a consistent shape inside each batch. For this reason, we pad the sequences dynamically at batch level rather than padding every sequence to a global fixed length.

The custom `collate_fn` sorts the samples in each batch by decreasing sequence length, pads only up to the longest sequence in that batch, and returns both the padded tensor and the original lengths. The embedding layer uses `padding_idx=0`, so the padding token has a dedicated embedding. Then, before the recurrent layer, we use `torch.nn.utils.rnn.pack_padded_sequence`, which tells the RNN to ignore padded positions during the recurrent computation.

This approach keeps the batch representation compatible with PyTorch while avoiding hidden-state updates on dummy tokens.

Q: Do you have to truncate the testing sequences? Justify your answer with your understanding of why it is/it is not the case. No. Unlike an FFNN, an RNN does not have a fixed input layer tied to a specific sequence length. The same recurrent weights are reused at every time step, so a longer test sequence simply produces more recurrent updates.

In our implementation, padding and `pack_padded_sequence` are used only to manage batches efficiently. They do not impose the fixed length of 90 used in the FFNN. Therefore, test sequences longer than the maximum training length do not need to be truncated for architectural reasons.

Avoiding truncation is important because API calls near the end of a trace may contain useful behavioral information. Truncating them would remove part of the execution context, while the

recurrent model can process the full sequence within normal memory limits.

[87]:
```
# Reusable training loop for RNN-like modules
def train_rnn_common(model, train_loader, val_loader, optimizer, criterion,␣
 ↪epochs, min_delta=None, patience=None, scheduler=None):
    """
    A generic training loop for RNN-based models that supports early stopping.
    Now also tracks validation F1 macro score.
    """
    device = next(model.parameters()).device

    best_val = math.inf
    best_state = None
    trigger = 0
    history = {'train_loss': [], 'val_loss': [], 'val_f1_macro': []}
    start_time = time.time()

    for epoch in range(1, epochs+1):
        # --- Training Phase ---
        model.train()
        train_losses = []
        for X_batch, lengths, y_batch in train_loader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            optimizer.zero_grad()
            outputs = model(X_batch, lengths)
            loss = criterion(outputs, y_batch)
            loss.backward()
            optimizer.step()
            train_losses.append(loss.item())
        train_loss = float(np.mean(train_losses)) if train_losses else 0.0

        # --- Validation Phase ---
        model.eval()
        val_losses = []
        all_preds = []
        all_targets = []
        with torch.no_grad():
            for X_batch, lengths, y_batch in val_loader:
                X_batch, y_batch = X_batch.to(device), y_batch.to(device)
                outputs = model(X_batch, lengths)
                loss = criterion(outputs, y_batch)
                val_losses.append(loss.item())

                # Collect predictions for F1 score
                probs = torch.sigmoid(outputs).view(-1)
                preds = (probs > 0.5).long()
                all_preds.extend(preds.cpu().tolist())
```

```
                all_targets.extend(y_batch.view(-1).cpu().tolist())

        val_loss = float(np.mean(val_losses)) if val_losses else 0.0
        val_f1_macro = f1_score(all_targets, all_preds, average='macro',␣
↪zero_division=0)

        history['train_loss'].append(train_loss)
        history['val_loss'].append(val_loss)
        history['val_f1_macro'].append(val_f1_macro)

        # --- Scheduler Step ---
        if scheduler is not None:
            last_lr = optimizer.param_groups[0]['lr']
            if isinstance(scheduler, torch.optim.lr_scheduler.
↪ReduceLROnPlateau):
                scheduler.step(val_loss)
            else:
                scheduler.step()
            curr_lr = optimizer.param_groups[0]['lr']
            if curr_lr != last_lr:
                print(f"Epoch {epoch}: LR reduced from {last_lr:.6f} to␣
↪{curr_lr:.6f}")

        if (epoch % 5 == 0) or (epoch == 1) or (epoch == epochs):
            print(f'Epoch {epoch}/{epochs} - Train Loss: {train_loss:.4f} -␣
↪Val Loss: {val_loss:.4f} - Val F1: {val_f1_macro:.4f}')

        # --- Early Stopping ---
        if val_loss < best_val - (min_delta if min_delta is not None else 0):
            best_val = val_loss
            best_state = {k: v.cpu().clone() for k, v in model.state_dict().
↪items()}
            trigger = 0
        else:
            trigger += 1
            if trigger >= patience:
                print(f'Early stopping at epoch {epoch} (best val loss␣
↪{best_val:.6f})')
                break

    elapsed = time.time() - start_time

    if best_state is not None:
        model.load_state_dict(best_state)

    return model, history, elapsed
```

[88]:
```python
# Prediction helper (binary classification threshold)
def predict_loader(model, loader):
    """
    Run the model on a DataLoader and return integer arrays for
    ground-truth labels and predicted labels (0/1).
    """
    # Set model to evaluation mode (e.g., disables dropout)
    model.eval()

    # Detect device from model parameters
    device = next(model.parameters()).device

    ys_true = [] # To store all true labels
    ys_pred = [] # To store all predicted labels

    # Disable gradient calculations to save memory and computation
    with torch.no_grad():
        # Iterate over all batches in the data loader
        for X_batch, lengths, y_batch in loader:
            # Move batch to device
            X_batch = X_batch.to(device)

            # Get model outputs (logits)
            out = model(X_batch, lengths)

            # Convert logits to probabilities using sigmoid
            probs = torch.sigmoid(out).cpu().numpy().ravel()

            # Apply threshold for binary classification
            preds = (probs >= 0.5).astype(int)

            # Store the ground-truth and predicted labels from this batch
            ys_true.extend(y_batch.cpu().numpy().ravel().astype(int).tolist())
            ys_pred.extend(preds.tolist())

    # Convert the lists of all labels/preds into final numpy arrays
    return np.array(ys_true, dtype=int), np.array(ys_pred, dtype=int)
```

[89]:
```python
def plot_cm_task3(model, loader, model_name, save_dir=None):
    """
    Specific for RNNs. Uses your existing 'predict_loader' logic.
    """
    # Use the helper function you already defined in the notebook
    y_true, y_pred = predict_loader(model, loader)

    # Plot
    _internal_plot_cm(y_true, y_pred, model_name, save_dir)
```

[90]:
```
# Function to train a model with a specific learning rate
def train_with_lr(model_class, model_params, initial_lr, train_loader,␣
 ↪val_loader, criterion, num_epochs, min_delta, patience, scheduler_params,␣
 ↪device):
    """
    Train a model with a specific initial learning rate and return the history.
    """
    # Create a fresh instance of the model
    model = model_class(**model_params).to(device)

    # Create optimizer with the specified learning rate
    optimizer = optim.AdamW(model.parameters(), lr=initial_lr)

    # Create scheduler
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
        optimizer,
        **scheduler_params
    )

    # Train the model
    model, history, elapsed = train_rnn_common(
        model,
        train_loader,
        val_loader,
        optimizer,
        criterion,
        epochs=num_epochs,
        min_delta=min_delta,
        patience=patience,
        scheduler=scheduler
    )

    return model, history, elapsed
```

Q: Is the RNN padding more memory efficient compared to the FFNN's one? Why? Yes, the RNN padding strategy is more efficient than the FFNN fixed-length strategy. In Task 2, every sequence must be converted into the same global length of 90 before entering the FFNN. This means that shorter sequences still occupy the full fixed-size input vector, and longer test sequences must be truncated.

In Task 3, padding is performed only inside each mini-batch, up to the longest sequence in that batch. Moreover, `pack_padded_sequence` prevents the recurrent layer from computing hidden-state transitions on padding tokens. The padded tensor still exists for batching, but the expensive recurrent computation is focused on real API calls.

This makes the RNN approach more flexible and usually more memory/computation efficient for variable-length sequences, especially when sequence lengths differ substantially across samples.

[91]:
```
# --- Variation 1: Simple one-directional RNN (nn.RNN) ---

class SimpleRNNClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim=32, hidden_size=128, num_layers=2,␣
↪num_classes=1, bidirectional=False, dropout=0.25, padding=0):
        super(SimpleRNNClassifier, self).__init__()

        # Embedding layer: Maps vocabulary indices (integers) to dense vectors␣
↪(embeddings).
        # padding_idx=0 ensures the padding token doesn't contribute to␣
↪gradients.
        self.embedding = nn.Embedding(vocab_size, embedding_dim=embed_dim,␣
↪padding_idx=padding)

        # Simple RNN layer: Processes the embedded sequences.
        self.rnn = nn.RNN(input_size=embed_dim, hidden_size=hidden_size,␣
↪num_layers=num_layers,
                          batch_first=True, nonlinearity='tanh',␣
↪bidirectional=bidirectional)

        self.bidirectional = bidirectional
        fc_in = hidden_size * (2 if bidirectional else 1)

        # Final fully connected layer for classification.
        self.fc = nn.Linear(fc_in, num_classes)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, lengths):
        # Embed the input indices
        emb = self.embedding(x) # Shape: (batch_size, max_seq_len, embed_dim)
        emb = self.dropout(emb)

        # Pack the sequence to let the RNN ignore padding
        packed = pack_padded_sequence(emb, lengths.cpu(), batch_first=True,␣
↪enforce_sorted=True)

        # Process the packed sequence through the RNN
        packed_out, hn = self.rnn(packed) # hn is the final hidden state(s)

        # hn shape: (num_layers * num_directions, batch_size, hidden_size)

        # Get the final hidden state to represent the whole sequence.
        if self.bidirectional:
            # Concatenate the final forward (hn[-2]) and backward (hn[-1])␣
↪states
            h_last = torch.cat([hn[-2], hn[-1]], dim=1)
```

```
        else:
            # Use the final hidden state from the last layer (hn[-1])
            h_last = hn[-1]

        # Pass the final hidden state through the classifier
        out = self.fc(h_last)
        return out
```

[92]:
```
# --- Variation 1: Simple Unidirectional RNN ---

print("\n" + "=" * 80)
print("Training Simple RNN with different learning rates")
print("=" * 80)

# Define learning rates to test
learning_rates = [2e-04, 1e-04, 8e-05, 5e-05]

# Model parameters
vocab_size_rnn = len(api_to_id)
simple_rnn_params = {
    'vocab_size': vocab_size_rnn,
    'embed_dim': 32,
    'hidden_size': 128,
    'num_layers': 2,
    'num_classes': 1,
    'bidirectional': False,
    'dropout': 0.3
}

# Initialize early stopping parameters
min_delta = 0.00001
patience = 20

# Compute class balance from training subset and set pos_weight for␣
 ↪BCEWithLogitsLoss
pos = sum(train_labels_sub) # number of positive (1) samples
neg = len(train_labels_sub) - pos

# Compute weights
pos_weight = compute_pos_weight(train_labels_sub).to(device)

criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)

# Setup Scheduler
scheduler_params_simple = {
    'mode': 'min', # reduce when metric stops decreasing (val_loss)
    'factor': 0.5, # halve the LR when triggered
```

```
    'patience': 8, # wait 8 epochs with no improvement before cutting LR
    'min_lr': 1e-7
}

# Store histories and models for each LR
histories_simple = {}
models_simple = {}
elapsed_simple = {}

for lr in learning_rates:
    print(f"\n--- Training Simple RNN with lr={lr:.0e} ---")
    model, history, elapsed = train_with_lr(
        model_class=SimpleRNNClassifier,
        model_params=simple_rnn_params,
        initial_lr=lr,
        train_loader=train_loader,
        val_loader=val_loader,
        criterion=nn.BCEWithLogitsLoss(pos_weight=pos_weight),
        num_epochs=num_epochs,
        min_delta=min_delta,
        patience=patience,
        scheduler_params=scheduler_params_simple,
        device=device
    )
    histories_simple[lr] = history
    models_simple[lr] = model
    elapsed_simple[lr] = elapsed
    elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed))
    print(f"Completed. Training time: {elapsed_str} ({elapsed:.2f}s) - Final␣
 ↪val loss: {history['val_loss'][-1]:.4f}")
```

```
================================================================================
Training Simple RNN with different learning rates
================================================================================
Negatives: 489, Positives: 12571
pos_weight (applied to y=1 samples): 0.2000

--- Training Simple RNN with lr=2e-04 ---
Epoch 1/150 - Train Loss: 0.0963 - Val Loss: 0.0859 - Val F1: 0.6193
Epoch 5/150 - Train Loss: 0.0760 - Val Loss: 0.0765 - Val F1: 0.6235
Epoch 10/150 - Train Loss: 0.0726 - Val Loss: 0.0748 - Val F1: 0.6285
Epoch 15/150 - Train Loss: 0.0653 - Val Loss: 0.0713 - Val F1: 0.6411
Epoch 20/150 - Train Loss: 0.0574 - Val Loss: 0.0711 - Val F1: 0.6605
Epoch 25/150 - Train Loss: 0.0536 - Val Loss: 0.0653 - Val F1: 0.6758
Epoch 30/150 - Train Loss: 0.0499 - Val Loss: 0.0672 - Val F1: 0.6358
Epoch 35/150 - Train Loss: 0.0445 - Val Loss: 0.0653 - Val F1: 0.7178
Epoch 40/150 - Train Loss: 0.0382 - Val Loss: 0.0687 - Val F1: 0.6973
```

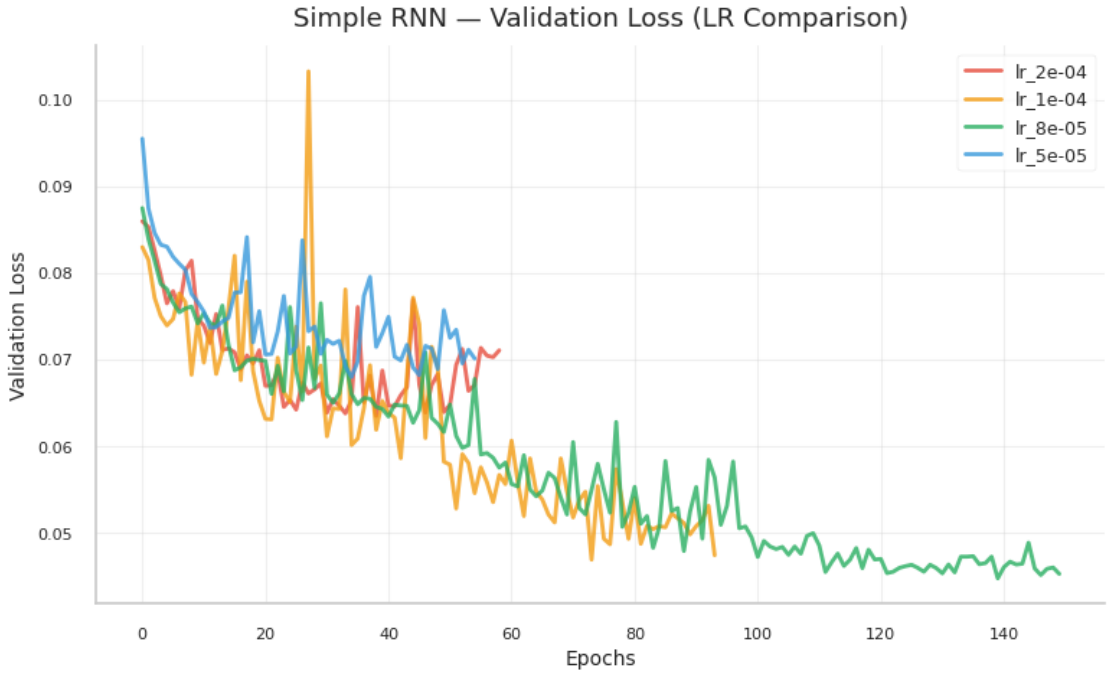
```
Epoch 45/150 - Train Loss: 0.0384 - Val Loss: 0.0769 - Val F1: 0.6227
Epoch 48: LR reduced from 0.000200 to 0.000100
Epoch 50/150 - Train Loss: 0.0303 - Val Loss: 0.0640 - Val F1: 0.6837
Epoch 55/150 - Train Loss: 0.0286 - Val Loss: 0.0671 - Val F1: 0.7380
Epoch 57: LR reduced from 0.000100 to 0.000050
Early stopping at epoch 59 (best val loss 0.063513)
Completed. Training time: 00:02:43 (163.34s) - Final val loss: 0.0711

--- Training Simple RNN with lr=1e-04 ---
Epoch 1/150 - Train Loss: 0.1035 - Val Loss: 0.0830 - Val F1: 0.6394
Epoch 5/150 - Train Loss: 0.0795 - Val Loss: 0.0739 - Val F1: 0.6221
Epoch 10/150 - Train Loss: 0.0686 - Val Loss: 0.0745 - Val F1: 0.6131
Epoch 15/150 - Train Loss: 0.0637 - Val Loss: 0.0759 - Val F1: 0.6129
Epoch 20/150 - Train Loss: 0.0597 - Val Loss: 0.0653 - Val F1: 0.6743
Epoch 25/150 - Train Loss: 0.0571 - Val Loss: 0.0651 - Val F1: 0.7097
Epoch 30/150 - Train Loss: 0.0537 - Val Loss: 0.0693 - Val F1: 0.6853
Epoch 35/150 - Train Loss: 0.0507 - Val Loss: 0.0602 - Val F1: 0.7100
Epoch 40/150 - Train Loss: 0.0467 - Val Loss: 0.0652 - Val F1: 0.7183
Epoch 45/150 - Train Loss: 0.0450 - Val Loss: 0.0772 - Val F1: 0.6359
Epoch 50/150 - Train Loss: 0.0422 - Val Loss: 0.0582 - Val F1: 0.6693
Epoch 55/150 - Train Loss: 0.0383 - Val Loss: 0.0546 - Val F1: 0.7200
Epoch 60/150 - Train Loss: 0.0371 - Val Loss: 0.0557 - Val F1: 0.7766
Epoch 61: LR reduced from 0.000100 to 0.000050
Epoch 65/150 - Train Loss: 0.0317 - Val Loss: 0.0548 - Val F1: 0.7239
Epoch 70/150 - Train Loss: 0.0312 - Val Loss: 0.0549 - Val F1: 0.7243
Epoch 75/150 - Train Loss: 0.0299 - Val Loss: 0.0554 - Val F1: 0.7386
Epoch 80/150 - Train Loss: 0.0280 - Val Loss: 0.0493 - Val F1: 0.7812
Epoch 83: LR reduced from 0.000050 to 0.000025
Epoch 85/150 - Train Loss: 0.0262 - Val Loss: 0.0508 - Val F1: 0.7866
Epoch 90/150 - Train Loss: 0.0259 - Val Loss: 0.0498 - Val F1: 0.7890
Epoch 92: LR reduced from 0.000025 to 0.000013
Early stopping at epoch 94 (best val loss 0.046940)
Completed. Training time: 00:04:19 (259.54s) - Final val loss: 0.0474

--- Training Simple RNN with lr=8e-05 ---
Epoch 1/150 - Train Loss: 0.1047 - Val Loss: 0.0875 - Val F1: 0.5844
Epoch 5/150 - Train Loss: 0.0809 - Val Loss: 0.0781 - Val F1: 0.6296
Epoch 10/150 - Train Loss: 0.0720 - Val Loss: 0.0742 - Val F1: 0.6584
Epoch 15/150 - Train Loss: 0.0692 - Val Loss: 0.0717 - Val F1: 0.6570
Epoch 20/150 - Train Loss: 0.0665 - Val Loss: 0.0700 - Val F1: 0.6716
Epoch 25/150 - Train Loss: 0.0659 - Val Loss: 0.0761 - Val F1: 0.5957
Epoch 30/150 - Train Loss: 0.0589 - Val Loss: 0.0765 - Val F1: 0.6037
Epoch 35/150 - Train Loss: 0.0574 - Val Loss: 0.0660 - Val F1: 0.6996
Epoch 40/150 - Train Loss: 0.0521 - Val Loss: 0.0643 - Val F1: 0.6689
Epoch 45/150 - Train Loss: 0.0502 - Val Loss: 0.0627 - Val F1: 0.6948
Epoch 50/150 - Train Loss: 0.0472 - Val Loss: 0.0617 - Val F1: 0.6926
Epoch 55/150 - Train Loss: 0.0447 - Val Loss: 0.0678 - Val F1: 0.6452
Epoch 60/150 - Train Loss: 0.0419 - Val Loss: 0.0581 - Val F1: 0.6726
```

```
Epoch 65/150 - Train Loss: 0.0396 - Val Loss: 0.0542 - Val F1: 0.6905
Epoch 70/150 - Train Loss: 0.0376 - Val Loss: 0.0522 - Val F1: 0.7261
Epoch 75/150 - Train Loss: 0.0368 - Val Loss: 0.0580 - Val F1: 0.7064
Epoch 80/150 - Train Loss: 0.0354 - Val Loss: 0.0523 - Val F1: 0.7222
Epoch 85/150 - Train Loss: 0.0322 - Val Loss: 0.0507 - Val F1: 0.6969
Epoch 90/150 - Train Loss: 0.0307 - Val Loss: 0.0525 - Val F1: 0.7112
Epoch 95/150 - Train Loss: 0.0278 - Val Loss: 0.0510 - Val F1: 0.7359
Epoch 98: LR reduced from 0.000080 to 0.000040
Epoch 100/150 - Train Loss: 0.0267 - Val Loss: 0.0495 - Val F1: 0.7680
Epoch 105/150 - Train Loss: 0.0251 - Val Loss: 0.0484 - Val F1: 0.7442
Epoch 110: LR reduced from 0.000040 to 0.000020
Epoch 110/150 - Train Loss: 0.0240 - Val Loss: 0.0500 - Val F1: 0.7519
Epoch 115/150 - Train Loss: 0.0233 - Val Loss: 0.0462 - Val F1: 0.7525
Epoch 120/150 - Train Loss: 0.0254 - Val Loss: 0.0470 - Val F1: 0.7523
Epoch 121: LR reduced from 0.000020 to 0.000010
Epoch 125/150 - Train Loss: 0.0207 - Val Loss: 0.0462 - Val F1: 0.7580
Epoch 130/150 - Train Loss: 0.0204 - Val Loss: 0.0460 - Val F1: 0.7639
Epoch 135/150 - Train Loss: 0.0202 - Val Loss: 0.0473 - Val F1: 0.7494
Epoch 140/150 - Train Loss: 0.0221 - Val Loss: 0.0448 - Val F1: 0.7442
Epoch 145/150 - Train Loss: 0.0195 - Val Loss: 0.0489 - Val F1: 0.7355
Epoch 149: LR reduced from 0.000010 to 0.000005
Epoch 150/150 - Train Loss: 0.0198 - Val Loss: 0.0453 - Val F1: 0.7677
Completed. Training time: 00:06:52 (412.86s) - Final val loss: 0.0453

--- Training Simple RNN with lr=5e-05 ---
Epoch 1/150 - Train Loss: 0.1180 - Val Loss: 0.0955 - Val F1: 0.4905
Epoch 5/150 - Train Loss: 0.0835 - Val Loss: 0.0830 - Val F1: 0.6318
Epoch 10/150 - Train Loss: 0.0763 - Val Loss: 0.0766 - Val F1: 0.6425
Epoch 15/150 - Train Loss: 0.0707 - Val Loss: 0.0748 - Val F1: 0.6474
Epoch 20/150 - Train Loss: 0.0676 - Val Loss: 0.0756 - Val F1: 0.6016
Epoch 25/150 - Train Loss: 0.0674 - Val Loss: 0.0707 - Val F1: 0.6323
Epoch 30: LR reduced from 0.000050 to 0.000025
Epoch 30/150 - Train Loss: 0.0635 - Val Loss: 0.0707 - Val F1: 0.6256
Epoch 35/150 - Train Loss: 0.0626 - Val Loss: 0.0680 - Val F1: 0.6688
Epoch 40/150 - Train Loss: 0.0609 - Val Loss: 0.0731 - Val F1: 0.6623
Epoch 44: LR reduced from 0.000025 to 0.000013
Epoch 45/150 - Train Loss: 0.0588 - Val Loss: 0.0691 - Val F1: 0.6508
Epoch 50/150 - Train Loss: 0.0595 - Val Loss: 0.0757 - Val F1: 0.6170
Epoch 53: LR reduced from 0.000013 to 0.000006
Epoch 55/150 - Train Loss: 0.0585 - Val Loss: 0.0701 - Val F1: 0.6302
Early stopping at epoch 55 (best val loss 0.067989)
Completed. Training time: 00:02:32 (152.53s) - Final val loss: 0.0701
```

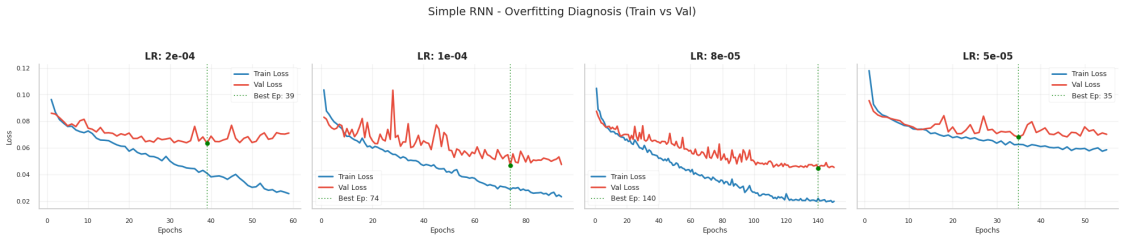
[93]:
```python
# --- Plot loss curves ---

# Plot comparison
plot_lr_comparison(histories_simple, "Simple RNN", save_dir)
```

```
#Plot Train vs Validation
plot_train_val_grid(histories_simple, "Simple RNN", save_dir)
```

```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/simple_rnn_lr_comparison.png
```

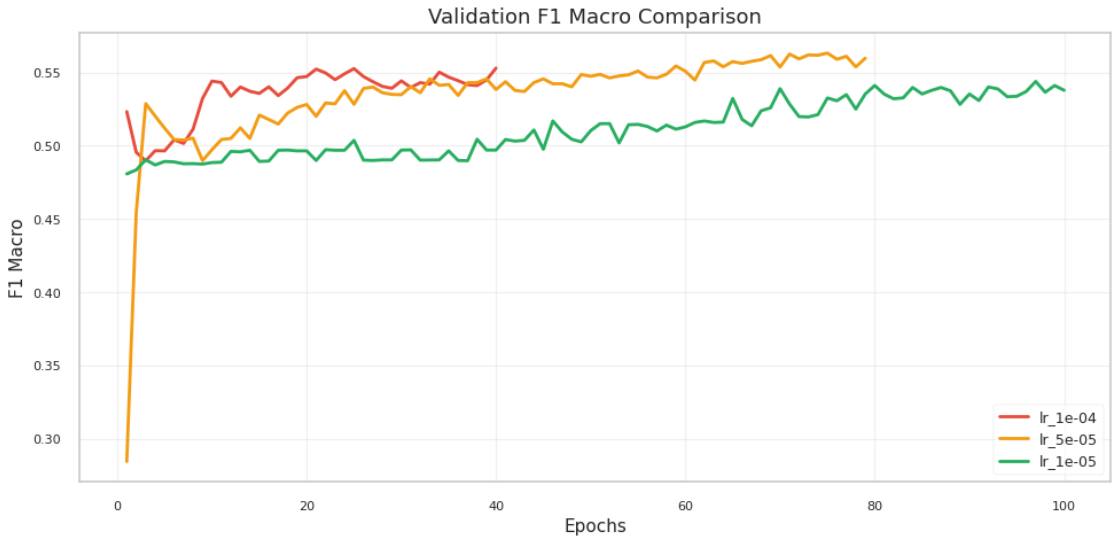



```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/simple_rnn_diagnosis.png
```



[94]:
```
# --- Plot Validation F1 Macro for Simple RNN ---
plot_training_metrics(histories_simple, save_dir=save_dir,␣
  ↪plot_name="simple_rnn_f1_comparison")
```

```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/simple_rnn_f1_comparison.png
```



[95]:
```
# --- Evaluate validation and test sets for ALL learning rates ---
print("\n" + "=" * 80)
print("Evaluation for ALL Simple RNN learning rates")
print("=" * 80)

for lr in learning_rates:
    model = models_simple[lr]

    print(f"\n{'='*60}")
    print(f"Simple RNN with lr={lr:.0e}")
    print(f"{'='*60}")

    # Validation evaluation
    y_val_true, y_val_pred = predict_loader(model, val_loader)
    print(f'\nValidation report (Simple RNN - lr={lr:.0e}):')
    print(classification_report(y_val_true, y_val_pred, digits=4,␣
 ↪zero_division=0))

    # Test evaluation
    y_test_true, y_test_pred = predict_loader(model, test_loader)
    print(f'Test report (Simple RNN - lr={lr:.0e}):')
    print(classification_report(y_test_true, y_test_pred, digits=4,␣
 ↪zero_division=0))

# Find and highlight best model
best_lr_simple = min(histories_simple.items(), key=lambda x:␣
 ↪min(x[1]['val_loss']))
print(f"\n{'='*60}")
```

```
print(f"  Best Simple RNN - LR: {best_lr_simple[0]:.0e} (Min Val Loss:␣
  ↪{min(best_lr_simple[1]['val_loss']):.4f})")
print(f"{'='*60}")
```

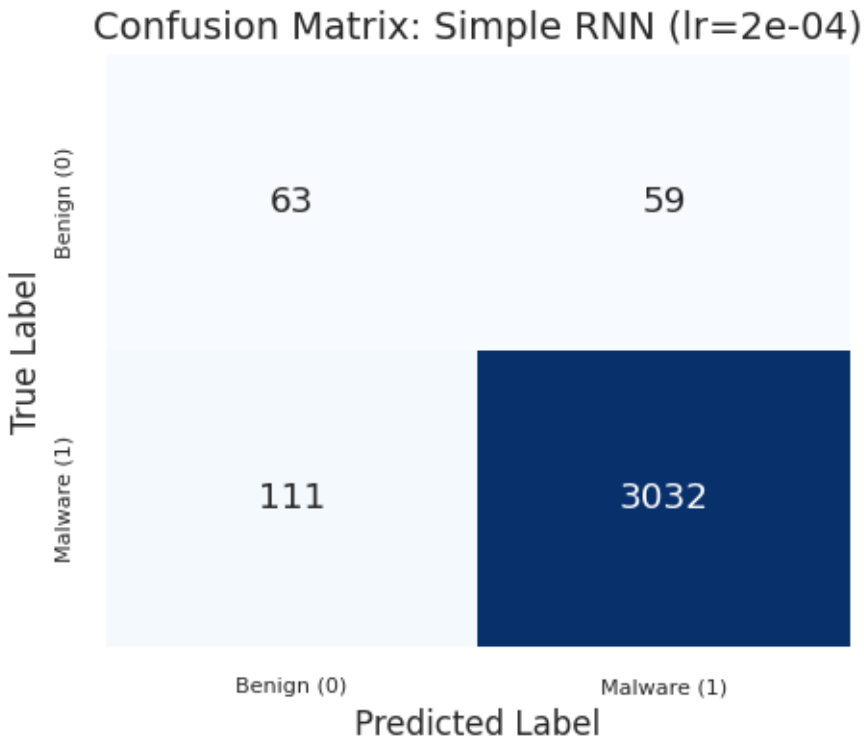
```
================================================================================
Evaluation for ALL Simple RNN learning rates
================================================================================

============================================================
Simple RNN with lr=2e-04
============================================================

Validation report (Simple RNN - lr=2e-04):
              precision    recall  f1-score   support

           0     0.3621    0.5164    0.4257       122
           1     0.9809    0.9647    0.9727      3143

    accuracy                         0.9479      3265
   macro avg     0.6715    0.7405    0.6992      3265
weighted avg     0.9578    0.9479    0.9523      3265

Test report (Simple RNN - lr=2e-04):
              precision    recall  f1-score   support

           0     0.3325    0.5720    0.4206       243
           1     0.9829    0.9554    0.9690      6262

    accuracy                         0.9411      6505
   macro avg     0.6577    0.7637    0.6948      6505
weighted avg     0.9586    0.9411    0.9485      6505


============================================================
Simple RNN with lr=1e-04
============================================================

Validation report (Simple RNN - lr=1e-04):
              precision    recall  f1-score   support

           0     0.4762    0.7377    0.5788       122
           1     0.9896    0.9685    0.9789      3143

    accuracy                         0.9599      3265
   macro avg     0.7329    0.8531    0.7789      3265
weighted avg     0.9704    0.9599    0.9640      3265
```

```
Test report (Simple RNN - lr=1e-04):
              precision    recall  f1-score   support

           0     0.4307    0.7284    0.5413       243
           1     0.9892    0.9626    0.9757      6262

    accuracy                         0.9539      6505
   macro avg     0.7099    0.8455    0.7585      6505
weighted avg     0.9683    0.9539    0.9595      6505


============================================================
Simple RNN with lr=8e-05
============================================================

Validation report (Simple RNN - lr=8e-05):
              precision    recall  f1-score   support

           0     0.3908    0.7623    0.5167       122
           1     0.9904    0.9539    0.9718      3143

    accuracy                         0.9467      3265
   macro avg     0.6906    0.8581    0.7442      3265
weighted avg     0.9680    0.9467    0.9548      3265

Test report (Simple RNN - lr=8e-05):
              precision    recall  f1-score   support

           0     0.3691    0.7366    0.4918       243
           1     0.9894    0.9511    0.9699      6262

    accuracy                         0.9431      6505
   macro avg     0.6792    0.8439    0.7308      6505
weighted avg     0.9662    0.9431    0.9520      6505


============================================================
Simple RNN with lr=5e-05
============================================================

Validation report (Simple RNN - lr=5e-05):
              precision    recall  f1-score   support

           0     0.3140    0.4426    0.3673       122
           1     0.9780    0.9625    0.9702      3143

    accuracy                         0.9430      3265
   macro avg     0.6460    0.7025    0.6688      3265
```

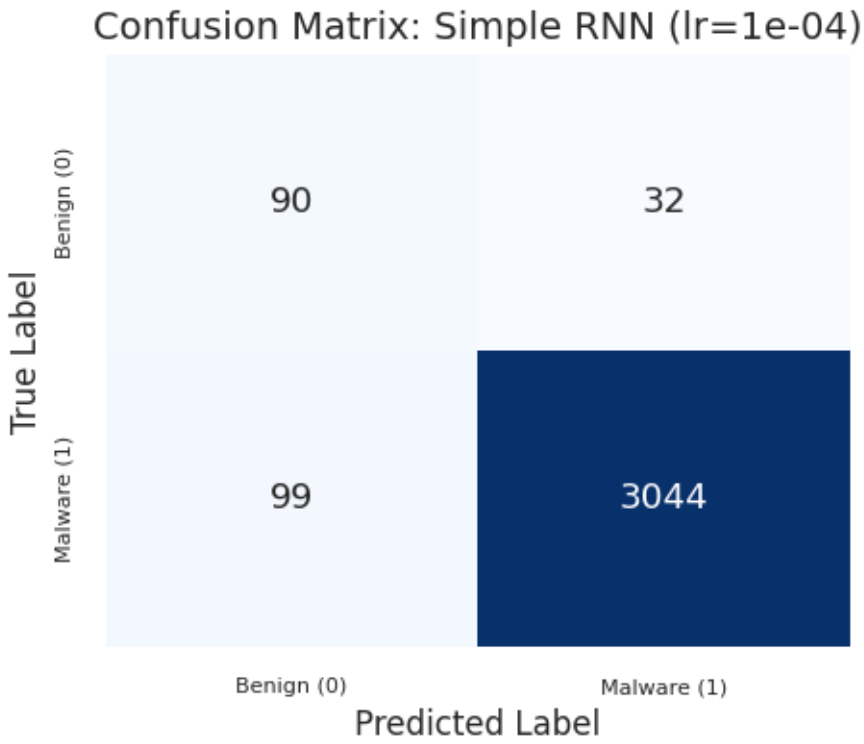
```
weighted avg     0.9532    0.9430    0.9476      3265

Test report (Simple RNN - lr=5e-05):
              precision    recall  f1-score   support

           0     0.2958    0.4650    0.3616       243
           1     0.9788    0.9570    0.9678      6262

    accuracy                         0.9387      6505
   macro avg     0.6373    0.7110    0.6647      6505
weighted avg     0.9533    0.9387    0.9451      6505


============================================================
 Best Simple RNN - LR: 8e-05 (Min Val Loss: 0.0448)
============================================================
```

[96]:
```python
# --- Plot confusion matrices for ALL learning rates - VALIDATION ---

print("\n" + "=" * 80)
print("Confusion matrices for ALL Simple RNN learning rates - VALIDATION")
print("=" * 80)

for lr in learning_rates:
    model = models_simple[lr]
    print(f"\nGenerating confusion matrix for Simple RNN with lr={lr:.0e}...")

    plot_cm_task3(model=model, loader=val_loader, model_name=f"Simple RNN␣
 ↪(lr={lr:.0e})", save_dir=save_dir)
```

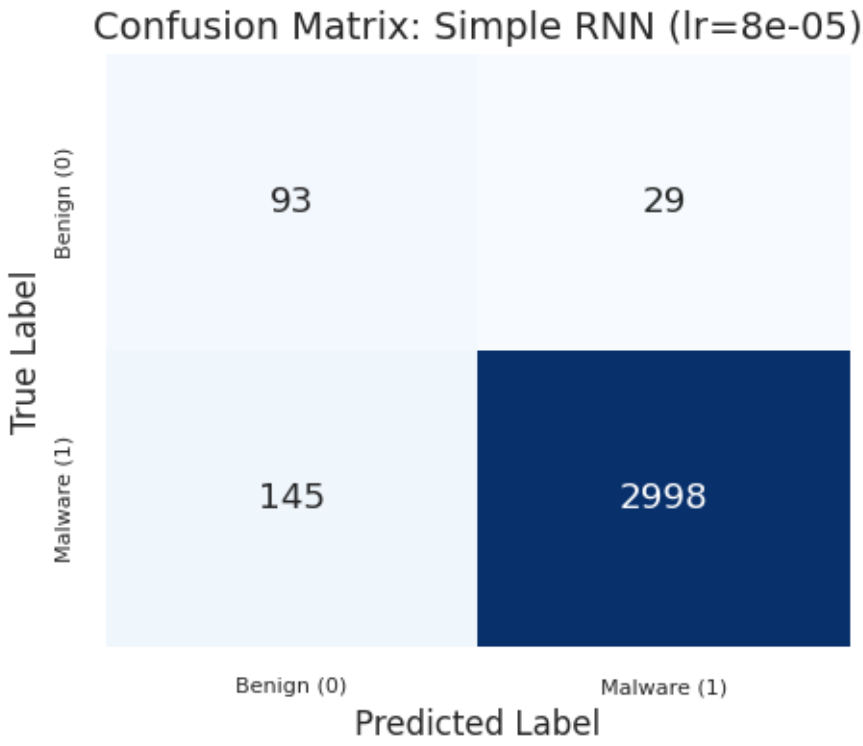
```
================================================================================
Confusion matrices for ALL Simple RNN learning rates - VALIDATION
================================================================================

Generating confusion matrix for Simple RNN with lr=2e-04…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_simple_rnn_(lr=2e_04).png
```



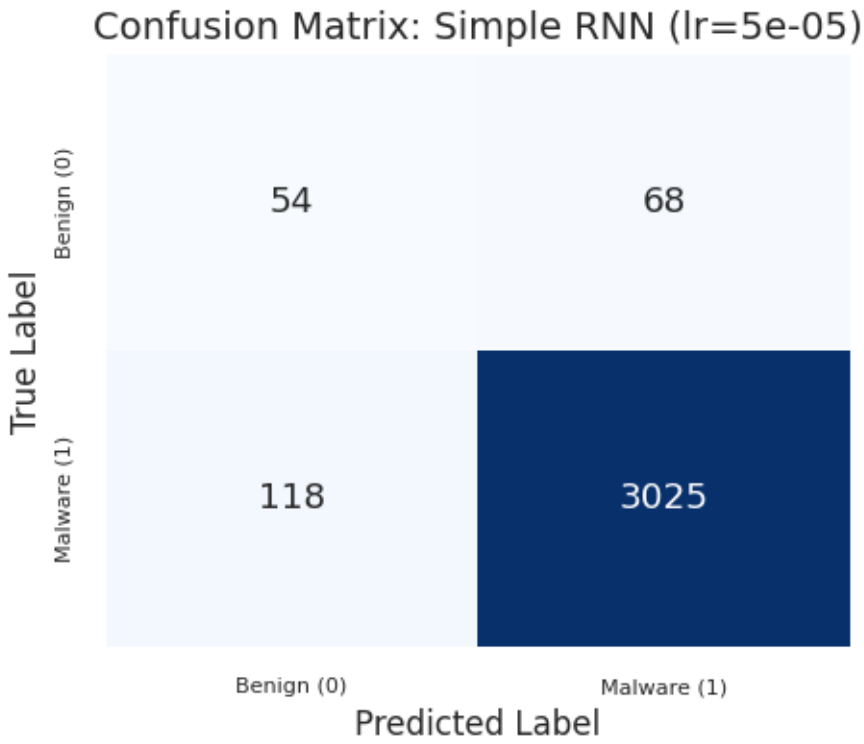

```
Generating confusion matrix for Simple RNN with lr=1e-04…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_simple_rnn_(lr=1e_04).png
```




```
Generating confusion matrix for Simple RNN with lr=8e-05…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_simple_rnn_(lr=8e_05).png
```

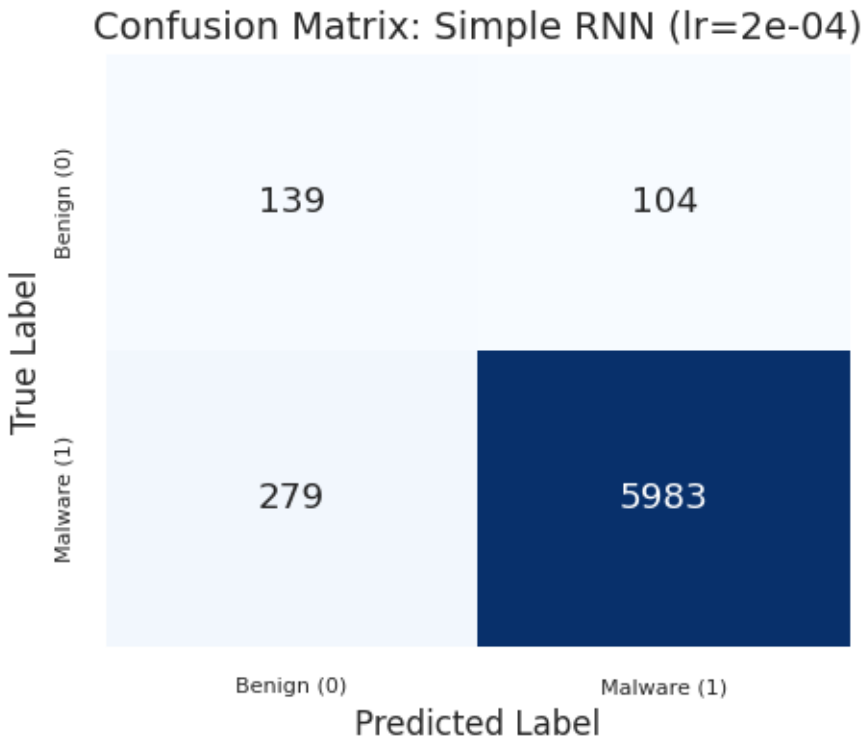



```
Generating confusion matrix for Simple RNN with lr=5e-05…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_simple_rnn_(lr=5e_05).png
```

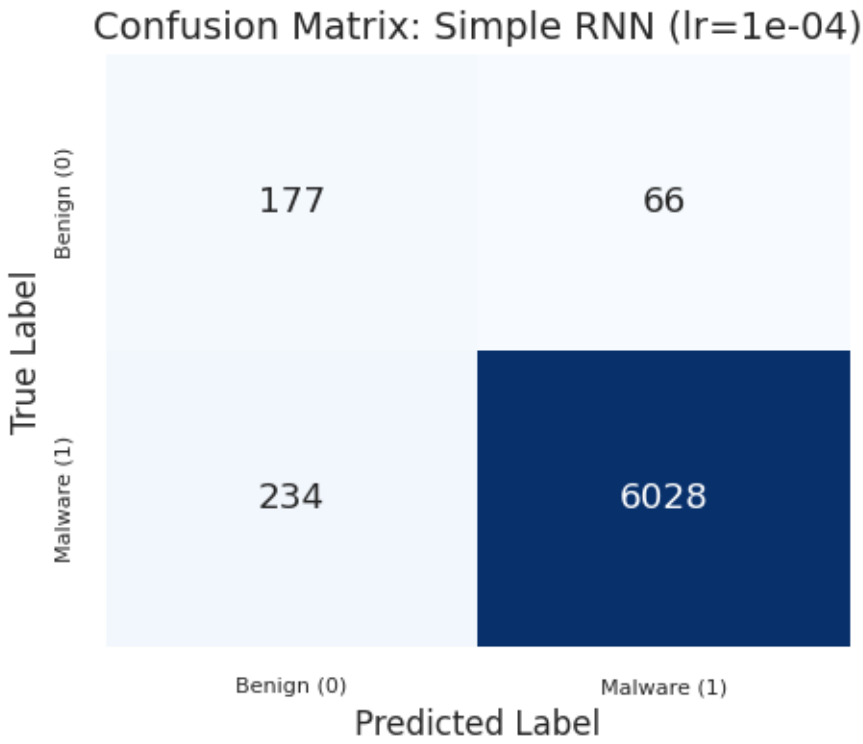


[97]:
```
# --- Plot confusion matrices for ALL learning rates - TEST ---

print("\n" + "=" * 80)
print("Confusion matrices for ALL Simple RNN learning rates - TEST")
print("=" * 80)

for lr in learning_rates:
    model = models_simple[lr]
    print(f"\nGenerating confusion matrix for Simple RNN with lr={lr:.0e}...")

    plot_cm_task3(model=model, loader=test_loader, model_name=f"Simple RNN␣
 ↪(lr={lr:.0e})", save_dir=save_dir)
```

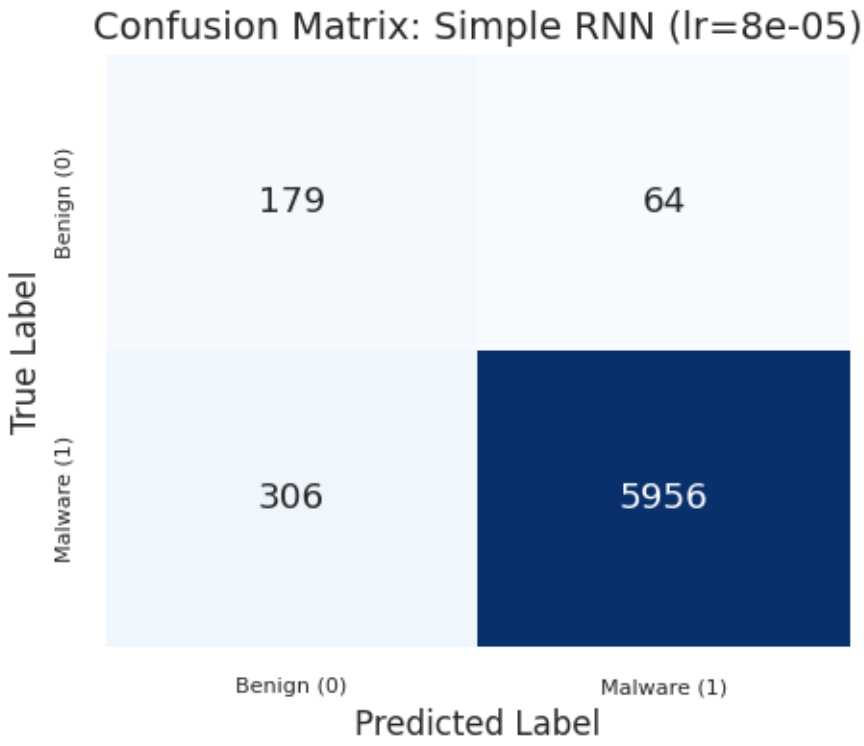
```
================================================================================
Confusion matrices for ALL Simple RNN learning rates - TEST
================================================================================

Generating confusion matrix for Simple RNN with lr=2e-04…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_simple_rnn_(lr=2e_04).png
```



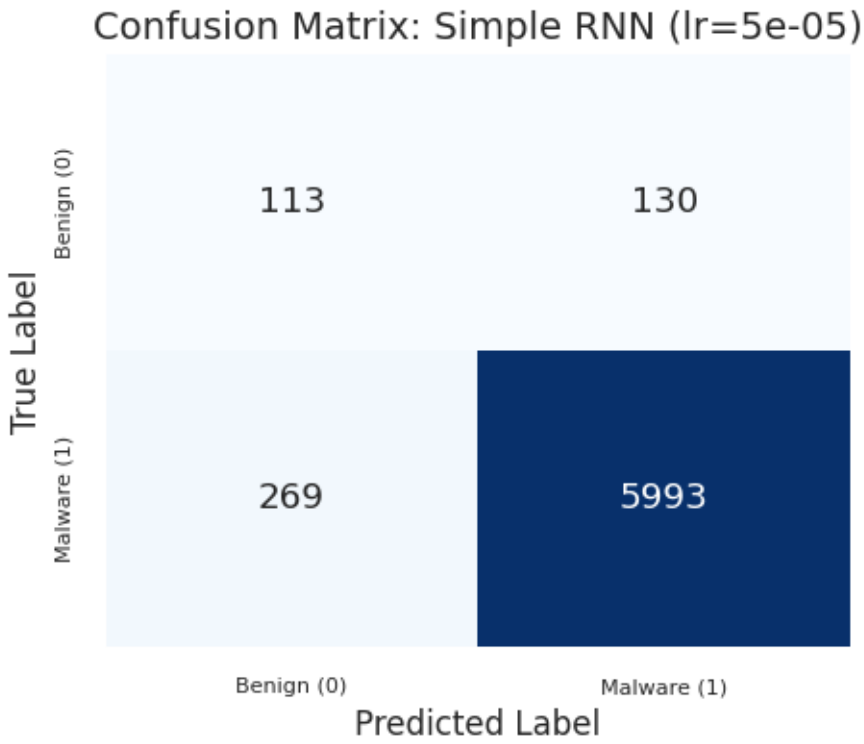

```
Generating confusion matrix for Simple RNN with lr=1e-04…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_simple_rnn_(lr=1e_04).png
```




```
Generating confusion matrix for Simple RNN with lr=8e-05…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_simple_rnn_(lr=8e_05).png
```




```
Generating confusion matrix for Simple RNN with lr=5e-05…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_simple_rnn_(lr=5e_05).png
```



Q: Start with a simple one-directional RNN. Is your network as fast as the FFNN? If not, why? No. The simple one-directional RNN is clearly slower than the FFNN models. In this run, the selected FFNN with sequential IDs trained in about `48` seconds, the selected FFNN with learnable embeddings in about `35` seconds, while the selected Simple RNN required about `6 min 52 s`.

The main reason is the sequential dependency across time steps. An FFNN processes the whole fixed-length input vector through dense matrix operations, which are highly parallelizable. An RNN instead updates its hidden state step by step, and each step depends on the previous hidden state. This limits parallelization across the temporal dimension.

Training is also more expensive because backpropagation through time must propagate gradients through the entire sequence, and the data loader performs sorting, dynamic padding, and packed-sequence handling. Even though packing avoids wasted computation on padding tokens, the recurrent structure remains substantially slower than the FFNN.

1.4.2 Network Variations

[98]:
```
# --- Variation 2: Bidirectional RNN ---

print("\n" + "=" * 80)
print("Training Bidirectional RNN with different learning rates")
print("=" * 80)
```

```
# Define learning rates to test
learning_rates = [3e-04, 1e-04, 8e-05, 5e-05]

# Model parameters
bi_rnn_params = {
    'vocab_size': vocab_size_rnn,
    'embed_dim': 32,
    'hidden_size': 128,
    'num_layers': 2,
    'num_classes': 1,
    'bidirectional': True,
    'dropout': 0.35
}

# Initialize early stopping parameters
min_delta = 0.00001
patience = 20

# Scheduler parameters
scheduler_params_bi = {
    'mode': 'min',
    'factor': 0.7,
    'patience': 7,
    'min_lr': 1e-7
}

# Store histories and models for each LR
histories_bi = {}
models_bi = {}
elapsed_bi = {}

for lr in learning_rates:
    print(f"\n--- Training Bi-RNN with lr={lr:.0e} ---")
    model, history, elapsed = train_with_lr(
        model_class=SimpleRNNClassifier,
        model_params=bi_rnn_params,
        initial_lr=lr,
        train_loader=train_loader,
        val_loader=val_loader,
        criterion=nn.BCEWithLogitsLoss(pos_weight=pos_weight),
        num_epochs=num_epochs,
        min_delta=min_delta,
        patience=patience,
        scheduler_params=scheduler_params_bi,
        device=device
    )
```

```
    histories_bi[lr] = history
    models_bi[lr] = model
    elapsed_bi[lr] = elapsed
    elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed))
    print(f"Completed. Training time: {elapsed_str} ({elapsed:.2f}s) - Final␣
↪val loss: {history['val_loss'][-1]:.4f}")
```

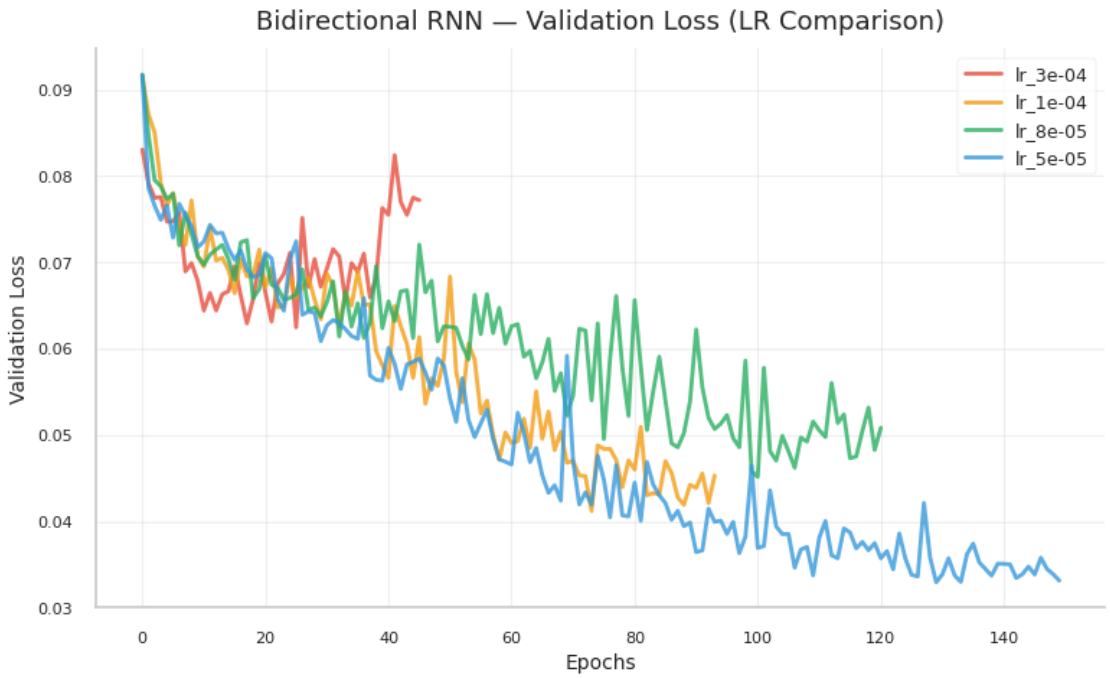
```
================================================================================
Training Bidirectional RNN with different learning rates
================================================================================

--- Training Bi-RNN with lr=3e-04 ---
Epoch 1/150 - Train Loss: 0.0925 - Val Loss: 0.0831 - Val F1: 0.6002
Epoch 5/150 - Train Loss: 0.0716 - Val Loss: 0.0747 - Val F1: 0.6373
Epoch 10/150 - Train Loss: 0.0615 - Val Loss: 0.0679 - Val F1: 0.6731
Epoch 15/150 - Train Loss: 0.0524 - Val Loss: 0.0667 - Val F1: 0.6708
Epoch 20/150 - Train Loss: 0.0439 - Val Loss: 0.0699 - Val F1: 0.6981
Epoch 25/150 - Train Loss: 0.0381 - Val Loss: 0.0711 - Val F1: 0.7044
Epoch 30/150 - Train Loss: 0.0306 - Val Loss: 0.0672 - Val F1: 0.7146
Epoch 34: LR reduced from 0.000300 to 0.000210
Epoch 35/150 - Train Loss: 0.0242 - Val Loss: 0.0699 - Val F1: 0.7190
Epoch 40/150 - Train Loss: 0.0215 - Val Loss: 0.0763 - Val F1: 0.7217
Epoch 42: LR reduced from 0.000210 to 0.000147
Epoch 45/150 - Train Loss: 0.0161 - Val Loss: 0.0775 - Val F1: 0.7407
Early stopping at epoch 46 (best val loss 0.062478)
Completed. Training time: 00:03:07 (187.07s) - Final val loss: 0.0772

--- Training Bi-RNN with lr=1e-04 ---
Epoch 1/150 - Train Loss: 0.1025 - Val Loss: 0.0916 - Val F1: 0.4984
Epoch 5/150 - Train Loss: 0.0753 - Val Loss: 0.0766 - Val F1: 0.6421
Epoch 10/150 - Train Loss: 0.0684 - Val Loss: 0.0707 - Val F1: 0.6556
Epoch 15/150 - Train Loss: 0.0626 - Val Loss: 0.0691 - Val F1: 0.6764
Epoch 20/150 - Train Loss: 0.0580 - Val Loss: 0.0715 - Val F1: 0.6969
Epoch 25/150 - Train Loss: 0.0565 - Val Loss: 0.0684 - Val F1: 0.6785
Epoch 30/150 - Train Loss: 0.0502 - Val Loss: 0.0634 - Val F1: 0.7097
Epoch 35/150 - Train Loss: 0.0457 - Val Loss: 0.0650 - Val F1: 0.7076
Epoch 40/150 - Train Loss: 0.0437 - Val Loss: 0.0581 - Val F1: 0.6890
Epoch 45/150 - Train Loss: 0.0398 - Val Loss: 0.0566 - Val F1: 0.7407
Epoch 50/150 - Train Loss: 0.0367 - Val Loss: 0.0589 - Val F1: 0.7377
Epoch 55: LR reduced from 0.000100 to 0.000070
Epoch 55/150 - Train Loss: 0.0327 - Val Loss: 0.0587 - Val F1: 0.7677
Epoch 60/150 - Train Loss: 0.0305 - Val Loss: 0.0503 - Val F1: 0.7566
Epoch 65/150 - Train Loss: 0.0278 - Val Loss: 0.0551 - Val F1: 0.7568
Epoch 67: LR reduced from 0.000070 to 0.000049
Epoch 70/150 - Train Loss: 0.0235 - Val Loss: 0.0468 - Val F1: 0.7687
Epoch 75/150 - Train Loss: 0.0237 - Val Loss: 0.0488 - Val F1: 0.7836
Epoch 80/150 - Train Loss: 0.0222 - Val Loss: 0.0470 - Val F1: 0.7765
```

```
Epoch 82: LR reduced from 0.000049 to 0.000034
Epoch 85/150 - Train Loss: 0.0217 - Val Loss: 0.0431 - Val F1: 0.7713
Epoch 90: LR reduced from 0.000034 to 0.000024
Epoch 90/150 - Train Loss: 0.0197 - Val Loss: 0.0442 - Val F1: 0.7936
Early stopping at epoch 94 (best val loss 0.041190)
Completed. Training time: 00:06:16 (376.47s) - Final val loss: 0.0453

--- Training Bi-RNN with lr=8e-05 ---
Epoch 1/150 - Train Loss: 0.1032 - Val Loss: 0.0918 - Val F1: 0.4905
Epoch 5/150 - Train Loss: 0.0733 - Val Loss: 0.0773 - Val F1: 0.6203
Epoch 10/150 - Train Loss: 0.0683 - Val Loss: 0.0707 - Val F1: 0.6782
Epoch 15/150 - Train Loss: 0.0615 - Val Loss: 0.0704 - Val F1: 0.6590
Epoch 20/150 - Train Loss: 0.0594 - Val Loss: 0.0669 - Val F1: 0.6776
Epoch 25/150 - Train Loss: 0.0573 - Val Loss: 0.0659 - Val F1: 0.6838
Epoch 30/150 - Train Loss: 0.0528 - Val Loss: 0.0637 - Val F1: 0.6832
Epoch 35/150 - Train Loss: 0.0500 - Val Loss: 0.0626 - Val F1: 0.6848
Epoch 40/150 - Train Loss: 0.0486 - Val Loss: 0.0624 - Val F1: 0.7120
Epoch 45/150 - Train Loss: 0.0445 - Val Loss: 0.0612 - Val F1: 0.6952
Epoch 50/150 - Train Loss: 0.0412 - Val Loss: 0.0626 - Val F1: 0.7305
Epoch 55/150 - Train Loss: 0.0389 - Val Loss: 0.0662 - Val F1: 0.7186
Epoch 60/150 - Train Loss: 0.0363 - Val Loss: 0.0606 - Val F1: 0.7146
Epoch 62: LR reduced from 0.000080 to 0.000056
Epoch 65/150 - Train Loss: 0.0329 - Val Loss: 0.0566 - Val F1: 0.7309
Epoch 70/150 - Train Loss: 0.0327 - Val Loss: 0.0522 - Val F1: 0.7322
Epoch 75/150 - Train Loss: 0.0285 - Val Loss: 0.0629 - Val F1: 0.7663
Epoch 80/150 - Train Loss: 0.0265 - Val Loss: 0.0522 - Val F1: 0.7591
Epoch 84: LR reduced from 0.000056 to 0.000039
Epoch 85/150 - Train Loss: 0.0245 - Val Loss: 0.0591 - Val F1: 0.7689
Epoch 90/150 - Train Loss: 0.0267 - Val Loss: 0.0539 - Val F1: 0.7649
Epoch 95/150 - Train Loss: 0.0230 - Val Loss: 0.0513 - Val F1: 0.7911
Epoch 96: LR reduced from 0.000039 to 0.000027
Epoch 100/150 - Train Loss: 0.0209 - Val Loss: 0.0458 - Val F1: 0.7908
Epoch 105/150 - Train Loss: 0.0190 - Val Loss: 0.0499 - Val F1: 0.8089
Epoch 109: LR reduced from 0.000027 to 0.000019
Epoch 110/150 - Train Loss: 0.0184 - Val Loss: 0.0516 - Val F1: 0.8128
Epoch 115/150 - Train Loss: 0.0179 - Val Loss: 0.0524 - Val F1: 0.8184
Epoch 117: LR reduced from 0.000019 to 0.000013
Epoch 120/150 - Train Loss: 0.0178 - Val Loss: 0.0483 - Val F1: 0.8225
Early stopping at epoch 121 (best val loss 0.045145)
Completed. Training time: 00:08:08 (488.91s) - Final val loss: 0.0508

--- Training Bi-RNN with lr=5e-05 ---
Epoch 1/150 - Train Loss: 0.1119 - Val Loss: 0.0917 - Val F1: 0.4905
Epoch 5/150 - Train Loss: 0.0766 - Val Loss: 0.0766 - Val F1: 0.5973
Epoch 10/150 - Train Loss: 0.0702 - Val Loss: 0.0718 - Val F1: 0.6501
Epoch 15/150 - Train Loss: 0.0658 - Val Loss: 0.0716 - Val F1: 0.6147
Epoch 20/150 - Train Loss: 0.0628 - Val Loss: 0.0685 - Val F1: 0.6434
Epoch 25/150 - Train Loss: 0.0584 - Val Loss: 0.0703 - Val F1: 0.6173
```

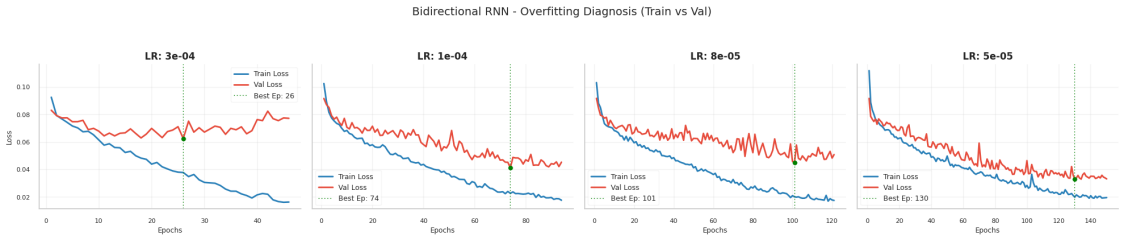
```
Epoch 30/150 - Train Loss: 0.0558 - Val Loss: 0.0609 - Val F1: 0.7102
Epoch 35/150 - Train Loss: 0.0532 - Val Loss: 0.0615 - Val F1: 0.6564
Epoch 40/150 - Train Loss: 0.0489 - Val Loss: 0.0563 - Val F1: 0.6971
Epoch 45/150 - Train Loss: 0.0478 - Val Loss: 0.0585 - Val F1: 0.6727
Epoch 50/150 - Train Loss: 0.0498 - Val Loss: 0.0581 - Val F1: 0.7166
Epoch 55/150 - Train Loss: 0.0434 - Val Loss: 0.0498 - Val F1: 0.7120
Epoch 60/150 - Train Loss: 0.0411 - Val Loss: 0.0469 - Val F1: 0.7817
Epoch 65/150 - Train Loss: 0.0398 - Val Loss: 0.0485 - Val F1: 0.7164
Epoch 70/150 - Train Loss: 0.0375 - Val Loss: 0.0592 - Val F1: 0.6665
Epoch 75/150 - Train Loss: 0.0358 - Val Loss: 0.0476 - Val F1: 0.7052
Epoch 80/150 - Train Loss: 0.0336 - Val Loss: 0.0406 - Val F1: 0.7576
Epoch 85/150 - Train Loss: 0.0324 - Val Loss: 0.0430 - Val F1: 0.7316
Epoch 90/150 - Train Loss: 0.0320 - Val Loss: 0.0399 - Val F1: 0.7850
Epoch 95/150 - Train Loss: 0.0297 - Val Loss: 0.0401 - Val F1: 0.7749
Epoch 100/150 - Train Loss: 0.0284 - Val Loss: 0.0464 - Val F1: 0.8409
Epoch 105/150 - Train Loss: 0.0274 - Val Loss: 0.0385 - Val F1: 0.8107
Epoch 106: LR reduced from 0.000050 to 0.000035
Epoch 110/150 - Train Loss: 0.0244 - Val Loss: 0.0337 - Val F1: 0.8123
Epoch 115/150 - Train Loss: 0.0232 - Val Loss: 0.0392 - Val F1: 0.8328
Epoch 118: LR reduced from 0.000035 to 0.000024
Epoch 120/150 - Train Loss: 0.0219 - Val Loss: 0.0375 - Val F1: 0.8134
Epoch 125/150 - Train Loss: 0.0235 - Val Loss: 0.0357 - Val F1: 0.8249
Epoch 126: LR reduced from 0.000024 to 0.000017
Epoch 130/150 - Train Loss: 0.0198 - Val Loss: 0.0329 - Val F1: 0.8402
Epoch 135/150 - Train Loss: 0.0215 - Val Loss: 0.0362 - Val F1: 0.8332
Epoch 138: LR reduced from 0.000017 to 0.000012
Epoch 140/150 - Train Loss: 0.0209 - Val Loss: 0.0351 - Val F1: 0.8344
Epoch 145/150 - Train Loss: 0.0203 - Val Loss: 0.0347 - Val F1: 0.8197
Epoch 146: LR reduced from 0.000012 to 0.000008
Epoch 150/150 - Train Loss: 0.0196 - Val Loss: 0.0331 - Val F1: 0.8280
Early stopping at epoch 150 (best val loss 0.032936)
Completed. Training time: 00:10:06 (606.34s) - Final val loss: 0.0331
```

[99]:
```python
# --- Plot loss curves ---

# Plot comparison
plot_lr_comparison(histories_bi, "Bidirectional RNN", save_dir)

#Plot Train vs Validation
plot_train_val_grid(histories_bi, "Bidirectional RNN", save_dir)
```

```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/bidirectional_rnn_lr_comparison.png
```

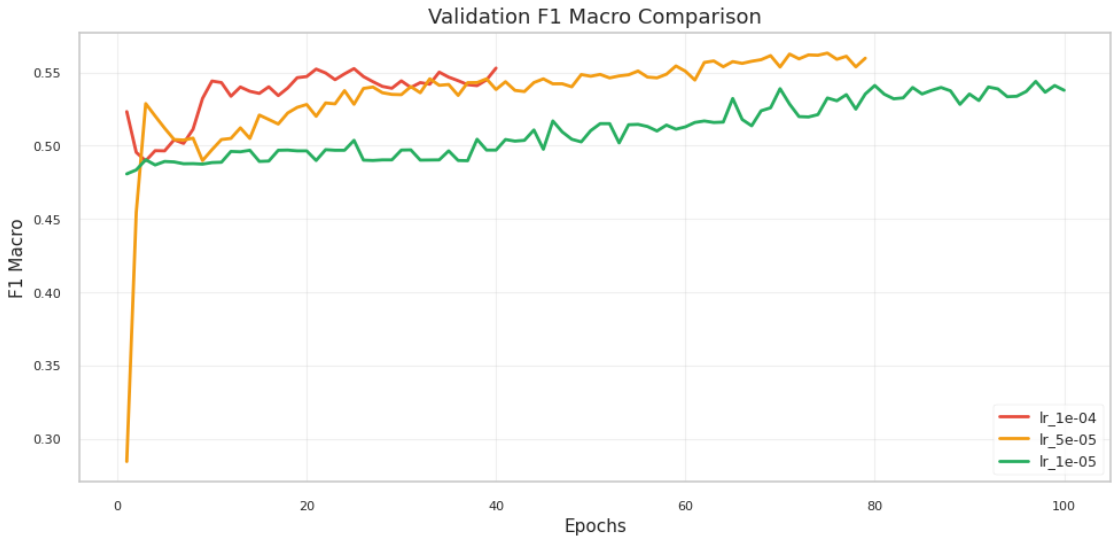


Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/bidirectional_rnn_diagnosis.png



[100]:
```
# --- Plot Validation F1 Macro for Bidirectional RNN ---
plot_training_metrics(histories_bi, save_dir=save_dir,␣
 ↪plot_name="bidirectional_rnn_f1_comparison")
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/bidirectional_rnn_f1_comparison.png



[101]:
```
# --- Evaluate validation and test sets for ALL learning rates ---

print("\n" + "=" * 80)
print("Evaluation for ALL Bi-RNN learning rates")
print("=" * 80)

for lr in learning_rates:
    model = models_bi[lr]

    print(f"\n{'='*60}")
    print(f"Bi-RNN with lr={lr:.0e}")
    print(f"{'='*60}")

    # Validation evaluation
    y_val_true, y_val_pred = predict_loader(model, val_loader)
    print(f'\nValidation report (Bi-RNN - lr={lr:.0e}):')
    print(classification_report(y_val_true, y_val_pred, digits=4,␣
 ↪zero_division=0))

    # Test evaluation
    y_test_true, y_test_pred = predict_loader(model, test_loader)
    print(f'Test report (Bi-RNN - lr={lr:.0e}):')
    print(classification_report(y_test_true, y_test_pred, digits=4,␣
 ↪zero_division=0))

# Find and highlight best model
best_lr_bi = min(histories_bi.items(), key=lambda x: min(x[1]['val_loss']))
print(f"\n{'='*60}")
```

```
print(f"  Best Bi-RNN - LR: {best_lr_bi[0]:.0e} (Min Val Loss:␣
  ↪{min(best_lr_bi[1]['val_loss']):.4f})")
print(f"{'='*60}")
```

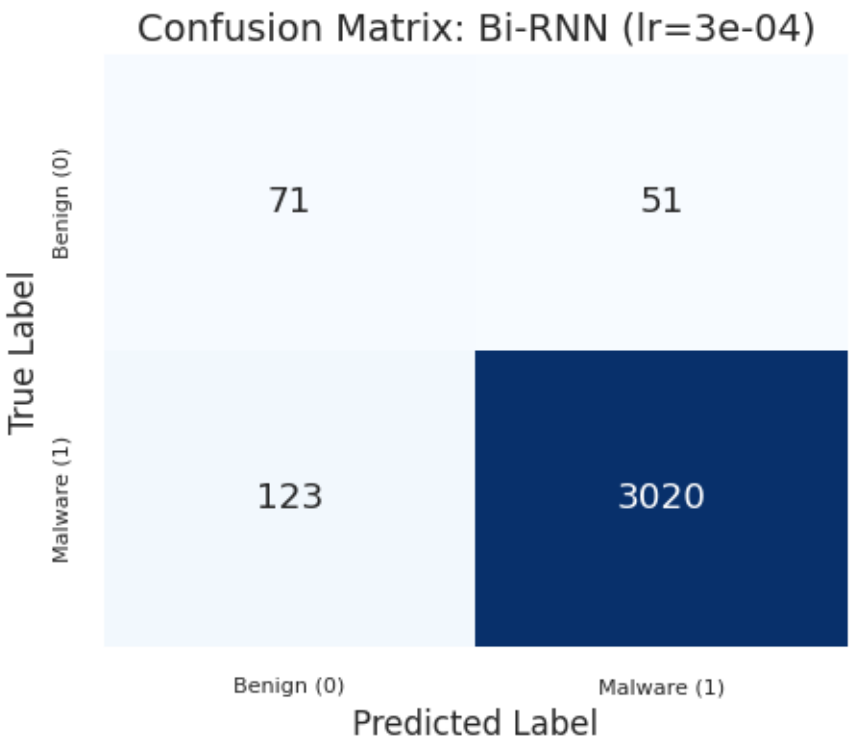
```
================================================================================
Evaluation for ALL Bi-RNN learning rates
================================================================================

============================================================
Bi-RNN with lr=3e-04
============================================================

Validation report (Bi-RNN - lr=3e-04):
              precision    recall  f1-score   support

           0     0.3660    0.5820    0.4494       122
           1     0.9834    0.9609    0.9720      3143

    accuracy                         0.9467      3265
   macro avg     0.6747    0.7714    0.7107      3265
weighted avg     0.9603    0.9467    0.9525      3265

Test report (Bi-RNN - lr=3e-04):
              precision    recall  f1-score   support

           0     0.3620    0.5720    0.4434       243
           1     0.9830    0.9609    0.9718      6262

    accuracy                         0.9463      6505
   macro avg     0.6725    0.7664    0.7076      6505
weighted avg     0.9598    0.9463    0.9521      6505


============================================================
Bi-RNN with lr=1e-04
============================================================

Validation report (Bi-RNN - lr=1e-04):
              precision    recall  f1-score   support

           0     0.4839    0.7377    0.5844       122
           1     0.9896    0.9695    0.9794      3143

    accuracy                         0.9608      3265
   macro avg     0.7367    0.8536    0.7819      3265
weighted avg     0.9707    0.9608    0.9647      3265
```

```
Test report (Bi-RNN - lr=1e-04):
              precision    recall  f1-score   support

           0     0.4715    0.6461    0.5451       243
           1     0.9861    0.9719    0.9789      6262

    accuracy                         0.9597      6505
   macro avg     0.7288    0.8090    0.7620      6505
weighted avg     0.9668    0.9597    0.9627      6505


============================================================
Bi-RNN with lr=8e-05
============================================================

Validation report (Bi-RNN - lr=8e-05):
              precision    recall  f1-score   support

           0     0.5366    0.7213    0.6154       122
           1     0.9890    0.9758    0.9824      3143

    accuracy                         0.9663      3265
   macro avg     0.7628    0.8486    0.7989      3265
weighted avg     0.9721    0.9663    0.9687      3265

Test report (Bi-RNN - lr=8e-05):
              precision    recall  f1-score   support

           0     0.4787    0.6461    0.5499       243
           1     0.9861    0.9727    0.9793      6262

    accuracy                         0.9605      6505
   macro avg     0.7324    0.8094    0.7646      6505
weighted avg     0.9671    0.9605    0.9633      6505


============================================================
Bi-RNN with lr=5e-05
============================================================

Validation report (Bi-RNN - lr=5e-05):
              precision    recall  f1-score   support

           0     0.6309    0.7705    0.6937       122
           1     0.9910    0.9825    0.9867      3143

    accuracy                         0.9746      3265
   macro avg     0.8109    0.8765    0.8402      3265
```

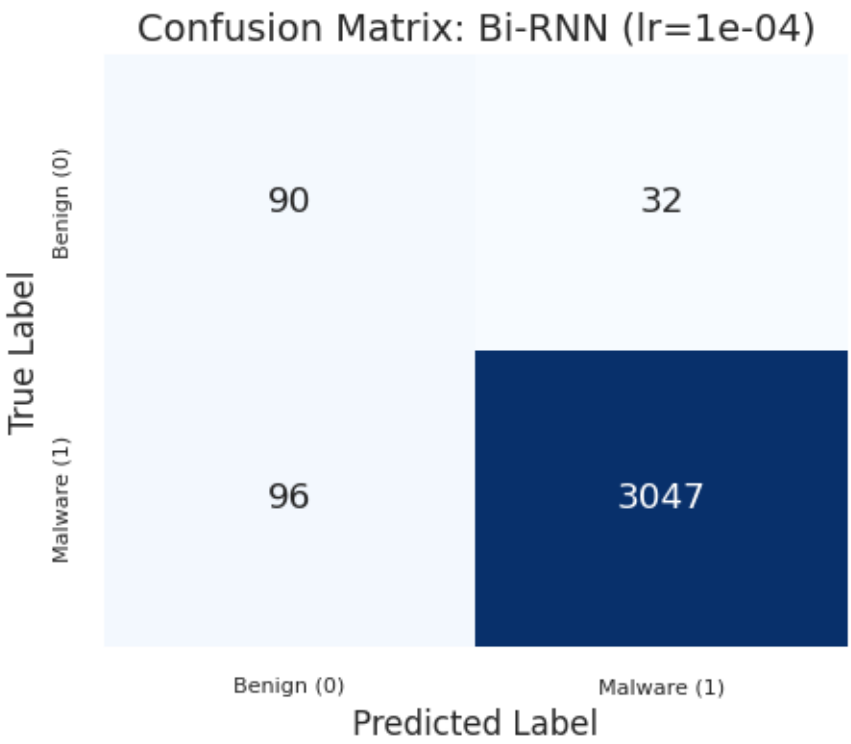
```
weighted avg     0.9776    0.9746    0.9758      3265

Test report (Bi-RNN - lr=5e-05):
              precision    recall  f1-score   support

           0     0.5223    0.6749    0.5889       243
           1     0.9872    0.9760    0.9816      6262

    accuracy                         0.9648      6505
   macro avg     0.7548    0.8255    0.7852      6505
weighted avg     0.9699    0.9648    0.9669      6505


============================================================
 Best Bi-RNN - LR: 5e-05 (Min Val Loss: 0.0329)
============================================================
```

[102]:
```python
# --- Plot confusion matrices for ALL learning rates - VALIDATION ---

print("\n" + "=" * 80)
print("Confusion matrices for ALL Bi-RNN learning rates - VALIDATION")
print("=" * 80)

for lr in learning_rates:
    model = models_bi[lr]
    print(f"\nGenerating confusion matrix for Bi-RNN with lr={lr:.0e}...")

    plot_cm_task3(model=model, loader=val_loader, model_name=f"Bi-RNN (lr={lr:.
 ↪0e})", save_dir=save_dir)
```

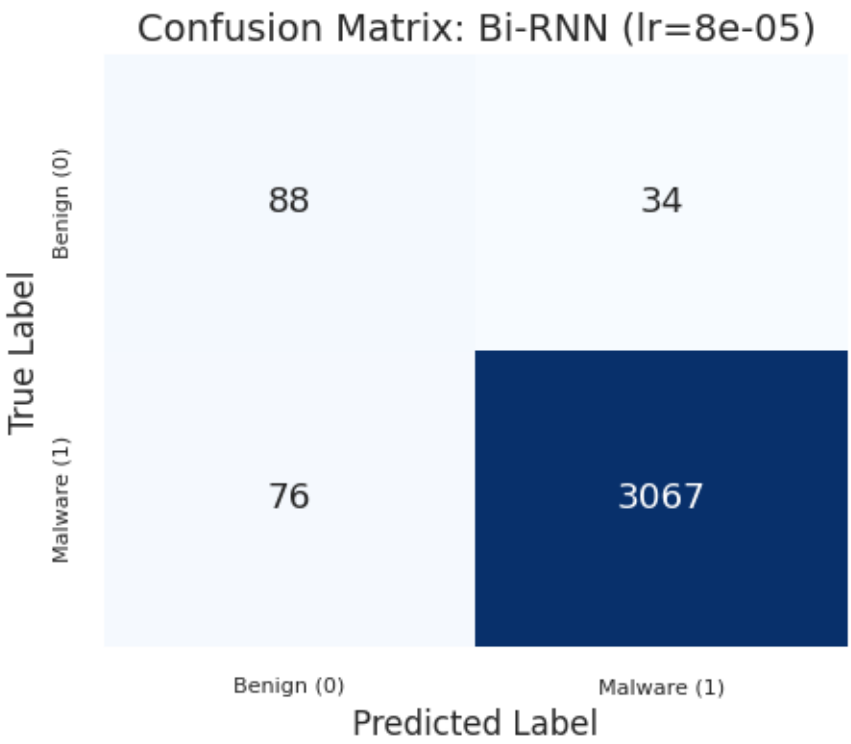
```
================================================================================
Confusion matrices for ALL Bi-RNN learning rates - VALIDATION
================================================================================

Generating confusion matrix for Bi-RNN with lr=3e-04…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_bi_rnn_(lr=3e_04).png
```



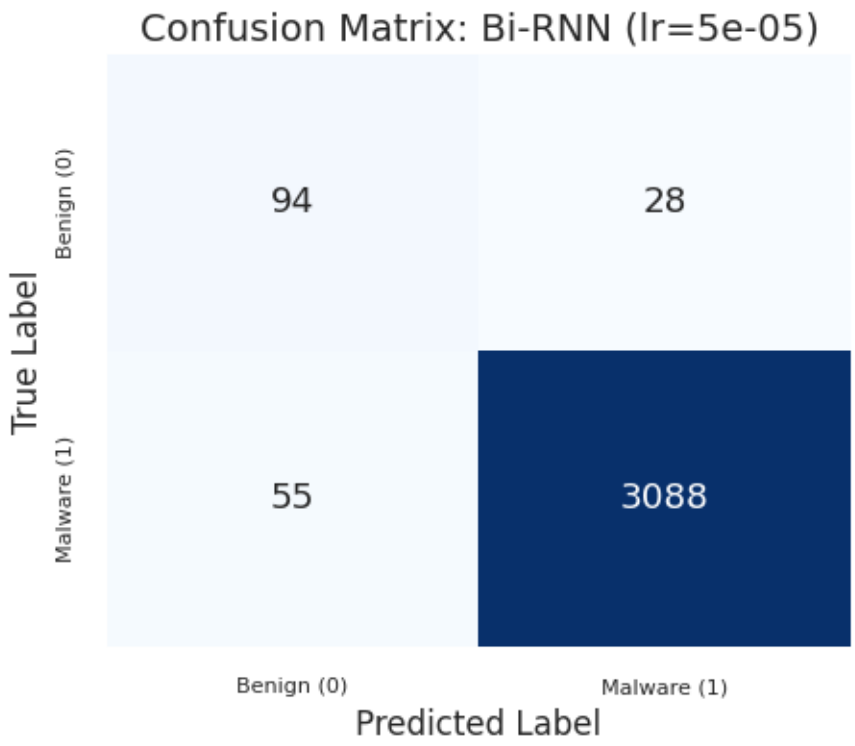

```
Generating confusion matrix for Bi-RNN with lr=1e-04…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_bi_rnn_(lr=1e_04).png
```




```
Generating confusion matrix for Bi-RNN with lr=8e-05…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_bi_rnn_(lr=8e_05).png
```

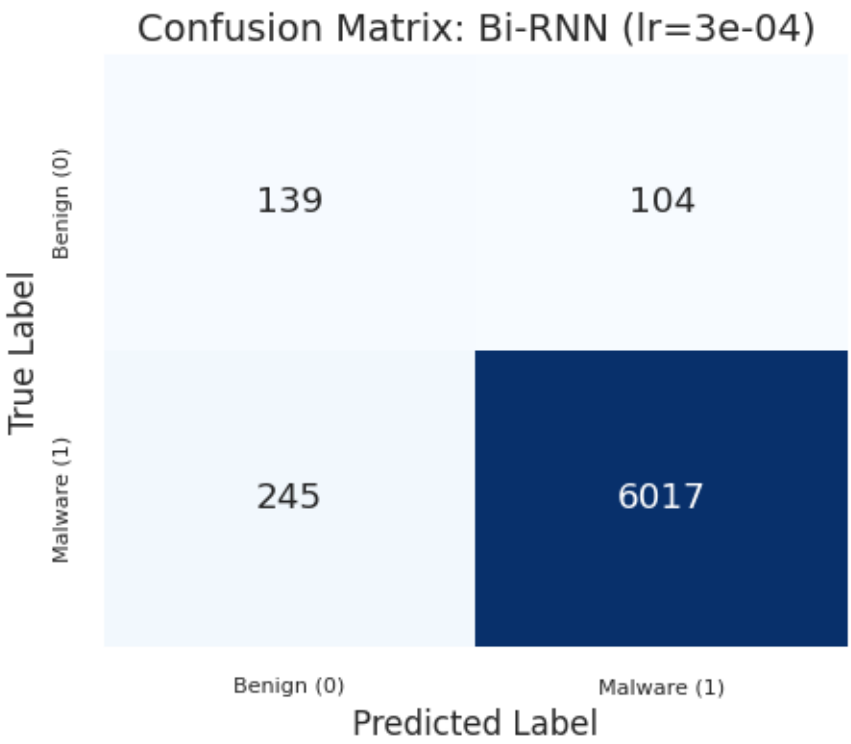



```
Generating confusion matrix for Bi-RNN with lr=5e-05…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_bi_rnn_(lr=5e_05).png
```

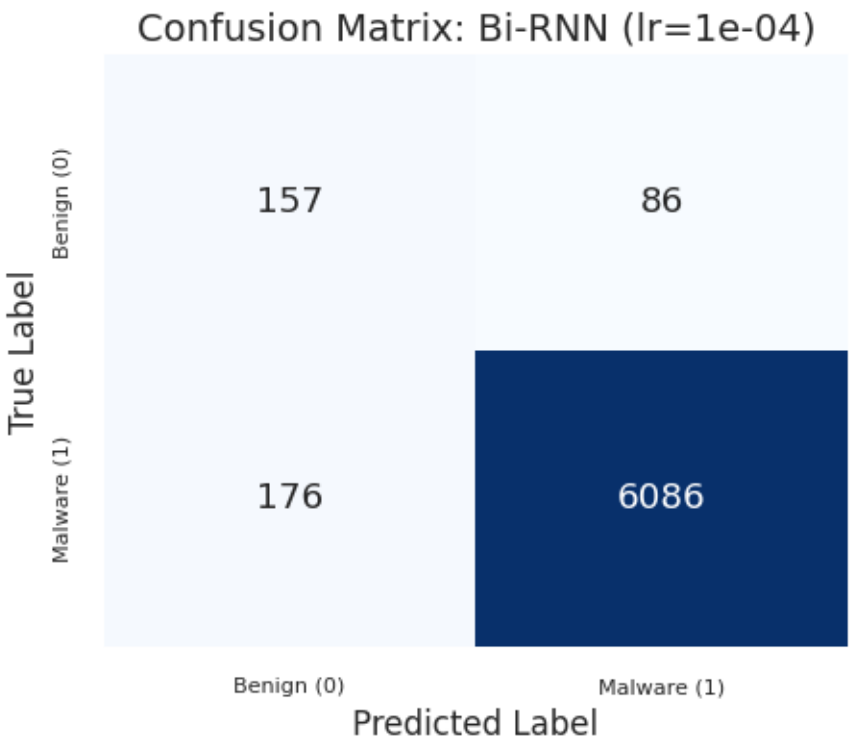


[103]:
```
# --- Plot confusion matrices for ALL learning rates - TEST ---

print("\n" + "=" * 80)
print("Confusion matrices for ALL Bi-RNN learning rates - TEST")
print("=" * 80)

for lr in learning_rates:
    model = models_bi[lr]
    print(f"\nGenerating confusion matrix for Bi-RNN with lr={lr:.0e}...")

    plot_cm_task3(model=model, loader=test_loader, model_name=f"Bi-RNN (lr={lr:.
 ↪0e})", save_dir=save_dir)
```

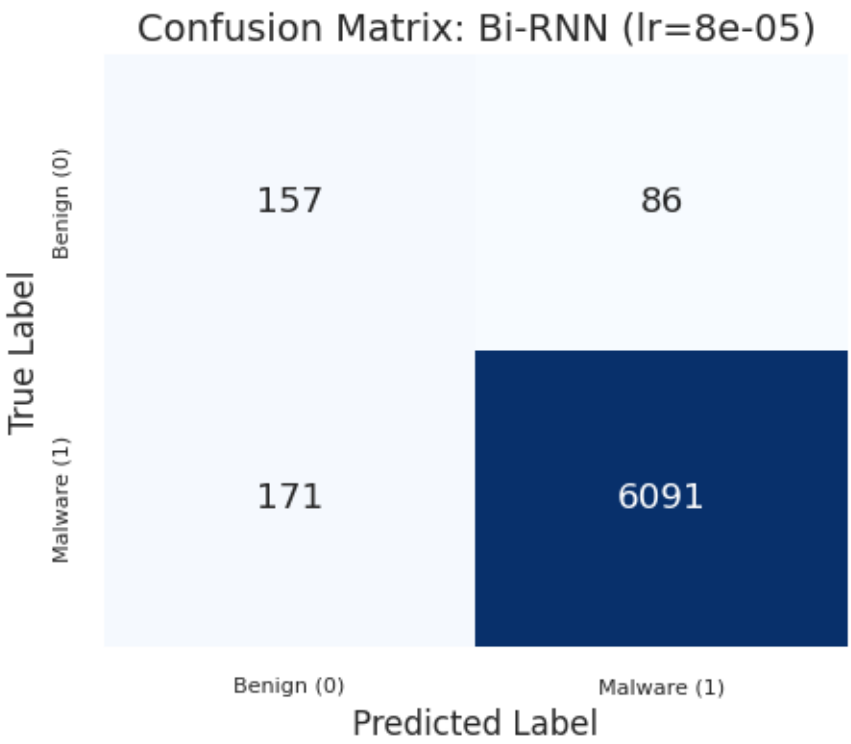
```
================================================================================
Confusion matrices for ALL Bi-RNN learning rates - TEST
================================================================================

Generating confusion matrix for Bi-RNN with lr=3e-04…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_bi_rnn_(lr=3e_04).png
```



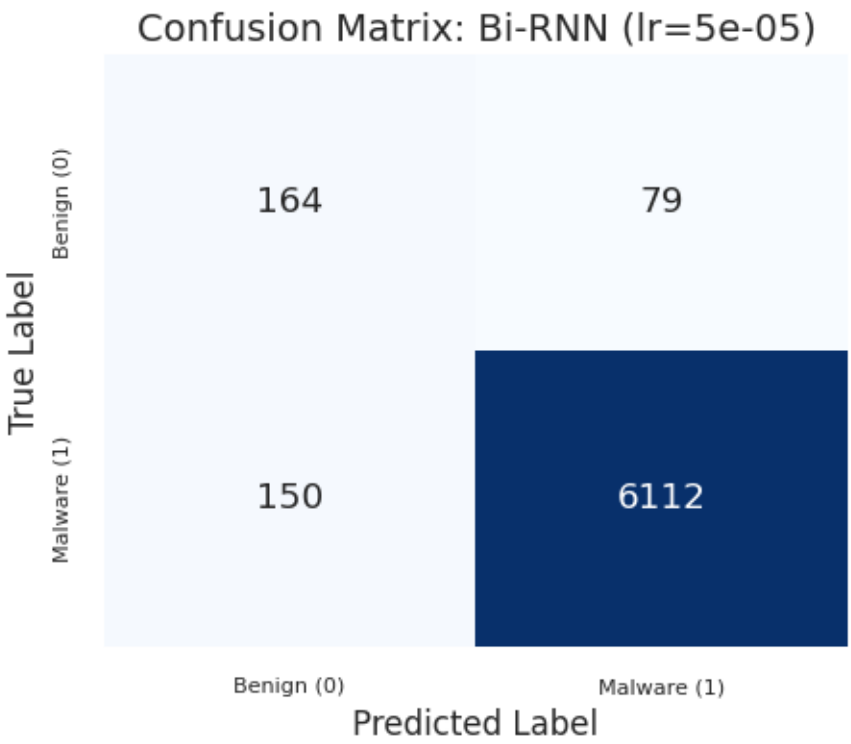

```
Generating confusion matrix for Bi-RNN with lr=1e-04…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_bi_rnn_(lr=1e_04).png
```




```
Generating confusion matrix for Bi-RNN with lr=8e-05…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_bi_rnn_(lr=8e_05).png
```




```
Generating confusion matrix for Bi-RNN with lr=5e-05…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_bi_rnn_(lr=5e_05).png
```



[104]:
```
# --- Variation 3: LSTM (bidirectional) ---

class LSTMClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim=32, hidden_size=128, num_layers=2,␣
 ↪num_classes=1, bidirectional=True, dropout=0.3, padding=0):
        super(LSTMClassifier, self).__init__()

        # Embedding layer: Maps API call indices to dense vectors. padding_idx␣
 ↪is set.
        self.embedding = nn.Embedding(vocab_size, embedding_dim=embed_dim,␣
 ↪padding_idx=padding)

        # LSTM layer: A more complex RNN unit that helps prevent vanishing␣
 ↪gradients.
        # batch_first=True expects input shape (batch_size, seq_len, features).
        self.lstm = nn.LSTM(input_size=embed_dim, hidden_size=hidden_size,␣
 ↪num_layers=num_layers,
                            batch_first=True, bidirectional=bidirectional)

        self.bidirectional = bidirectional
        fc_in = hidden_size * (2 if bidirectional else 1)
```

```
        self.fc = nn.Linear(fc_in, num_classes)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, lengths):
        # Embed the input indices
        emb = self.embedding(x) # (batch_size, max_seq_len, embed_dim)
        emb = self.dropout(emb)

        # Pack the padded sequence to efficiently process variable lengths
        packed = pack_padded_sequence(emb, lengths.cpu(), batch_first=True,␣
↪enforce_sorted=True)

        # LSTM returns packed_output, (final_hidden_state, final_cell_state)
        packed_out, (hn, cn) = self.lstm(packed)

        # hn shape: (num_layers * num_directions, batch_size, hidden_size)

        # Get the final hidden state from the last layer
        if self.bidirectional:
            # Concatenate the final forward state (hn[-2]) and backward state␣
↪(hn[-1])
            h_last = torch.cat([hn[-2], hn[-1]], dim=1)
        else:
            # Use the final hidden state of the last layer (hn[-1])
            h_last = hn[-1]

        # Pass the final hidden state through the classifier
        out = self.fc(h_last)
        return out
```

[105]:
```
# --- Variation 3: LSTM (bidirectional) ---

print("\n" + "=" * 80)
print("Training LSTM with different learning rates")
print("=" * 80)

# Model parameters
lstm_params = {
    'vocab_size': vocab_size_rnn,
    'embed_dim': 32,
    'hidden_size': 128,
    'num_layers': 2,
    'num_classes': 1,
    'bidirectional': True,
    'dropout': 0.4
}
```

```
# Initialize early stopping parameters
min_delta = 0.00001
patience = 30

# Scheduler parameters
scheduler_params_lstm = {
    'mode': 'min',
    'factor': 0.5,
    'patience': 6,
    'min_lr': 1e-7
}

# Test slightly different LRs for LSTM (typically handles higher LRs better)
learning_rates_lstm = [5e-04, 3e-04, 1e-04, 8e-05]

# Store histories and models for each LR
histories_lstm = {}
models_lstm = {}
elapsed_lstm = {}

for lr in learning_rates_lstm:
    print(f"\n--- Training LSTM with lr={lr:.0e} ---")
    model, history, elapsed = train_with_lr(
        model_class=LSTMClassifier,
        model_params=lstm_params,
        initial_lr=lr,
        train_loader=train_loader,
        val_loader=val_loader,
        criterion=nn.BCEWithLogitsLoss(pos_weight=pos_weight),
        num_epochs=num_epochs,
        min_delta=min_delta,
        patience=patience,
        scheduler_params=scheduler_params_lstm,
        device=device
    )
    histories_lstm[lr] = history
    models_lstm[lr] = model
    elapsed_lstm[lr] = elapsed
    elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed))
    print(f"Completed. Training time: {elapsed_str} ({elapsed:.2f}s) - Final␣
 ↪val loss: {history['val_loss'][-1]:.4f}")
```

```
================================================================================
Training LSTM with different learning rates
================================================================================

--- Training LSTM with lr=5e-04 ---
```

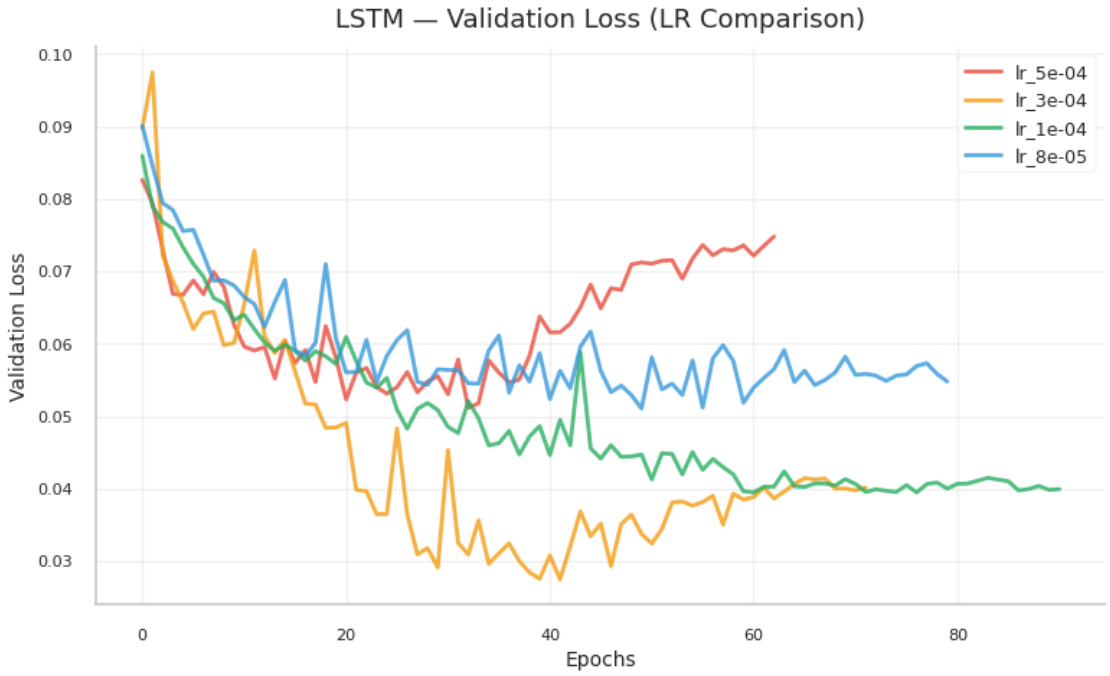
```
Epoch 1/150 - Train Loss: 0.0953 - Val Loss: 0.0826 - Val F1: 0.5636
Epoch 5/150 - Train Loss: 0.0677 - Val Loss: 0.0668 - Val F1: 0.6727
Epoch 10/150 - Train Loss: 0.0573 - Val Loss: 0.0626 - Val F1: 0.6654
Epoch 15/150 - Train Loss: 0.0468 - Val Loss: 0.0604 - Val F1: 0.6858
Epoch 20/150 - Train Loss: 0.0413 - Val Loss: 0.0581 - Val F1: 0.6815
Epoch 25/150 - Train Loss: 0.0289 - Val Loss: 0.0531 - Val F1: 0.7338
Epoch 28: LR reduced from 0.000500 to 0.000250
Epoch 30/150 - Train Loss: 0.0240 - Val Loss: 0.0555 - Val F1: 0.7248
Epoch 35/150 - Train Loss: 0.0184 - Val Loss: 0.0577 - Val F1: 0.7557
Epoch 40: LR reduced from 0.000250 to 0.000125
Epoch 40/150 - Train Loss: 0.0148 - Val Loss: 0.0638 - Val F1: 0.7660
Epoch 45/150 - Train Loss: 0.0125 - Val Loss: 0.0682 - Val F1: 0.7711
Epoch 47: LR reduced from 0.000125 to 0.000063
Epoch 50/150 - Train Loss: 0.0105 - Val Loss: 0.0712 - Val F1: 0.7689
Epoch 54: LR reduced from 0.000063 to 0.000031
Epoch 55/150 - Train Loss: 0.0102 - Val Loss: 0.0718 - Val F1: 0.7754
Epoch 60/150 - Train Loss: 0.0092 - Val Loss: 0.0736 - Val F1: 0.7700
Epoch 61: LR reduced from 0.000031 to 0.000016
Early stopping at epoch 63 (best val loss 0.051150)
Completed. Training time: 00:04:47 (287.98s) - Final val loss: 0.0748

--- Training LSTM with lr=3e-04 ---
Epoch 1/150 - Train Loss: 0.0971 - Val Loss: 0.0898 - Val F1: 0.5348
Epoch 5/150 - Train Loss: 0.0661 - Val Loss: 0.0657 - Val F1: 0.6851
Epoch 10/150 - Train Loss: 0.0554 - Val Loss: 0.0601 - Val F1: 0.6974
Epoch 15/150 - Train Loss: 0.0466 - Val Loss: 0.0606 - Val F1: 0.7306
Epoch 20/150 - Train Loss: 0.0435 - Val Loss: 0.0484 - Val F1: 0.7477
Epoch 25/150 - Train Loss: 0.0343 - Val Loss: 0.0365 - Val F1: 0.8022
Epoch 30/150 - Train Loss: 0.0294 - Val Loss: 0.0291 - Val F1: 0.8298
Epoch 35/150 - Train Loss: 0.0230 - Val Loss: 0.0296 - Val F1: 0.8325
Epoch 37: LR reduced from 0.000300 to 0.000150
Epoch 40/150 - Train Loss: 0.0180 - Val Loss: 0.0276 - Val F1: 0.8234
Epoch 45/150 - Train Loss: 0.0158 - Val Loss: 0.0334 - Val F1: 0.8499
Epoch 49: LR reduced from 0.000150 to 0.000075
Epoch 50/150 - Train Loss: 0.0114 - Val Loss: 0.0337 - Val F1: 0.8569
Epoch 55/150 - Train Loss: 0.0118 - Val Loss: 0.0376 - Val F1: 0.8402
Epoch 56: LR reduced from 0.000075 to 0.000037
Epoch 60/150 - Train Loss: 0.0112 - Val Loss: 0.0385 - Val F1: 0.8438
Epoch 63: LR reduced from 0.000037 to 0.000019
Epoch 65/150 - Train Loss: 0.0104 - Val Loss: 0.0406 - Val F1: 0.8409
Epoch 70: LR reduced from 0.000019 to 0.000009
Epoch 70/150 - Train Loss: 0.0099 - Val Loss: 0.0400 - Val F1: 0.8470
Early stopping at epoch 72 (best val loss 0.027468)
Completed. Training time: 00:05:28 (328.61s) - Final val loss: 0.0401

--- Training LSTM with lr=1e-04 ---
Epoch 1/150 - Train Loss: 0.1066 - Val Loss: 0.0860 - Val F1: 0.4905
Epoch 5/150 - Train Loss: 0.0736 - Val Loss: 0.0733 - Val F1: 0.6015
```

```
Epoch 10/150 - Train Loss: 0.0659 - Val Loss: 0.0633 - Val F1: 0.6652
Epoch 15/150 - Train Loss: 0.0601 - Val Loss: 0.0599 - Val F1: 0.6741
Epoch 20/150 - Train Loss: 0.0526 - Val Loss: 0.0572 - Val F1: 0.6853
Epoch 25/150 - Train Loss: 0.0479 - Val Loss: 0.0553 - Val F1: 0.6887
Epoch 30/150 - Train Loss: 0.0450 - Val Loss: 0.0508 - Val F1: 0.7038
Epoch 35/150 - Train Loss: 0.0419 - Val Loss: 0.0460 - Val F1: 0.7253
Epoch 40/150 - Train Loss: 0.0383 - Val Loss: 0.0486 - Val F1: 0.7448
Epoch 45/150 - Train Loss: 0.0365 - Val Loss: 0.0456 - Val F1: 0.7391
Epoch 50/150 - Train Loss: 0.0333 - Val Loss: 0.0447 - Val F1: 0.7470
Epoch 55/150 - Train Loss: 0.0317 - Val Loss: 0.0450 - Val F1: 0.7336
Epoch 58: LR reduced from 0.000100 to 0.000050
Epoch 60/150 - Train Loss: 0.0279 - Val Loss: 0.0396 - Val F1: 0.7915
Epoch 65/150 - Train Loss: 0.0261 - Val Loss: 0.0403 - Val F1: 0.7795
Epoch 68: LR reduced from 0.000050 to 0.000025
Epoch 70/150 - Train Loss: 0.0252 - Val Loss: 0.0413 - Val F1: 0.7541
Epoch 75: LR reduced from 0.000025 to 0.000013
Epoch 75/150 - Train Loss: 0.0233 - Val Loss: 0.0395 - Val F1: 0.7861
Epoch 80/150 - Train Loss: 0.0223 - Val Loss: 0.0400 - Val F1: 0.7853
Epoch 84: LR reduced from 0.000013 to 0.000006
Epoch 85/150 - Train Loss: 0.0207 - Val Loss: 0.0412 - Val F1: 0.7882
Epoch 90/150 - Train Loss: 0.0207 - Val Loss: 0.0398 - Val F1: 0.7836
Epoch 91: LR reduced from 0.000006 to 0.000003
Early stopping at epoch 91 (best val loss 0.039486)
Completed. Training time: 00:06:44 (404.84s) - Final val loss: 0.0399

--- Training LSTM with lr=8e-05 ---
Epoch 1/150 - Train Loss: 0.1134 - Val Loss: 0.0901 - Val F1: 0.4905
Epoch 5/150 - Train Loss: 0.0756 - Val Loss: 0.0756 - Val F1: 0.6085
Epoch 10/150 - Train Loss: 0.0667 - Val Loss: 0.0680 - Val F1: 0.6756
Epoch 15/150 - Train Loss: 0.0590 - Val Loss: 0.0688 - Val F1: 0.6724
Epoch 20/150 - Train Loss: 0.0599 - Val Loss: 0.0608 - Val F1: 0.6796
Epoch 25/150 - Train Loss: 0.0519 - Val Loss: 0.0583 - Val F1: 0.7121
Epoch 30/150 - Train Loss: 0.0507 - Val Loss: 0.0565 - Val F1: 0.6845
Epoch 35/150 - Train Loss: 0.0467 - Val Loss: 0.0591 - Val F1: 0.6574
Epoch 36: LR reduced from 0.000080 to 0.000040
Epoch 40/150 - Train Loss: 0.0426 - Val Loss: 0.0587 - Val F1: 0.7226
Epoch 45/150 - Train Loss: 0.0416 - Val Loss: 0.0617 - Val F1: 0.7317
Epoch 48: LR reduced from 0.000040 to 0.000020
Epoch 50/150 - Train Loss: 0.0390 - Val Loss: 0.0511 - Val F1: 0.7318
Epoch 55/150 - Train Loss: 0.0383 - Val Loss: 0.0577 - Val F1: 0.7491
Epoch 57: LR reduced from 0.000020 to 0.000010
Epoch 60/150 - Train Loss: 0.0385 - Val Loss: 0.0519 - Val F1: 0.7422
Epoch 64: LR reduced from 0.000010 to 0.000005
Epoch 65/150 - Train Loss: 0.0355 - Val Loss: 0.0547 - Val F1: 0.7561
Epoch 70/150 - Train Loss: 0.0362 - Val Loss: 0.0582 - Val F1: 0.7519
Epoch 71: LR reduced from 0.000005 to 0.000003
Epoch 75/150 - Train Loss: 0.0356 - Val Loss: 0.0556 - Val F1: 0.7490
Epoch 78: LR reduced from 0.000003 to 0.000001
```

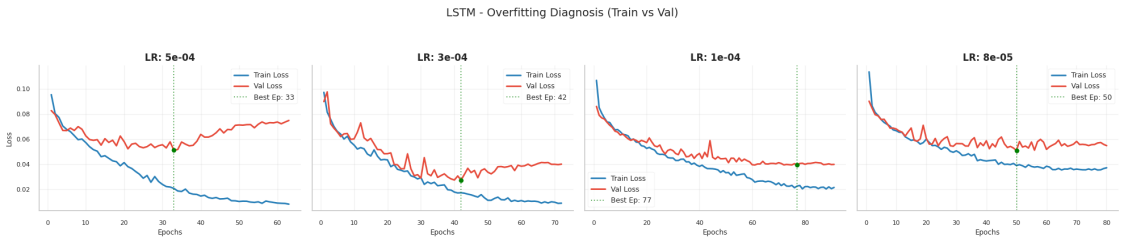
```
Epoch 80/150 - Train Loss: 0.0373 - Val Loss: 0.0548 - Val F1: 0.7479
Early stopping at epoch 80 (best val loss 0.051051)
Completed. Training time: 00:05:56 (356.60s) - Final val loss: 0.0548
```

[106]:
```python
# --- Plot loss curves ---

# Plot comparison
plot_lr_comparison(histories_lstm, "LSTM", save_dir)

#Plot Train vs Validation
plot_train_val_grid(histories_lstm, "LSTM", save_dir)
```

```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/lstm_lr_comparison.png
```

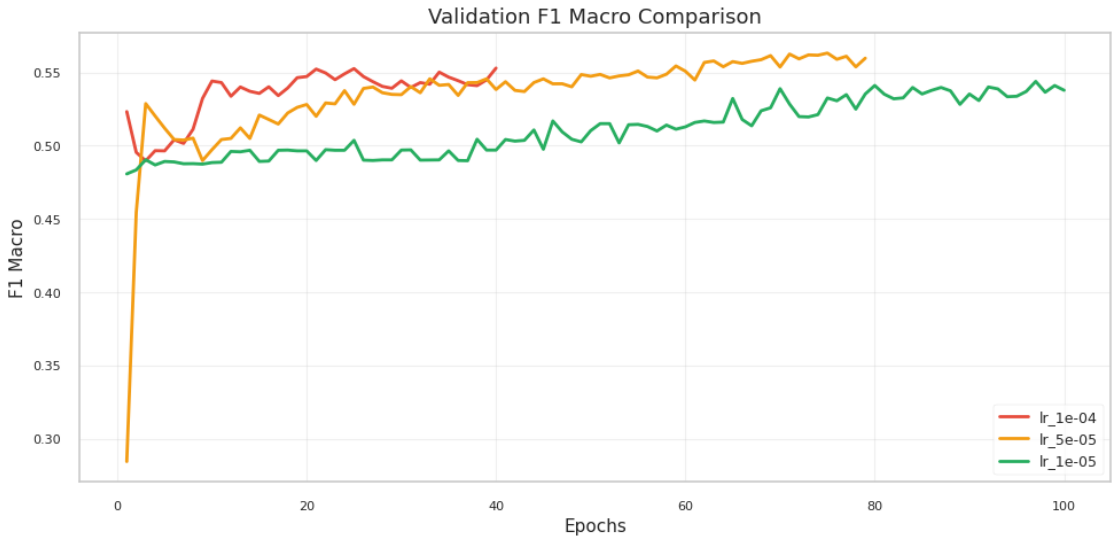



```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/lstm_diagnosis.png
```



[107]:
```
# --- Plot Validation F1 Macro for LSTM ---
plot_training_metrics(histories_lstm, save_dir=save_dir,␣
↪plot_name="lstm_f1_comparison")
```

```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/lstm_f1_comparison.png
```



[108]:
```
# --- Evaluate validation and test sets for ALL learning rates ---

print("\n" + "=" * 80)
print("Evaluation for ALL LSTM RNN learning rates")
print("=" * 80)

for lr in learning_rates_lstm: # Changed 'learning_rates' to␣
↪'learning_rates_lstm'
    model = models_lstm[lr]

    print(f"\n{'='*60}")
    print(f"LSTM RNN with lr={lr:.0e}")
    print(f"{'='*60}")

    # Validation evaluation
    y_val_true, y_val_pred = predict_loader(model, val_loader)
    print(f'\nValidation report (LSTM RNN - lr={lr:.0e}):')
    print(classification_report(y_val_true, y_val_pred, digits=4,␣
↪zero_division=0))
```

```
    # Test evaluation
    y_test_true, y_test_pred = predict_loader(model, test_loader)
    print(f'Test report (LSTM RNN - lr={lr:.0e}):')
    print(classification_report(y_test_true, y_test_pred, digits=4,␣
 ↪zero_division=0))

# Find and highlight best model
best_lr_lstm = min(histories_lstm.items(), key=lambda x: min(x[1]['val_loss']))
print(f"\n{'='*60}")
print(f"  Best LSTM RNN - LR: {best_lr_lstm[0]:.0e} (Min Val Loss:␣
 ↪{min(best_lr_lstm[1]['val_loss']):.4f})")
print(f"{'='*60}")
```

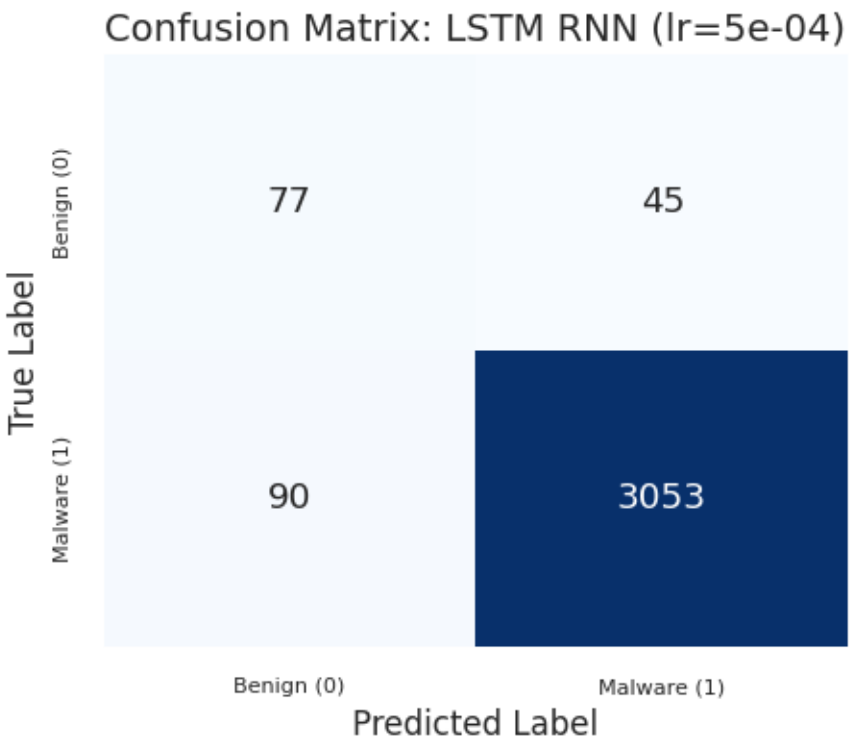
```
================================================================================
Evaluation for ALL LSTM RNN learning rates
================================================================================

============================================================
LSTM RNN with lr=5e-04
============================================================

Validation report (LSTM RNN - lr=5e-04):
              precision    recall  f1-score   support

           0     0.4611    0.6311    0.5329       122
           1     0.9855    0.9714    0.9784      3143

    accuracy                         0.9587      3265
   macro avg     0.7233    0.8013    0.7556      3265
weighted avg     0.9659    0.9587    0.9617      3265

Test report (LSTM RNN - lr=5e-04):
              precision    recall  f1-score   support

           0     0.4794    0.6214    0.5412       243
           1     0.9851    0.9738    0.9794      6262

    accuracy                         0.9606      6505
   macro avg     0.7323    0.7976    0.7603      6505
weighted avg     0.9662    0.9606    0.9631      6505


============================================================
LSTM RNN with lr=3e-04
============================================================
```

```
Validation report (LSTM RNN - lr=3e-04):
              precision    recall  f1-score   support

           0     0.5839    0.7705    0.6643       122
           1     0.9910    0.9787    0.9848      3143

    accuracy                         0.9709      3265
   macro avg     0.7874    0.8746    0.8246      3265
weighted avg     0.9758    0.9709    0.9728      3265

Test report (LSTM RNN - lr=3e-04):
              precision    recall  f1-score   support

           0     0.5967    0.7366    0.6593       243
           1     0.9897    0.9807    0.9852      6262

    accuracy                         0.9716      6505
   macro avg     0.7932    0.8587    0.8222      6505
weighted avg     0.9750    0.9716    0.9730      6505


============================================================
LSTM RNN with lr=1e-04
============================================================

Validation report (LSTM RNN - lr=1e-04):
              precision    recall  f1-score   support

           0     0.4208    0.8279    0.5580       122
           1     0.9931    0.9558    0.9741      3143

    accuracy                         0.9510      3265
   macro avg     0.7069    0.8918    0.7660      3265
weighted avg     0.9717    0.9510    0.9585      3265

Test report (LSTM RNN - lr=1e-04):
              precision    recall  f1-score   support

           0     0.4352    0.7325    0.5460       243
           1     0.9893    0.9631    0.9760      6262

    accuracy                         0.9545      6505
   macro avg     0.7123    0.8478    0.7610      6505
weighted avg     0.9686    0.9545    0.9600      6505


============================================================
LSTM RNN with lr=8e-05
```

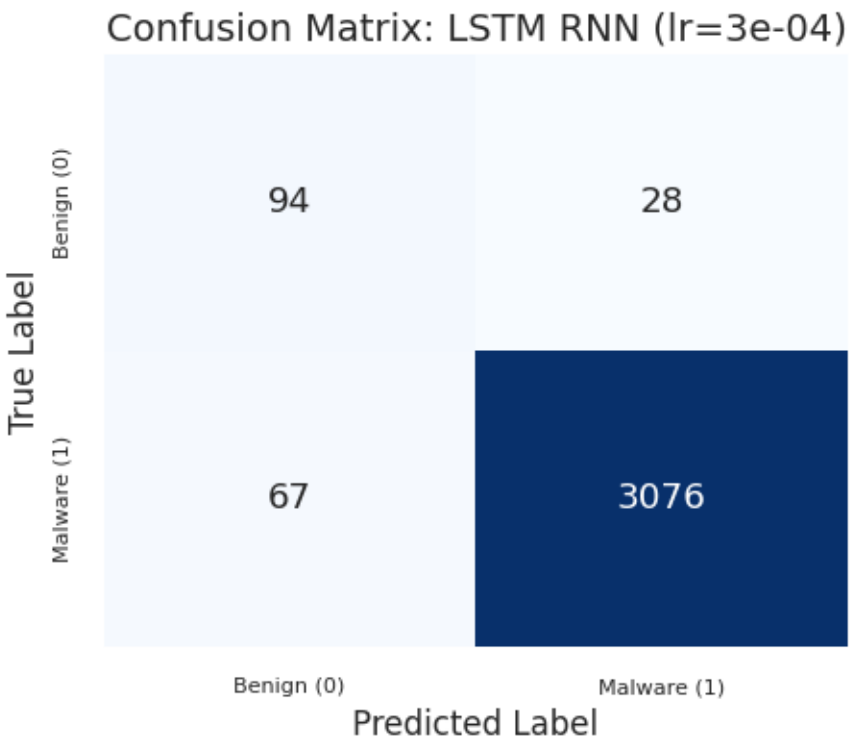
```
============================================================

Validation report (LSTM RNN - lr=8e-05):
              precision    recall  f1-score   support

           0     0.4088    0.6066    0.4884       122
           1     0.9844    0.9660    0.9751      3143

    accuracy                         0.9525      3265
   macro avg     0.6966    0.7863    0.7318      3265
weighted avg     0.9629    0.9525    0.9569      3265

Test report (LSTM RNN - lr=8e-05):
              precision    recall  f1-score   support

           0     0.4557    0.6132    0.5228       243
           1     0.9848    0.9716    0.9781      6262

    accuracy                         0.9582      6505
   macro avg     0.7202    0.7924    0.7505      6505
weighted avg     0.9650    0.9582    0.9611      6505


============================================================
 Best LSTM RNN - LR: 3e-04 (Min Val Loss: 0.0275)
============================================================
```

[109]:
```python
# --- Plot confusion matrices for ALL learning rates - VALIDATION ---

print("\n" + "=" * 80)
print("Confusion matrices for ALL LSTM RNN learning rates - VALIDATION")
print("=" * 80)

for lr in learning_rates_lstm:
    model = models_lstm[lr]
    print(f"\nGenerating confusion matrix for LSTM RNN with lr={lr:.0e}...")

    plot_cm_task3(model=model, loader=val_loader, model_name=f"LSTM RNN (lr={lr:
 ↪.0e})", save_dir=save_dir)
```

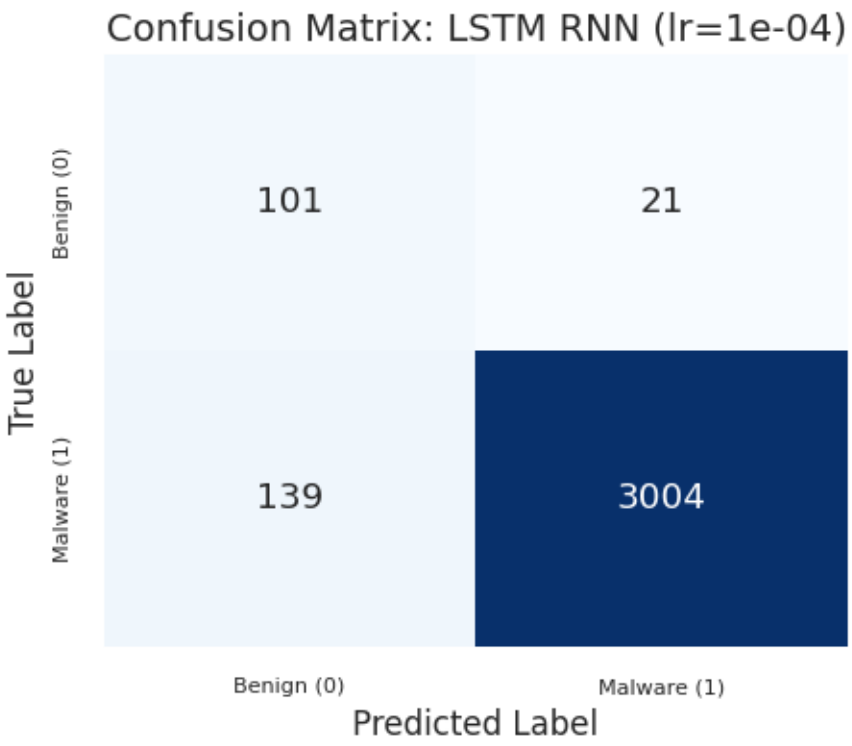
```
================================================================================
Confusion matrices for ALL LSTM RNN learning rates - VALIDATION
================================================================================

Generating confusion matrix for LSTM RNN with lr=5e-04…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_lstm_rnn_(lr=5e_04).png
```



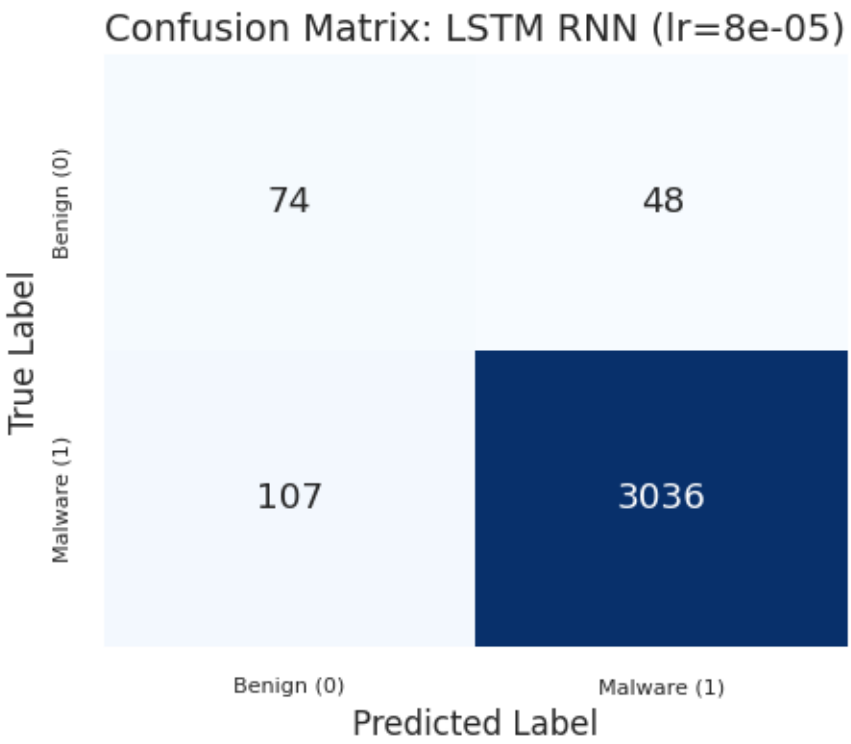

```
Generating confusion matrix for LSTM RNN with lr=3e-04…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_lstm_rnn_(lr=3e_04).png
```




```
Generating confusion matrix for LSTM RNN with lr=1e-04…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_lstm_rnn_(lr=1e_04).png
```

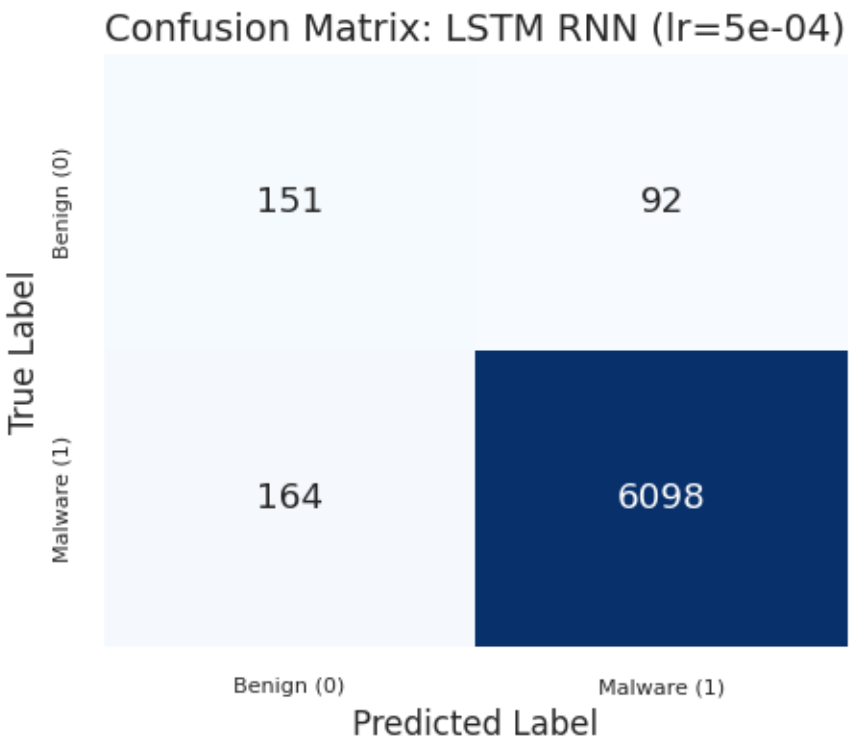



```
Generating confusion matrix for LSTM RNN with lr=8e-05…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_lstm_rnn_(lr=8e_05).png
```

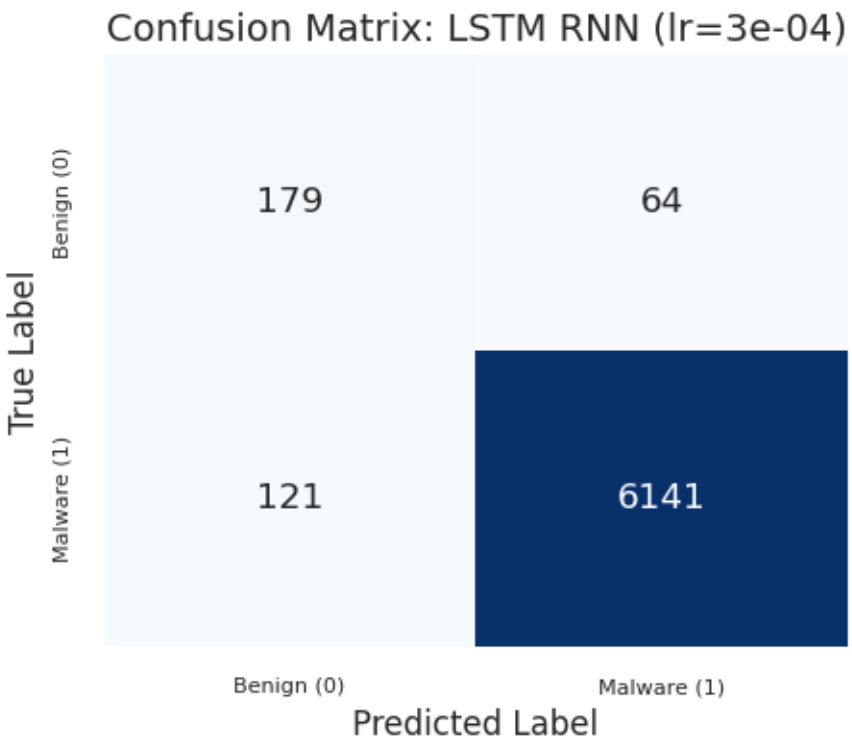


[110]:
```
# --- Plot confusion matrices for ALL learning rates - TEST ---

print("\n" + "=" * 80)
print("Confusion matrices for ALL LSTM RNN learning rates - TEST")
print("=" * 80)

for lr in learning_rates_lstm:
    model = models_lstm[lr]
    print(f"\nGenerating confusion matrix for LSTM RNN with lr={lr:.0e}...")

    plot_cm_task3(model=model, loader=test_loader, model_name=f"LSTM RNN␣
 ↪(lr={lr:.0e})", save_dir=save_dir)
```

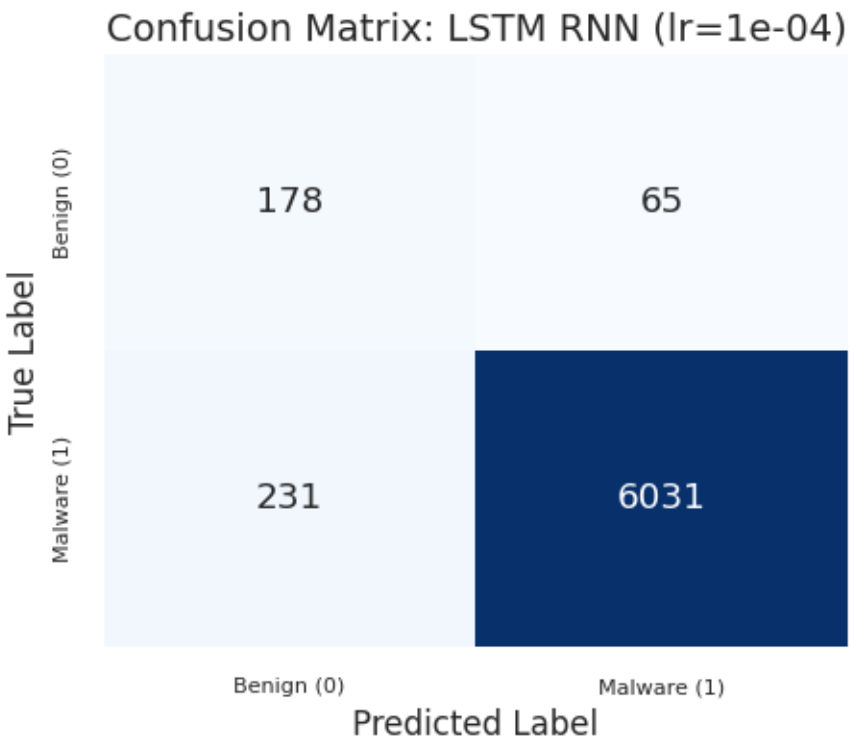
```
================================================================================
Confusion matrices for ALL LSTM RNN learning rates - TEST
================================================================================

Generating confusion matrix for LSTM RNN with lr=5e-04…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_lstm_rnn_(lr=5e_04).png
```



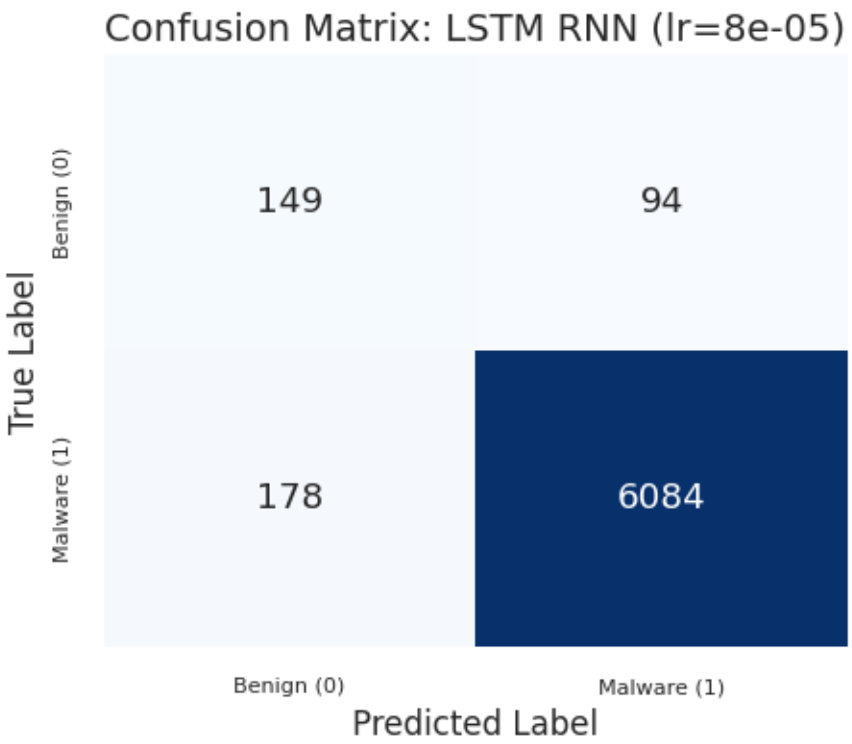

```
Generating confusion matrix for LSTM RNN with lr=3e-04…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_lstm_rnn_(lr=3e_04).png
```




```
Generating confusion matrix for LSTM RNN with lr=1e-04…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_lstm_rnn_(lr=1e_04).png
```




```
Generating confusion matrix for LSTM RNN with lr=8e-05…
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_lstm_rnn_(lr=8e_05).png
```



Q: Is the RNN training as stable as the FFNN's one? No. RNN training is less stable than FFNN training. The FFNN validation-loss curves are comparatively smoother, while the recurrent models show more oscillations and stronger sensitivity to the learning rate.

This happens because recurrent models backpropagate through time, producing a deeper computational graph whose effective depth depends on the sequence length. This increases the risk of vanishing or unstable gradients, especially for vanilla RNNs.

Among the recurrent architectures, the LSTM is the most stable and effective in our experiments. With `lr=3e-4`, it reaches the lowest validation loss (`0.0275`) and the best validation macro F1 among Task 3 models (`0.8246`). The gating mechanism of the LSTM helps regulate what information is kept or forgotten, making optimization more reliable than in simple RNN variants.

Q: How does your model's performance compare to the simple frequency baseline? The frequency baseline remains very competitive in accuracy, reaching `96.97%` on the test set, but its macro F1 is `0.7223` and its benign recall is only `0.3457`. This means that it is excellent at recognizing malware (`0.9939` recall), but it misclassifies many benign samples as malware.

The recurrent models improve the balance between the two classes. The selected Simple RNN reaches test macro F1 `0.7308` and benign recall `0.7366`, already slightly improving over the frequency baseline in macro F1. The selected Bi-RNN improves further, reaching test macro F1 `0.7852`, accuracy `96.48%`, and benign recall `0.6749`.

The selected LSTM is the strongest recurrent model in this run. With `lr=3e-4`, it reaches test

accuracy 97.16%, macro F1 0.8222, and benign recall 0.7366. It therefore beats the frequency baseline not only in macro F1, but also slightly in accuracy, while providing a much better balance between benign and malware recognition. Overall, the recurrent results show that temporal order contains discriminative information that the frequency baseline cannot capture.

[111]:
```
# ============================================================================
# FINAL SUMMARY
# ============================================================================

# Helper function to extract macro avg from the best model
def get_macro_avg_stats(model, loader):
    # Predict using your existing helper function
    y_true, y_pred = predict_loader(model, loader)
    # Get report as a DICTIONARY, not a string
    report_dict = classification_report(y_true, y_pred, output_dict=True,␣
 ↪zero_division=0)
    return report_dict['macro avg']

# --- 1. Prepare Stats for Simple RNN ---
# Retrieve the actual model object using the best LR key
best_model_simple_obj = models_simple[best_lr_simple[0]]
macro_simple = get_macro_avg_stats(best_model_simple_obj, val_loader)

# --- 2. Prepare Stats for Bi-RNN ---
best_model_bi_obj = models_bi[best_lr_bi[0]]
macro_bi = get_macro_avg_stats(best_model_bi_obj, val_loader)

# --- 3. Prepare Stats for LSTM ---
best_model_lstm_obj = models_lstm[best_lr_lstm[0]]
macro_lstm = get_macro_avg_stats(best_model_lstm_obj, val_loader)

# --- PRINTING THE REPORT ---

print("\n" + "=" * 90)
print("TASK 3 SUMMARY: Best Learning Rates & Macro Avg Performance")
print("=" * 90)
# Header for the manual table we are creating
print(f"{'Model':<20} | {'Best LR':<10} | {'Val Loss':<10} | {'Macro P':<8}␣
 ↪{'Macro R':<8} {'Macro F1':<8} | {'Time':<10}")
print("-" * 90)

# 1. Simple RNN
t_simple = time.strftime('%H:%M:%S', time.
 ↪gmtime(elapsed_simple[best_lr_simple[0]]))
print(f"{'Simple RNN':<20} | {best_lr_simple[0]:.0e}      |␣
 ↪{min(best_lr_simple[1]['val_loss']):.4f}     | "
```

```
      f"{macro_simple['precision']:.4f}   {macro_simple['recall']:.4f}  ␣
 ↪{macro_simple['f1-score']:.4f}   | {t_simple}")

# 2. Bi-RNN
t_bi = time.strftime('%H:%M:%S', time.gmtime(elapsed_bi[best_lr_bi[0]]))
print(f"{'Bidirectional RNN':<20} | {best_lr_bi[0]:.0e}      |␣
 ↪{min(best_lr_bi[1]['val_loss']):.4f}     | "
      f"{macro_bi['precision']:.4f}   {macro_bi['recall']:.4f}  ␣
 ↪{macro_bi['f1-score']:.4f}   | {t_bi}")

# 3. LSTM
t_lstm = time.strftime('%H:%M:%S', time.gmtime(elapsed_lstm[best_lr_lstm[0]]))
print(f"{'LSTM':<20} | {best_lr_lstm[0]:.0e}      |␣
 ↪{min(best_lr_lstm[1]['val_loss']):.4f}     | "
      f"{macro_lstm['precision']:.4f}   {macro_lstm['recall']:.4f}  ␣
 ↪{macro_lstm['f1-score']:.4f}   | {t_lstm}")
```

```
================================================================================
==========
TASK 3 SUMMARY: Best Learning Rates & Macro Avg Performance
================================================================================
==========
Model                | Best LR    | Val Loss   | Macro P  Macro R  Macro F1 |
Time
--------------------------------------------------------------------------------
----------
Simple RNN           | 8e-05      | 0.0448     | 0.6906   0.8581   0.7442   |
00:06:52
Bidirectional RNN    | 5e-05      | 0.0329     | 0.8109   0.8765   0.8402   |
00:10:06
LSTM                 | 3e-04      | 0.0275     | 0.7874   0.8746   0.8246   |
00:05:28
```

1.5 Task 4 - Graph Neural Network (GNN)

In this task we model API-call sequences as graphs and train GNN architectures (GCN, GraphSAGE, GAT) to classify samples. We reuse the lab preprocessing and reporting utilities for consistency.

[112]:
```
# Create directory for plots
save_dir = results_path + 'images/' + 'task4_plots/'
os.makedirs(save_dir, exist_ok=True)
```

1.5.1 Modeling API Sequences as Graphs

[113]:
```
df_train
```

[113]:
```
                                      api_call_sequence  is_malware  \
0      [LdrGetDllHandle, LdrGetProcedureAddress, LdrL…           1
1      [NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr…           1
2      [FindResourceExW, LoadResource, FindResourceEx…           1
3      [FindResourceExW, LoadResource, FindResourceEx…           1
4      [LdrGetProcedureAddress, SetErrorMode, LdrLoad…           1
…                                                    …         …
16320  [LdrGetProcedureAddress, LdrLoadDll, LdrGetPro…           1
16321  [NtClose, LdrGetProcedureAddress, CryptCreateH…           1
16322  [LdrGetProcedureAddress, LdrGetDllHandle, LdrG…           1
16323  [LdrGetProcedureAddress, LdrGetDllHandle, LdrG…           1
16324  [SetFilePointer, NtReadFile, SetFilePointer, N…           1

       seq_length                                          seq_fixed  \
0              73  [LdrGetDllHandle, LdrGetProcedureAddress, LdrL…
1              88  [NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr…
2              79  [FindResourceExW, LoadResource, FindResourceEx…
3              71  [FindResourceExW, LoadResource, FindResourceEx…
4              63  [LdrGetProcedureAddress, SetErrorMode, LdrLoad…
…           …                                                   …
16320          64  [LdrGetProcedureAddress, LdrLoadDll, LdrGetPro…
16321          78  [NtClose, LdrGetProcedureAddress, CryptCreateH…
16322          84  [LdrGetProcedureAddress, LdrGetDllHandle, LdrG…
16323          62  [LdrGetProcedureAddress, LdrGetDllHandle, LdrG…
16324          77  [SetFilePointer, NtReadFile, SetFilePointer, N…

                                    seq_sequential_ids
0      [117, 118, 119, 118, 117, 118, 86, 123, 201, 2…
1      [133, 119, 118, 82, 80, 137, 82, 152, 153, 163…
2      [46, 121, 46, 121, 46, 121, 46, 121, 46, 121, …
3      [46, 121, 46, 121, 46, 121, 46, 121, 46, 121, …
4      [118, 219, 119, 219, 81, 46, 121, 46, 121, 46,…
…                                                    …
16320  [118, 119, 118, 119, 118, 119, 118, 119, 118, …
16321  [134, 118, 24, 118, 30, 118, 135, 66, 221, 166…
16322  [118, 117, 118, 81, 216, 119, 118, 46, 121, 46…
16323  [118, 117, 118, 117, 118, 117, 118, 117, 118, …
16324  [221, 166, 221, 166, 221, 45, 121, 133, 146, 1…

[16325 rows x 5 columns]
```

[114]:
```
df_train_split, df_val = train_test_split(
    df_train,
    test_size=0.2,
```

```
    random_state=42,
    stratify=df_train['is_malware'],  # Preserve class balance
    shuffle=True
)
```

[115]:
```
# Use original api_call_sequence (not truncated seq_sequential_ids)
# GNNs can handle variable-length graphs, so we use full sequences
X_train_split = df_train_split['api_call_sequence'].values
y_train_split = df_train_split['is_malware'].values

X_val = df_val['api_call_sequence'].values
y_val = df_val['is_malware'].values

X_test = df_test['api_call_sequence'].values
y_test = df_test['is_malware'].values

print(f"Train length: {len(X_train_split)}")
print(f"Val length: {len(X_val)}")
print(f"Test length: {len(X_test)}")
```

```
Train length: 13060
Val length: 3265
Test length: 6505
```

[116]:
```
# --- Graph Construction ---

def create_graph_from_sequence(sequence, api_to_id, label, window_size=1):
    """
    Create a sparse graph containing ONLY the nodes present in the sequence.
    Accepts API name strings and converts them to integer IDs using api_to_id␣
 ↪mapping.
    Unknown APIs are mapped to UNK_IDX (index 1).
    """
    UNK_IDX = 1  # Same as defined in Task 3

    # 1. Convert API names to integer IDs (handle unknown APIs with UNK_IDX)
    sequence_ids = [api_to_id.get(api, UNK_IDX) for api in sequence]

    # 2. Identify unique node IDs present in this sequence
    # IMPORTANT: Filter out padding (0) before creating node set
    unique_api_ids = sorted(list(set([api_id for api_id in sequence_ids if␣
 ↪api_id != 0])))
    num_nodes = len(unique_api_ids)

    # 3. Create a mapping from Global API ID -> Local Graph Node Index
    global_to_local = {global_id: local_id for local_id, global_id in␣
 ↪enumerate(unique_api_ids)}
```

```
    # 4. Create Node Features (The Global IDs to be fed into the Embedding␣
↪Layer)
    x = torch.tensor(unique_api_ids, dtype=torch.long).unsqueeze(1)

    # 5. Create Edges (Transition Graph)
    src_list = []
    dst_list = []

    # Sliding window to find transitions
    for i in range(len(sequence_ids) - window_size):
        global_src = sequence_ids[i]
        global_dst = sequence_ids[i + window_size]

        # Skip padding (0) if present
        if global_src == 0 or global_dst == 0:
            continue

        # Map to local indices
        src_list.append(global_to_local[global_src])
        dst_list.append(global_to_local[global_dst])

    edge_index = torch.tensor([src_list, dst_list], dtype=torch.long)

    data = Data(
        x=x,
        edge_index=edge_index,
        y=torch.tensor([label], dtype=torch.float32),
        num_nodes=num_nodes
    )

    return data

def create_graphs_from_sequences(sequences, api_to_id, labels, window_size=1):
    graph_data_list = []
    for sequence, label in zip(sequences, labels):
        # Ensure sequence is a list (handles numpy arrays)
        seq_list = sequence if isinstance(sequence, list) else list(sequence)
        data = create_graph_from_sequence(seq_list, api_to_id, label,␣
↪window_size)
        graph_data_list.append(data)

    # Sanity check for each graph
    if data.edge_index.numel() > 0 and data.edge_index.max().item() >= data.
↪num_nodes:
        print(f"Warning: Graph {idx} has invalid edge index!")
```

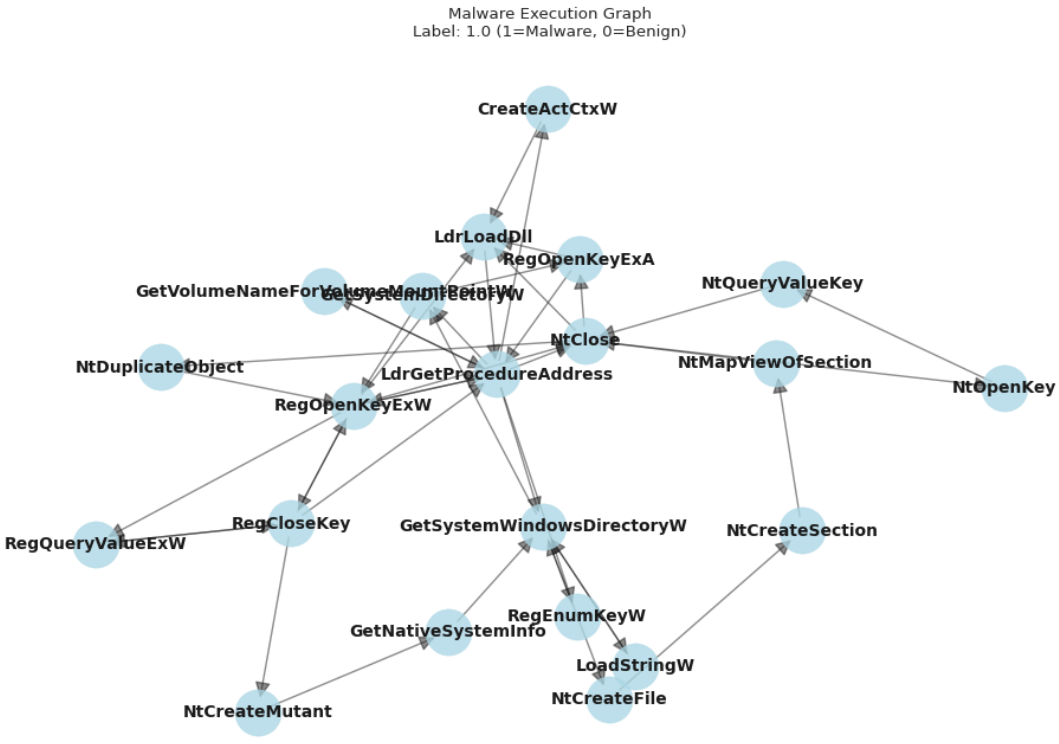
```
    return graph_data_list

# --- Execute Creation ---
print("Creating sparse graphs...")
train_graphs = create_graphs_from_sequences(X_train_split, api_to_id,␣
 ↪y_train_split)
val_graphs = create_graphs_from_sequences(X_val, api_to_id, y_val)
test_graphs = create_graphs_from_sequences(X_test, api_to_id, y_test)

print(f"Training graphs: {len(train_graphs)}")
print(f"First graph nodes: {train_graphs[0].num_nodes} (Subset of total vocab)")
print(f"First graph edges: {train_graphs[0].num_edges}")
```

```
Creating sparse graphs…
Training graphs: 13060
First graph nodes: 21 (Subset of total vocab)
First graph edges: 80
```

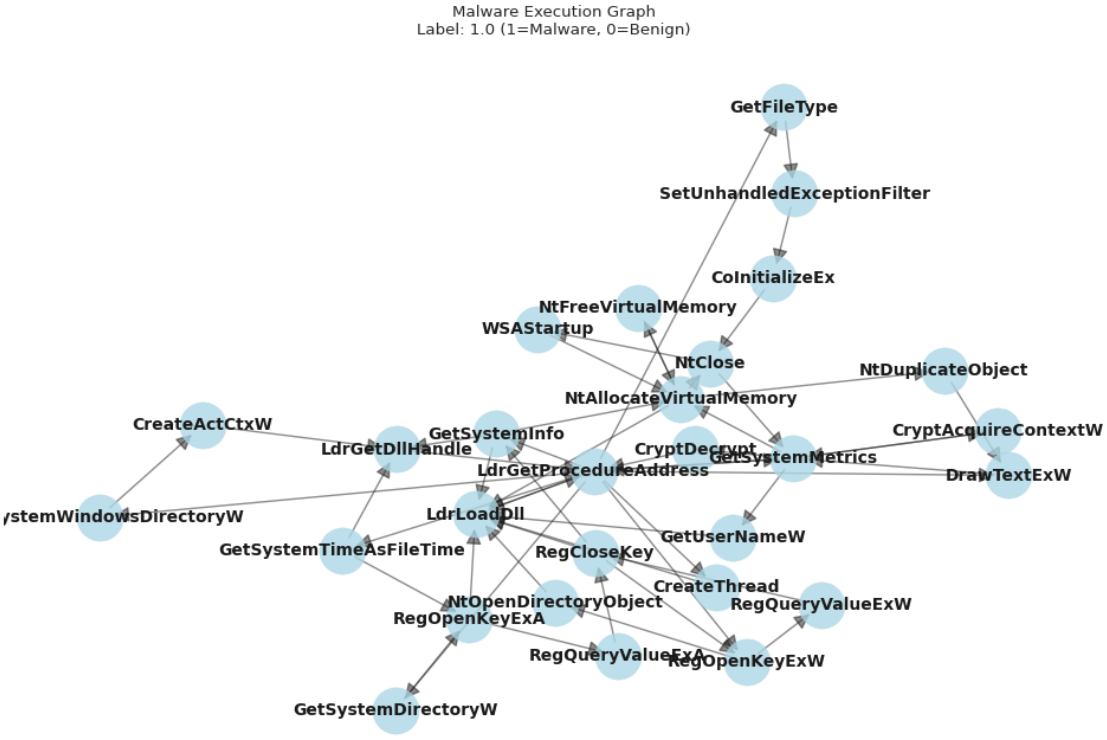
[117]:
```
import networkx as nx
import matplotlib.pyplot as plt
from torch_geometric.utils import to_networkx

def visualize_graph(data, id_to_api_mapping=None):
    # Convert PyG data to NetworkX
    G = to_networkx(data, to_undirected=False) # Keep it directed to see flow

    plt.figure(figsize=(12, 8))

    # Layout (Spring layout tends to show clusters and loops well)
    pos = nx.spring_layout(G, seed=42)

    # Draw nodes
    nx.draw_networkx_nodes(G, pos, node_size=800, node_color='lightblue',␣
 ↪alpha=0.8)

    # Draw edges (with arrows)
    nx.draw_networkx_edges(G, pos, width=1.0, alpha=0.5, arrowstyle='-|>',␣
 ↪arrowsize=20)

    # Labels: If you have the mapping, show API names. Otherwise show IDs.
    if id_to_api_mapping:
        # Get the Global API ID from data.x for each local node index
        labels = {i: id_to_api_mapping.get(data.x[i].item(), str(i)) for i in␣
 ↪range(data.num_nodes)}
        nx.draw_networkx_labels(G, pos, labels, font_size=10,␣
 ↪font_weight="bold")
    else:
```

```
        nx.draw_networkx_labels(G, pos)

    plt.title(f"Malware Execution Graph\nLabel: {data.y.item()} (1=Malware,␣
 ↪0=Benign)")
    plt.axis('off')
    plt.show()

# --- RUN THE CHECK ---
# Plot example graphs from each dataset
print("Plotting example graphs...\n")
print("Training set example:")
visualize_graph(train_graphs[0], id_to_api)

print("\nValidation set example:")
visualize_graph(val_graphs[0], id_to_api)

print("\nTest set example:")
visualize_graph(test_graphs[0], id_to_api)
```

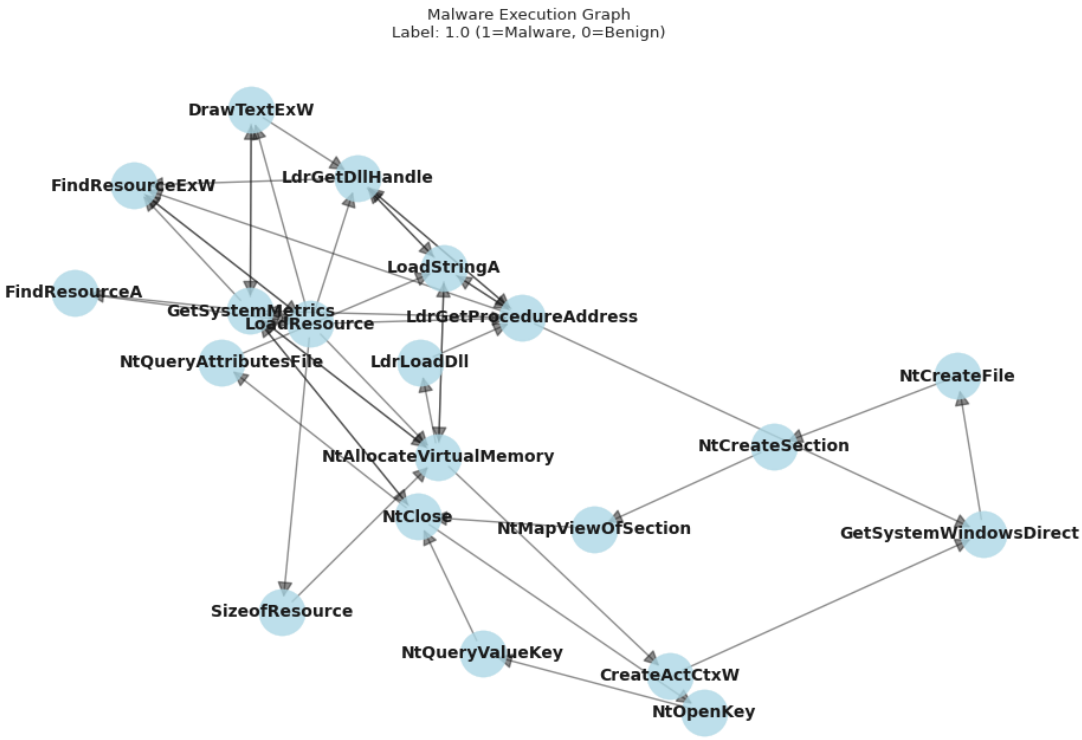
```
Plotting example graphs...

Training set example:
```



Validation set example:



Test set example:



[118]:
```
data = train_graphs[0]

print(f"1. Node Features Shape: {data.x.shape}")
print(f"   → {data.num_nodes} unique APIs in this sequence")

print(f"\n2. Edge Index Shape: {data.edge_index.shape}")
print(f"   → {data.num_edges} transitions (edges) in the sequence")

print(f"\n3. Max Node Index: {data.edge_index.max().item()}")
print(f"   → Valid range: [0, {data.num_nodes - 1}]")
if data.edge_index.numel() > 0:
    is_valid = data.edge_index.max().item() < data.num_nodes
    print(f"   → {'  VALID' if is_valid else '  INVALID - edges point to␣
 ↪non-existent nodes'}")
else:
    print(f"   → No edges in this graph")

print(f"\n4. Label Shape: {data.y.shape}")
print(f"   → Label value: {data.y.item()} ({'Malware' if data.y.item() == 1␣
 ↪else 'Benign'})")

print(f"\n5. Summary:")
```

```
print(f"   → Graph has {data.num_nodes} nodes and {data.num_edges} edges")
print(f"   → Sparsity: {(data.num_edges / (data.num_nodes * (data.num_nodes - 1))) * 100:.2f}% (if fully connected)")
```

```
1. Node Features Shape: torch.Size([21, 1])
   → 21 unique APIs in this sequence

2. Edge Index Shape: torch.Size([2, 80])
   → 80 transitions (edges) in the sequence

3. Max Node Index: 20
   → Valid range: [0, 20]
   →  VALID

4. Label Shape: torch.Size([1])
   → Label value: 1.0 (Malware)

5. Summary:
   → Graph has 21 nodes and 80 edges
   → Sparsity: 19.05% (if fully connected)
```

[119]:
```
# Get the raw sequence and the graph
raw_seq_ids = X_train_split[0] # The list of IDs: [117, 118, 119...]
graph = train_graphs[0]

# Check 1: Unique counts
unique_apis_raw = len(set(raw_seq_ids))
# Padding (0) is usually removed in your graph logic, so:
if 0 in raw_seq_ids: unique_apis_raw -= 1

print(f"Unique APIs in raw sequence: {unique_apis_raw}")
print(f"Nodes in Graph:              {graph.num_nodes}")
# THESE SHOULD MATCH EXACTLY.

# Check 2: Transitions vs Edges
# Raw edges = length of sequence - 1 (minus padding transitions)
# Note: Your graph combines duplicate edges.
# If raw sequence is A->B->A->B, graph has edges (A,B) and (B,A).
# This check is qualitative: Does graph.num_edges roughly track with sequence complexity?
print(f"Raw sequence length: {len([x for x in raw_seq_ids if x != 0])}")
print(f"Graph edges:         {graph.num_edges}")
```

```
Unique APIs in raw sequence: 21
Nodes in Graph:              21
Raw sequence length: 81
Graph edges:         80
```

Q: Do you still have to pad your data? If yes, how? No. The graph-based representation eliminates the need for padding. Each API sequence is converted into its own graph, where nodes correspond to the unique API calls appearing in the sequence and edges represent transitions between consecutive calls.

Longer sequences simply create more transitions and possibly more nodes. Shorter sequences create smaller graphs. PyTorch Geometric can batch these variable-sized graphs by merging them into a larger disconnected graph and using a `batch` vector to keep track of graph membership.

As a result, no dummy padding nodes or padding edges are required. The model works directly on the sparse graph structure of each execution trace.

Q: Do you have to truncate the testing sequences? Justify your answer. No. GNNs do not require test-time truncation because there is no fixed-size input vector that would overflow when a sequence is longer than expected. A longer sequence simply generates additional edges, and possibly additional nodes, in its graph representation.

Truncating a test sequence would remove transitions from the graph and would therefore discard part of the behavioral structure we are trying to model. Since the GNN can process variable-sized graphs through PyTorch Geometric batching and graph-level pooling, keeping the full test sequence is the correct choice.

Q: What is the advantage of modeling your problem with a GNN compared to an RNN? What do you lose? The advantage of a GNN is that it models the API trace as a transition structure rather than only as a left-to-right chain. If an API call repeatedly transitions to several other calls, or if a small cycle appears multiple times, the GNN can aggregate information from those local neighborhoods and learn structural motifs that may be discriminative for malware detection.

This representation also handles variable-length sequences naturally: graphs can have different numbers of nodes and edges, and batching does not require padding or truncation. Message passing can summarize repeated transition patterns and relationships between API calls in a way that is different from the hidden-state memory of an RNN.

What we lose is strict chronological information. After converting the sequence into a graph and applying graph-level pooling, the model knows that certain API calls are connected, but it does not know exactly whether a transition happened early or late in the trace. RNNs preserve this temporal evolution explicitly through their hidden states, while GNNs emphasize transition structure.

1.5.2 GNN Training and Helper Functions

We need new training and prediction functions, as the `PyGDataLoader` returns batches differently from the standard `DataLoader`.

[120]:
```
# Create DataLoader for batching
train_loader = PyGDataLoader(train_graphs, batch_size=32, shuffle=True)
val_loader = PyGDataLoader(val_graphs, batch_size=32, shuffle=False)
test_loader = PyGDataLoader(test_graphs, batch_size=32, shuffle=False)
```

```
print(f"Train loader: {len(train_loader)}, Validation loader:␣
 ↪{len(val_loader)}, Test loader: {len(test_loader)}")
```

```
Train loader: 409, Validation loader: 103, Test loader: 204
```

[121]:
```
# ============================================================================
# GNN TRAINING FUNCTION (with Macro F1 tracking)
# ============================================================================

def train_gnn_common(model, train_loader, val_loader, optimizer, criterion,␣
 ↪epochs=50, min_delta=0.00001, patience=20):
    """
    Train a GNN model with early stopping and track macro F1 score.
    """
    device = next(model.parameters()).device

    best_val = math.inf
    best_state = None
    trigger = 0
    history = {'train_loss': [], 'val_loss': [], 'val_f1_macro': []}
    start_time = time.time()

    for epoch in range(1, epochs + 1):
        model.train()
        train_losses = []
        for data in train_loader:
            data = data.to(device)
            optimizer.zero_grad()
            outputs = model(data)
            loss = criterion(outputs, data.y.unsqueeze(1).float())
            loss.backward()
            optimizer.step()
            train_losses.append(loss.item())
        train_loss = float(np.mean(train_losses)) if train_losses else 0.0

        # Validation
        model.eval()
        val_losses = []
        all_preds = []
        all_targets = []
        with torch.no_grad():
            for data in val_loader:
                data = data.to(device)
                outputs = model(data)
                loss = criterion(outputs, data.y.unsqueeze(1).float())
                val_losses.append(loss.item())
```

```
                # Collect predictions for F1 score
                # FIX: use view(-1) instead of squeeze() to handle␣
↪single-element batches
                probs = torch.sigmoid(outputs).view(-1)
                preds = (probs > 0.5).long()

                # FIX: convert to list properly to handle both single and␣
↪multi-element tensors
                all_preds.extend(preds.cpu().tolist())
                all_targets.extend(data.y.cpu().tolist())

        val_loss = float(np.mean(val_losses)) if val_losses else 0.0
        val_f1_macro = f1_score(all_targets, all_preds, average='macro',␣
↪zero_division=0)

        history['train_loss'].append(train_loss)
        history['val_loss'].append(val_loss)
        history['val_f1_macro'].append(val_f1_macro)

        if (epoch % 10 == 0) or (epoch == 1) or (epoch == epochs):
            print(f'Epoch {epoch}/{epochs} - Train Loss: {train_loss:.4f} - Val␣
↪Loss: {val_loss:.4f} - Val F1: {val_f1_macro:.4f}')

        # Early stopping logic
        if val_loss < best_val - min_delta:
            best_val = val_loss
            best_state = {k: v.cpu().clone() for k, v in model.state_dict().
↪items()}
            trigger = 0
        else:
            trigger += 1
            if trigger >= patience:
                print(f'Early stopping at epoch {epoch} (best val loss␣
↪{best_val:.6f})')
                break

    elapsed = time.time() - start_time

    if best_state is not None:
        model.load_state_dict(best_state)
        model.to(device)

    return model, history, elapsed
```

[122]:
```
# --- GNN Prediction Helper ---

def predict_loader_gnn(model, loader, threshold=0.5):
```

```
    model.eval()

    # Detect device from model parameters
    device = next(model.parameters()).device

    ys_true = []
    ys_pred = []
    with torch.no_grad():
        for data in loader:
            # Move batch to device
            data = data.to(device)

            out = model(data)
            probs = torch.sigmoid(out).cpu().numpy().ravel()
            preds = (probs >= threshold).astype(int)

            # Get true labels from the DataBatch object
            ys_true.extend(data.y.cpu().numpy().ravel().astype(int).tolist())
            ys_pred.extend(preds.tolist())

    return np.array(ys_true, dtype=int), np.array(ys_pred, dtype=int)
```

[123]:
```
def plot_cm_task4(model, loader, model_name, save_dir=None):
    """
    Specific for GNNs. Uses your existing 'predict_loader_gnn' logic.
    """
    # Use the GNN specific helper
    y_true, y_pred = predict_loader_gnn(model, loader)

    # Plot
    _internal_plot_cm(y_true, y_pred, model_name, save_dir)
```

1.5.3 GNN Variations

Now we define and train the three GNN architectures as requested by the lab PDF. We will use the same hyperparameters (`vocab_size`, `embed_dim`, `hidden_dim`) for a fair comparison. We also re-use the `pos_weight` from Task 3 to handle the imbalanced dataset.

[124]:
```
# Hyperparameters for all GNNs
VOCAB_SIZE_GNN = len(api_to_id)
EMBED_DIM_GNN = 32
HIDDEN_DIM_GNN = 64
OUTPUT_DIM_GNN = 1
EPOCHS_GNN = 100
PATIENCE_GNN = 15
MIN_DELTA_GNN = 0.0001
LR_GNN = 5e-4 # Learning rate
```

```
# Re-compute pos_weight for the GNN training subset
pos_weight_gnn = compute_pos_weight(y_train_split).to(device)

criterion_gnn = nn.BCEWithLogitsLoss(pos_weight=pos_weight_gnn)


def select_best_gnn_model(architecture_name, models, histories, elapsed_times,␣
 ↪val_loader):
    """
    Select the best Task 4 model automatically.

    Primary criterion: final validation macro F1 from the training history.
    Tie-breakers: validation accuracy of the restored model, then minimum␣
 ↪validation loss.
    This avoids manually hardcoding the selected learning rate.
    """
    selection_rows = []

    for lr_label, model in models.items():
        history = histories[lr_label]
        y_val_true, y_val_pred = predict_loader_gnn(model, val_loader)

        final_val_f1 = history['val_f1_macro'][-1] if history.
 ↪get('val_f1_macro') else float('-inf')
        val_accuracy = accuracy_score(y_val_true, y_val_pred)
        eval_val_f1 = f1_score(y_val_true, y_val_pred, average='macro',␣
 ↪zero_division=0)
        min_val_loss = min(history['val_loss']) if history.get('val_loss') else␣
 ↪float('inf')

        selection_rows.append({
            'lr_label': lr_label,
            'final_val_f1_macro': final_val_f1,
            'val_accuracy': val_accuracy,
            'eval_val_f1_macro': eval_val_f1,
            'min_val_loss': min_val_loss,
            'elapsed_s': elapsed_times[lr_label],
        })

    selection_table = pd.DataFrame(selection_rows).sort_values(
        by=['final_val_f1_macro', 'val_accuracy', 'min_val_loss'],
        ascending=[False, False, True]
    ).reset_index(drop=True)

    best_lr = selection_table.loc[0, 'lr_label']
    best_model = models[best_lr]
```

```
    best_history = histories[best_lr]
    best_elapsed = elapsed_times[best_lr]

    print(f"Automatic selection for {architecture_name}")
    print("Ranking criterion: final validation macro F1, then validation␣
 ↪accuracy, then minimum validation loss")
    print(selection_table.to_string(index=False))
    print(f"\nSelected best {architecture_name} model: {best_lr}")
    print(f"Final Val F1 Macro: {selection_table.loc[0, 'final_val_f1_macro']:.
 ↪4f}")
    print(f"Validation Accuracy: {selection_table.loc[0, 'val_accuracy']:.4f}")

    return best_lr, best_model, best_history, best_elapsed, selection_table


print(f"GNN Vocab Size: {VOCAB_SIZE_GNN}")
print(f"GNN Positive Weight: {pos_weight_gnn.item():.4f}") # Use .item() for␣
 ↪scalar tensor print
```

```
Negatives: 489, Positives: 12571
pos_weight (applied to y=1 samples): 0.2000
GNN Vocab Size: 260
GNN Positive Weight: 0.2000
```

1. Simple GCN This model uses Graph Convolutional Networks (GCN).

[125]:
```
# --- 1. GCN Model Definition ---

class GCNmodel(torch.nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, output_dim):
        super(GCNmodel, self).__init__()

        # Embedding: Maps Global API IDs -> Dense Vector
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)

        # GCN Layers
        self.conv1 = GCNConv(embed_dim, hidden_dim)
        self.conv2 = GCNConv(hidden_dim, hidden_dim)
        self.conv3 = GCNConv(hidden_dim, hidden_dim)

        # Pooling
        self.pool = global_mean_pool

        # Classifier
        self.classifier = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
```

```
            nn.Dropout(0.5),
            nn.Linear(hidden_dim, output_dim)
        )

    def forward(self, data):
        x, edge_index, batch = data.x, data.edge_index, data.batch

        # x comes in as [num_nodes_in_batch, 1]
        x = x.squeeze(1) # [num_nodes_in_batch]

        # 1. Embedding
        x = self.embedding(x) # [num_nodes, embed_dim]

        # 2. GCN
        x = self.conv1(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=0.2, training=self.training)

        x = self.conv2(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=0.2, training=self.training)

        x = self.conv3(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=0.2, training=self.training)

        # 3. Global Pooling
        # Averages node vectors belonging to the same graph
        x = self.pool(x, batch)

        # 4. Classification
        out = self.classifier(x)
        return out
```

[126]:
```
# ============================================================================
# GCN: Training with Multiple Learning Rates
# ============================================================================

gcn_learning_rates = [1e-4, 5e-4, 1e-3]
gcn_lr_labels = ['LR=1e-4', 'LR=5e-4', 'LR=1e-3']

# Storage for results
gcn_histories = {}
gcn_models = {}
gcn_elapsed_times = {}

for lr, lr_label in zip(gcn_learning_rates, gcn_lr_labels):
```

```
    print(f"\n{'='*60}")
    print(f"Training GCN with {lr_label}")
    print('='*60)

    # Set seed for reproducibility
    torch.manual_seed(42)
    np.random.seed(42)

    # Create model
    model_gcn_temp = GCNmodel(
        vocab_size=VOCAB_SIZE_GNN,
        embed_dim=EMBED_DIM_GNN,
        hidden_dim=HIDDEN_DIM_GNN,
        output_dim=OUTPUT_DIM_GNN
    ).to(device)

    optimizer_temp = optim.AdamW(model_gcn_temp.parameters(), lr=lr)

    model_gcn_temp, history_temp, elapsed_temp = train_gnn_common(
        model_gcn_temp,
        train_loader,
        val_loader,
        optimizer_temp,
        criterion_gnn,
        epochs=EPOCHS_GNN,
        min_delta=MIN_DELTA_GNN,
        patience=PATIENCE_GNN
    )

    gcn_models[lr_label] = model_gcn_temp
    gcn_histories[lr_label] = history_temp
    gcn_elapsed_times[lr_label] = elapsed_temp

    elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed_temp))
    print(f'\n{lr_label} - Training time: {elapsed_str}')
```

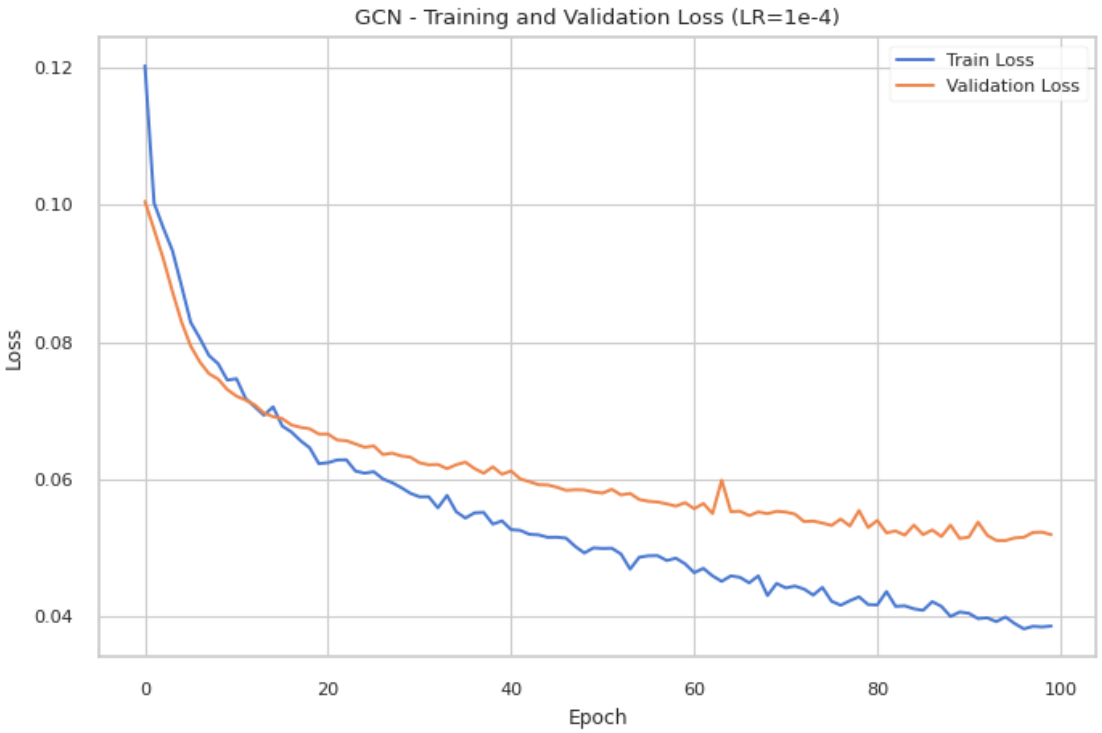
```
============================================================
Training GCN with LR=1e-4
============================================================
Epoch 1/100 - Train Loss: 0.1204 - Val Loss: 0.1005 - Val F1: 0.4905
Epoch 10/100 - Train Loss: 0.0745 - Val Loss: 0.0731 - Val F1: 0.6072
Epoch 20/100 - Train Loss: 0.0622 - Val Loss: 0.0666 - Val F1: 0.6562
Epoch 30/100 - Train Loss: 0.0579 - Val Loss: 0.0632 - Val F1: 0.6871
Epoch 40/100 - Train Loss: 0.0539 - Val Loss: 0.0607 - Val F1: 0.6815
Epoch 50/100 - Train Loss: 0.0500 - Val Loss: 0.0581 - Val F1: 0.7123
Epoch 60/100 - Train Loss: 0.0476 - Val Loss: 0.0566 - Val F1: 0.7123
Epoch 70/100 - Train Loss: 0.0448 - Val Loss: 0.0553 - Val F1: 0.7354
```

```
Epoch 80/100 - Train Loss: 0.0417 - Val Loss: 0.0529 - Val F1: 0.7302
Epoch 90/100 - Train Loss: 0.0406 - Val Loss: 0.0513 - Val F1: 0.7375
Epoch 100/100 - Train Loss: 0.0385 - Val Loss: 0.0519 - Val F1: 0.7446

LR=1e-4 - Training time: 00:05:43

============================================================
Training GCN with LR=5e-4
============================================================
Epoch 1/100 - Train Loss: 0.1031 - Val Loss: 0.0874 - Val F1: 0.4905
Epoch 10/100 - Train Loss: 0.0530 - Val Loss: 0.0576 - Val F1: 0.7012
Epoch 20/100 - Train Loss: 0.0386 - Val Loss: 0.0513 - Val F1: 0.7506
Epoch 30/100 - Train Loss: 0.0324 - Val Loss: 0.0510 - Val F1: 0.7999
Epoch 40/100 - Train Loss: 0.0268 - Val Loss: 0.0475 - Val F1: 0.7674
Early stopping at epoch 49 (best val loss 0.043461)

LR=5e-4 - Training time: 00:02:49

============================================================
Training GCN with LR=1e-3
============================================================
Epoch 1/100 - Train Loss: 0.0957 - Val Loss: 0.0753 - Val F1: 0.5886
Epoch 10/100 - Train Loss: 0.0468 - Val Loss: 0.0550 - Val F1: 0.7338
Epoch 20/100 - Train Loss: 0.0304 - Val Loss: 0.0560 - Val F1: 0.7977
Epoch 30/100 - Train Loss: 0.0243 - Val Loss: 0.0634 - Val F1: 0.7981
Epoch 40/100 - Train Loss: 0.0206 - Val Loss: 0.0614 - Val F1: 0.8148
Early stopping at epoch 44 (best val loss 0.048973)

LR=1e-3 - Training time: 00:02:31
```

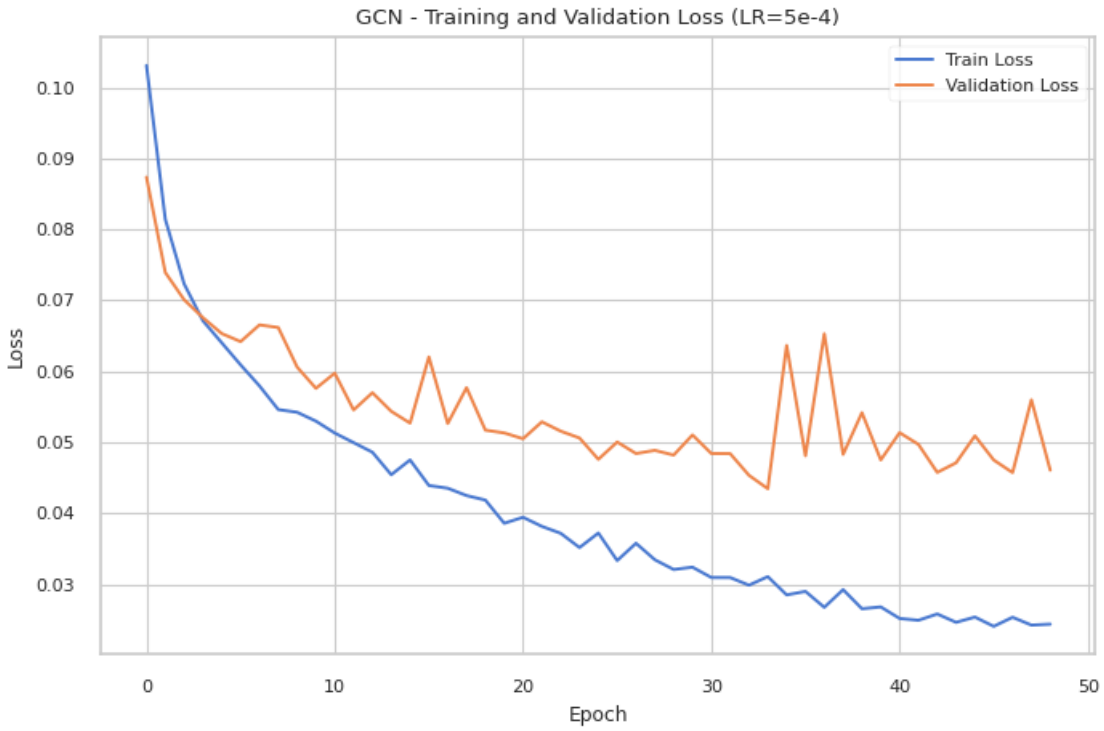
[127]:
```python
# ============================================================================
# Plot Training/Validation Loss per OGNI learning rate (GCN)
# ============================================================================

for lr_label, history in gcn_histories.items():
    plt.figure(figsize=(8, 5))
    plt.plot(history['train_loss'], label='Train Loss')
    plt.plot(history['val_loss'], label='Validation Loss')
    plt.title(f'GCN - Training and Validation Loss ({lr_label})')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend()
    plt.grid(True)

    # Nome file unico per ogni lr
    save_plot(plt.gcf(), f'task4_gcn_loss_curves_{lr_label.replace("=", "")}',␣
 ↪save_dir)
```

```
    plt.show()
```

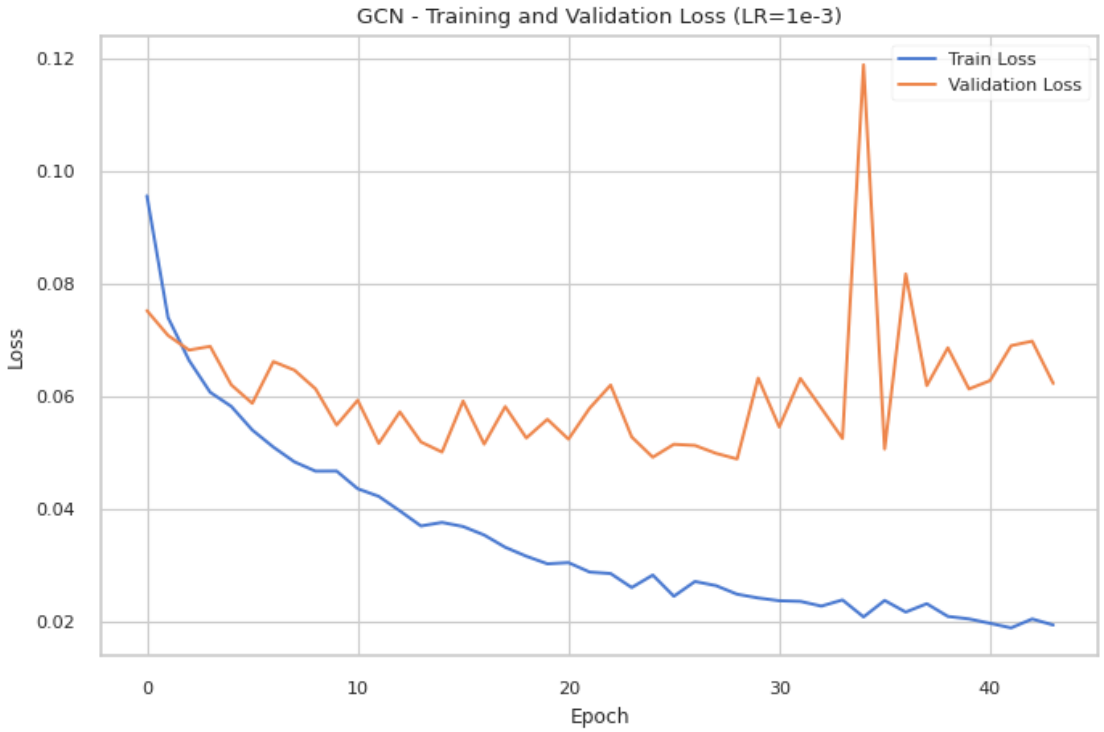
```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task4_plots/task4_gcn_loss_curves_LR1e-4.png
```




```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task4_plots/task4_gcn_loss_curves_LR5e-4.png
```



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/task4_gcn_loss_curves_LR1e-3.png



[128]:
```
# ============================================================================
# GCN: Plot Learning Rate Comparison (Validation Loss + Macro F1)
# ============================================================================

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

colors = ['#1b9e77', '#d95f02', '#7570b3']

# Plot Validation Loss
for (lr_label, history), color in zip(gcn_histories.items(), colors):
    epochs_range = range(1, len(history['val_loss']) + 1)
    ax1.plot(epochs_range, history['val_loss'], label=lr_label, linewidth=2,␣
 ↪color=color)

ax1.set_title('GCN - Validation Loss', fontsize=14, fontweight='bold')
ax1.set_xlabel('Epoch', fontsize=12)
ax1.set_ylabel('Loss', fontsize=12)
ax1.legend()
ax1.grid(alpha=0.3)

# Plot Validation Macro F1
for (lr_label, history), color in zip(gcn_histories.items(), colors):
    epochs_range = range(1, len(history['val_f1_macro']) + 1)
    ax2.plot(epochs_range, history['val_f1_macro'], label=lr_label,␣
 ↪linewidth=2, color=color)

ax2.set_title('GCN - Validation Macro F1 Score', fontsize=14, fontweight='bold')
ax2.set_xlabel('Epoch', fontsize=12)
ax2.set_ylabel('Macro F1 Score', fontsize=12)
ax2.legend()
ax2.grid(alpha=0.3)

plt.tight_layout()
save_plot(plt.gcf(), f"task4_gcn_lr_comparison", save_dir)
plt.show()
```

```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task4_plots/task4_gcn_lr_comparison.png
```

[129]:
```
# --- Evaluate GCN ---
for lr_label, model_gcn in gcn_models.items():
    print(f"\n{'='*60}")
    print(f"Evaluation for GCN - {lr_label}")
    print('='*60)

    y_val_true_gcn, y_val_pred_gcn = predict_loader_gnn(model_gcn, val_loader)
    print('\nValidation report (GCN):')
    print(classification_report(y_val_true_gcn, y_val_pred_gcn, digits=4,␣
 ↪zero_division=0))

    y_test_true_gcn, y_test_pred_gcn = predict_loader_gnn(model_gcn,␣
 ↪test_loader)
    print('Test report (GCN):')
    print(classification_report(y_test_true_gcn, y_test_pred_gcn, digits=4,␣
 ↪zero_division=0))
```

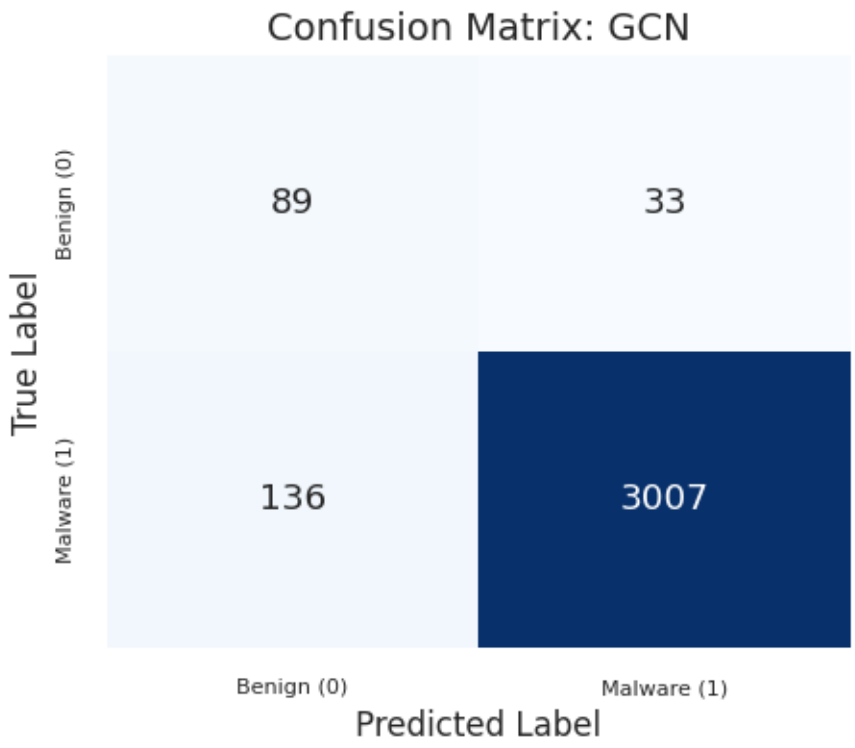
```
============================================================
Evaluation for GCN - LR=1e-4
============================================================

Validation report (GCN):
              precision    recall  f1-score   support

           0     0.3956    0.7295    0.5130       122
           1     0.9891    0.9567    0.9727      3143

    accuracy                         0.9482      3265
   macro avg     0.6924    0.8431    0.7428      3265
weighted avg     0.9670    0.9482    0.9555      3265

Test report (GCN):
```

```
              precision    recall  f1-score   support

           0     0.3863    0.6502    0.4847       243
           1     0.9861    0.9599    0.9728      6262

    accuracy                         0.9483      6505
   macro avg     0.6862    0.8051    0.7287      6505
weighted avg     0.9637    0.9483    0.9546      6505


============================================================
Evaluation for GCN - LR=5e-4
============================================================

Validation report (GCN):
              precision    recall  f1-score   support

           0     0.4847    0.7787    0.5975       122
           1     0.9912    0.9679    0.9794      3143

    accuracy                         0.9608      3265
   macro avg     0.7379    0.8733    0.7884      3265
weighted avg     0.9723    0.9608    0.9651      3265

Test report (GCN):
              precision    recall  f1-score   support

           0     0.4803    0.7037    0.5710       243
           1     0.9883    0.9705    0.9793      6262

    accuracy                         0.9605      6505
   macro avg     0.7343    0.8371    0.7751      6505
weighted avg     0.9693    0.9605    0.9640      6505


============================================================
Evaluation for GCN - LR=1e-3
============================================================

Validation report (GCN):
              precision    recall  f1-score   support

           0     0.4762    0.8197    0.6024       122
           1     0.9928    0.9650    0.9787      3143

    accuracy                         0.9596      3265
   macro avg     0.7345    0.8923    0.7906      3265
weighted avg     0.9735    0.9596    0.9646      3265
```

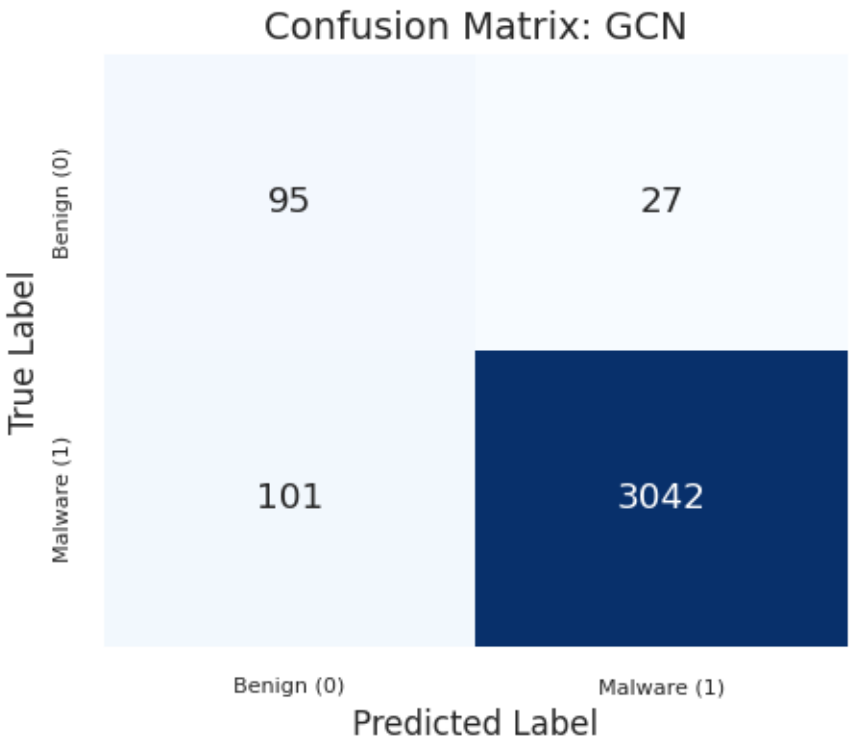
```
Test report (GCN):
              precision    recall  f1-score   support

           0     0.4572    0.7037    0.5543       243
           1     0.9883    0.9676    0.9778      6262

    accuracy                         0.9577      6505
   macro avg     0.7227    0.8356    0.7661      6505
weighted avg     0.9684    0.9577    0.9620      6505
```

[130]:
```
for lr_label, model_gcn in gcn_models.items():
  plot_cm_task4(model=model_gcn,
                loader=val_loader,
                model_name="GCN",
                save_dir=results_path + 'images/task4_plots/')
```

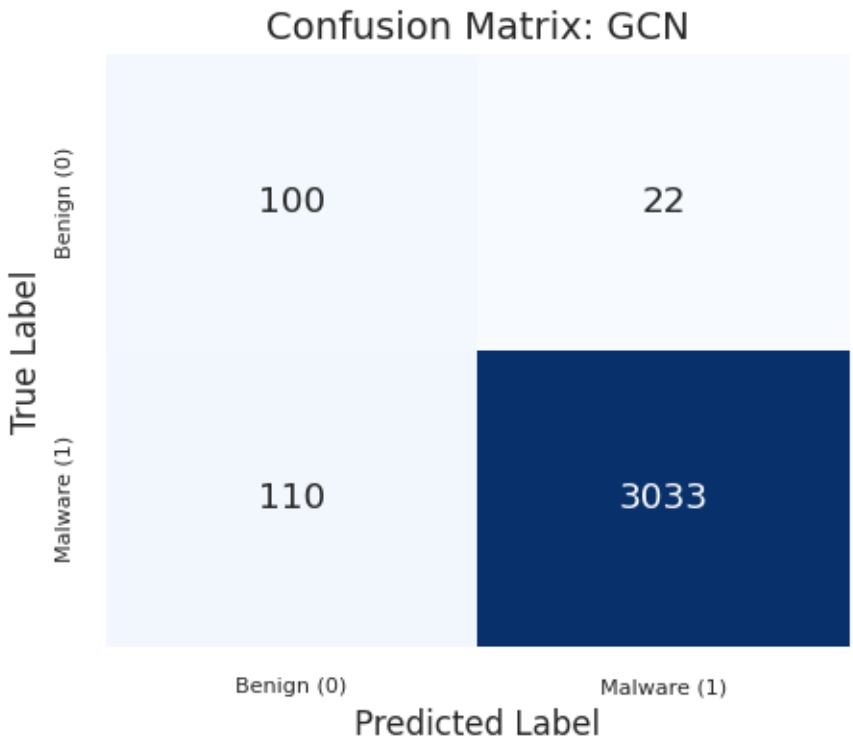
```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task4_plots/cm_gcn.png
```




```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task4_plots/cm_gcn.png
```



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_gcn.png



[131]:
```
# ============================================================================
# GCN: Select Best Model and Evaluate
# ============================================================================

best_gcn_lr, model_gcn, history_gcn, elapsed_gcn, gcn_selection_table =␣
 ↪select_best_gnn_model(
    architecture_name='GCN',
    models=gcn_models,
    histories=gcn_histories,
    elapsed_times=gcn_elapsed_times,
    val_loader=val_loader,
)

# Evaluate selected model
y_val_true_gcn, y_val_pred_gcn = predict_loader_gnn(model_gcn, val_loader)
print('\nValidation report (GCN):')
print(classification_report(y_val_true_gcn, y_val_pred_gcn, digits=4,␣
 ↪zero_division=0))

y_test_true_gcn, y_test_pred_gcn = predict_loader_gnn(model_gcn, test_loader)
print('Test report (GCN):')
```

```
print(classification_report(y_test_true_gcn, y_test_pred_gcn, digits=4,␣
 ↪zero_division=0))
```

```
Automatic selection for GCN
Ranking criterion: final validation macro F1, then validation accuracy, then
minimum validation loss
lr_label  final_val_f1_macro  val_accuracy  eval_val_f1_macro  min_val_loss
elapsed_s
 LR=1e-3            0.806204      0.959571           0.790556      0.048973
151.882000
 LR=5e-4            0.773322      0.960796           0.788439      0.043461
169.093862
 LR=1e-4            0.744575      0.948239           0.742818      0.051019
343.416907

Selected best GCN model: LR=1e-3
Final Val F1 Macro: 0.8062
Validation Accuracy: 0.9596

Validation report (GCN):
              precision    recall  f1-score   support

           0     0.4762    0.8197    0.6024       122
           1     0.9928    0.9650    0.9787      3143

    accuracy                         0.9596      3265
   macro avg     0.7345    0.8923    0.7906      3265
weighted avg     0.9735    0.9596    0.9646      3265

Test report (GCN):
              precision    recall  f1-score   support

           0     0.4572    0.7037    0.5543       243
           1     0.9883    0.9676    0.9778      6262

    accuracy                         0.9577      6505
   macro avg     0.7227    0.8356    0.7661      6505
weighted avg     0.9684    0.9577    0.9620      6505
```

2. GraphSAGE This model uses the GraphSAGE architecture

[132]:
```
# --- 2. GraphSAGE Model Definition ---

class GraphSAGEmodel(torch.nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, output_dim):
        super(GraphSAGEmodel, self).__init__()
```

```
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)

        # SAGEConv layers
        self.conv1 = SAGEConv(embed_dim, hidden_dim)
        self.conv2 = SAGEConv(hidden_dim, hidden_dim)
        self.conv3 = SAGEConv(hidden_dim, hidden_dim)

        self.pool = global_mean_pool # or global_max_pool

        self.classifier = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(hidden_dim, output_dim)
        )

    def forward(self, data):
        x, edge_index, batch = data.x, data.edge_index, data.batch

        x = self.embedding(x.squeeze(1))

        x = self.conv1(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=0.2, training=self.training)

        x = self.conv2(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=0.2, training=self.training)

        x = self.conv3(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=0.2, training=self.training)

        x = self.pool(x, batch)
        out = self.classifier(x)
        return out
```

[133]:
```
# ============================================================================
# GraphSAGE: Training with Multiple Learning Rates
# ============================================================================

sage_learning_rates = [1e-4, 5e-4, 1e-3]
sage_lr_labels = ['LR=1e-4', 'LR=5e-4', 'LR=1e-3']

sage_histories = {}
sage_models = {}
```

```python
sage_elapsed_times = {}

for lr, lr_label in zip(sage_learning_rates, sage_lr_labels):
    print(f"\n{'='*60}")
    print(f"Training GraphSAGE with {lr_label}")
    print('='*60)

    torch.manual_seed(42)
    np.random.seed(42)

    model_sage_temp = GraphSAGEmodel(
        vocab_size=VOCAB_SIZE_GNN,
        embed_dim=EMBED_DIM_GNN,
        hidden_dim=HIDDEN_DIM_GNN,
        output_dim=OUTPUT_DIM_GNN
    ).to(device)

    optimizer_temp = optim.AdamW(model_sage_temp.parameters(), lr=lr)

    model_sage_temp, history_temp, elapsed_temp = train_gnn_common(
        model_sage_temp,
        train_loader,
        val_loader,
        optimizer_temp,
        criterion_gnn,
        epochs=EPOCHS_GNN,
        min_delta=MIN_DELTA_GNN,
        patience=PATIENCE_GNN
    )

    sage_models[lr_label] = model_sage_temp
    sage_histories[lr_label] = history_temp
    sage_elapsed_times[lr_label] = elapsed_temp

    elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed_temp))
    print(f'\n{lr_label} - Training time: {elapsed_str}')
```

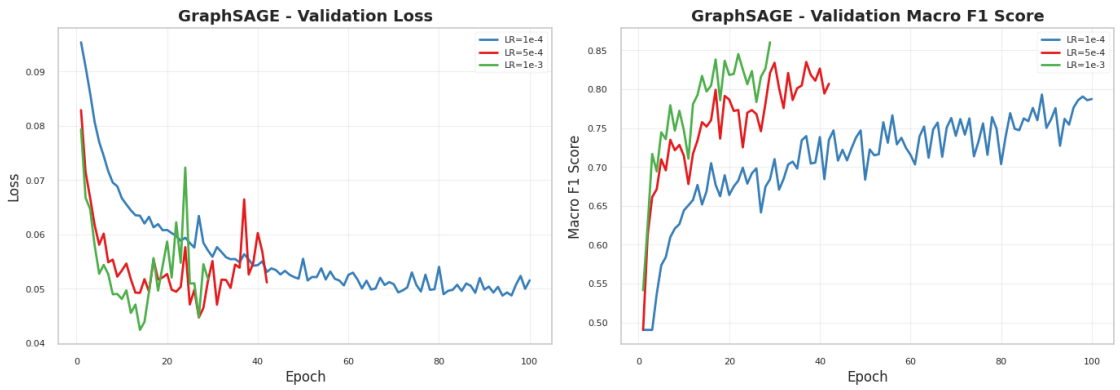
```
============================================================
Training GraphSAGE with LR=1e-4
============================================================
Epoch 1/100 - Train Loss: 0.1102 - Val Loss: 0.0954 - Val F1: 0.4905
Epoch 10/100 - Train Loss: 0.0657 - Val Loss: 0.0667 - Val F1: 0.6442
Epoch 20/100 - Train Loss: 0.0536 - Val Loss: 0.0608 - Val F1: 0.6640
Epoch 30/100 - Train Loss: 0.0474 - Val Loss: 0.0559 - Val F1: 0.7102
Epoch 40/100 - Train Loss: 0.0430 - Val Loss: 0.0543 - Val F1: 0.7385
Epoch 50/100 - Train Loss: 0.0386 - Val Loss: 0.0555 - Val F1: 0.6836
Epoch 60/100 - Train Loss: 0.0356 - Val Loss: 0.0525 - Val F1: 0.7156
```

```
Epoch 70/100 - Train Loss: 0.0322 - Val Loss: 0.0508 - Val F1: 0.7401
Epoch 80/100 - Train Loss: 0.0297 - Val Loss: 0.0540 - Val F1: 0.7036
Epoch 90/100 - Train Loss: 0.0267 - Val Loss: 0.0498 - Val F1: 0.7503
Epoch 100/100 - Train Loss: 0.0269 - Val Loss: 0.0515 - Val F1: 0.7874

LR=1e-4 - Training time: 00:05:00

============================================================
Training GraphSAGE with LR=5e-4
============================================================
Epoch 1/100 - Train Loss: 0.0974 - Val Loss: 0.0829 - Val F1: 0.4905
Epoch 10/100 - Train Loss: 0.0415 - Val Loss: 0.0533 - Val F1: 0.7142
Epoch 20/100 - Train Loss: 0.0290 - Val Loss: 0.0527 - Val F1: 0.7868
Epoch 30/100 - Train Loss: 0.0228 - Val Loss: 0.0551 - Val F1: 0.8340
Epoch 40/100 - Train Loss: 0.0197 - Val Loss: 0.0602 - Val F1: 0.8266
Early stopping at epoch 42 (best val loss 0.044698)

LR=5e-4 - Training time: 00:02:05

============================================================
Training GraphSAGE with LR=1e-3
============================================================
Epoch 1/100 - Train Loss: 0.0913 - Val Loss: 0.0794 - Val F1: 0.5415
Epoch 10/100 - Train Loss: 0.0328 - Val Loss: 0.0481 - Val F1: 0.7469
Epoch 20/100 - Train Loss: 0.0232 - Val Loss: 0.0587 - Val F1: 0.8184
Early stopping at epoch 29 (best val loss 0.042405)

LR=1e-3 - Training time: 00:01:25
```

[134]:
```python
# ============================================================================
# GraphSAGE: Plot Learning Rate Comparison
# ============================================================================

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

colors = ['#377eb8', '#e41a1c', '#4daf4a']

for (lr_label, history), color in zip(sage_histories.items(), colors):
    epochs_range = range(1, len(history['val_loss']) + 1)
    ax1.plot(epochs_range, history['val_loss'], label=lr_label, linewidth=2,␣
 ↪color=color)

ax1.set_title('GraphSAGE - Validation Loss', fontsize=14, fontweight='bold')
ax1.set_xlabel('Epoch', fontsize=12)
ax1.set_ylabel('Loss', fontsize=12)
ax1.legend()
ax1.grid(alpha=0.3)
```

```
for (lr_label, history), color in zip(sage_histories.items(), colors):
    epochs_range = range(1, len(history['val_f1_macro']) + 1)
    ax2.plot(epochs_range, history['val_f1_macro'], label=lr_label,␣
 ↪linewidth=2, color=color)

ax2.set_title('GraphSAGE - Validation Macro F1 Score', fontsize=14,␣
 ↪fontweight='bold')
ax2.set_xlabel('Epoch', fontsize=12)
ax2.set_ylabel('Macro F1 Score', fontsize=12)
ax2.legend()
ax2.grid(alpha=0.3)

plt.tight_layout()
save_plot(plt.gcf(), f"task4_sage_lr_comparison.png", save_dir)
plt.show()
```

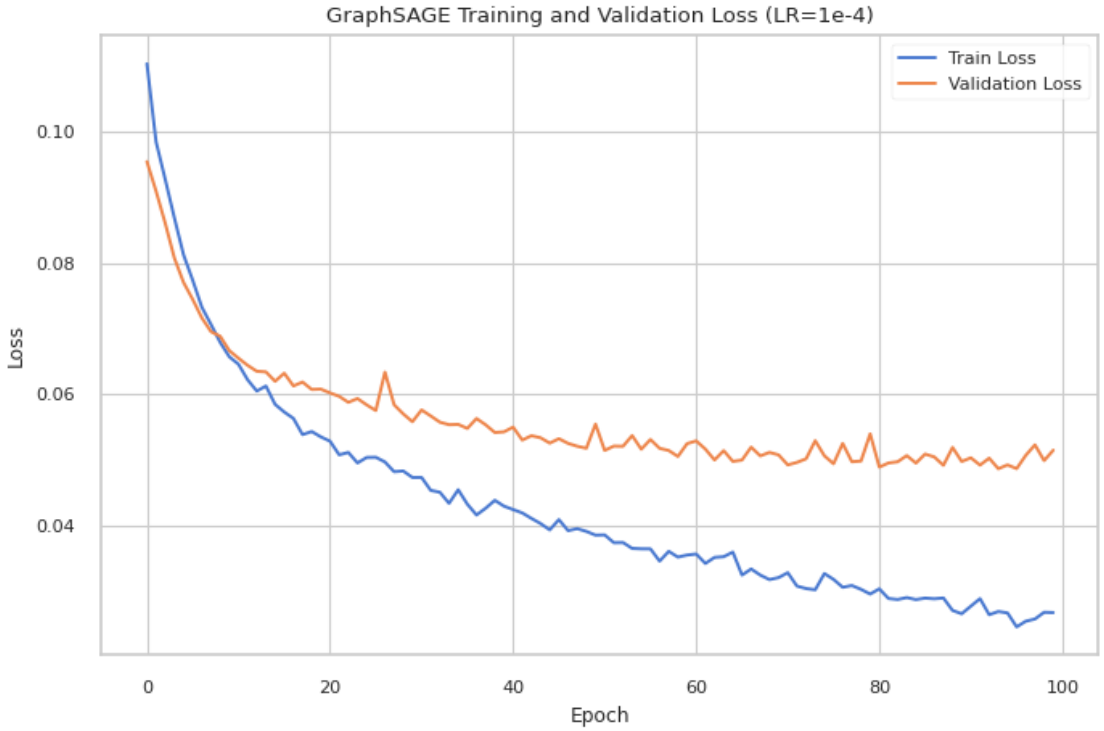
```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task4_plots/task4_sage_lr_comparison.png.png
```



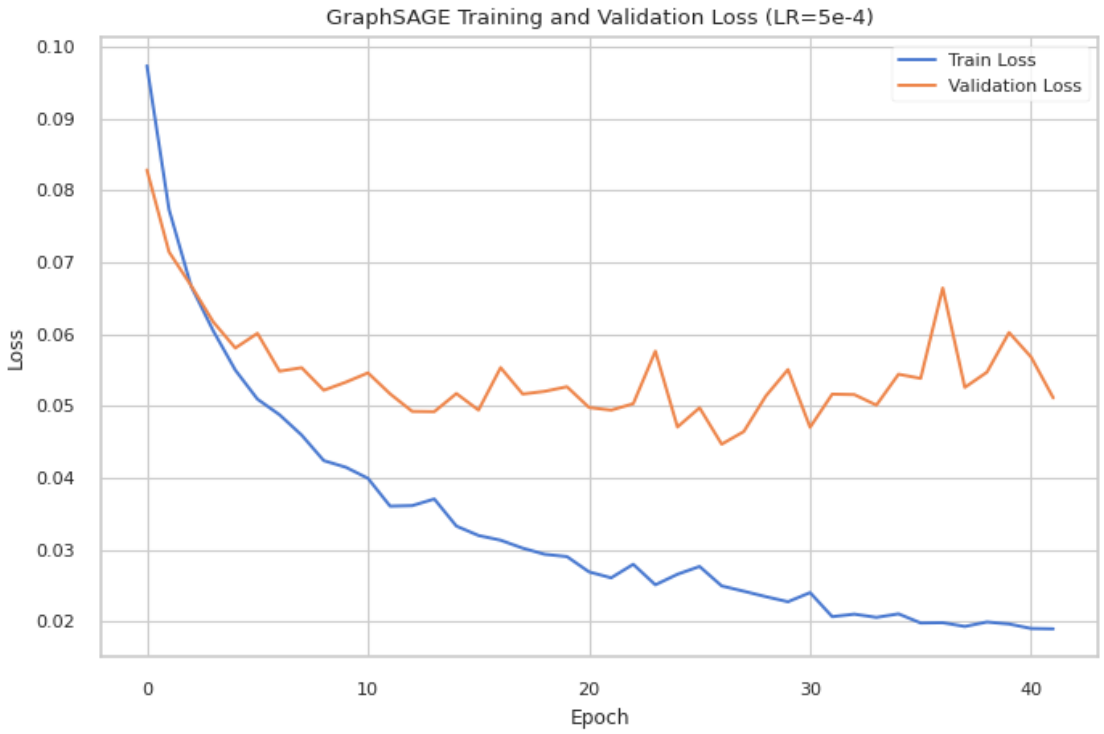
[135]:
```
for lr_label, model_sage in sage_models.items():
    history = sage_histories[lr_label]  # <-- Prendi la history per questo LR

    plt.figure(figsize=(8, 5))
    plt.plot(history['train_loss'], label='Train Loss')
    plt.plot(history['val_loss'], label='Validation Loss')
    plt.title(f'GraphSAGE Training and Validation Loss ({lr_label})')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend()
    plt.grid(True)
    save_plot(plt.gcf(), f'graphsage_loss_curves_{lr_label.replace("=", "")}',␣
 ↪save_dir)
```

```
    plt.show()
```

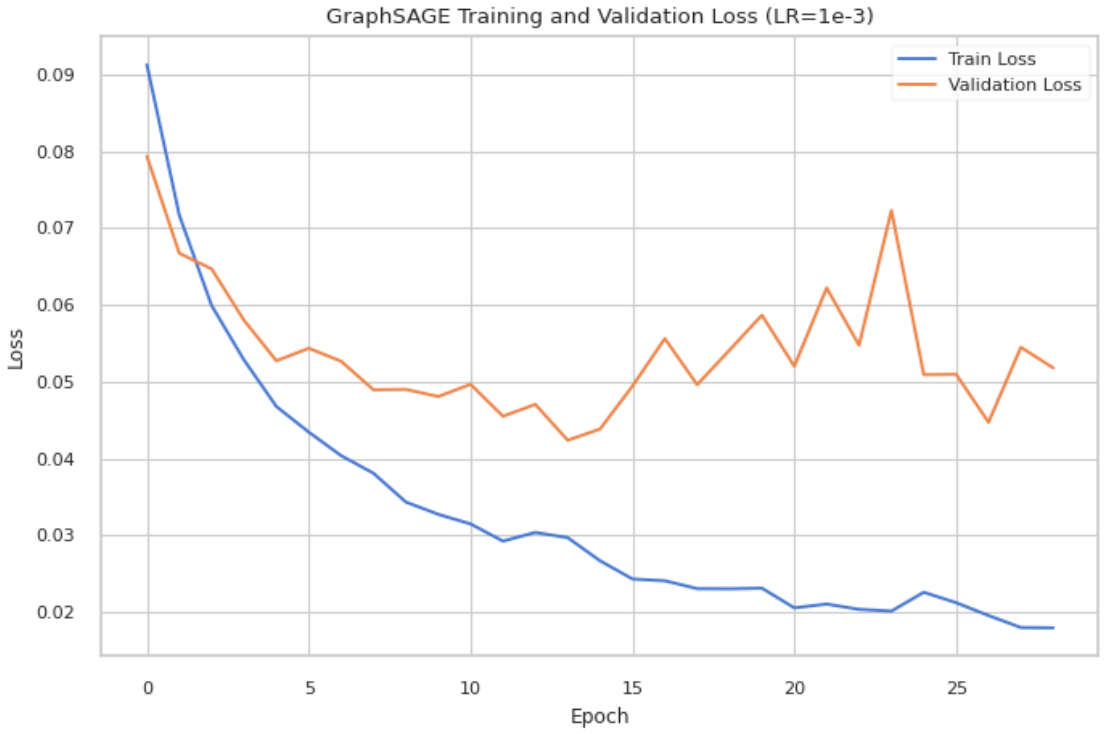
```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task4_plots/graphsage_loss_curves_LR1e-4.png
```




```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task4_plots/graphsage_loss_curves_LR5e-4.png
```



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/graphsage_loss_curves_LR1e-3.png



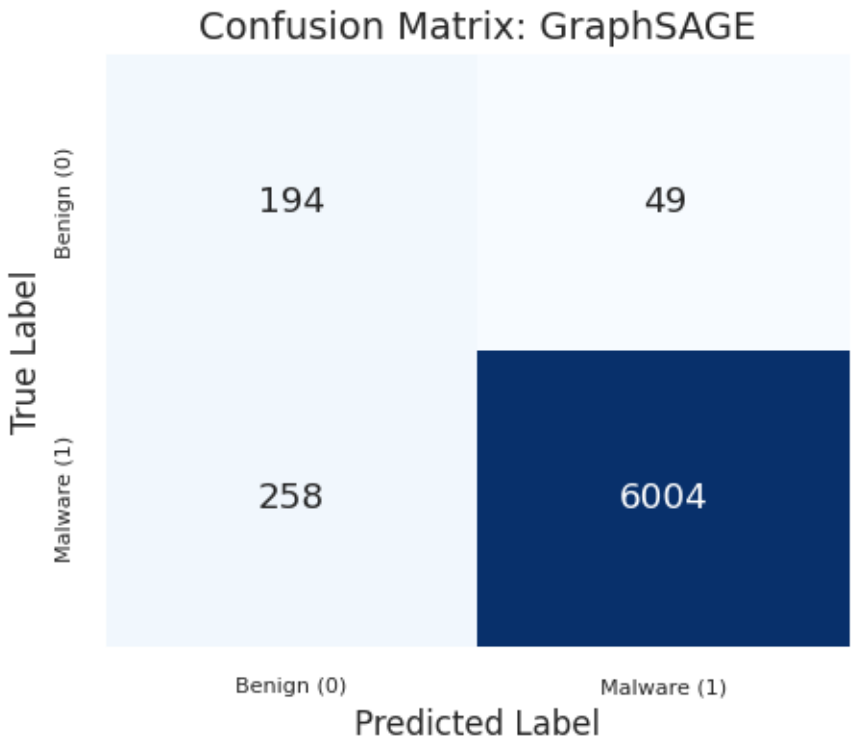
[136]:
```
# --- Evaluate GraphSAGE ---
for lr_label, model_sage in sage_models.items():
  y_val_true_sage, y_val_pred_sage = predict_loader_gnn(model_sage, val_loader)
  print('\nValidation report (GraphSAGE):')
  print(classification_report(y_val_true_sage, y_val_pred_sage, digits=4,␣
 ↪zero_division=0))

  y_test_true_sage, y_test_pred_sage = predict_loader_gnn(model_sage,␣
 ↪test_loader)
  print('Test report (GraphSAGE):')
  print(classification_report(y_test_true_sage, y_test_pred_sage, digits=4,␣
 ↪zero_division=0))
```

```
Validation report (GraphSAGE):
              precision    recall  f1-score   support

           0     0.4160    0.8115    0.5500       122
           1     0.9924    0.9558    0.9737      3143

    accuracy                         0.9504      3265
   macro avg     0.7042    0.8836    0.7619      3265
weighted avg     0.9709    0.9504    0.9579      3265

Test report (GraphSAGE):
              precision    recall  f1-score   support

           0     0.4292    0.7984    0.5583       243
           1     0.9919    0.9588    0.9751      6262

    accuracy                         0.9528      6505
   macro avg     0.7106    0.8786    0.7667      6505
weighted avg     0.9709    0.9528    0.9595      6505


Validation report (GraphSAGE):
              precision    recall  f1-score   support

           0     0.3811    0.8279    0.5220       122
           1     0.9930    0.9478    0.9699      3143

    accuracy                         0.9433      3265
   macro avg     0.6871    0.8878    0.7459      3265
weighted avg     0.9701    0.9433    0.9531      3265
```

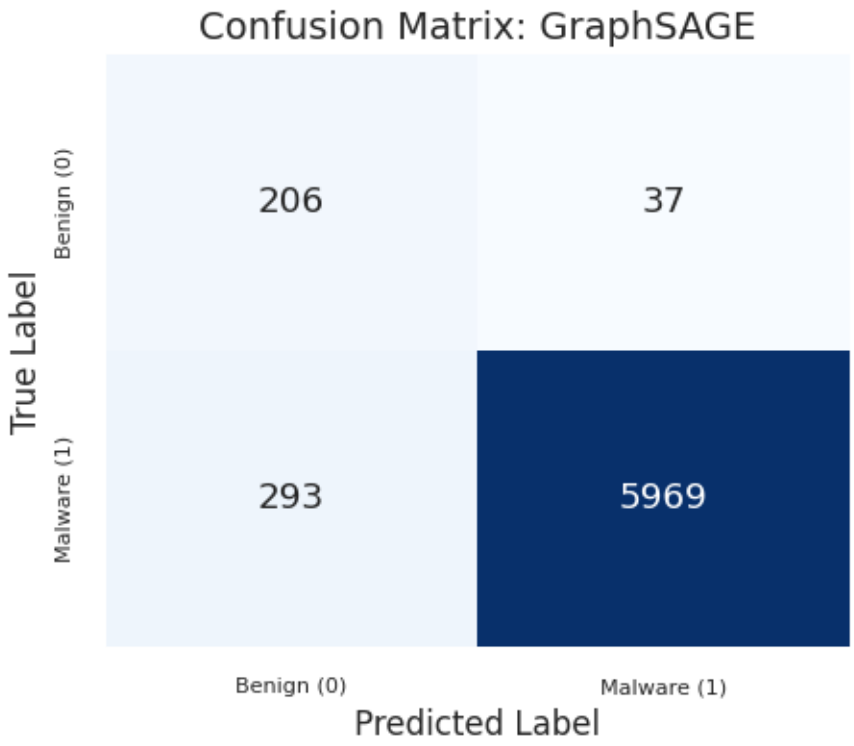
```
Test report (GraphSAGE):
              precision    recall  f1-score   support

           0     0.4128    0.8477    0.5553       243
           1     0.9938    0.9532    0.9731      6262

    accuracy                         0.9493      6505
   macro avg     0.7033    0.9005    0.7642      6505
weighted avg     0.9721    0.9493    0.9575      6505


Validation report (GraphSAGE):
              precision    recall  f1-score   support

           0     0.5759    0.7459    0.6500       122
           1     0.9900    0.9787    0.9843      3143

    accuracy                         0.9700      3265
   macro avg     0.7830    0.8623    0.8172      3265
weighted avg     0.9746    0.9700    0.9718      3265

Test report (GraphSAGE):
              precision    recall  f1-score   support

           0     0.5600    0.6914    0.6188       243
           1     0.9879    0.9789    0.9834      6262

    accuracy                         0.9682      6505
   macro avg     0.7740    0.8351    0.8011      6505
weighted avg     0.9719    0.9682    0.9698      6505
```

[137]:
```python
for lr_label, model_sage in sage_models.items():
  plot_cm_task4(model=model_sage,
                loader=test_loader,
                model_name="GraphSAGE",
                save_dir=results_path + 'images/task4_plots/')
```

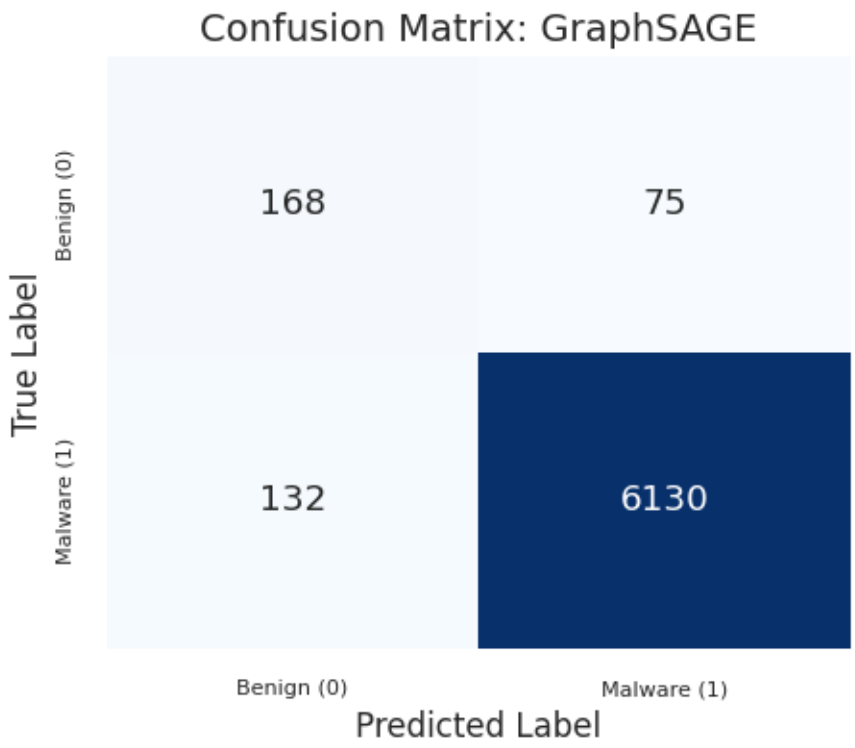
```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task4_plots/cm_graphsage.png
```




```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task4_plots/cm_graphsage.png
```




```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task4_plots/cm_graphsage.png
```



[138]:
```
# ============================================================================
# GraphSAGE: Select Best Model and Evaluate
# ============================================================================

best_sage_lr, model_sage, history_sage, elapsed_sage, sage_selection_table =␣
 ↪select_best_gnn_model(
    architecture_name='GraphSAGE',
    models=sage_models,
    histories=sage_histories,
    elapsed_times=sage_elapsed_times,
    val_loader=val_loader,
)

y_val_true_sage, y_val_pred_sage = predict_loader_gnn(model_sage, val_loader)
print('\nValidation report (GraphSAGE):')
print(classification_report(y_val_true_sage, y_val_pred_sage, digits=4,␣
 ↪zero_division=0))

y_test_true_sage, y_test_pred_sage = predict_loader_gnn(model_sage, test_loader)
print('Test report (GraphSAGE):')
print(classification_report(y_test_true_sage, y_test_pred_sage, digits=4,␣
 ↪zero_division=0))
```

```
Automatic selection for GraphSAGE
Ranking criterion: final validation macro F1, then validation accuracy, then
minimum validation loss
lr_label  final_val_f1_macro  val_accuracy  eval_val_f1_macro  min_val_loss
elapsed_s
 LR=1e-3            0.860371      0.969985           0.817160      0.042405
85.732514
 LR=5e-4            0.807045      0.943338           0.745924      0.044698
125.376887
 LR=1e-4            0.787388      0.950383           0.761872      0.048727
300.847718

Selected best GraphSAGE model: LR=1e-3
Final Val F1 Macro: 0.8604
Validation Accuracy: 0.9700

Validation report (GraphSAGE):
              precision    recall  f1-score   support

           0     0.5759    0.7459    0.6500       122
           1     0.9900    0.9787    0.9843      3143

    accuracy                         0.9700      3265
   macro avg     0.7830    0.8623    0.8172      3265
weighted avg     0.9746    0.9700    0.9718      3265

Test report (GraphSAGE):
              precision    recall  f1-score   support

           0     0.5600    0.6914    0.6188       243
           1     0.9879    0.9789    0.9834      6262

    accuracy                         0.9682      6505
   macro avg     0.7740    0.8351    0.8011      6505
weighted avg     0.9719    0.9682    0.9698      6505
```

3. GAT This model uses the Graph Attention Network (GAT) architecture.

[139]:
```python
class GATmodel(torch.nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, output_dim, heads=4):
        super(GATmodel, self).__init__()

        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)

        # GAT Layers
        # Layer 1: Output is hidden_dim * heads
```

```
        self.conv1 = GATConv(embed_dim, hidden_dim, heads=heads, dropout=0.2)

        # Layer 2: Input is hidden_dim * heads. We set heads=1 for the final␣
↪graph embedding before pooling.
        # concat=False means we average the heads or just output one head.
        self.conv2 = GATConv(hidden_dim * heads, hidden_dim, heads=1,␣
↪concat=False, dropout=0.2)

        # Layer 3: Input should now be hidden_dim (output of conv2)
        self.conv3 = GATConv(hidden_dim, hidden_dim, heads=1, concat=False,␣
↪dropout=0.2)

        self.pool = global_mean_pool

        self.classifier = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, output_dim)
        )

    def forward(self, data):
        x, edge_index, batch = data.x, data.edge_index, data.batch

        x = self.embedding(x.squeeze(1))

        # Conv 1
        x = self.conv1(x, edge_index)
        x = F.elu(x)

        # Conv 2
        x = self.conv2(x, edge_index)

        # Conv 3
        x = self.conv3(x, edge_index)

        # Pooling
        x = self.pool(x, batch)

        # Classifier
        out = self.classifier(x)
        return out
```

[140]:
```
# ============================================================================
# GAT: Training with Multiple Learning Rates
# ============================================================================

gat_learning_rates = [5e-5, 1e-4, 5e-4]
```

```
gat_lr_labels = ['LR=5e-5', 'LR=1e-4', 'LR=5e-4']

gat_histories = {}
gat_models = {}
gat_elapsed_times = {}

for lr, lr_label in zip(gat_learning_rates, gat_lr_labels):
    print(f"\n{'='*60}")
    print(f"Training GAT with {lr_label}")
    print('='*60)

    torch.manual_seed(42)
    np.random.seed(42)

    model_gat_temp = GATmodel(
        vocab_size=VOCAB_SIZE_GNN,
        embed_dim=EMBED_DIM_GNN,
        hidden_dim=HIDDEN_DIM_GNN,
        output_dim=OUTPUT_DIM_GNN,
        heads=4
    ).to(device)

    optimizer_temp = optim.AdamW(model_gat_temp.parameters(), lr=lr)

    model_gat_temp, history_temp, elapsed_temp = train_gnn_common(
        model_gat_temp,
        train_loader,
        val_loader,
        optimizer_temp,
        criterion_gnn,
        epochs=EPOCHS_GNN,
        min_delta=MIN_DELTA_GNN,
        patience=PATIENCE_GNN
    )

    gat_models[lr_label] = model_gat_temp
    gat_histories[lr_label] = history_temp
    gat_elapsed_times[lr_label] = elapsed_temp

    elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed_temp))
    print(f'\n{lr_label} - Training time: {elapsed_str}')
```

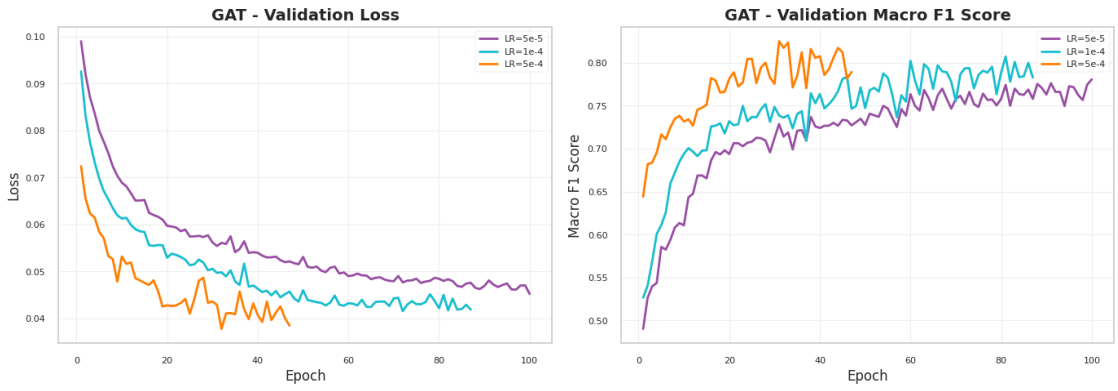
```
============================================================
Training GAT with LR=5e-5
============================================================
Epoch 1/100 - Train Loss: 0.1187 - Val Loss: 0.0990 - Val F1: 0.4902
Epoch 10/100 - Train Loss: 0.0725 - Val Loss: 0.0689 - Val F1: 0.6108
```

```
Epoch 20/100 - Train Loss: 0.0611 - Val Loss: 0.0597 - Val F1: 0.6937
Epoch 30/100 - Train Loss: 0.0557 - Val Loss: 0.0562 - Val F1: 0.7125
Epoch 40/100 - Train Loss: 0.0509 - Val Loss: 0.0539 - Val F1: 0.7240
Epoch 50/100 - Train Loss: 0.0485 - Val Loss: 0.0531 - Val F1: 0.7274
Epoch 60/100 - Train Loss: 0.0435 - Val Loss: 0.0490 - Val F1: 0.7633
Epoch 70/100 - Train Loss: 0.0432 - Val Loss: 0.0479 - Val F1: 0.7579
Epoch 80/100 - Train Loss: 0.0420 - Val Loss: 0.0484 - Val F1: 0.7578
Epoch 90/100 - Train Loss: 0.0413 - Val Loss: 0.0469 - Val F1: 0.7629
Epoch 100/100 - Train Loss: 0.0391 - Val Loss: 0.0452 - Val F1: 0.7806

LR=5e-5 - Training time: 00:07:33

============================================================
Training GAT with LR=1e-4
============================================================
Epoch 1/100 - Train Loss: 0.1088 - Val Loss: 0.0926 - Val F1: 0.5266
Epoch 10/100 - Train Loss: 0.0629 - Val Loss: 0.0612 - Val F1: 0.6938
Epoch 20/100 - Train Loss: 0.0523 - Val Loss: 0.0529 - Val F1: 0.7318
Epoch 30/100 - Train Loss: 0.0465 - Val Loss: 0.0505 - Val F1: 0.7486
Epoch 40/100 - Train Loss: 0.0424 - Val Loss: 0.0463 - Val F1: 0.7634
Epoch 50/100 - Train Loss: 0.0380 - Val Loss: 0.0460 - Val F1: 0.7471
Epoch 60/100 - Train Loss: 0.0343 - Val Loss: 0.0432 - Val F1: 0.8021
Epoch 70/100 - Train Loss: 0.0363 - Val Loss: 0.0442 - Val F1: 0.7551
Epoch 80/100 - Train Loss: 0.0326 - Val Loss: 0.0421 - Val F1: 0.7885
Early stopping at epoch 87 (best val loss 0.041501)

LR=1e-4 - Training time: 00:06:32

============================================================
Training GAT with LR=5e-4
============================================================
Epoch 1/100 - Train Loss: 0.0914 - Val Loss: 0.0724 - Val F1: 0.6441
Epoch 10/100 - Train Loss: 0.0491 - Val Loss: 0.0531 - Val F1: 0.7315
Epoch 20/100 - Train Loss: 0.0385 - Val Loss: 0.0428 - Val F1: 0.7813
Epoch 30/100 - Train Loss: 0.0333 - Val Loss: 0.0436 - Val F1: 0.7754
Epoch 40/100 - Train Loss: 0.0290 - Val Loss: 0.0407 - Val F1: 0.8075
Early stopping at epoch 47 (best val loss 0.037721)

LR=5e-4 - Training time: 00:03:34
```

[141]:
```python
# ============================================================================
# GAT: Plot Learning Rate Comparison
# ============================================================================

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

colors = ['#984ea3', '#17becf', '#ff7f00']
```

```
for (lr_label, history), color in zip(gat_histories.items(), colors):
    epochs_range = range(1, len(history['val_loss']) + 1)
    ax1.plot(epochs_range, history['val_loss'], label=lr_label, linewidth=2,␣
 ↪color=color)

ax1.set_title('GAT - Validation Loss', fontsize=14, fontweight='bold')
ax1.set_xlabel('Epoch', fontsize=12)
ax1.set_ylabel('Loss', fontsize=12)
ax1.legend()
ax1.grid(alpha=0.3)

for (lr_label, history), color in zip(gat_histories.items(), colors):
    epochs_range = range(1, len(history['val_f1_macro']) + 1)
    ax2.plot(epochs_range, history['val_f1_macro'], label=lr_label,␣
 ↪linewidth=2, color=color)

ax2.set_title('GAT - Validation Macro F1 Score', fontsize=14, fontweight='bold')
ax2.set_xlabel('Epoch', fontsize=12)
ax2.set_ylabel('Macro F1 Score', fontsize=12)
ax2.legend()
ax2.grid(alpha=0.3)

plt.tight_layout()
save_plot(plt.gcf(), f"task4_gat_lr_comparison.png", save_dir)
plt.show()
```

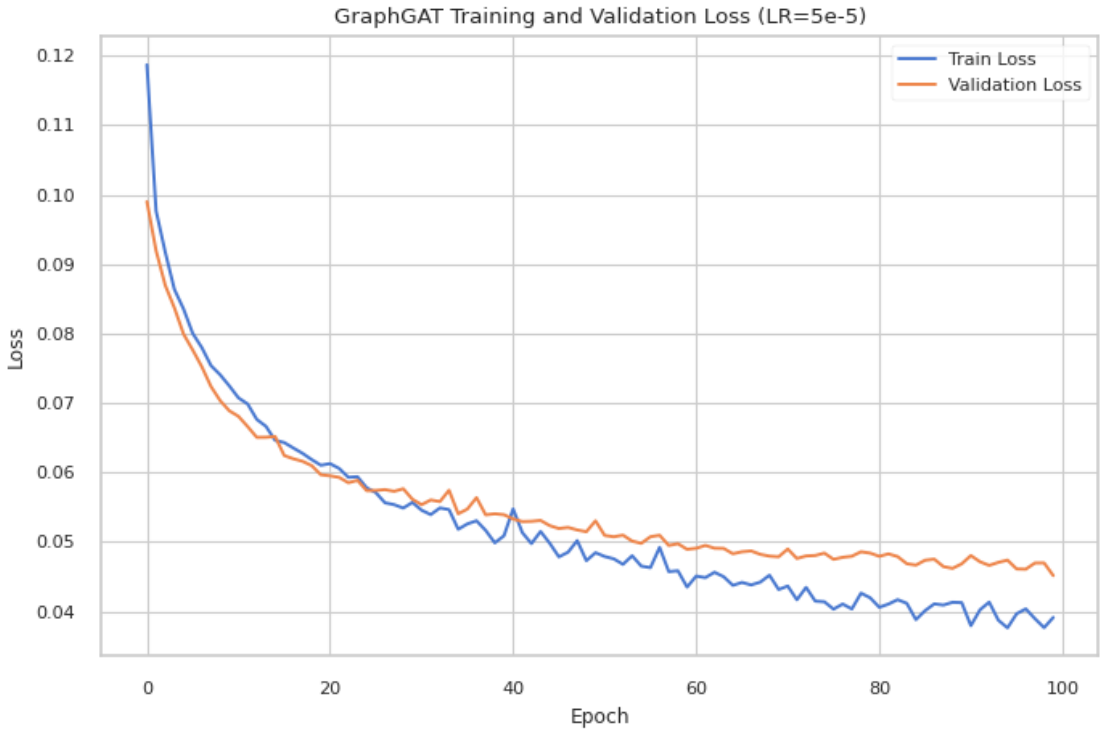
```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task4_plots/task4_gat_lr_comparison.png.png
```



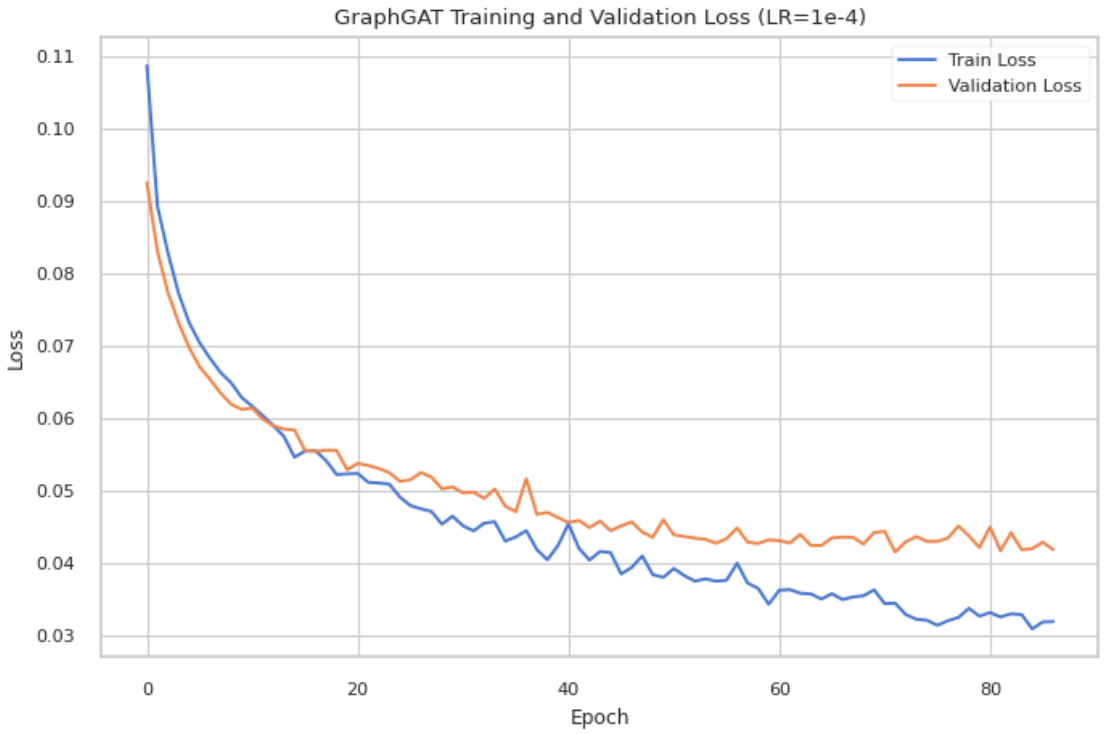
[142]:
```
for lr_label, model_gat in gat_models.items():
    history = gat_histories[lr_label]  # <-- Prendi la history per questo LR
```

```
    plt.figure(figsize=(8, 5))
    plt.plot(history['train_loss'], label='Train Loss')
    plt.plot(history['val_loss'], label='Validation Loss')
    plt.title(f'GraphGAT Training and Validation Loss ({lr_label})')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend()
    plt.grid(True)
    save_plot(plt.gcf(), f'graphgat_loss_curves_{lr_label.replace("=", "")}',␣
↪save_dir)
    plt.show()
```

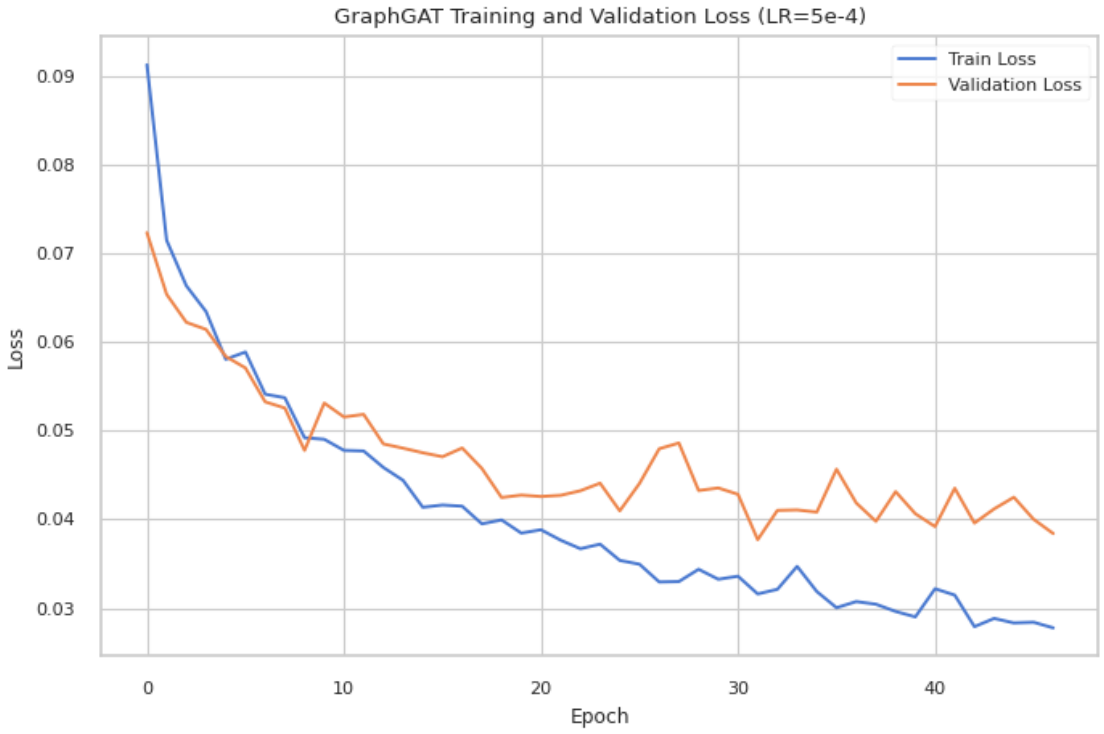
```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task4_plots/graphgat_loss_curves_LR5e-5.png
```




```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task4_plots/graphgat_loss_curves_LR1e-4.png
```



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/graphgat_loss_curves_LR5e-4.png



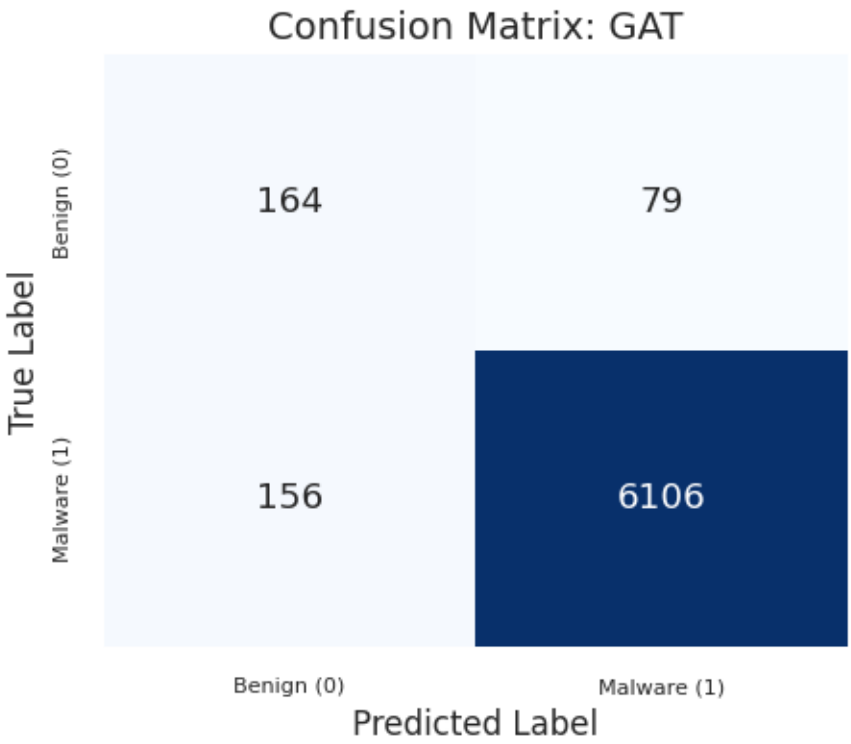
[143]:
```
# --- Evaluate GAT ---
for lr_label, model_gat in gat_models.items():
  y_val_true_gat, y_val_pred_gat = predict_loader_gnn(model_gat, val_loader)
  print('\nValidation report (GAT):')
  print(classification_report(y_val_true_gat, y_val_pred_gat, digits=4,␣
 ↪zero_division=0))

  y_test_true_gat, y_test_pred_gat = predict_loader_gnn(model_gat, test_loader)
  print('Test report (GAT):')
  print(classification_report(y_test_true_gat, y_test_pred_gat, digits=4,␣
 ↪zero_division=0))
```

```
Validation report (GAT):
              precision    recall  f1-score   support

           0     0.5061    0.6803    0.5804       122
           1     0.9874    0.9742    0.9808      3143

    accuracy                         0.9632      3265
   macro avg     0.7468    0.8273    0.7806      3265
weighted avg     0.9694    0.9632    0.9658      3265

Test report (GAT):
              precision    recall  f1-score   support

           0     0.5125    0.6749    0.5826       243
           1     0.9872    0.9751    0.9811      6262

    accuracy                         0.9639      6505
   macro avg     0.7499    0.8250    0.7819      6505
weighted avg     0.9695    0.9639    0.9662      6505


Validation report (GAT):
              precision    recall  f1-score   support

           0     0.5385    0.6885    0.6043       122
           1     0.9878    0.9771    0.9824      3143

    accuracy                         0.9663      3265
   macro avg     0.7631    0.8328    0.7934      3265
weighted avg     0.9710    0.9663    0.9683      3265

Test report (GAT):
```

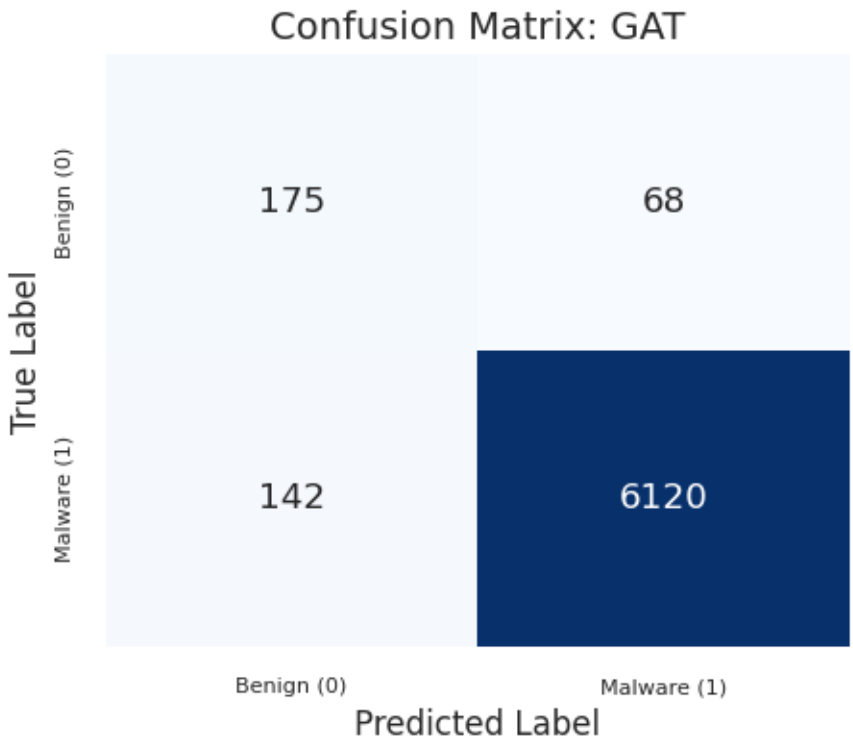
```
              precision    recall  f1-score   support

           0     0.5521    0.7202    0.6250       243
           1     0.9890    0.9773    0.9831      6262

    accuracy                         0.9677      6505
   macro avg     0.7705    0.8487    0.8041      6505
weighted avg     0.9727    0.9677    0.9698      6505


Validation report (GAT):
              precision    recall  f1-score   support

           0     0.5759    0.7459    0.6500       122
           1     0.9900    0.9787    0.9843      3143

    accuracy                         0.9700      3265
   macro avg     0.7830    0.8623    0.8172      3265
weighted avg     0.9746    0.9700    0.9718      3265

Test report (GAT):
              precision    recall  f1-score   support

           0     0.5307    0.6749    0.5942       243
           1     0.9872    0.9768    0.9820      6262

    accuracy                         0.9656      6505
   macro avg     0.7590    0.8259    0.7881      6505
weighted avg     0.9702    0.9656    0.9675      6505
```

[144]:
```python
for lr_label, model_gat in gat_models.items():
    plot_cm_task4(model=model_gat,
                  loader=test_loader,
                  model_name="GAT",
                  save_dir=results_path + 'images/task4_plots/')
```

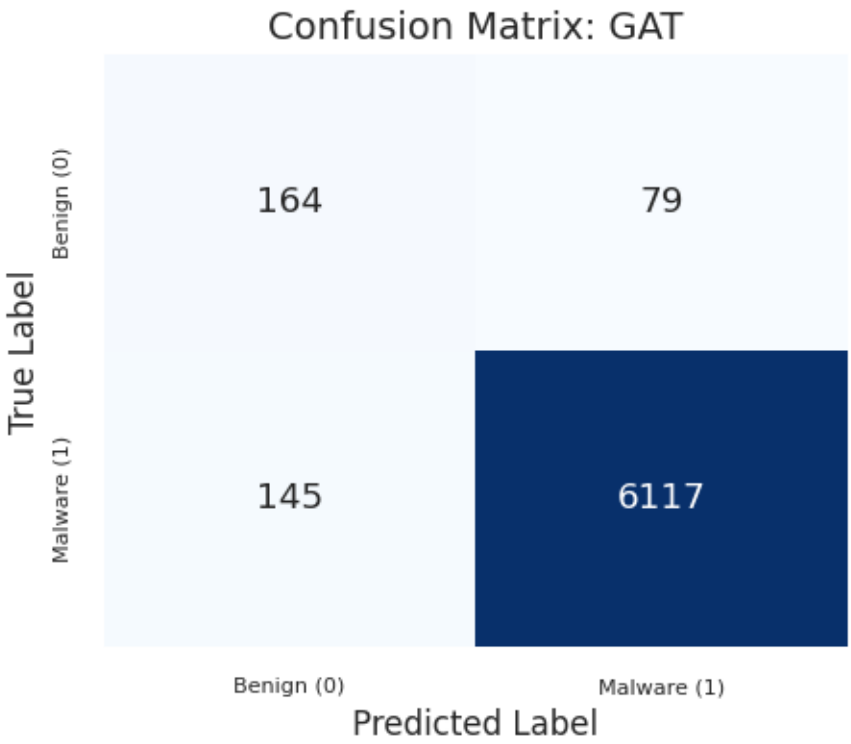
```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task4_plots/cm_gat.png
```



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_gat.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task4_plots/cm_gat.png



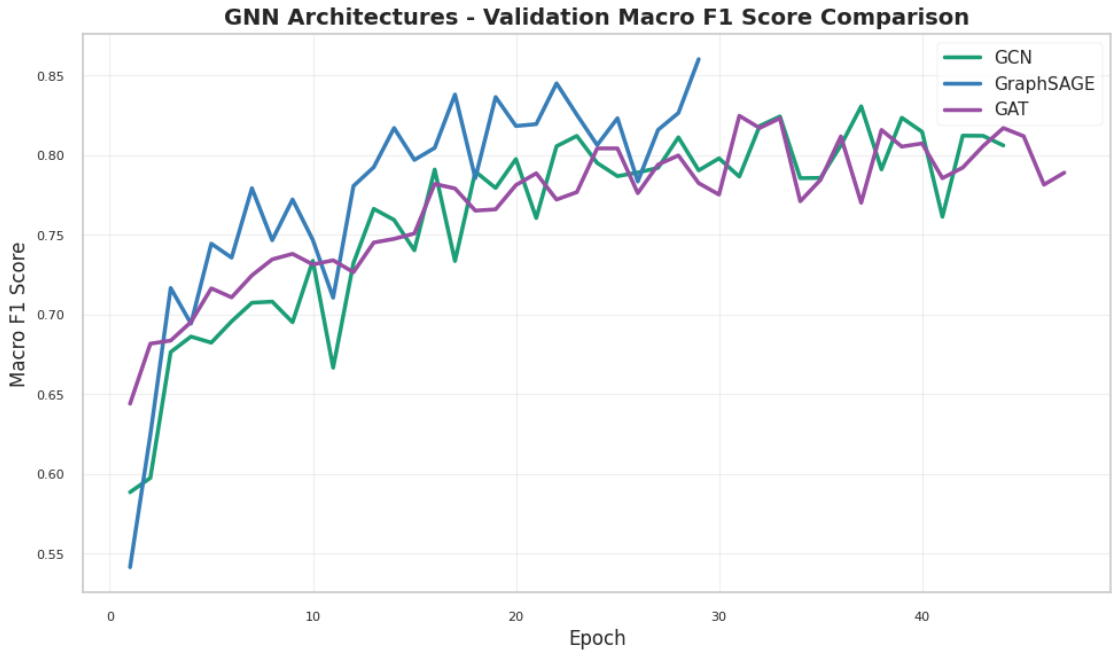
[145]:
```
# ============================================================================
# GAT: Select Best Model and Evaluate
# ============================================================================

best_gat_lr, model_gat, history_gat, elapsed_gat, gat_selection_table =␣
 ↪select_best_gnn_model(
    architecture_name='GAT',
    models=gat_models,
    histories=gat_histories,
    elapsed_times=gat_elapsed_times,
    val_loader=val_loader,
)

y_val_true_gat, y_val_pred_gat = predict_loader_gnn(model_gat, val_loader)
print('\nValidation report (GAT):')
print(classification_report(y_val_true_gat, y_val_pred_gat, digits=4,␣
 ↪zero_division=0))

y_test_true_gat, y_test_pred_gat = predict_loader_gnn(model_gat, test_loader)
print('Test report (GAT):')
print(classification_report(y_test_true_gat, y_test_pred_gat, digits=4,␣
 ↪zero_division=0))
```

```
Automatic selection for GAT
Ranking criterion: final validation macro F1, then validation accuracy, then
minimum validation loss
lr_label  final_val_f1_macro  val_accuracy  eval_val_f1_macro  min_val_loss
elapsed_s
 LR=5e-4            0.789135      0.969985           0.817160      0.037721
214.241665
 LR=1e-4            0.782644      0.966309           0.793361      0.041501
392.810866
 LR=5e-5            0.780601      0.963247           0.780601      0.045168
453.424327

Selected best GAT model: LR=5e-4
Final Val F1 Macro: 0.7891
Validation Accuracy: 0.9700

Validation report (GAT):
              precision    recall  f1-score   support

           0     0.5759    0.7459    0.6500       122
           1     0.9900    0.9787    0.9843      3143

    accuracy                         0.9700      3265
   macro avg     0.7830    0.8623    0.8172      3265
weighted avg     0.9746    0.9700    0.9718      3265

Test report (GAT):
              precision    recall  f1-score   support

           0     0.5307    0.6749    0.5942       243
           1     0.9872    0.9768    0.9820      6262

    accuracy                         0.9656      6505
   macro avg     0.7590    0.8259    0.7881      6505
weighted avg     0.9702    0.9656    0.9675      6505
```

[146]:
```python
# ============================================================================
# GNN Comparison: Validation Macro F1 Score (Best LR for each model)
# ============================================================================

fig, ax = plt.subplots(figsize=(10, 6))

# Plot best model for each architecture
best_models_gnn = {
    'GCN': history_gcn,
    'GraphSAGE': history_sage,
```

```
    'GAT': history_gat
}

colors_gnn = ['#1b9e77', '#377eb8', '#984ea3']

for (name, history), color in zip(best_models_gnn.items(), colors_gnn):
    epochs_range = range(1, len(history['val_f1_macro']) + 1)
    ax.plot(epochs_range, history['val_f1_macro'], label=name, linewidth=2.5,␣
 ↪color=color)

ax.set_title('GNN Architectures - Validation Macro F1 Score Comparison',␣
 ↪fontsize=14, fontweight='bold')
ax.set_xlabel('Epoch', fontsize=12)
ax.set_ylabel('Macro F1 Score', fontsize=12)
ax.legend(fontsize=11)
ax.grid(alpha=0.3)

plt.tight_layout()
save_plot(plt.gcf(), f"task4_gnn_comparison_f1_macro", save_dir)
plt.show()
```

```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task4_plots/task4_gnn_comparison_f1_macro.png
```



[147]:
```
# ============================================================================
# FINAL REPORT: GNN Architecture Comparison
# ============================================================================

def get_test_metrics(y_true, y_pred):
    """Extract test metrics from predictions."""
    report = classification_report(y_true, y_pred, output_dict=True,␣
 ↪zero_division=0)
    return {
        'precision': report['macro avg']['precision'],
        'recall': report['macro avg']['recall'],
        'f1_score': report['macro avg']['f1-score'],
        'accuracy': report['accuracy'],
        'benign_recall': report['0']['recall'],
        'malware_recall': report['1']['recall'],
    }

metrics_gcn = get_test_metrics(y_test_true_gcn, y_test_pred_gcn)
metrics_sage = get_test_metrics(y_test_true_sage, y_test_pred_sage)
metrics_gat = get_test_metrics(y_test_true_gat, y_test_pred_gat)

gnn_summary = pd.DataFrame([
    {
        'architecture': 'GCN',
        'selected_lr': best_gcn_lr,
        'val_final_macro_f1': history_gcn['val_f1_macro'][-1],
        'test_macro_f1': metrics_gcn['f1_score'],
        'test_accuracy': metrics_gcn['accuracy'],
        'test_benign_recall': metrics_gcn['benign_recall'],
        'test_malware_recall': metrics_gcn['malware_recall'],
        'training_time_s': elapsed_gcn,
    },
    {
        'architecture': 'GraphSAGE',
        'selected_lr': best_sage_lr,
        'val_final_macro_f1': history_sage['val_f1_macro'][-1],
        'test_macro_f1': metrics_sage['f1_score'],
        'test_accuracy': metrics_sage['accuracy'],
        'test_benign_recall': metrics_sage['benign_recall'],
        'test_malware_recall': metrics_sage['malware_recall'],
        'training_time_s': elapsed_sage,
    },
    {
        'architecture': 'GAT',
        'selected_lr': best_gat_lr,
        'val_final_macro_f1': history_gat['val_f1_macro'][-1],
        'test_macro_f1': metrics_gat['f1_score'],
```

```
        'test_accuracy': metrics_gat['accuracy'],
        'test_benign_recall': metrics_gat['benign_recall'],
        'test_malware_recall': metrics_gat['malware_recall'],
        'training_time_s': elapsed_gat,
    },
])

gnn_summary = gnn_summary.sort_values(
    by=['test_macro_f1', 'test_accuracy', 'test_benign_recall'],
    ascending=[False, False, False]
).reset_index(drop=True)

best_gnn_row = gnn_summary.iloc[0]

print("\n" + "="*120)
print("FINAL REPORT: GNN Architecture Comparison (Test Set)")
print("Ranking criterion: test macro F1, then test accuracy, then benign␣
 ↪recall")
print("="*120)
display(gnn_summary)
print(
    f"Best selected GNN: {best_gnn_row['architecture']} "
    f"({best_gnn_row['selected_lr']}) | "
    f"test macro F1={best_gnn_row['test_macro_f1']:.4f}, "
    f"accuracy={best_gnn_row['test_accuracy']:.4f}, "
    f"benign recall={best_gnn_row['test_benign_recall']:.4f}"
)
```

```
========================================================================================
========================================
FINAL REPORT: GNN Architecture Comparison (Test Set)
Ranking criterion: test macro F1, then test accuracy, then benign recall
========================================================================================
========================================

  architecture selected_lr  val_final_macro_f1  test_macro_f1  test_accuracy  \
0    GraphSAGE     LR=1e-3            0.860371       0.801090       0.968178
1          GAT     LR=5e-4            0.789135       0.788111       0.965565
2          GCN     LR=1e-3            0.806204       0.766053       0.957725

   test_benign_recall  test_malware_recall  training_time_s
0            0.691358             0.978920        85.732514
1            0.674897             0.976844       214.241665
2            0.703704             0.967582       151.882000

Best selected GNN: GraphSAGE (LR=1e-3) | test macro F1=0.8011, accuracy=0.9682,
benign recall=0.6914
```

Q: How does each model perform compared to the previous architectures? Can you beat the baseline? Yes, the graph models beat the frequency baseline in macro F1, although the baseline remains very competitive in accuracy. The baseline reaches test accuracy `96.97%` and macro F1 `0.7223`, but its benign recall is only `0.3457`. This confirms that accuracy alone is misleading on this imbalanced dataset.

Among the GNNs, GraphSAGE is the strongest selected architecture in this run. It selects `LR=1e-3` and reaches test macro F1 `0.8011`, accuracy `96.82%`, and benign recall `0.6914`. GAT selects `LR=5e-4` and obtains test macro F1 `0.7881`, while GCN selects `LR=1e-3` and obtains test macro F1 `0.7661`. All three GNNs substantially improve benign recall with respect to the frequency baseline, while keeping high malware recall.

Compared with the previous architectures, FFNNs remain the weakest family: even with learnable embeddings and embedding-size tuning, the best FFNN test macro F1 is around `0.59`, mainly because the model relies on fixed-length truncation and cannot explicitly model temporal or structural dependencies. RNNs improve substantially: the selected LSTM reaches test macro F1 `0.8222`, making it the best overall model in this run. GNNs are slightly below the LSTM, but clearly stronger than FFNNs and the frequency baseline in terms of balanced classification.

Overall, the experiments show that both sequential information and transition-graph structure are useful. The frequency baseline is strong because API-call counts are informative, but RNNs and GNNs provide a better balance between malware detection and benign recognition.

---

[147]: